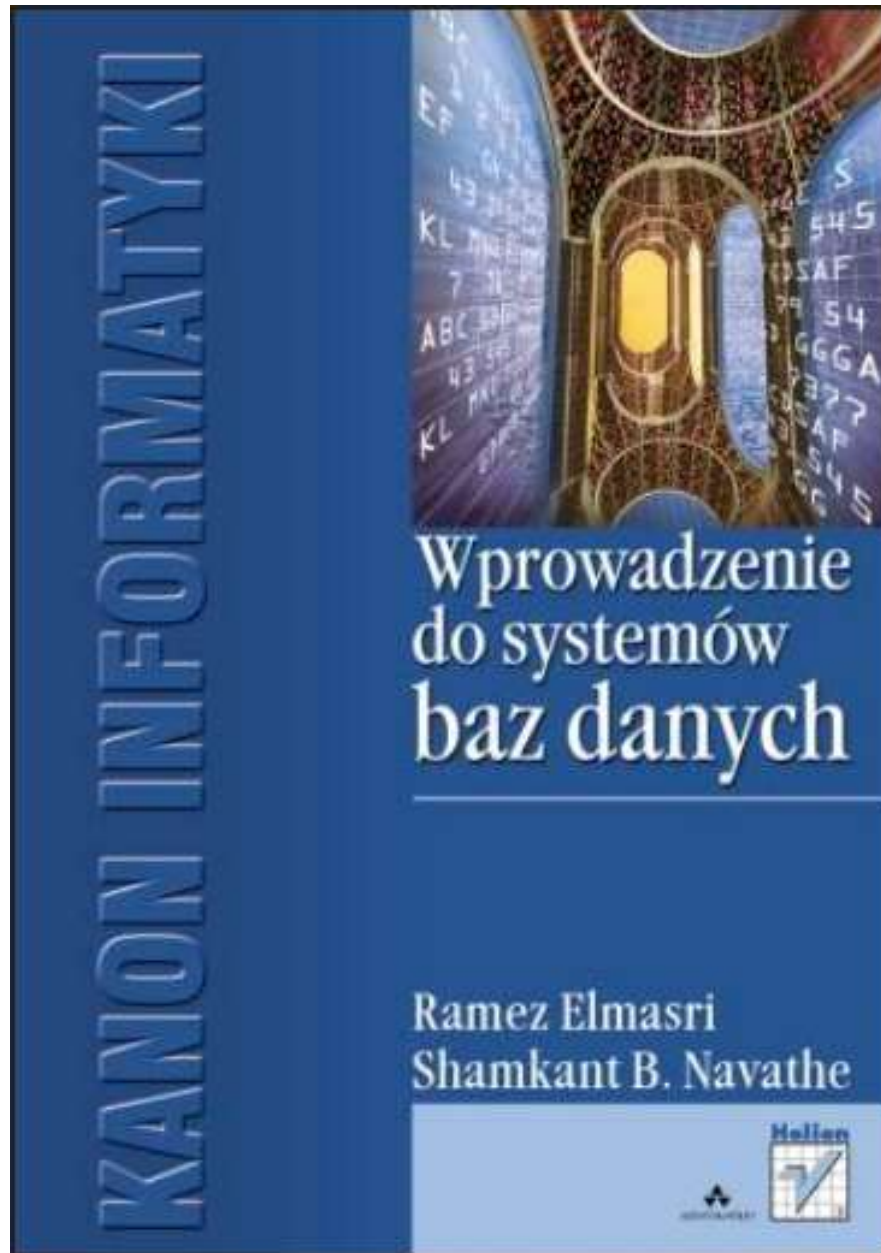


Podstawy baz danych



Warto zapoznać się z narzędziami CASE np.:

- ERDPlus
- MySQL Workbench
- pgModeler

Właściwości bazy danych

Współdzielenie danych

- Korzystanie z danych przez więcej niż jednego klienta – współbieżność dostępu do danych

Integracja danych

- Wszystkie dane przechowywane w jednym miejscu (logicznie), unikanie nadmiarowości danych, ułatwiona administracja danymi, ułatwione uzyskiwanie informacji wymagających łączenia ze sobą różnych zbiorów danych

Integralność (spójność) danych

- Zachowanie prawidłowych związków pomiędzy danymi mimo ich nieustannej aktualizacji

Właściwości bazy danych

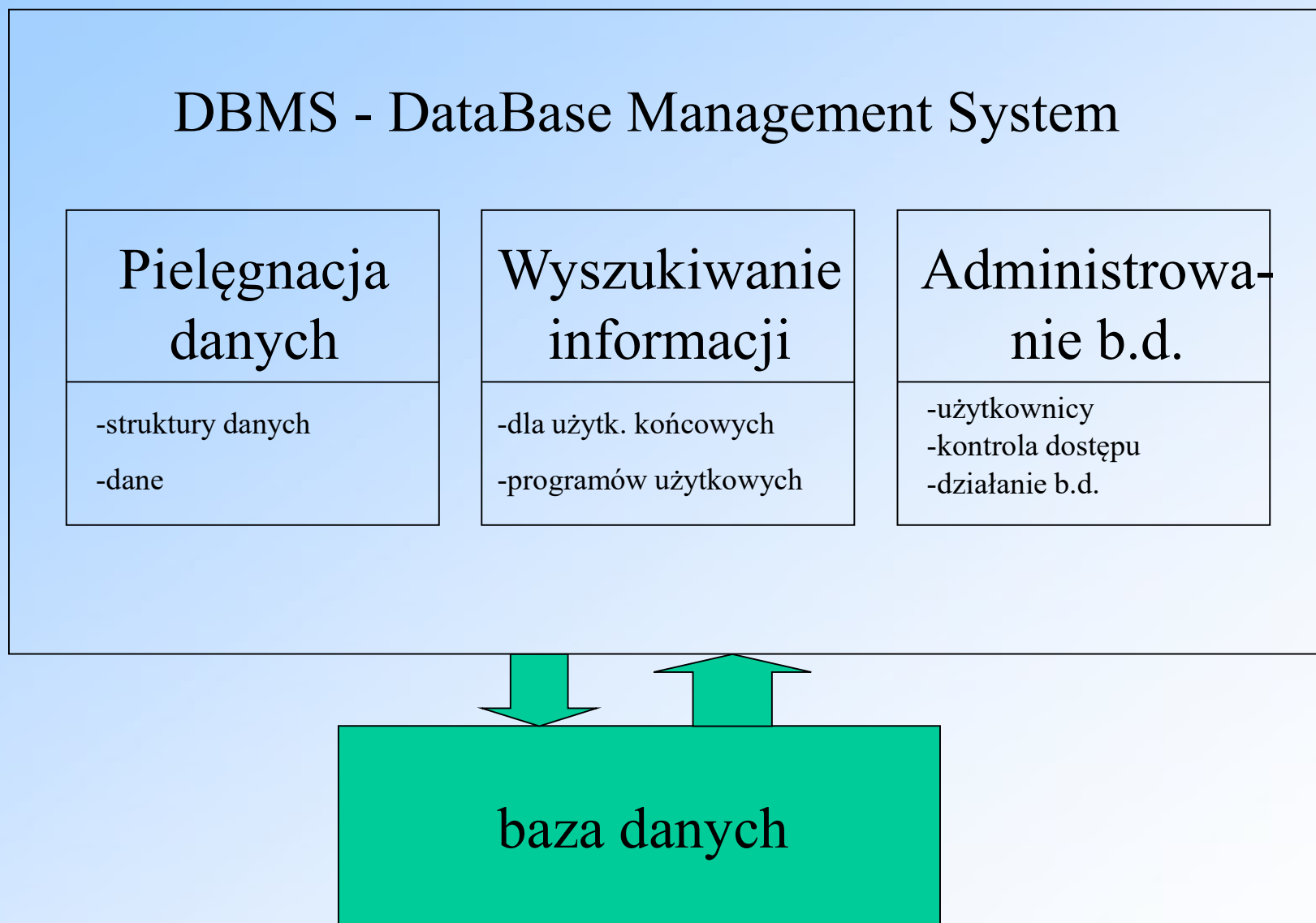
Bezpieczeństwo danych

- Ograniczenie dostępu do danych osobom nieuprawnionym, scentralizowany dostęp do danych umożliwia zastosowanie własnych mechanizmów kontroli dostępu do danych

Abstrakcja i niezależność danych

- Ukrycie wewnętrznej organizacji bazy danych przed użytkownikami – programami użytkowymi, zmiana wewnętrznej organizacji b.d. nie powoduje konieczności zmian w programach użytkowych

System zarządzania bazą danych



Zadania DBMS

Pielęgnacja danych:

- Zarządzanie strukturą danych (dodawanie, modyfikacja, usuwanie)
- Zarządzanie danymi w strukturach

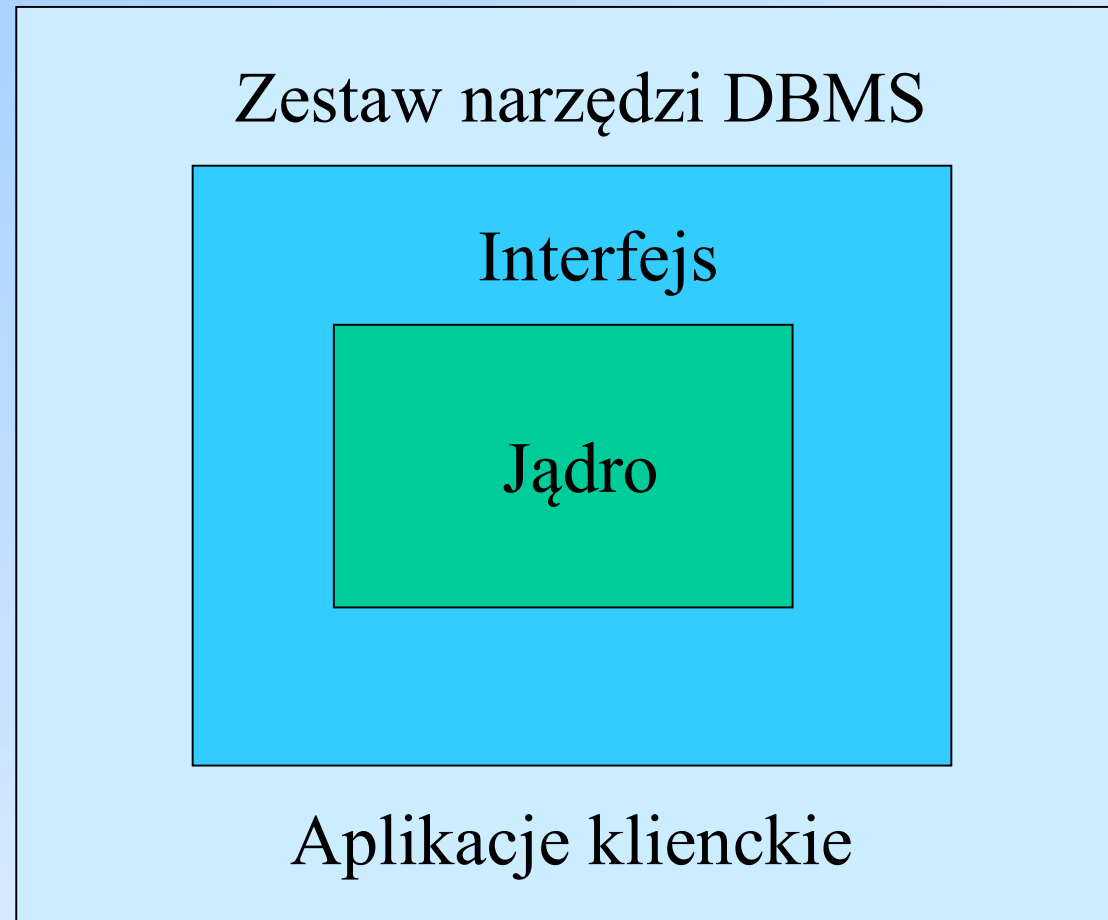
Wyszukiwanie danych:

- Wydobywanie istniejących danych przez użytkowników końcowych i programy użytkowe

Administrowanie bazą danych:

- Tworzenie i monitorowanie użytkowników b.d. (ich kont)
- Nadzorowanie działania b.d. (kopie ratunkowe, ciągłość pracy)
- Kontrolowanie dostępu do danych – definiowanie uprawnień użytkowników

Struktura DBMS



Struktura DBMS - interfejs

Jest to warstwa pośrednia umożliwiająca korzystanie z bazy danych programom klienckim

Język definiowania danych – DDL (Data Definition Language):

- tworzenie, modyfikacja, usuwanie struktur danych

Język operowanie danymi – DML (Data Manipulation Language):

- służy do tworzenia, modyfikacji, czytania i usuwania danych znajdujących się w strukturach bazy danych

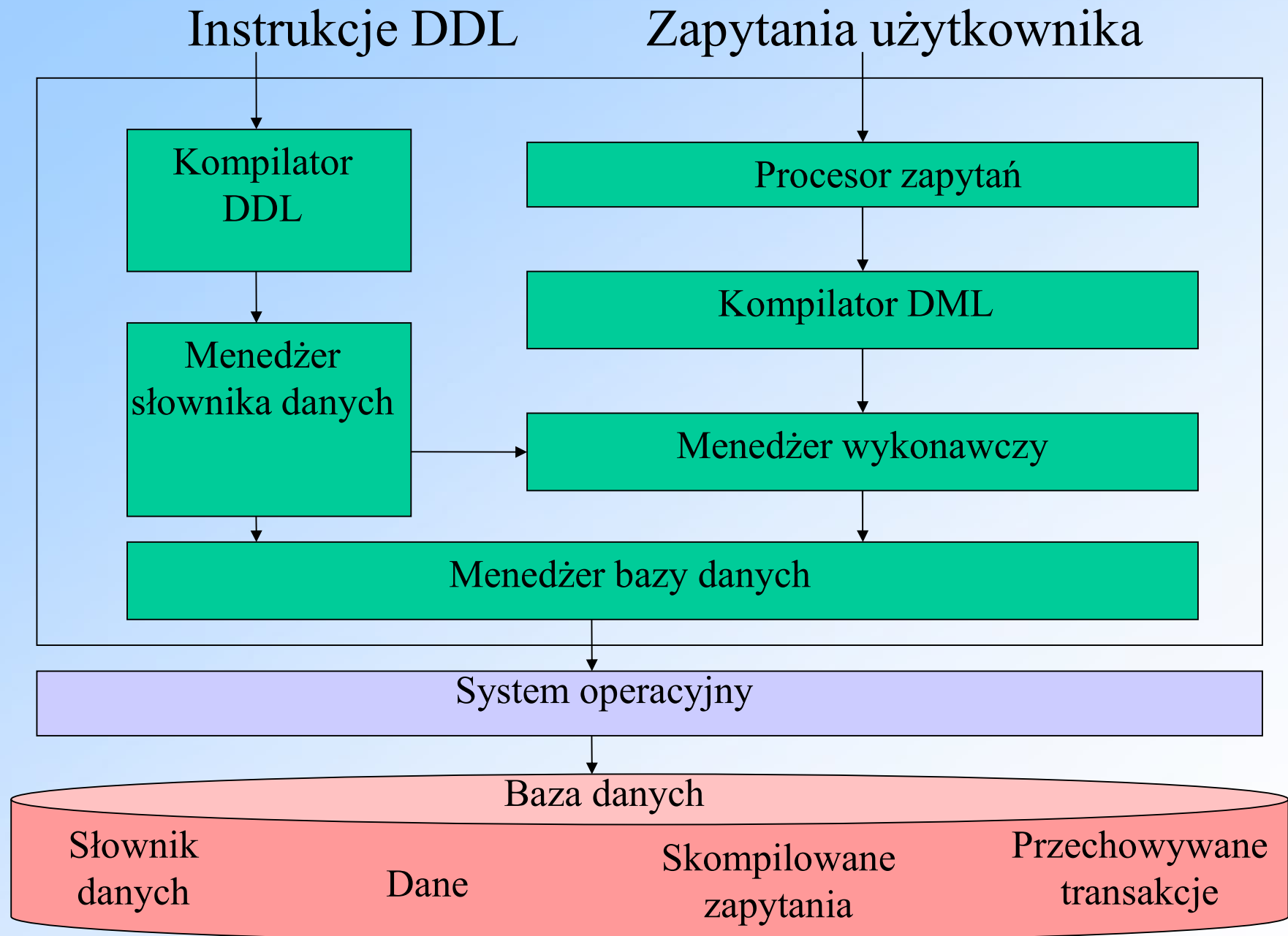
Język kontroli danych – DCL (Data Control Language):

- definiowanie użytkowników, przysługujących im uprawnień, tworzenie grup użytkowników, wykorzystywany przez administratora b.d

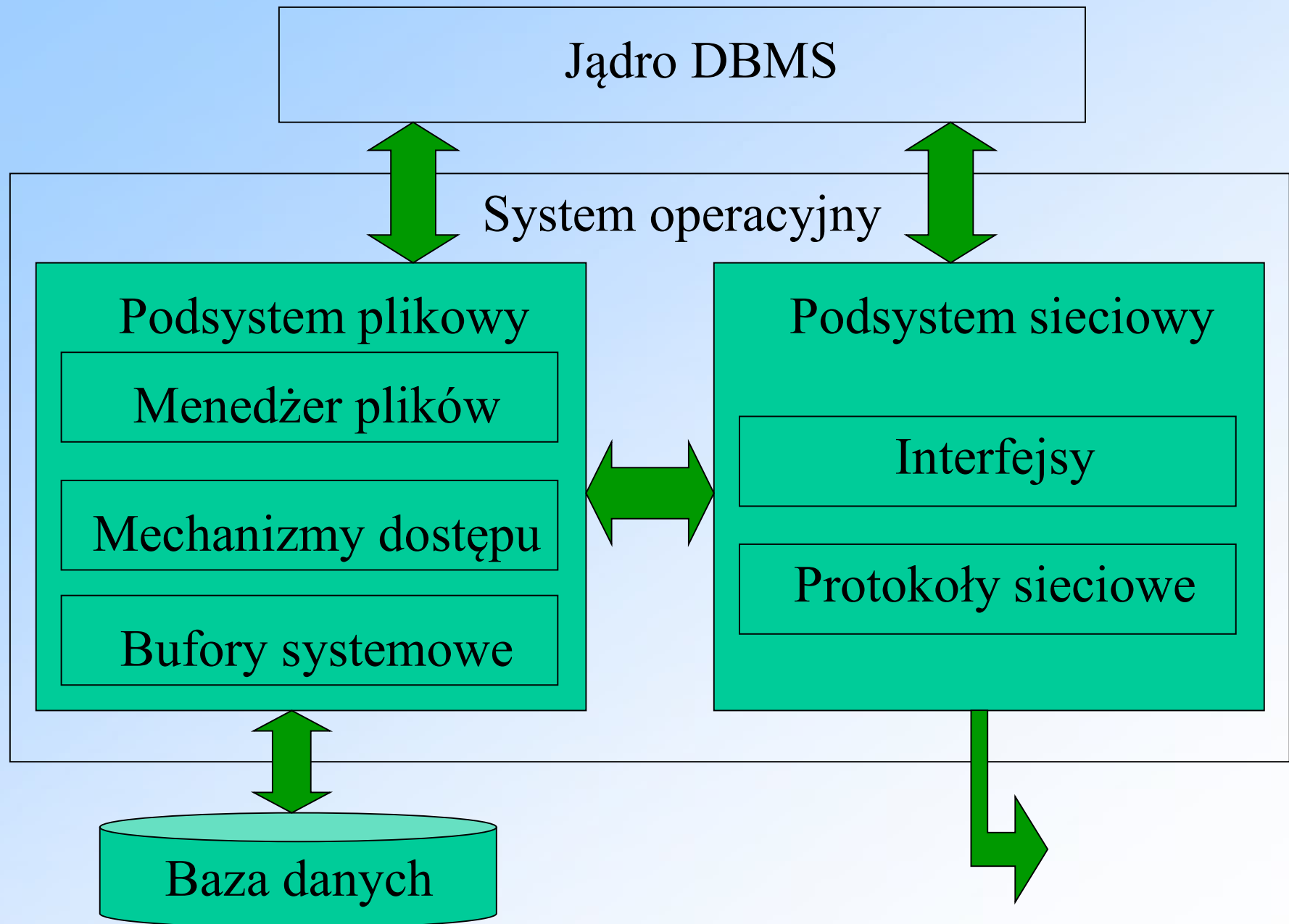
Język integralności danych – DIL (Data Integrity Language):

- definiowanie więzów integralności bazy danych

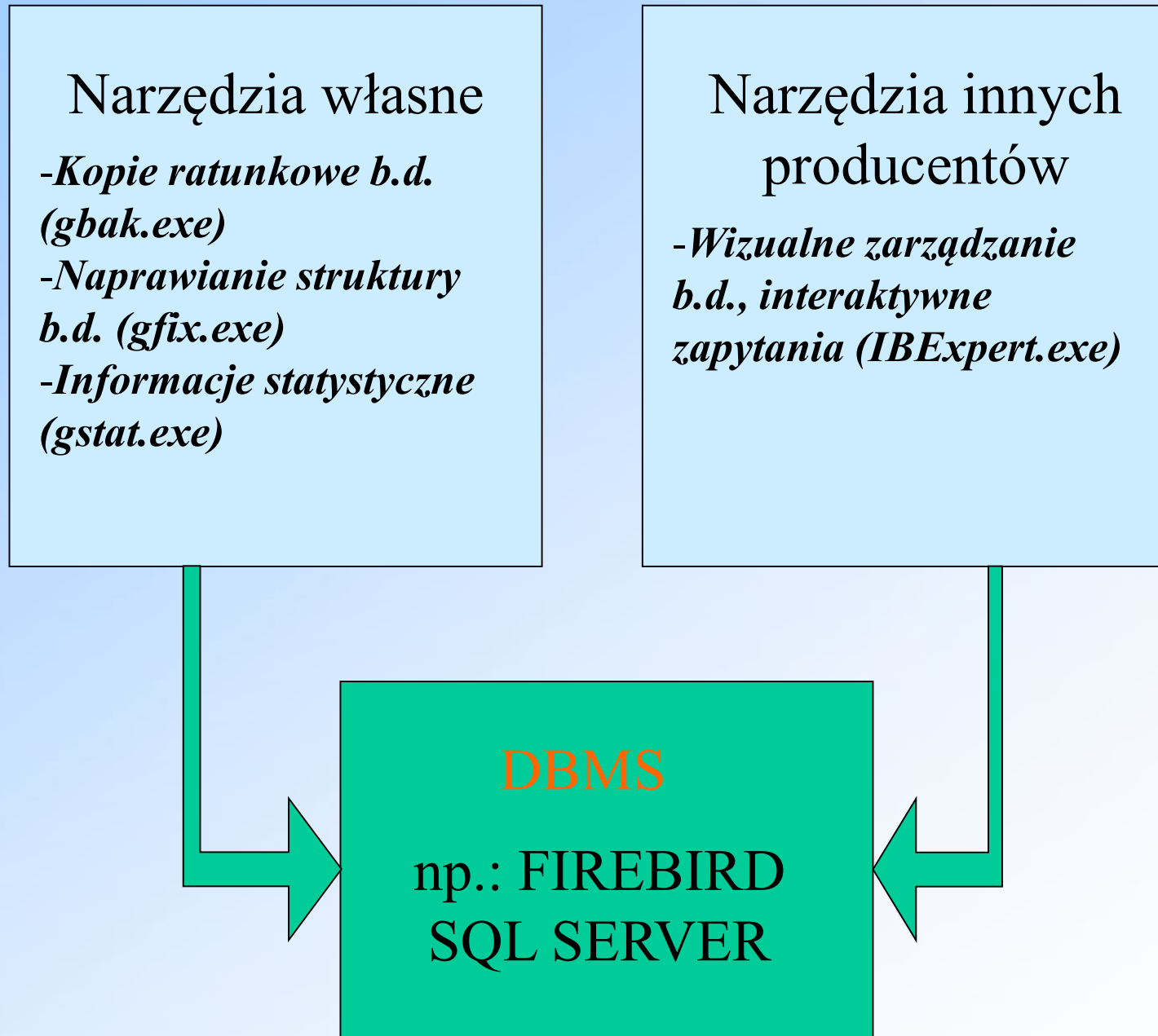
Struktura DBMS - jądro



Struktura DBMS - jądro



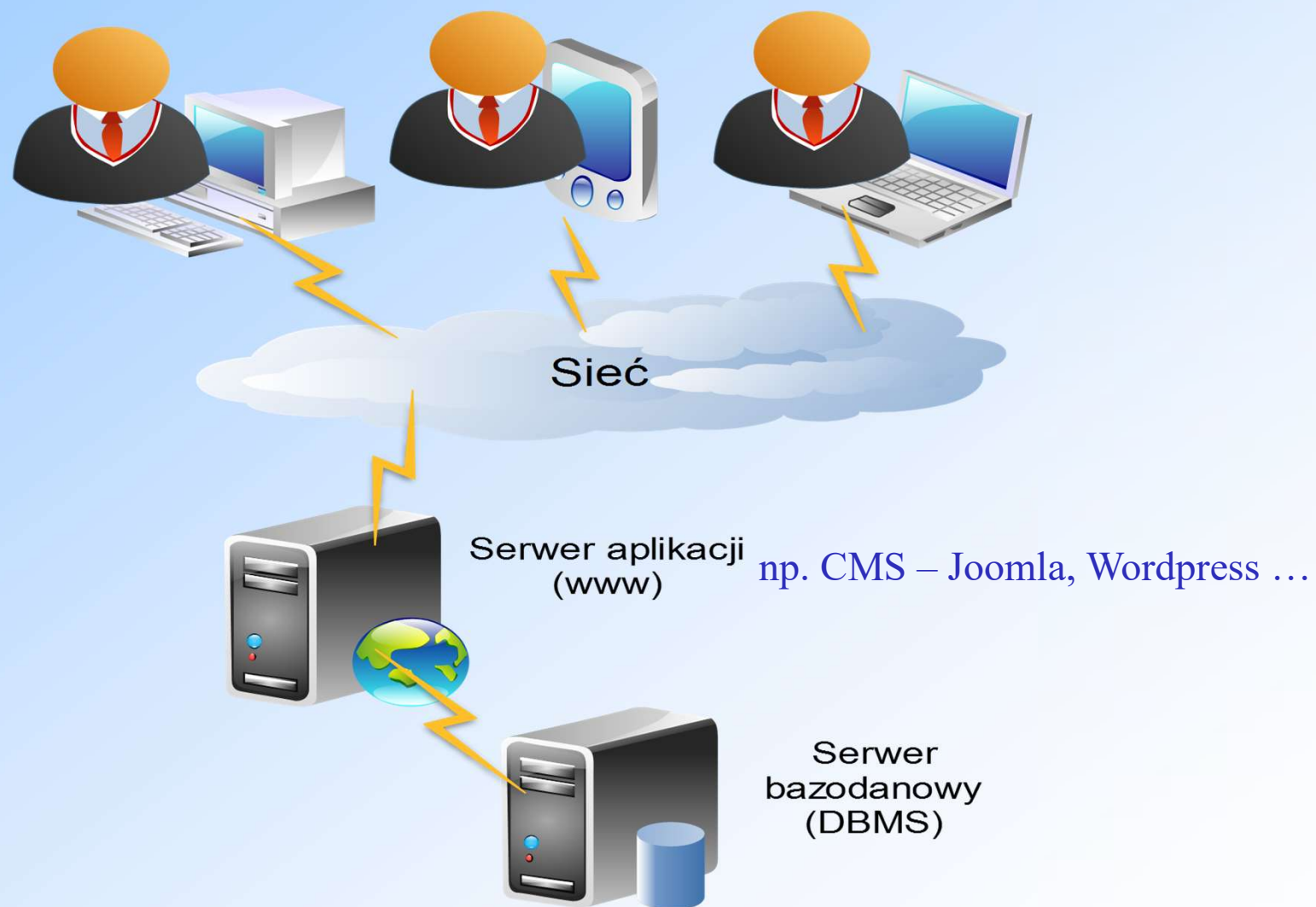
Struktura DBMS – narzędzia DBMS



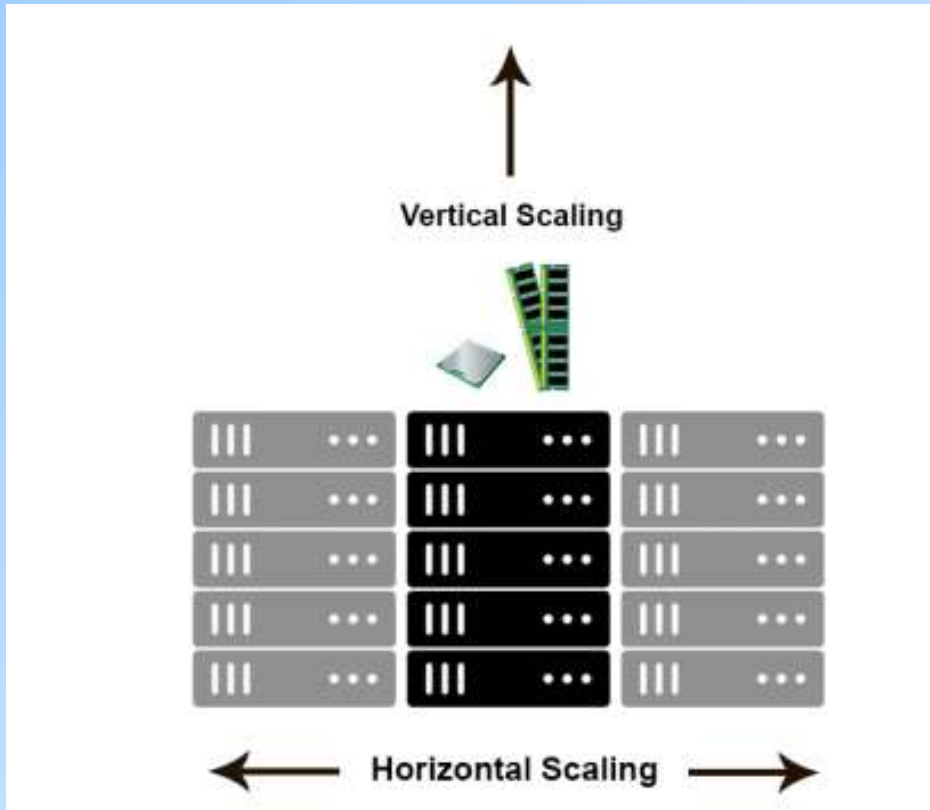
Architektura klient-serwer



Architektura 3-warstwowa



Skalowalność – pozioma, pionowa



Pionowa – dokładanie zasobów do istniejącego serwera np.: pamięć

Pozioma – dokładanie kolejnych serwerów

Skalowalność – pozioma, pionowa

Skalowalność pionowa

Zalety	Wady
• Mniejsze zużycie energii • Łatwiejsze utrzymanie spójności danych • Mniej miejsca w w serwerowni	• Pojedyncza awaria unieruchamia całą bazę danych • Koszty serwerów

Skalowalność pozioma

Zalety	Wady
• Odporność na pojedynczą awarię • Prosty sposób skalowania • Raczej niższe koszty	• Większe zużycie energii • Więcej miejsca • Trudniej utrzymać spójność danych

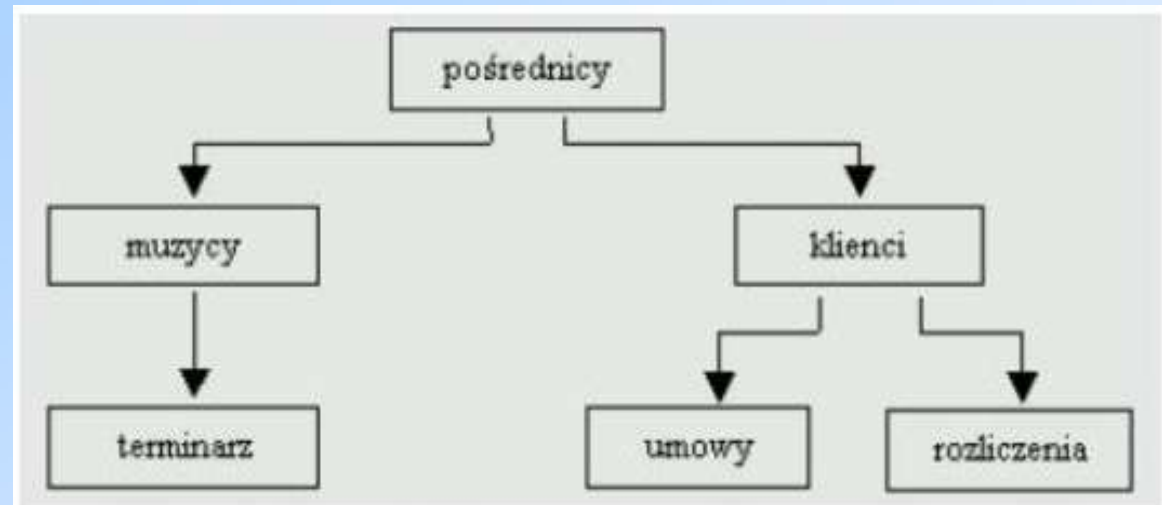
Podział SZBD

Kryteria podziału:

- Wykorzystywany model danych
- Liczba węzłów
- Cel stosowania

W/g wykorzystywanego modelu danych:

- hierarchiczne
- sieciowe



Rysunek 1 Diagram modelu hierarchicznego.

np.: IBM IMS, systemy plików, rejestr Windows, XML

Zalety:

- prostota – 1:N

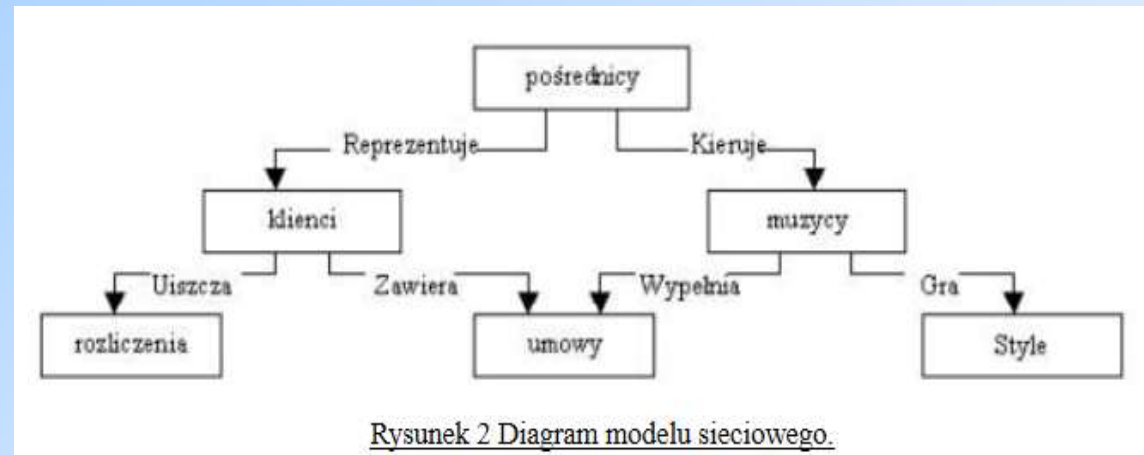
Wady:

- sztywna struktura 1:N (brak N:M)
- nadmiarowość jeśli do jednego korzenia kilka gałęzi
- trudniejszy dostęp do danych (przeszukiwanie drzewa)
- niezbyt przydatne do danych wielkoskalowych

```
<?xml version="1.0" encoding="UTF-8"?>
<catalog>
  <book id="bk101">
    <author>Chand, Mahesh</author>
    <title>A Programmer's Guide to ADO.NET</title>
    <genre>Database</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>A complete reference guide to database programming using C# and
  </book>
  <book id="bk102">
    <author>Chand, Mahesh</author>
    <title>Graphics Programming using GDI+</title>
    <genre>Graphics</genre>
    <price>55.95</price>
    <publish_date>2002-12-16</publish_date>
    <description>Complete graphics programming in C# and .NET using GDI+. Publis
  </book>
  <book id="bk103">
  <book id="bk104">
  <book id="bk105">
  <book id="bk106">
  <book id="bk107">
  <book id="bk108">
  <book id="bk109">
  <book id="bk110">
  <book id="bk111">
  <book id="bk112">
</catalog>
```

W/g wykorzystywanego modelu danych:

- hierarchiczne
- sieciowe



np.: IBM IMS, systemy plików, rejestr Windows, XML

Zalety:

- prostota – 1:N

Wady:

- sztywna struktura 1:N (brak N:M)
- nadmiarowość jeśli do jednego korzenia kilka gałęzi
- trudniejszy dostęp do danych (przeszukiwanie drzewa)
- niezbyt przydatne do danych wielkoskalowych

```
<?xml version="1.0" encoding="UTF-8"?>
<catalog>
  <book id="bk101">
    <author>Chand, Mahesh</author>
    <title>A Programmer's Guide to ADO.NET</title>
    <genre>Database</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>A complete reference guide to database programming using C# and
  </book>
  <book id="bk102">
    <author>Chand, Mahesh</author>
    <title>Graphics Programming using GDI+</title>
    <genre>Graphics</genre>
    <price>55.95</price>
    <publish_date>2002-12-16</publish_date>
    <description>Complete graphics programming in C# and .NET using GDI+. Publish
  </book>
  <book id="bk103">
  <book id="bk104">
  <book id="bk105">
  <book id="bk106">
  <book id="bk107">
  <book id="bk108">
  <book id="bk109">
  <book id="bk110">
  <book id="bk111">
  <book id="bk112">
</catalog>
```

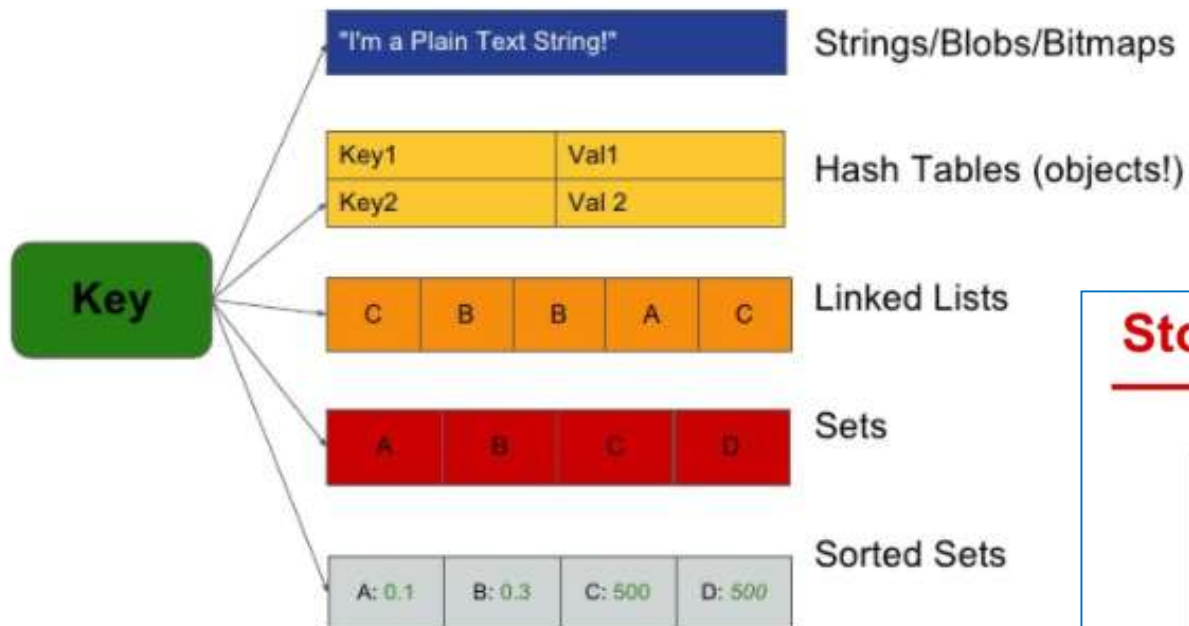
W/g wykorzystywanego modelu danych:

- relacyjne
- obiektowo-relacyjne
- obiektowe (db4o)
 - Zalety:
 - szeroki zakres obsługiwanych typów danych, łatwość definiowania nowych
 - integracja z językami obiektowymi
 - łatwy zapis i odczyt danych
 - Wady:
 - brak standardów
 - duży poziom skomplikowania – potencjalnie niższa wydajność
 - brak mechanizmów bezpieczeństwa (prawa dostępu do obiektu)
 - mniejsza popularność niż relacyjne
- semistrukturalne (XML)

W/g wykorzystywanego modelu danych – NoSQL (Not Only SQL)

- **Klucz-wartość (key-value)** Redis, Memcached, DynamoDB(AWS)

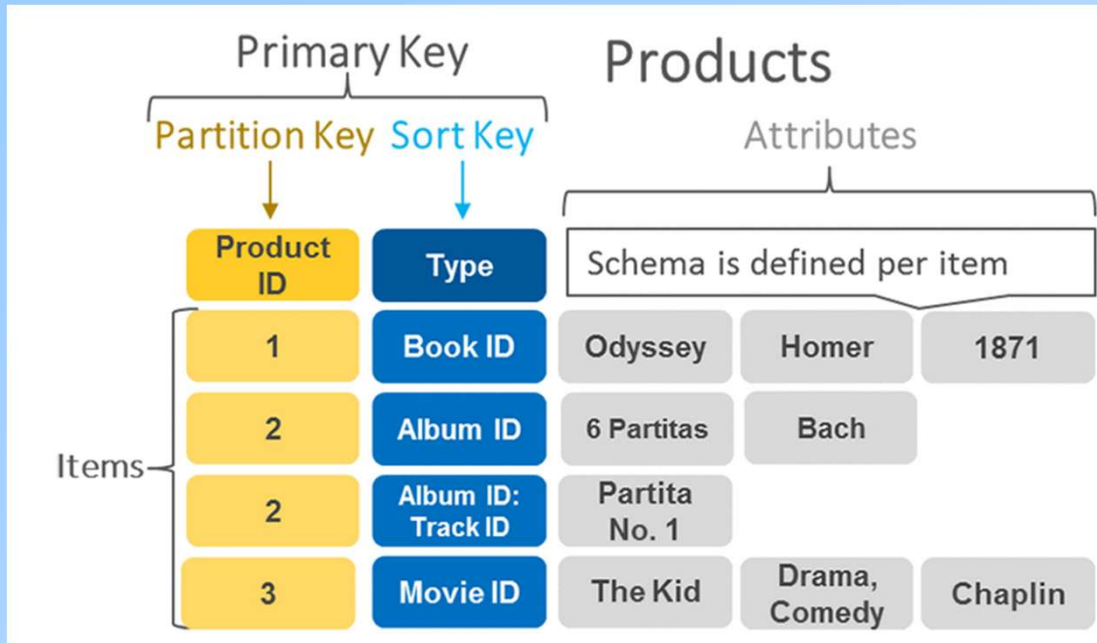
Key => { Data Structures }



Storing users as HASHes

users:1	email	john@domain.com
	name	John
	Password	ae6c65feae8b
	id	1
users:2	email	Jane@domain.com
	name	Jane
	Password	ae6c65ab117b
	id	2

Przykład systemu klucz- wartość- DynamoDB (AWS)



```
queryWithDateIndex: function(queryParams, callback){
  var params = {
    TableName: 'marsDemoImages',
    KeyConditions: [
      // Condition 1: images with "Mission+Instrument" hash key equal to the given value
      AWS.dynamodb.Condition('Mission+InstrumentID', 'EQ', queryParams.hashKey),
      // Condition 2: images with "TimeStamp" range key Less than the given value
      AWS.dynamodb.Condition('TimeStamp', 'LE', queryParams.rangeKey)
    ],
    IndexName: 'date-gsi',
    Limit: 5,
    ScanIndexForward: false // Sort in descending order
  };
  // Run the query
  AWS.dynamodb.query(params, callback);
}
```

Query

- Specify partition key name
- Condition on sort key
- Cheap with high cardinality keys

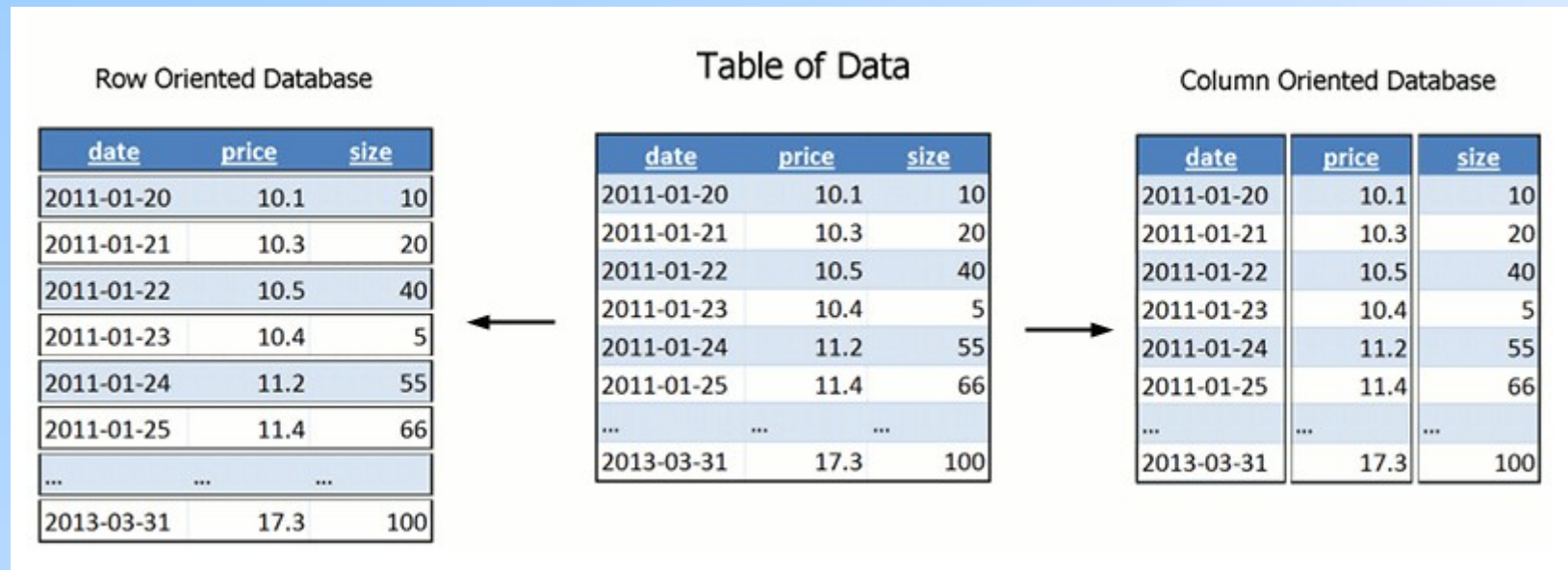
Scan

- Reads all data
- Conditions available through filters
- Expensive for large tables

Brak
dedykowanego
języka zapytań

W/g wykorzystywanego modelu danych – NoSQL

Kolumnowe (column oriented) Cassandra, Hadoop/Hbase, Vertica



- Zapis kolumnowy jest zoptymalizowany pod względem operacji odczytu, czytane są tylko wartości z interesujących nas kolumn (mniej operacji we/wy na pamięci masowej w porównaniu do zapisu wierszowego), szybsza np. agregacja danych
- Operacje zapisu są trudniejsze, ponieważ zapis pojedynczej krotki wymaga zapisu w kilku fizycznych miejscach (kolumnach)

W/g wykorzystywanego modelu danych – NoSQL

Kolumnowe (column oriented)

Cassandra Query Language CQL

```
cqlsh:my_keyspace> UPDATE user SET phone_numbers =  
    phone_numbers + [ '480-111-1111' ] WHERE first_name = 'Mary';  
cqlsh:my_keyspace> SELECT phone_numbers FROM user WHERE  
    first_name = 'Mary';  
  
phone_numbers  
-----  
['1-800-999-9999', '480-111-1111']  
  
(1 rows)
```

W/g wykorzystywanego modelu danych– NoSQL

- **Dokumentowe** (MongoDB,CouchDB)

```
{
  "_id": 2,
  "name": "Steven Jobs",
  "title": "VP, New Product Development",
  "company": "Apple Computer",
  "address": [
    {
      "street": "10260 Bandley Dr",
      "city": "Cupertino",
      "state": "CA",
      "zip_code": "95014",
      "country": "USA"
    },
    {
      "street": "1 Infinite Loop",
      "city": "Cupertino",
      "state": "CA"
    }
  ],
  "phone": "408-996-1010"
}
```

- brak sztywnego schematu
- łatwiejszy rozwój i migracja
- wady – zajętość przestrzeni dyskowej – dane tekstowe + redundancja danych

W/g wykorzystywanego modelu danych– NoSQL

- **Dokumentowe**

MongoDB Query Language

```
// All posts having "likes" field with numeric value greater than one:  
> db.post.find({likes: {$gt: 1}})
```

```
> db.users.find({"name.family": "Smith"}).count()  
1
```

```
> db.user.find().sort({age: 1})  
{  
  "_id": ObjectId("5ce45d7606444f199acfb1e"),  
  "name": {given: "Alex", family: "Smith"},  
  "email": "email@example.com",  
  "age": 27  
}  
{  
  _id: ObjectId("5effaa5662679b5af2c58829"),  
  email: "email@example.com",  
  name: {given: "Jesse", family: "Xiao"},  
  age: 31  
}
```


MongoDB Query Language - złączenie

Orders

```
[  
  { _id: 1, product_id: 154, status: 1 }  
]
```

```
dbo.collection('orders').aggregate([  
  { $lookup:  
    {  
      from: 'products',  
      localField: 'product_id',  
      foreignField: '_id',  
      as: 'orderdetails'  
    }  
  }  
]).toArray(function(err, res) {  
  if (err) throw err;  
  console.log(JSON.stringify(res));  
  db.close();  
});
```

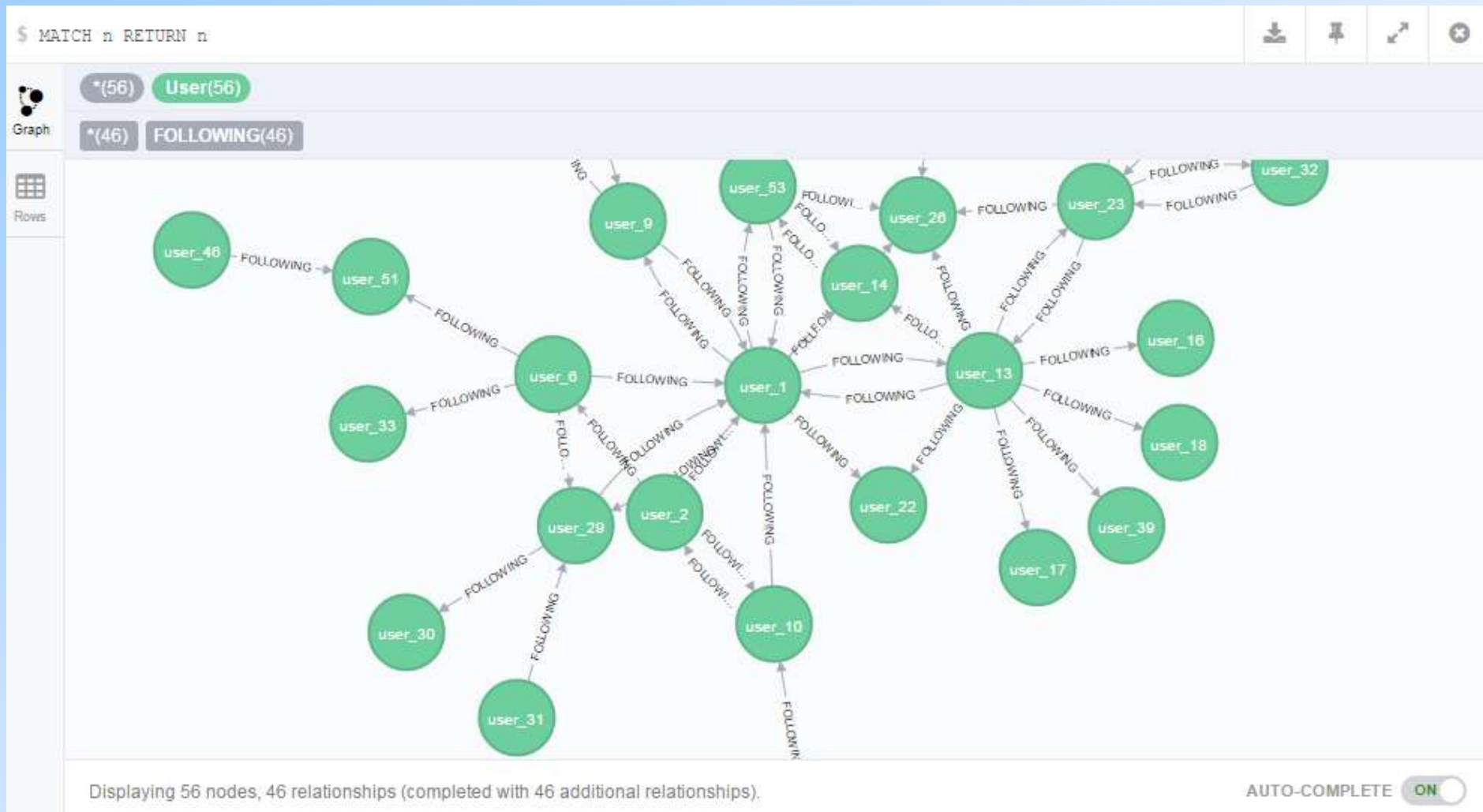
Products

```
[  
  { _id: 154, name: 'Chocolate Heaven' },  
  { _id: 155, name: 'Tasty Lemons' },  
  { _id: 156, name: 'Vanilla Dreams' }  
]
```

```
[  
  { "_id": 1, "product_id": 154, "status": 1, "orderdetails": [  
    { "_id": 154, "name": "Chocolate Heaven" } ]  
  }  
]
```

W/g wykorzystywanego modelu danych – NoSQL

- **Oparte na grafach (graph stores) (neo4j)**



W/g wykorzystywanego modelu danych – NoSQL

- Oparte na grafach

Cypher Query Language (Neo4J)

```
MATCH (:Person {name: 'Jennifer'})-[:WORKS_FOR]->(company:Company)  
RETURN company
```

\$ MATCH (:Person {name: "Jennifer"})-[:WORKS_FOR]->(compa...

Graph

Table

Text

Code

*(1)

Company(1)

Neo4j

Displaying 1 nodes, 0 relationships.

Zapytania SQL do danych JSON - Athena – Presto (AWS)

```
WITH dataset AS (  
  SELECT * FROM (VALUES  
    (JSON '{"name": "Bob Smith",  
          "org": "legal",  
          "projects": [{"name": "project1", "completed": false}]}'),  
    (JSON '{"name": "Susan Smith",  
          "org": "engineering",  
          "projects": [{"name": "project2", "completed": true},  
                      {"name": "project3", "completed": true}]}'),  
    (JSON '{"name": "Jane Smith",  
          "org": "finance",  
          "projects": [{"name": "project2", "completed": true}]}')  
  ) AS t (users)  
,  
employees AS (  
  SELECT users, CAST(json_extract(users, '$.projects') AS  
    ARRAY(MAP(VARCHAR, JSON))) AS projects_array  
  FROM dataset  
,  
names AS (  
  SELECT json_extract_scalar(users, '$.name') AS name, projects  
  FROM employees, UNNEST (projects_array) AS t(projects)  
)  
SELECT name, count(projects) AS completed_projects FROM names  
WHERE cast(element_at(projects, 'completed') AS BOOLEAN) = true  
GROUP BY name
```

name	completed_projects
Susan Smith	2
Jane Smith	1

Podział SZBD (2)

W/g liczby węzłów:

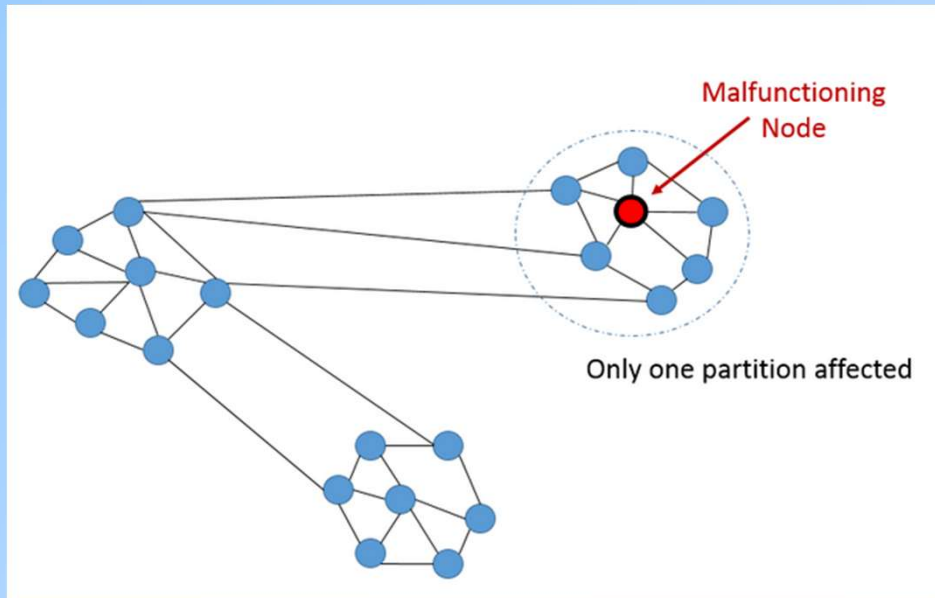
- Scentralizowane
- Rozproszone

Teoria CAP:

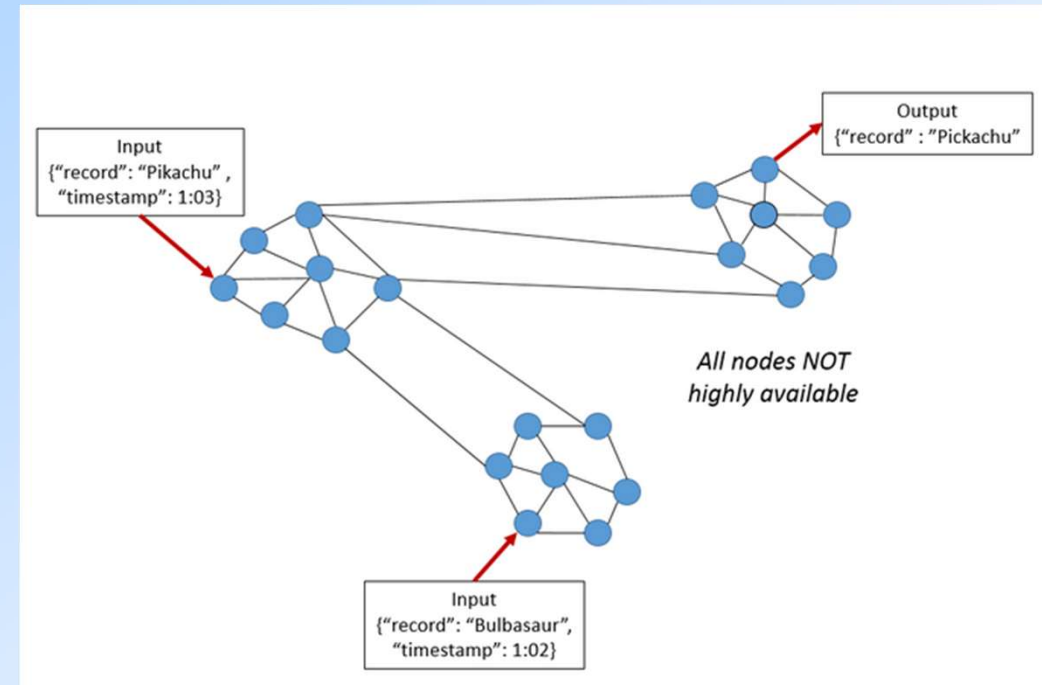
- **C**onsistency (Spójność)
- **A**vailability (Dostępność)
- **P**artition tolerance (Tolerancja na podział)

Podział SZBD – systemy rozproszone – Teoria CAP

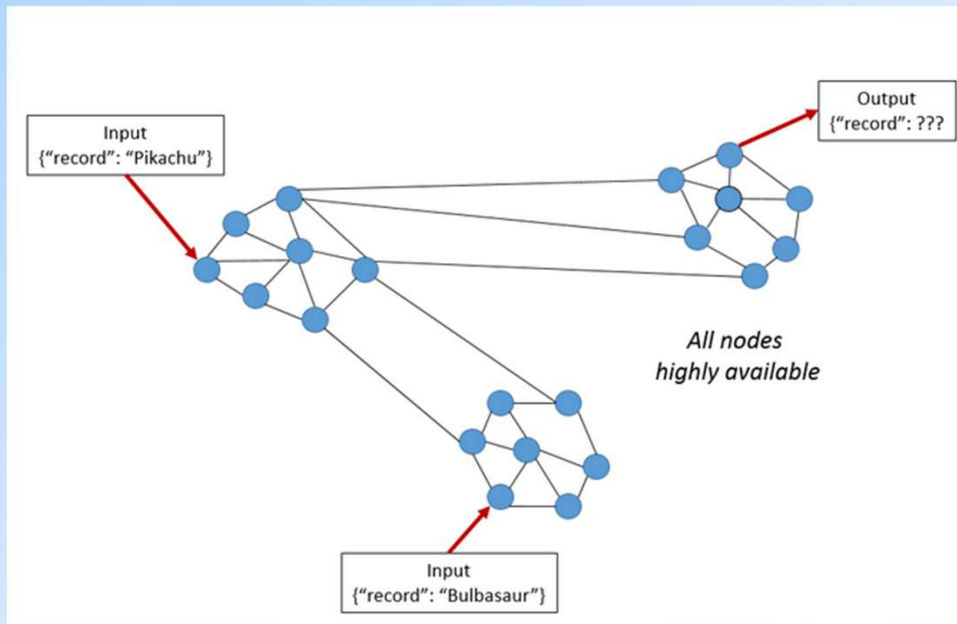
Partition tolerance (Tolerancja na podział)



Consistency (Spójność)



Availability (Dostępność)



Podział SZBD – systemy rozproszone – Teoria CAP

Consistency (Spójność) – wszystkie węzły „widzą” te same dane w tym samym czasie

- konieczność replikacji informacji modyfikowanej w jednym węźle do pozostałych węzłów

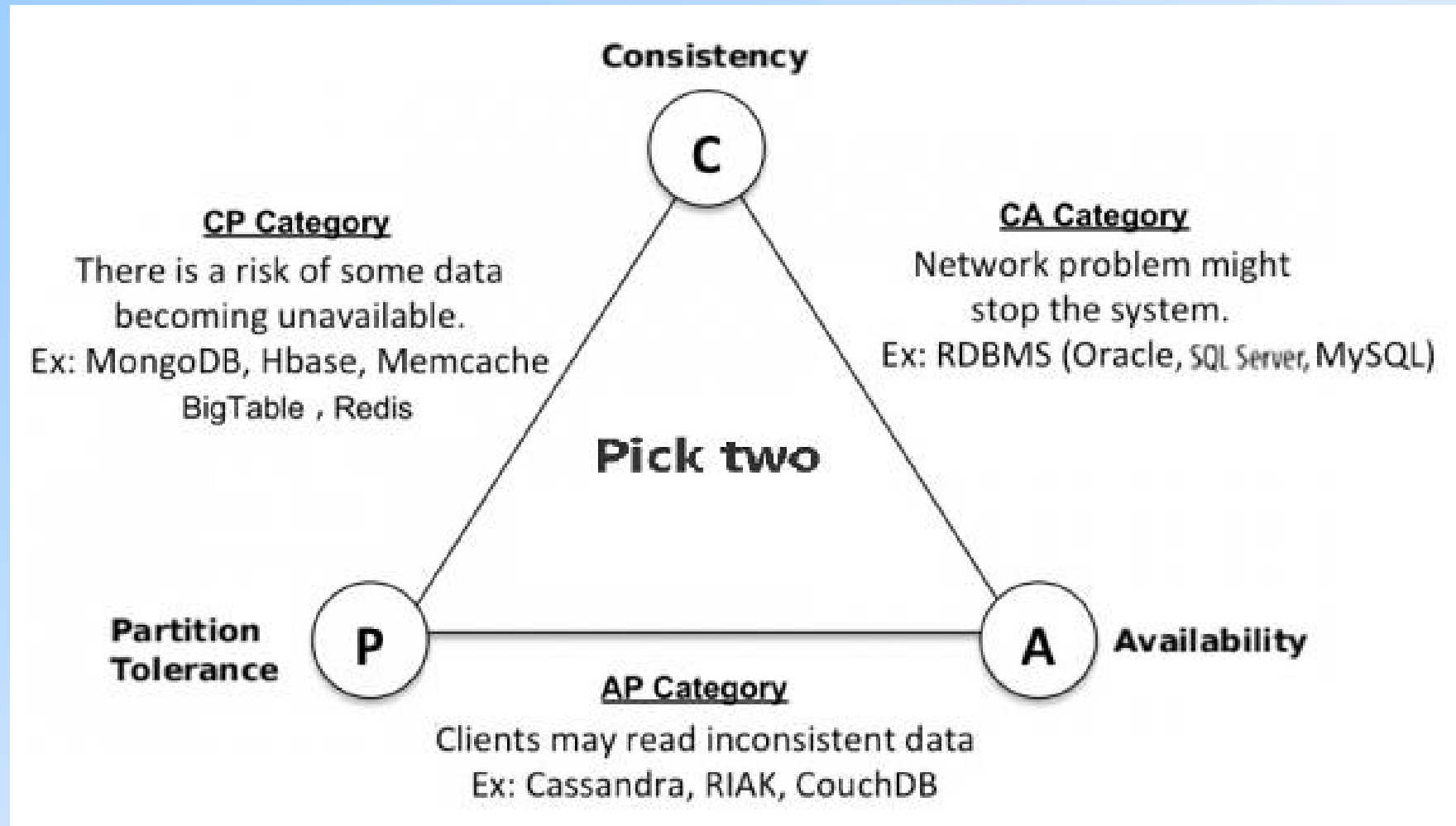
• **Availability** (Dostępność) – gwarancja, że każde zapytanie skierowane do systemu otrzyma odpowiedź

- Mimo aktualizacji danych w różnych węzłach, system musi wykonywać operacje zapisu/odczytu – rezygnacja ze spójności (chwilowej)

• **Partition tolerance** (Tolerancja na podział) – system powinien prawidłowo działać mimo awarii jego fragmentu

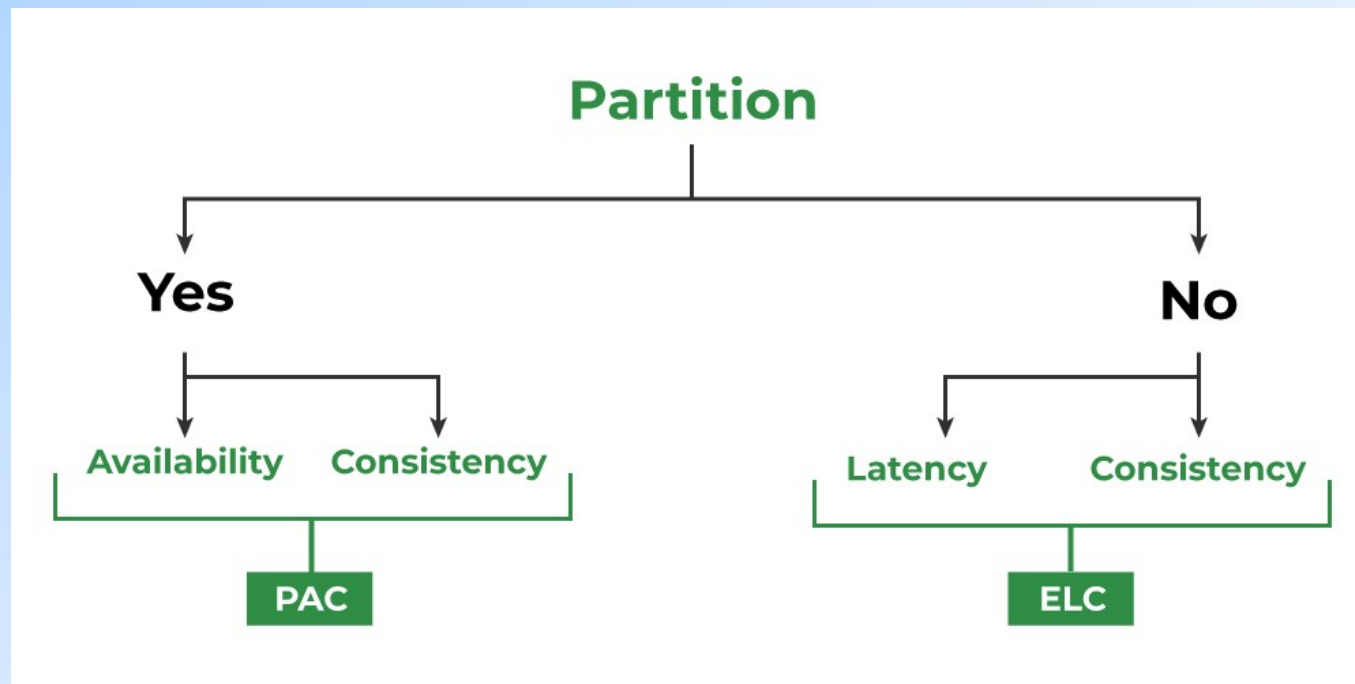
Rozproszone systemy bazodanowe mogą jednocześnie spełniać tylko 2 z 3 powyższych postulatów – czyli np. CA, AP, CA

Podział SZBD – systemy rozproszone – Teoria CAP



Podział SZBD – systemy rozproszone – Model PACELC

- Klasyczna teoria (CAP) mówi o zachowaniu systemu **tylko w momencie awarii sieci** (partycjonowanie), ignorując to, jak system działa podczas normalnej pracy
- W rzeczywistości, w typowych warunkach trzeba wybrać pomiędzy **natychmiastową odpowiedzią (niskie opóźnienie)**, ale być może z nieaktualnymi danymi, albo **większym opóźnieniem** (czas synchronizacji) ale za to **wysoką spójnością**



Przykład: Cassandra / DynamoDB: Zazwyczaj klasyfikowane jako **PA/EL** (w razie awarii wybierają dostępność, w normalnym trybie stawiają na niskie opóźnienia)
MySQL – std. replikacja – PA/EL, zastosowanie Group Replication - PC/EC

Systemy NoSQL

- Ponieważ struktura danych **nie jest sformalizowana**, pilnowanie poprawności struktur spoczywa na programiście
- **Poszczególne obiekty** przechowywane w bazie **mogą się różnić** pomiędzy sobą atrybutami
- Trudniej zapewnić transakcyjność operacji
- **Uboższy interfejs zapytań**, brak **unifikacji interfejsu**
- Część z nich oferuje tylko **Eventual Consistency** – dopuszczalne jest zwrócenie nie najnowszej informacji
- Zazwyczaj optymalizowane dla dużych zbiorów danych
- Wysoka skalowalność - horyzontalna

NoSQL a SQL

Feature	NoSQL Databases	Relational Databases
Performance	High	Low
Reliability (Transactions)	Poor	Good
Availability	Good	Good
Consistency	Poor	Good
Data Storage	Optimized for huge data	Medium sized to large
Scalability	High	High (but more expensive)

Podział SZBD (3)

W/g celu stosowania:

- **OLTP** (On-Line Transaction Processing) – przetwarzanie transakcyjne
- **OLAP** (On-Line Analytical Processing) – przetwarzanie analityczne – hurtownie danych
- **GIS** (Geographical Information Systems) –systemy informacji geograficznej (np. PostGIS)

Podział SZBD (3)

	OLTP	OLAP
Przeznaczenie	Typowe, proste operacje na danych	Wspieranie decyzji
Projekt bazy	Zazwyczaj wysoko znormalizowany, dużo tabel	Zdenormalizowany, mniej tabel
Dane	Bieżące, detaliczne, płaskie – relacyjne	Historyczne, zagregowane, wielowymiarowe
Sposób użycia	Powtarzające się zapytania	Ad-Hoc
Dostęp do danych	Odczyt/ zapis	Dużo odczytów
Jednostka przetwarzania	Krótką, prosta transakcja	Skomplikowane zapytanie
Typowe ilości czytanych rekordów	Dziesiątki	Miliony
Rozmiar bazy	100MB-GB	100GB-TB
Miara wydajności	Transakcje/sek	Prędkość przetw danych
Zapytania	Stosunkowo proste, zwracają relatywnie mało rekordów	Złożone zapytania, skomplikowane odmiany agregacji danych

Bazy danych GIS

Wsparcie dla operacji
związanych z
rozmieszczeniem
przestrzennym danych
Np.: PostGIS

```
SELECT * FROM artwork;
```



```
"Giaconda";      "painting";      "Leonardo Da Vinci";  48.860547; 2.338513;  
"David";         "sculpture";     "Michelangelo Buonarroti"; 43.776709; 11.258887;  
"Sunflowers";    "painting";      "Vincent Van Gogh";    48.149966; 11.570856;  
"Guernica";      "painting";      "Pablo Picasso";       40.407561; -3.694042; "
```

```
SELECT artwork1.artwork_name,  
       artwork2.artwork_name,  
       ST_DISTANCE(artwork1.where_is, artwork2.where_is) * 3.2808399  
FROM   artwork artwork1,  
       artwork artwork2
```



Artwork1	Artwork2	Distance (meters)
"Giaconda"	"Giaconda"	0
"Giaconda"	"David"	2915185.37043353
"Giaconda"	"Sunflowers"	2251773.82097004

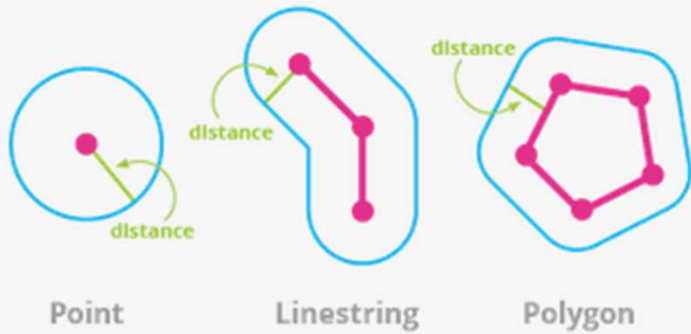
```
SELECT museum_name,  
       st_area(perimeter)  
FROM   museum  
ORDER BY 2 DESC
```



<u>museum_name</u>	<u>st_area (in square meters)</u>
"Museo Reina Sofia"	9412789265.14431
"Accademia Gallery"	8985388059.43758
"Munich Pinakothek"	8217590765.89231
"Musee du Louvre"	8217590765.89229

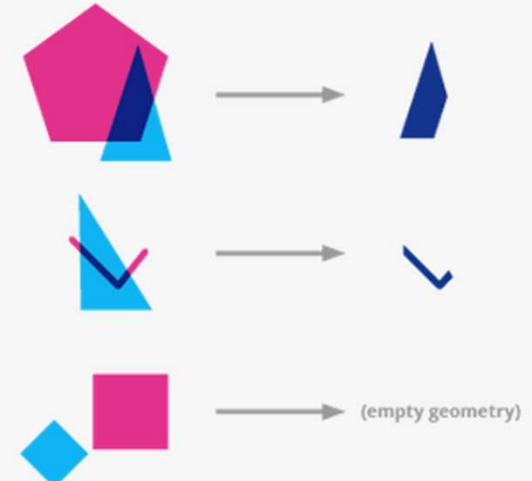
Bazy danych GIS

ST_Buffer (geometry, distance)

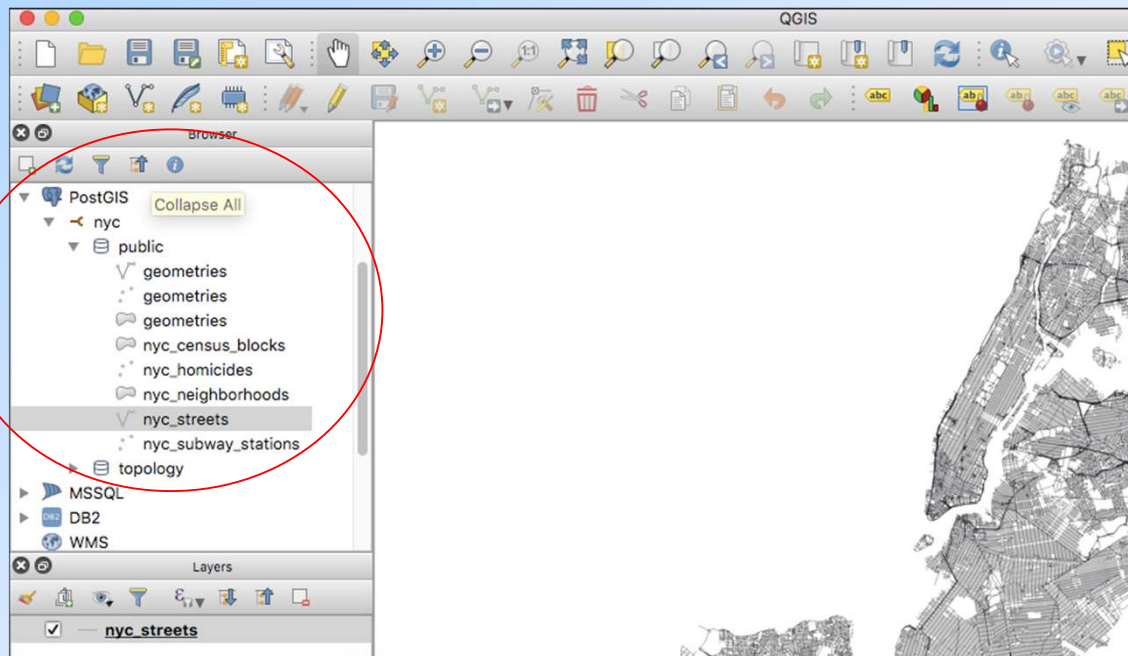


```
SELECT
  ST_Buffer(ST_Transform(
    coordinates,26910),50)
FROM sf_atms;
```

ST_Intersection



```
SELECT
  sfsr.id,
  sfpd.id,
  ST_Intersection(sfpd.boundaries,
sfsr.course)
FROM sf_selected_railways sfsr
JOIN sf_planning_districts sfpd
ON ST_Intersects(sfpd.boundaries,
sfsr.course);
```



Technologie implementacyjne aplikacji

Języki 3GL:

- komunikacja z bazą danych wymaga stosowania specjalizowanych bibliotek
- np: Visual Basic, C, C++ , C#, Java, JavaScript, PHP, PERL

Języki 4GL – niszowe - enterprise:

- bezpośrednio zawieranie zapytań i bezpośrednia obsługa ich wyników w kodzie aplikacji
- krótszy kod niż w 3GL
- np: Oracle Forms, Progress OpenEdge, Informix, Fantasy 4GL
- ewolucja w stronę low-code, no-code – wizualne kreatory, skrypty, np. Oracle APEX, MS PowerApps

Biblioteki 4GL do języków 3GL

- - np. .NET LinQ

Technologie implementacyjne aplikacji – języki 4GL

Fantasy 4GL

```
PROCEDURE Loan ();
{
  $today = system.call('current_date');
  read($x); // read Rno

  if (call(Rnheck($x)) == 0)
  {
    message("card invalid");
    exit();
  };

  read($y); # read Bno
  while ($y <> EndOfLoan)
  {
    call(Register_loan($today, $x, $y));
    read($y);
  }
}
```

```
int Rnheck ($x);
{
  SELECT COUNT (*)
  FROM Reader
  WHERE Rno = $x;
}
```

```
void Register_loan
($d, $x, $y);
{
  INSERT INTO Loan
  VALUES ($y, $x, $d, NULL);
}
```

► Report writing

```
SELECT Name, Address, ...
FROM Loan, Reader, Book
WHERE Loan.Rno = Reader.Rno
AND Loan_date < '01.12.2016'
AND Return_date IS NULL
```

@name
@address

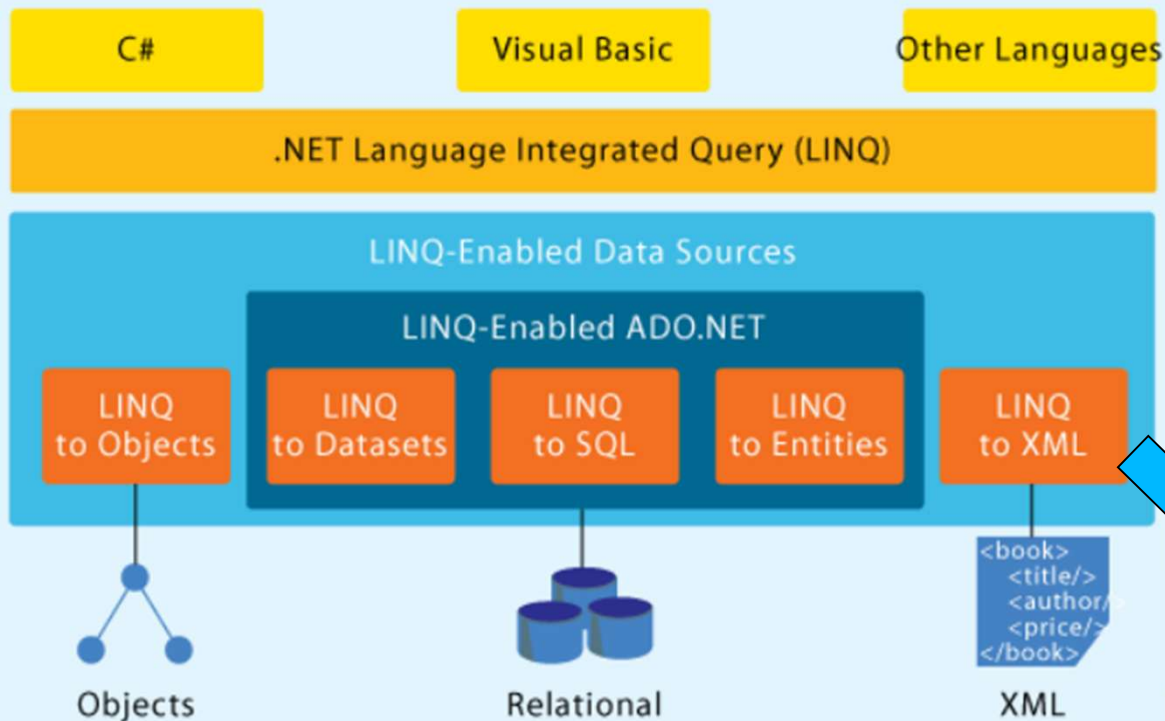
Dear mr/mrs @name,

On @loan_date you have borrowed the following book from our library:
@title by @author.

We kindly request you to return this book as soon as possible.

Hybrydowe technologie implementacyjne aplikacji – biblioteka 4GL, język 3GL

LINQ Architecture



```
public NamePriceModel[] AllProducts()
{
    try
    {
        using (UserDataDataContext db = new UserDataDataContext())
        {
            return db.mrobpProducts
                .Where(x => x.Status == 1)
                .Select(x => new NamePriceModel {
                    Name = x.Name,
                    Id = x.Id,
                    Price = x.Price
                })
                .OrderBy(x => x.Id)
                .ToArray();
        }
    }
    catch
    {
        return null;
    }
}
```

var result =

Query Expressions

from c in Customers
where c.City == "Boston"
select new { c.Name, c.Address };

var result =

Lambda expressions

Local variable type inference

Customers

Extension methods

.Where(c => c.City == "Boston")
.Select(c => new { c.Name, c.Address });

Anonymous types

Object initializers

Technologie implementacyjne aplikacji – język 3GL

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

public class Type_One
{
    public static void main(String[] args)
    {
        try
        {
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver"); //Load Driver
            Connection con = DriverManager.getConnection("jdbc:odbc:HOD_DATA"); //Create
            Connection with Data Source Name : HOD_DATA
            Statement s = con.createStatement(); // Create Statement
            String query = "select * from Data"; // Create Query
            s.execute(query); // Execute Query
            ResultSet rs = s.getResultSet(); //return the data from Statement into ResultSet
            while(rs.next()) // Retrieve data from ResultSet
            {
                System.out.print("Serial number : "+rs.getString(1)); //1st column of Table from database
                System.out.print(" , Name : "+rs.getString(2)); //2nd column of Table
                System.out.print(" , City : "+rs.getString(3)); //3rd column of Table
                System.out.println(" and Age : "+rs.getString(4)); //4th column of Table
            }
            s.close();
            con.close();
        }
        catch (Exception e)
        {
            System.out.println("Exception : "+e);
        }
    }
}
```


Przegląd SZBD

Komercyjne:

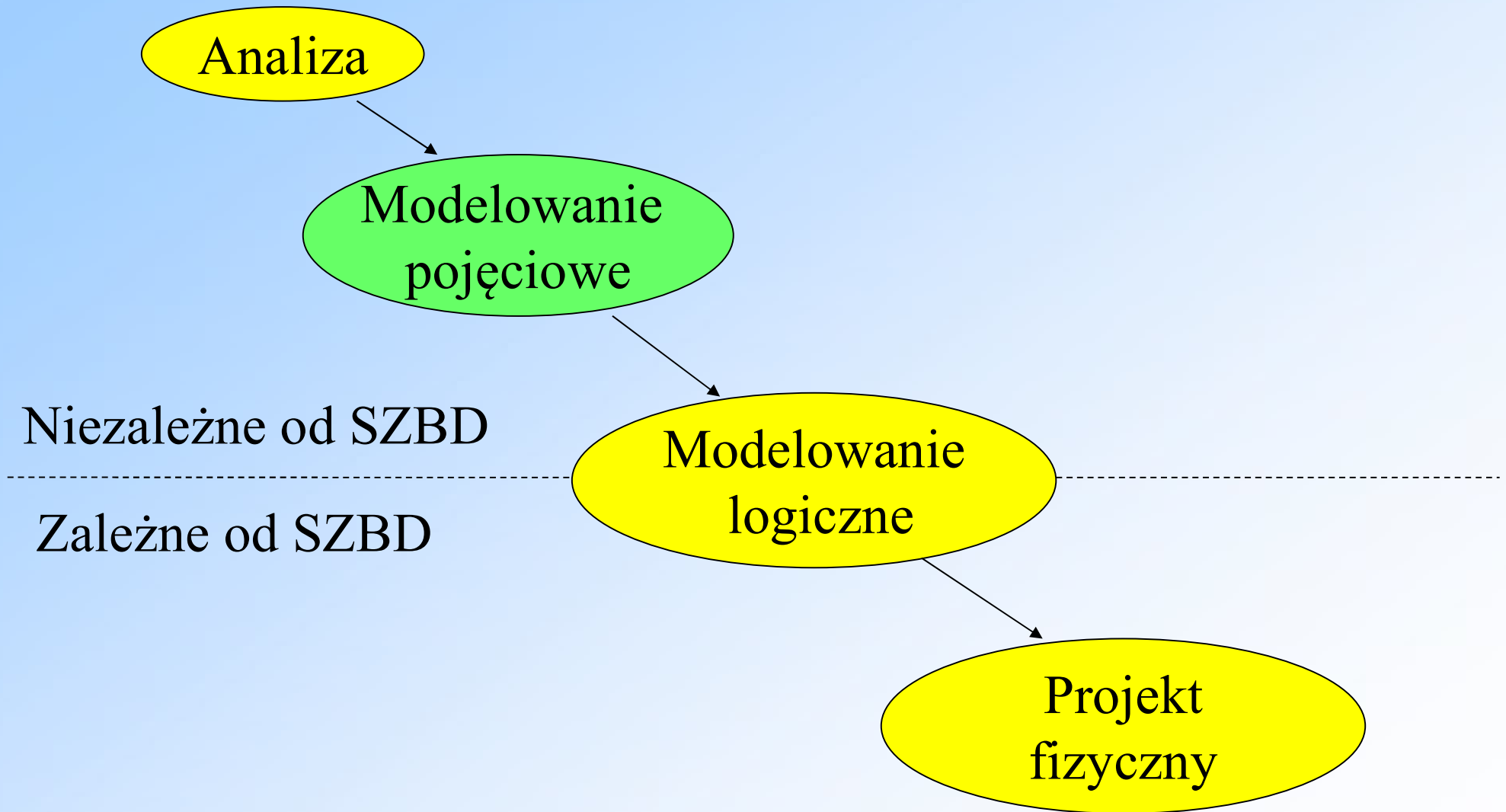
- Oracle
- IBM – DB2, Informix
- Microsoft – SQL Server 2022
- Sybase – Adaptive Server Enterprise

Niekomercyjne:

- Firebird
- MySQL
- PostgreSQL

Model związków encji E-R

Etapy projektowania bazodanowego systemu informatycznego



Modele pojęciowe

Model ERD (Entity – Relationship Diagrams) – diagramy związków encji, notacje:

- **Chena**
- **Barkera (Oracle)**
- **IDEF1X**
- ...
- **Diagramy UML (Unified Modeling Language)** – ujednolicony język modelowania

Model związków encji (E-R)

- Model E/R (Entity-Relationship) pozwala na utworzenie schematu bazy danych.
 - Schemat ten opisuje struktury danych oraz sposób w jaki poszczególne struktury są ze sobą połączone.
 - Nie opisuje działań na danych.
- Projekt b.d w modelu E-R nazywa się diagramem E-R.

Zbiory encji

- *Encja* = "rzecz" , obiekt
 - Podobieństwo do klasy w językach zorientowanych obiektowo.
- *Zbiór encji* = kolekcja podobnych encji.
- *Atrybut* = pewna właściwość encji.

Cechy encji

- każda encja posiada atrybuty (własności)
- każda encja posiada unikalną nazwę (identyfikator)
- dowolny obiekt świata rzeczywistego jest reprezentowany wyłącznie przez jedną encję
- encje są ze sobą powiązane

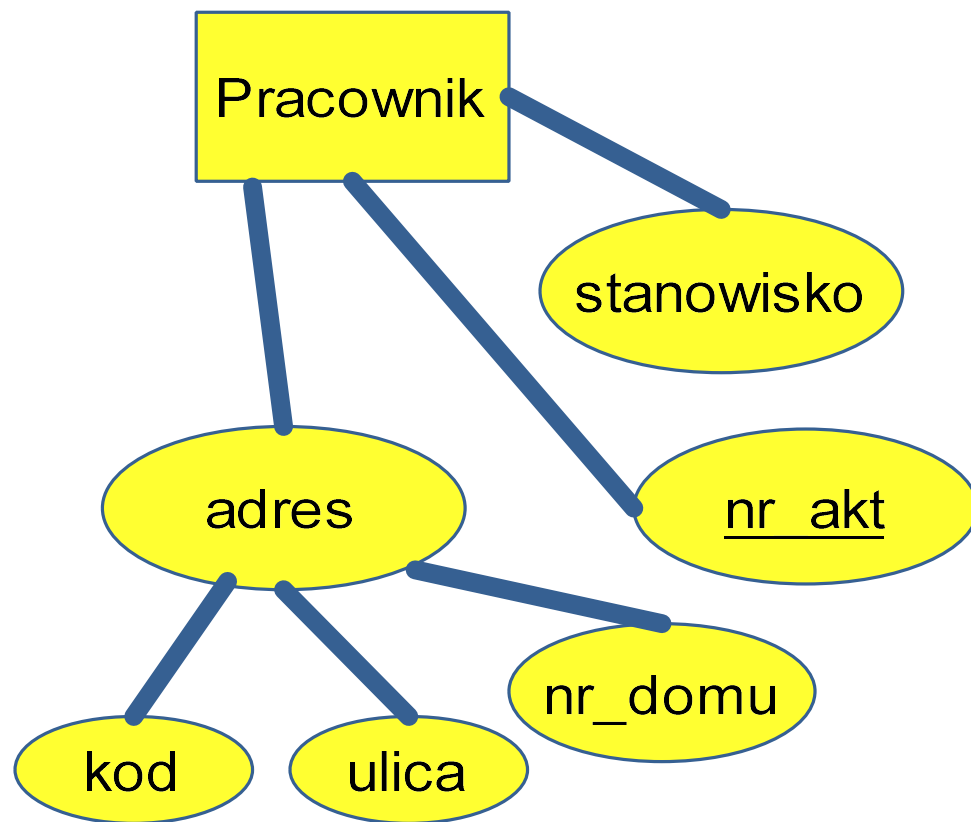
Atrybuty

- Każdy atrybut przybiera wartości z odpowiedniego zbioru wartości
- Typy atrybutów:
 - proste lub złożone
 - kluczowe
 - jedno lub wielowartościowe
- Atrybuty kluczowe
 - każda encja musi posiadać atrybut lub zbiór atrybutów będących jej **unikalnym identyfikatorem**
 - atrybuty kluczowe przyjmują unikalne wartości w ramach wszystkich wystąpień encji

Atrybuty (2)

- Wszystkie niekluczowe atrybuty encji to deskryptory.
- Definicja atrybutu obejmuje:
 - nazwę
 - określenie dziedziny (typ i zbiór dozwolonych wartości)
 - dopuszczenie wartości pustych
 - opcjonalną unikatowość

Atrybuty (3)

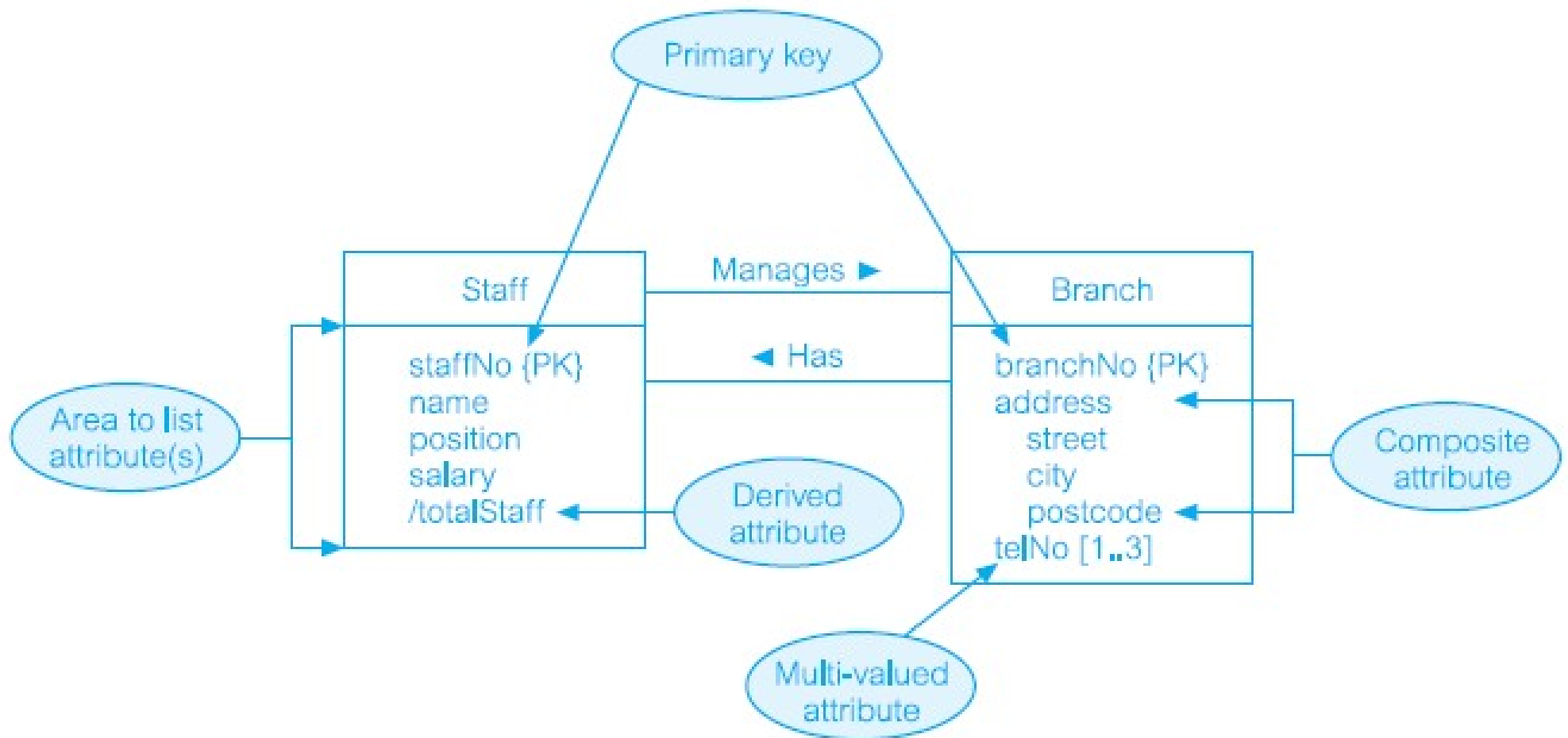


Notacja Chena



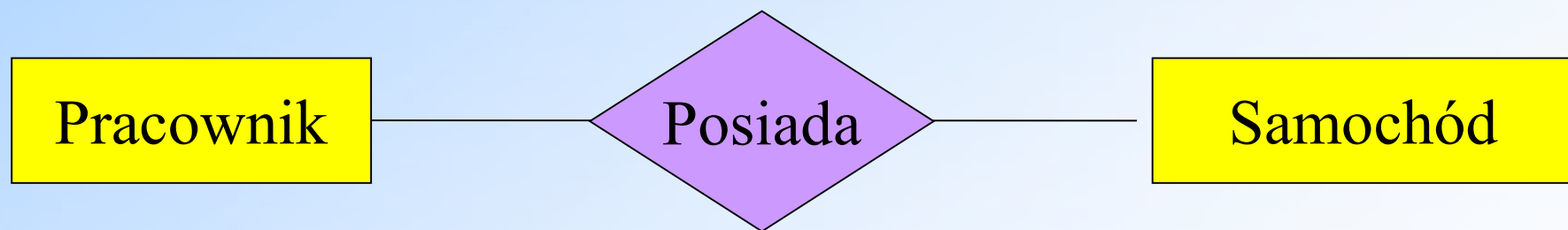
Notacja Barkera

Atrybuty - UML₍₄₎



Związki

- **Związki** określają powiązania pomiędzy instancjami encji (pomiędzy obiektami świata rzeczywistego)
- Liczba uczestników (encji) w związku nazywana jest jego **stopniem**.

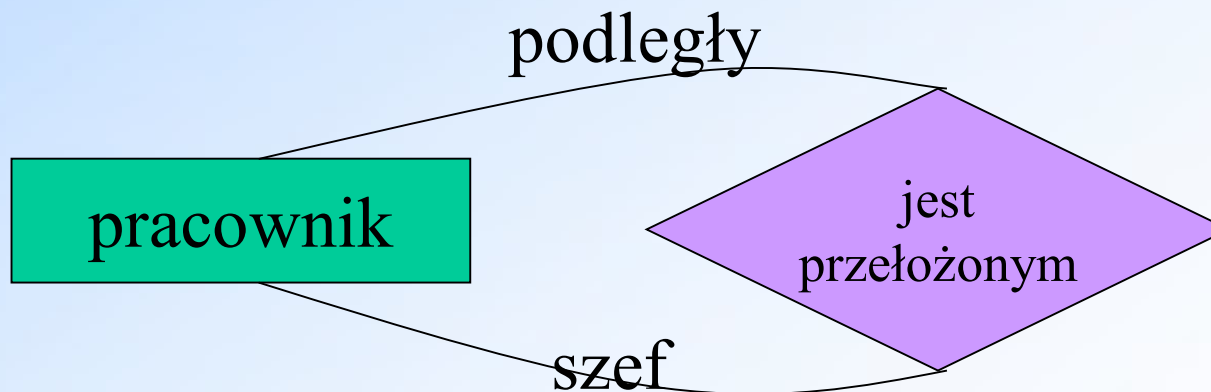
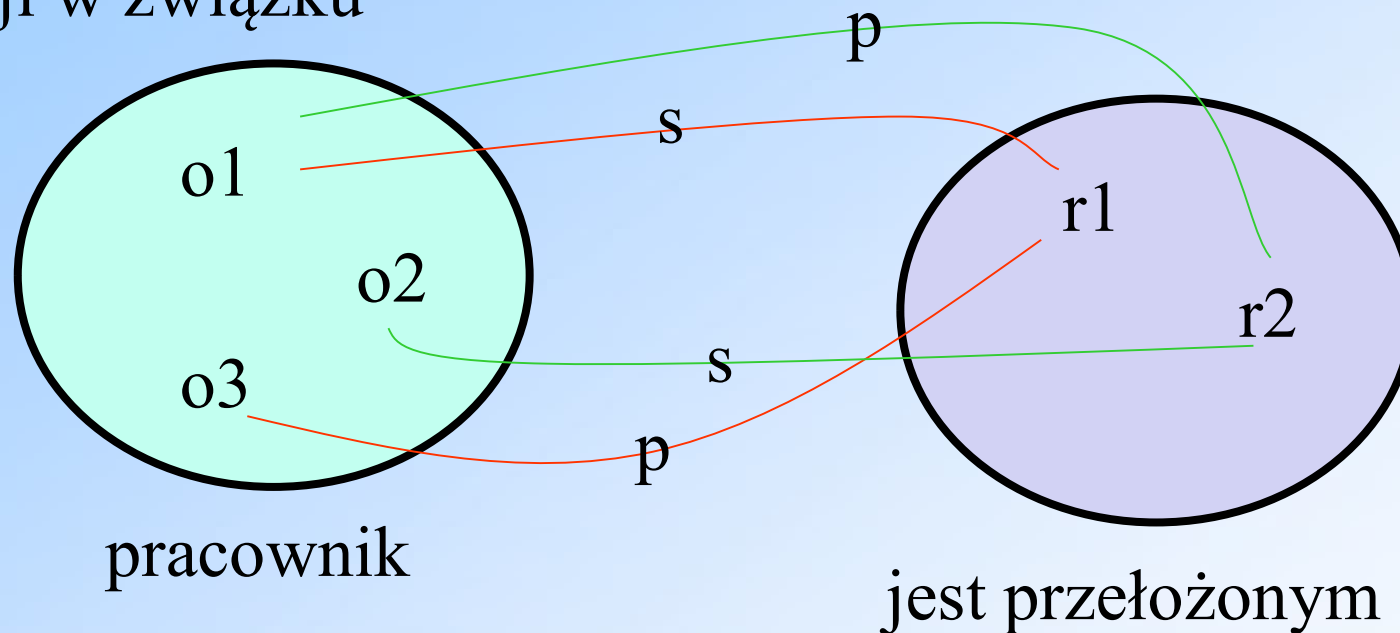


Cechy związku

- **Stopień związku**
 - unarny (binarny rekursywny)
 - binarny
 - ternarny
 - n-arny
- **Typ asocjacji**
 - jeden-do-jeden (1:1)
 - jeden-do-wiele (1:M)
 - wiele-do-wiele (M:N)
- **Istnienie**
 - opcjonalny
 - obowiązkowy

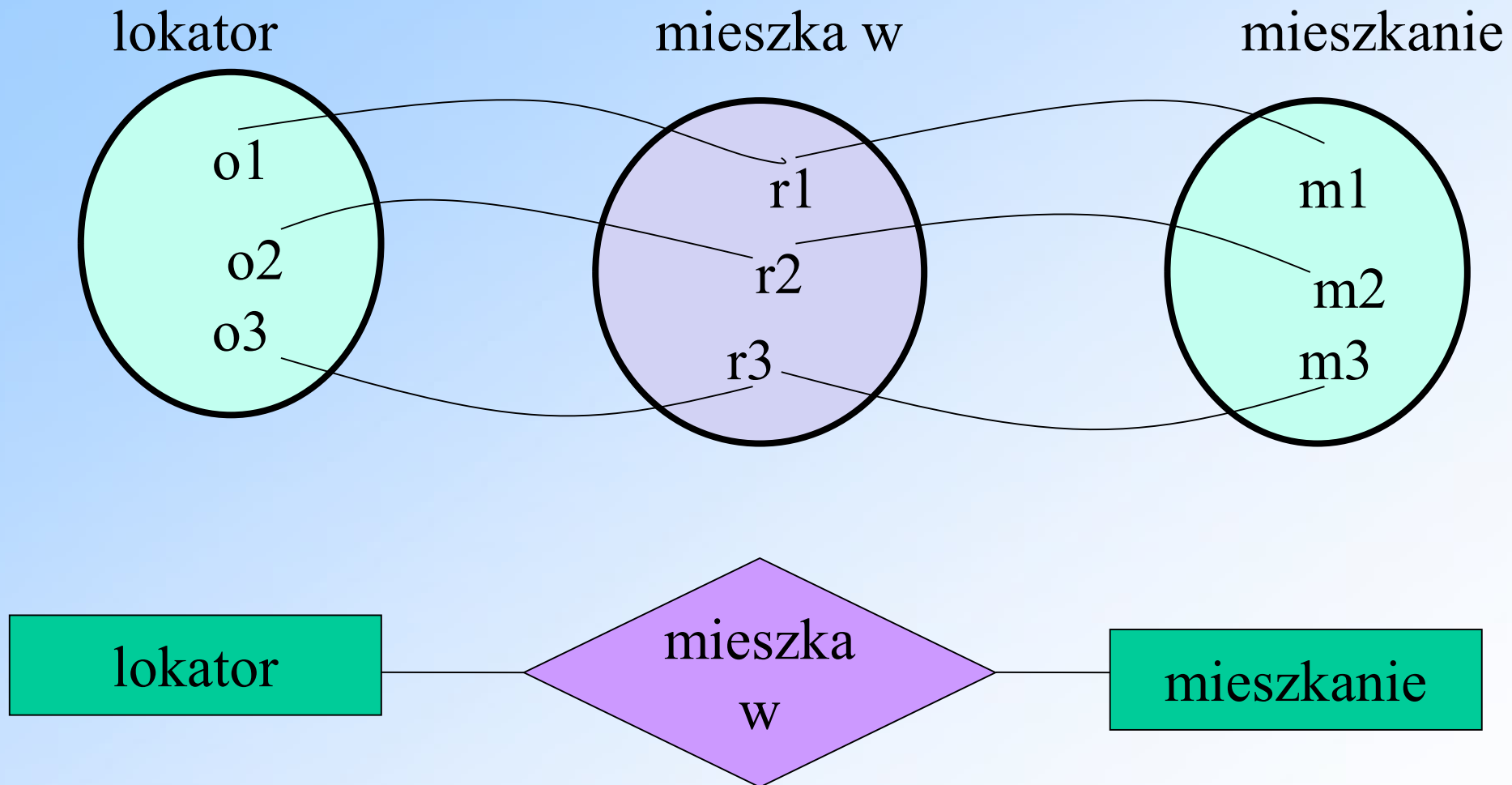
Związki unarne

Wiążą instancję pewnej encji samą ze sobą, w przypadku takich związków należy określić rolę jaką gra każda instancja encji w związku

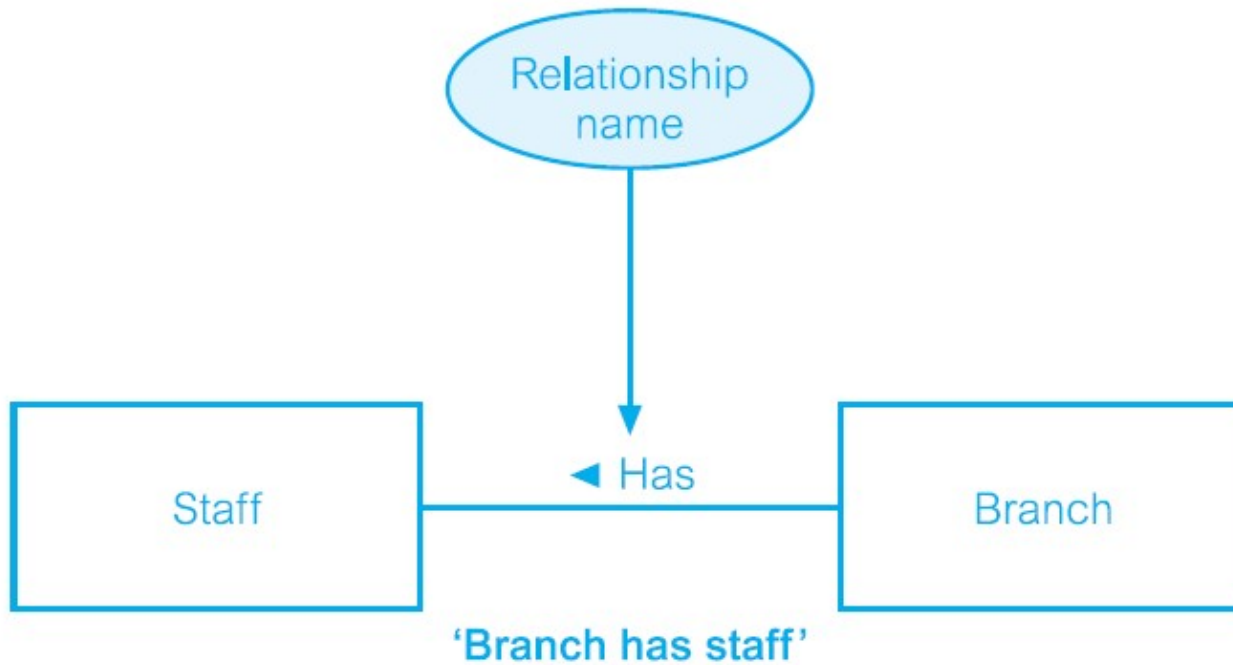


Związki binarne

Wiążą instancje dwóch encji

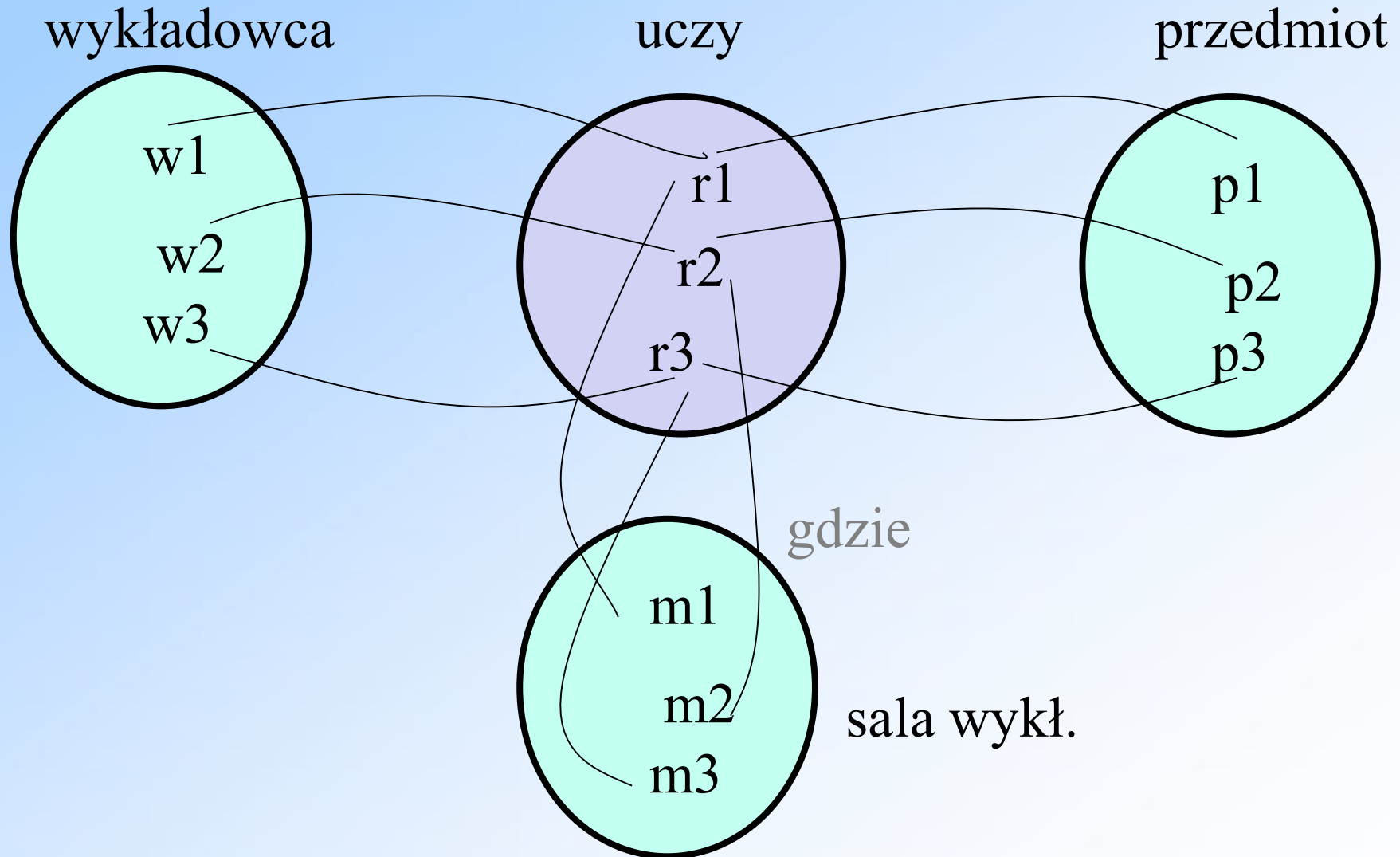


Związki binarne - UML

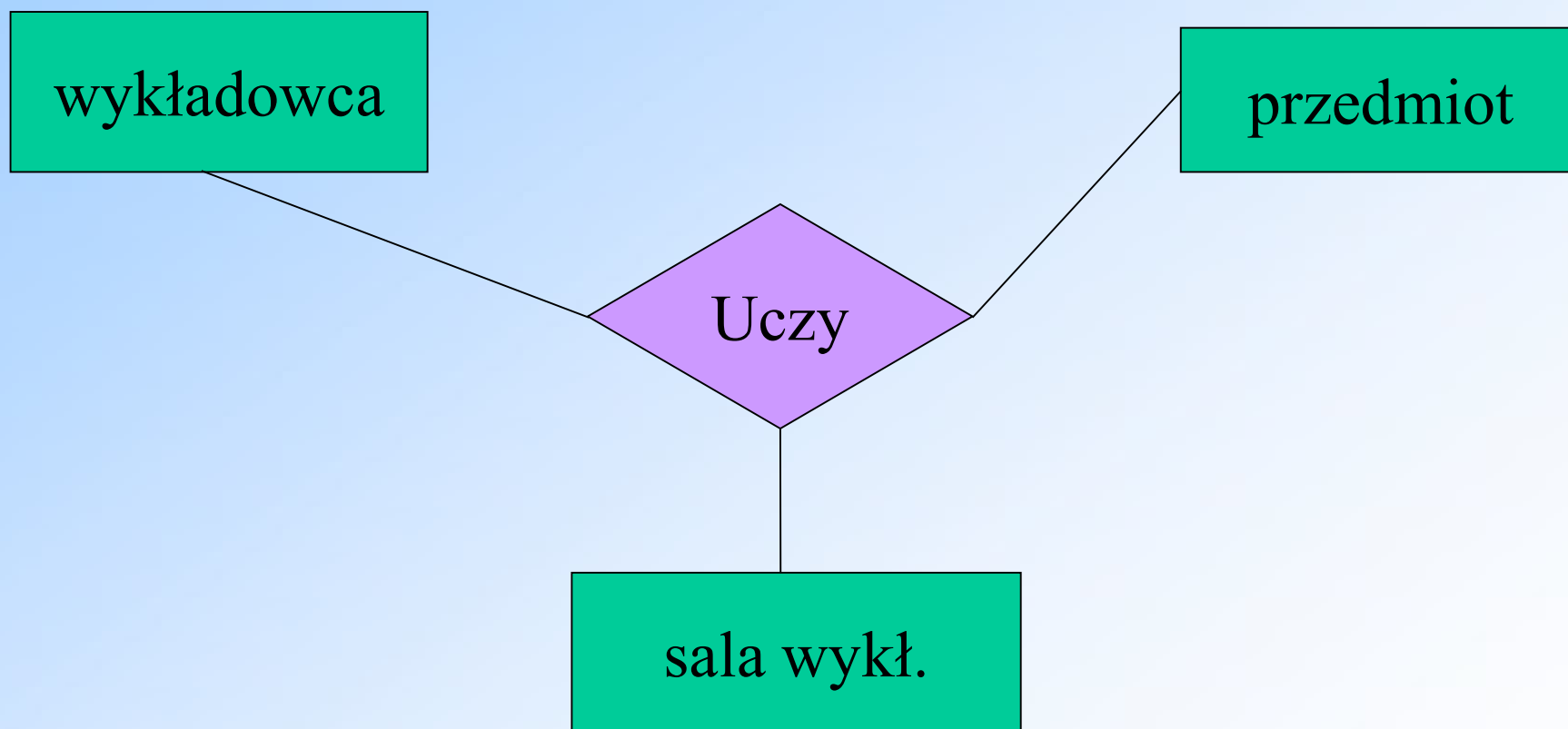


Związki ternarne

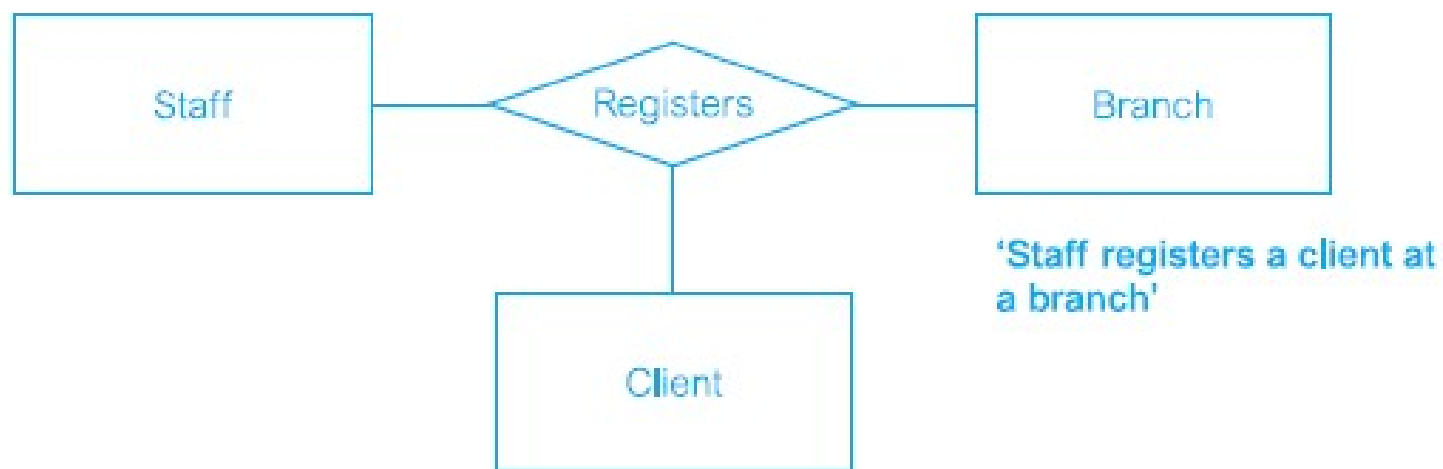
Wiążą instancje trzech encji



Związki ternarne (2)

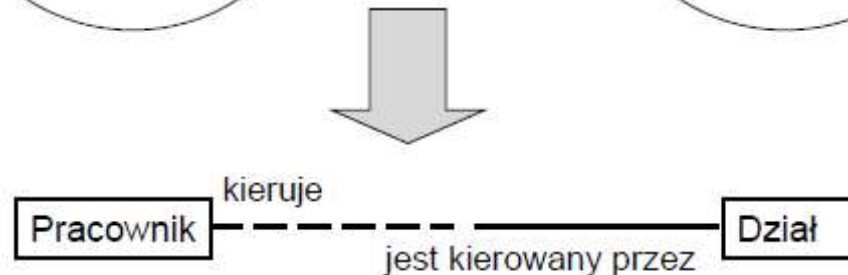
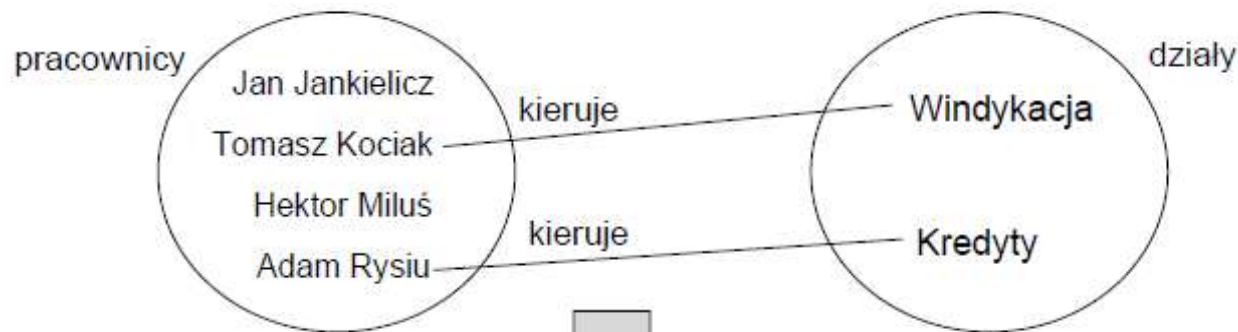


Związki ternarne (2) - UML

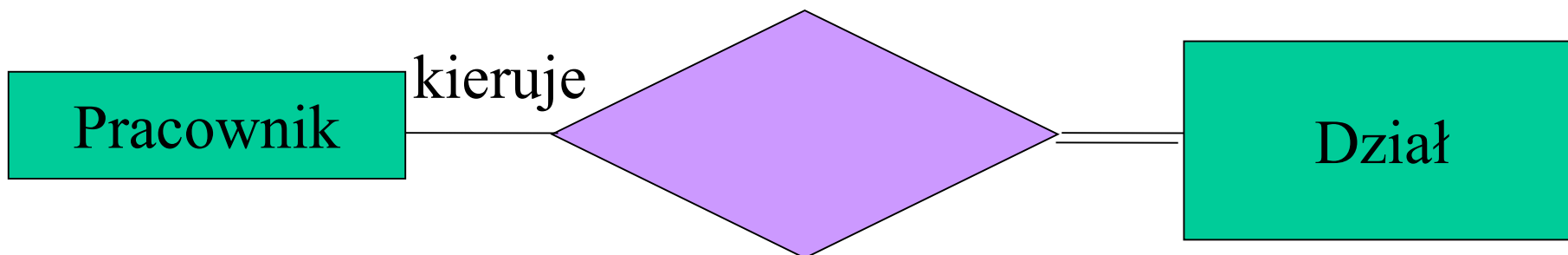


Typ asocjacji 1:1

Każda instancja encji bierze udział w co najwyżej jednej instancji związku

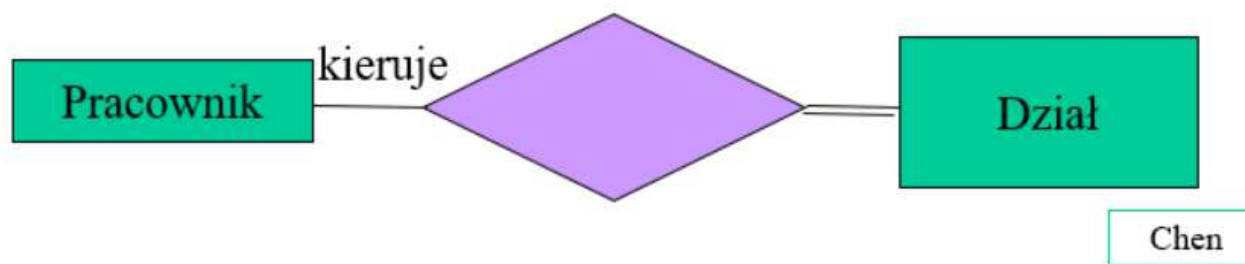
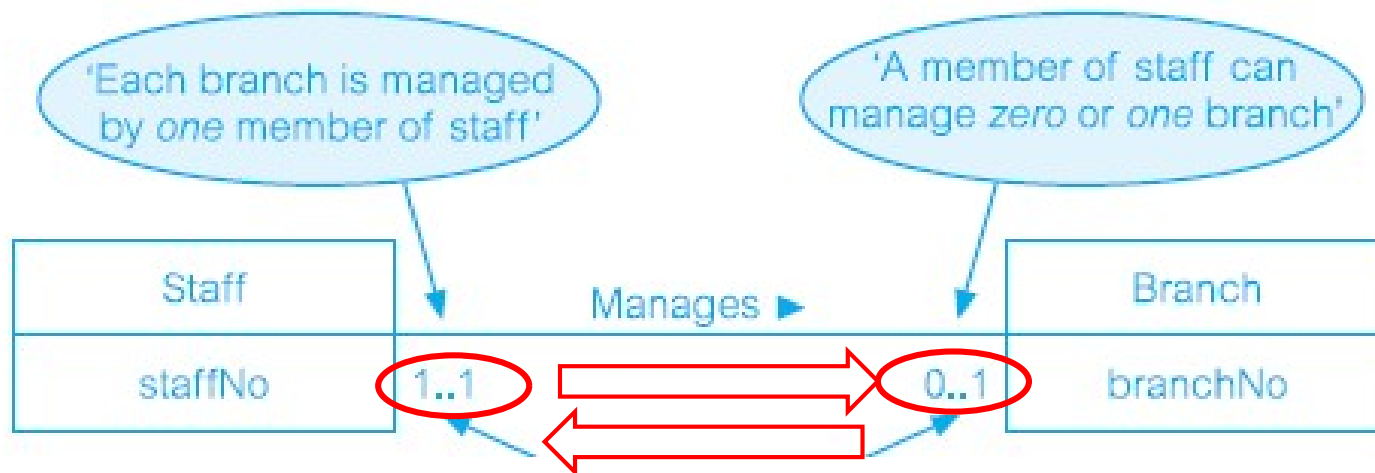
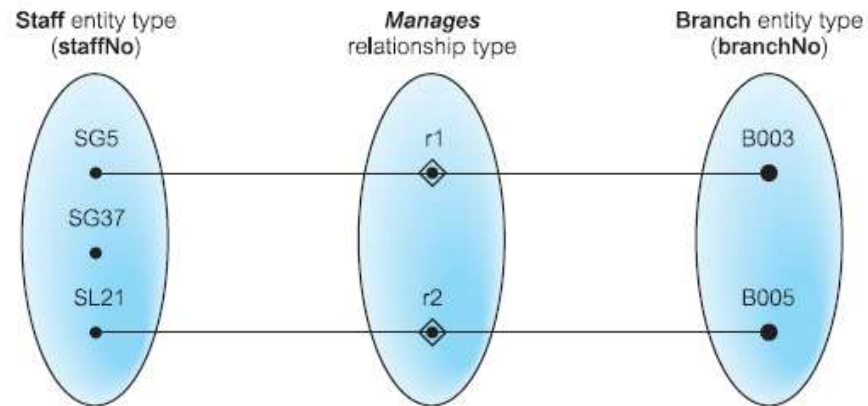


Barker



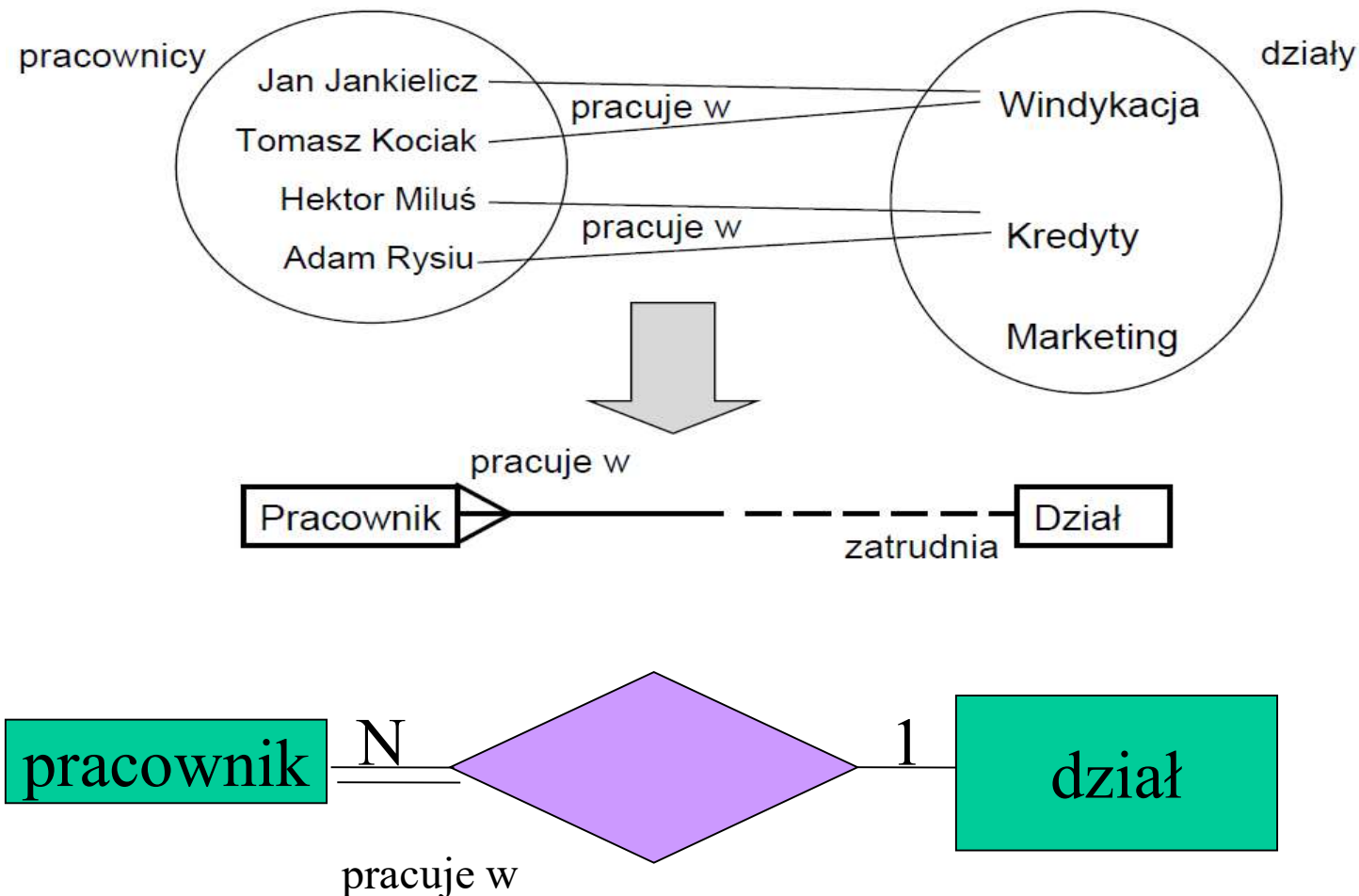
Chen

Typ asocjacji 1:1 - UML

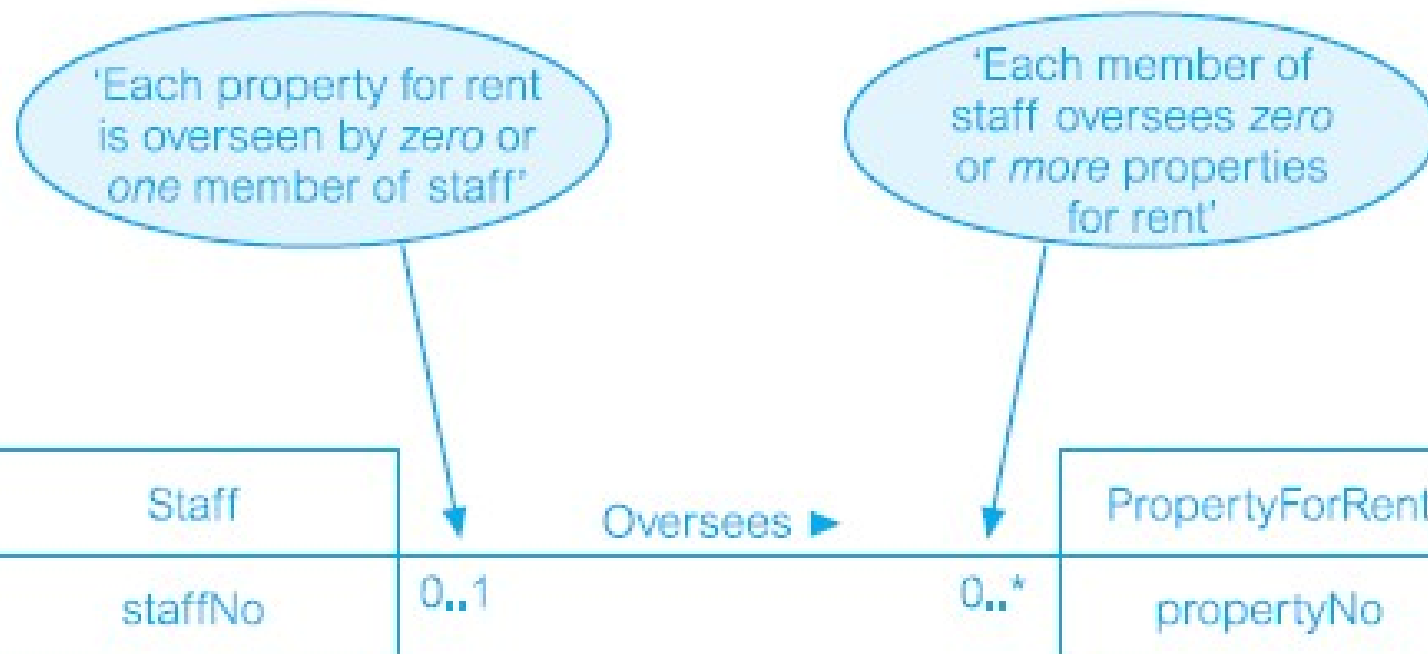
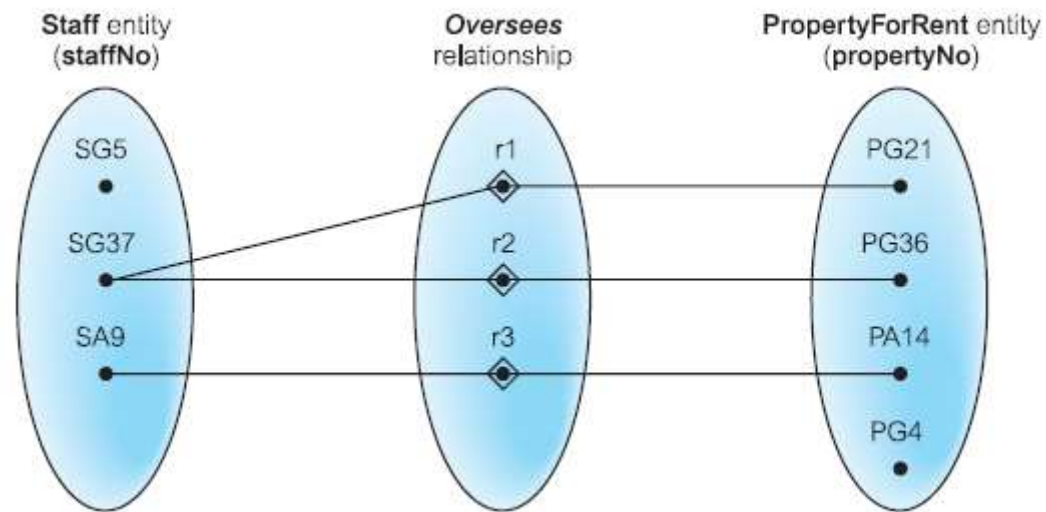


Typ asocjacji 1:N

Instancja jednej encji bierze udział w co najwyżej jednej instancji związku, instancja drugiej w dowolnej liczbie instancji związku

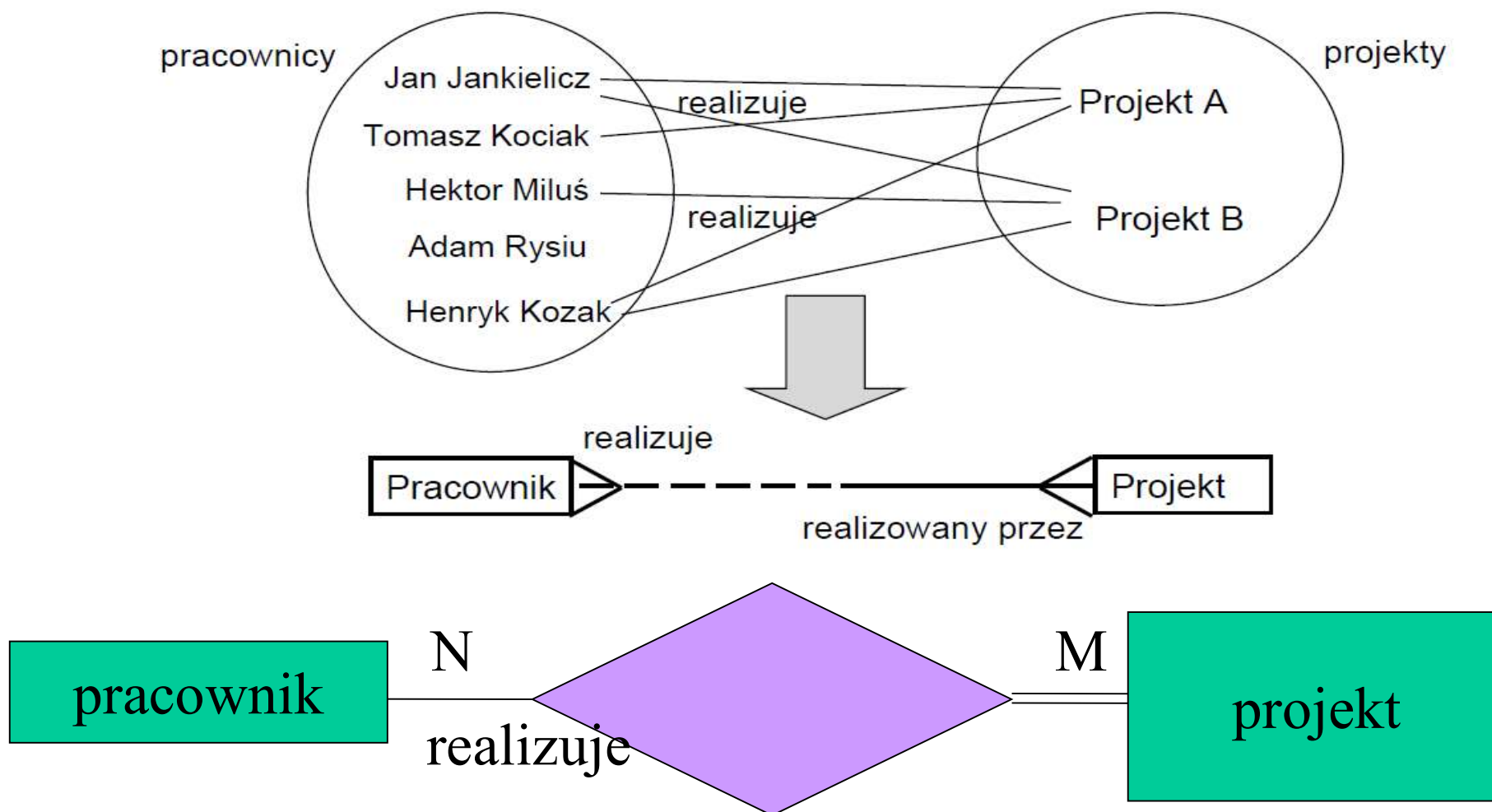


Typ asocjacji 1:N - UML

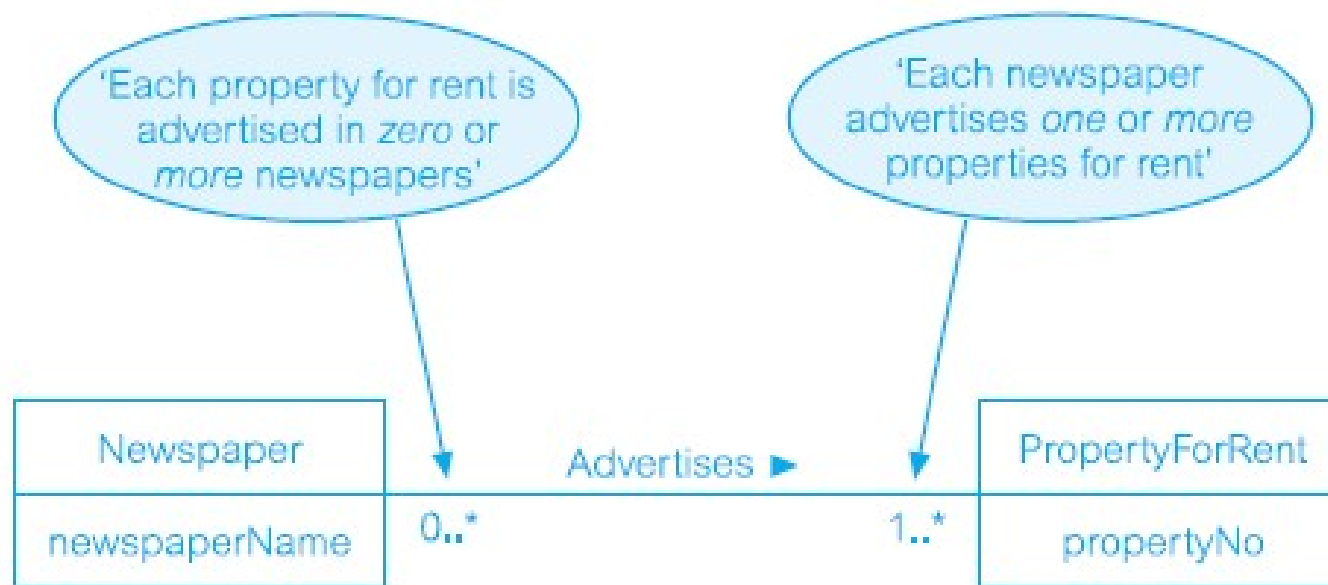
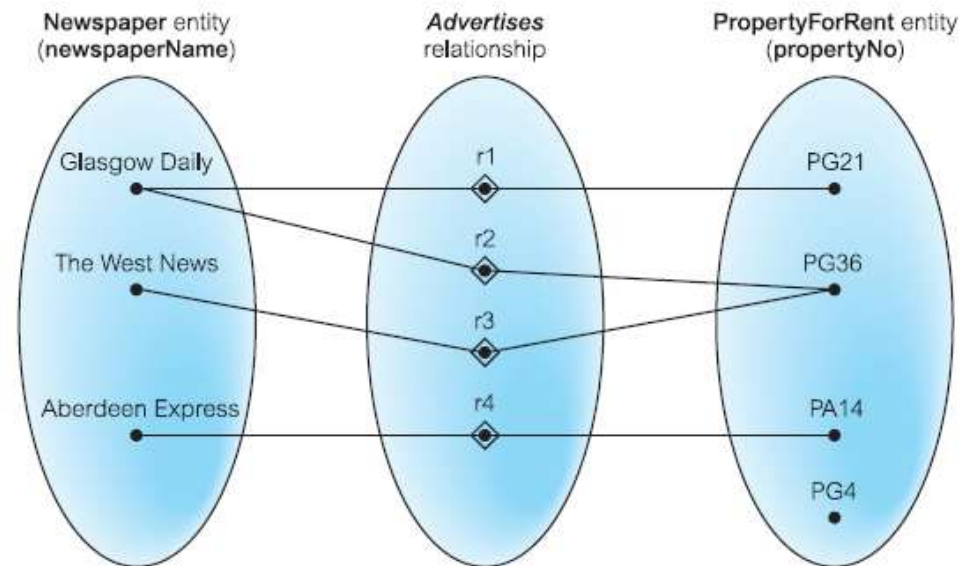


Typ asocjacji N:M

Instancja obu encji biorą udział w dowolnej liczbie instancji związku

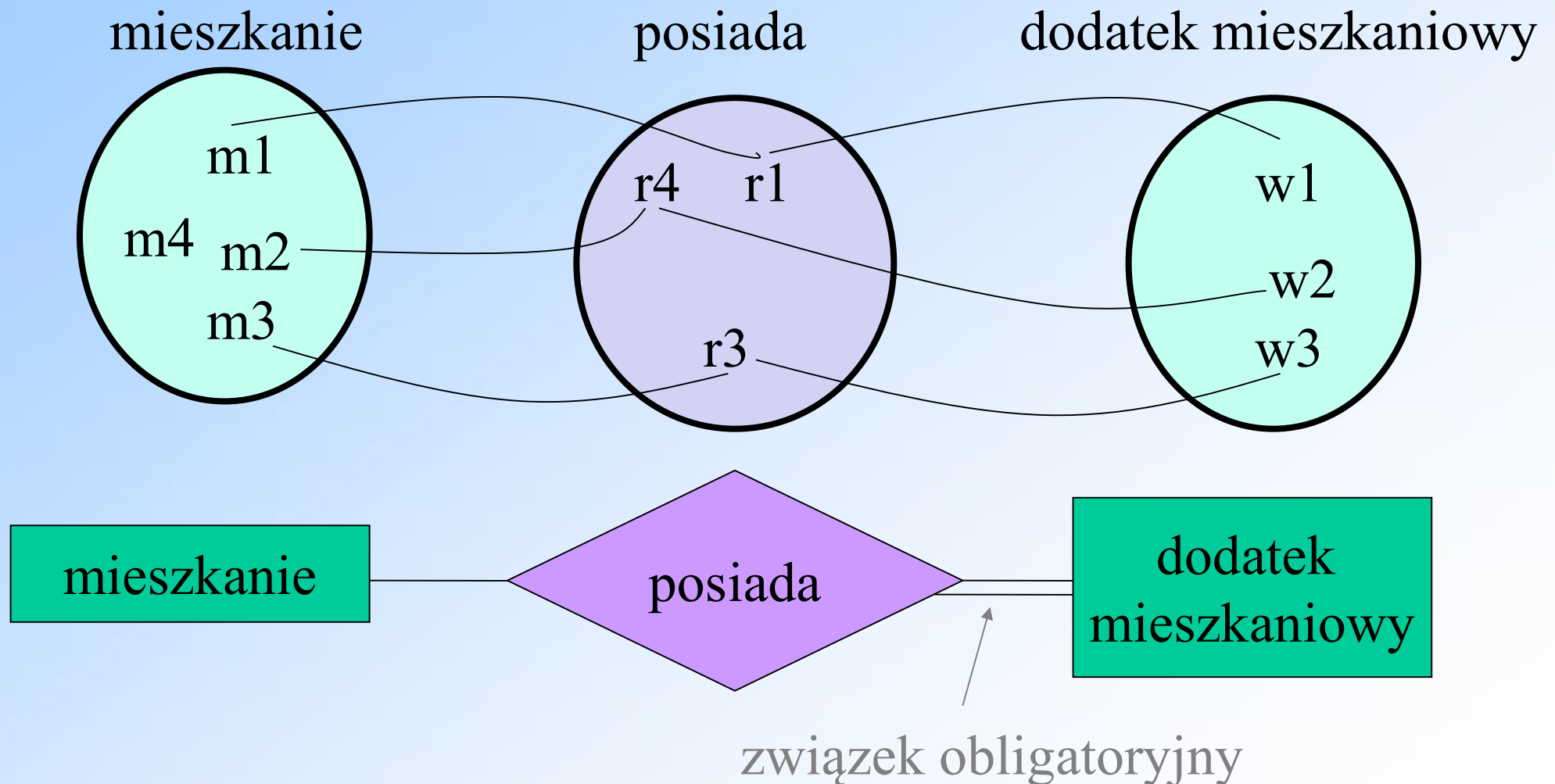


Typ asocjacji N:M - UML



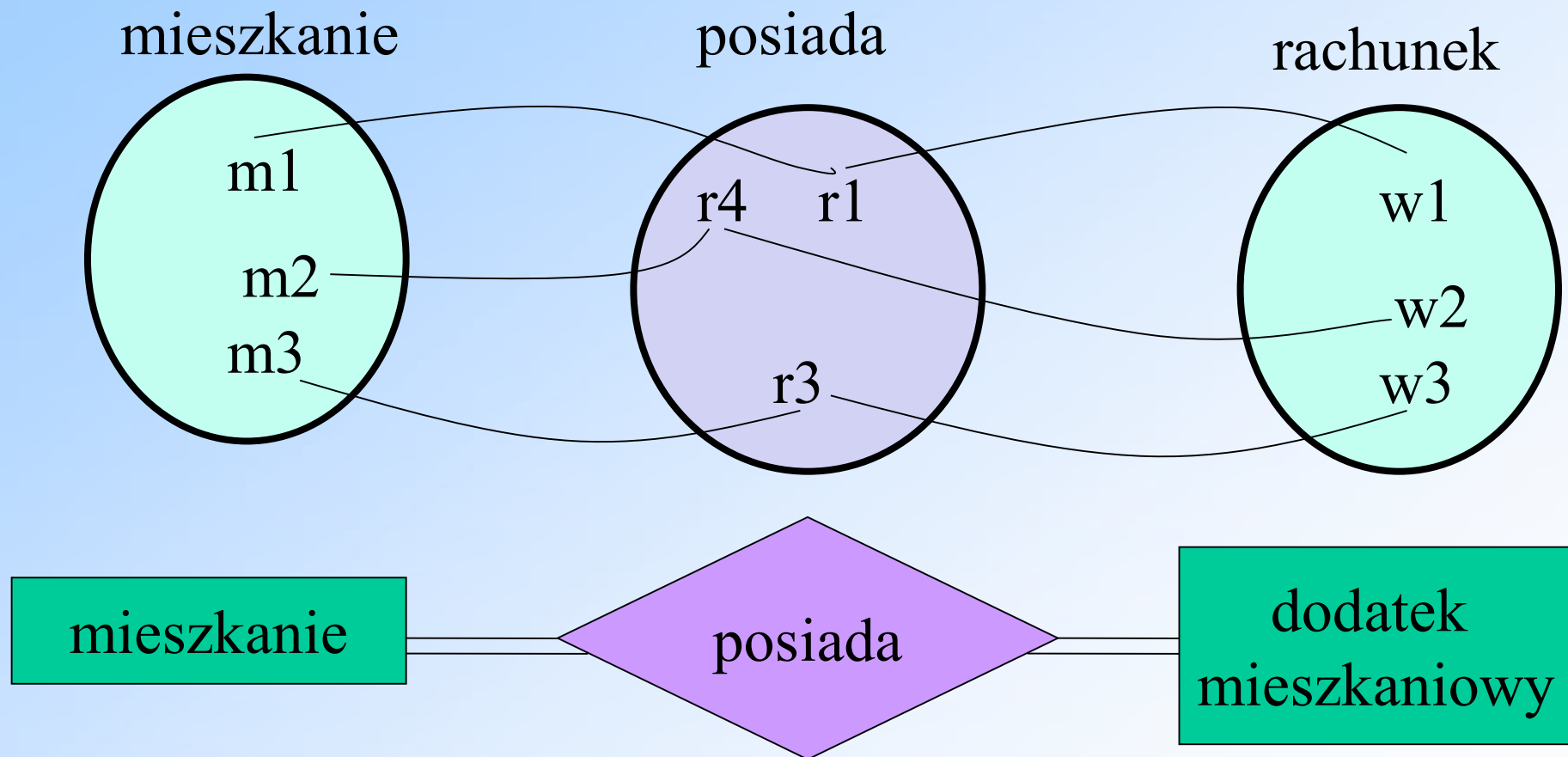
Opcjonalna przynależność do związku

Mogą istnieć encje nie biorące udziału w żadnej instancji związku



Obligatoryjna przynależność do związku

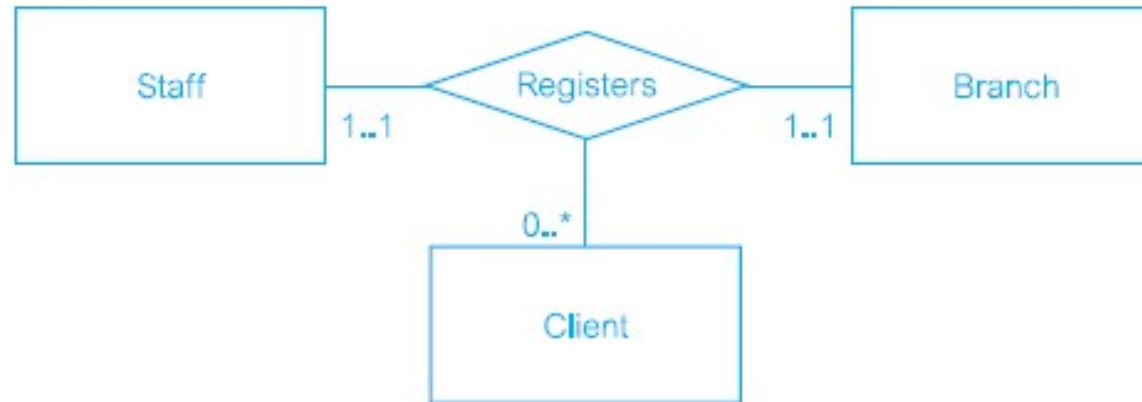
Wszystkie instancje encji muszą brać udział w instancji związku



Notacje Barkera



Notacja UML



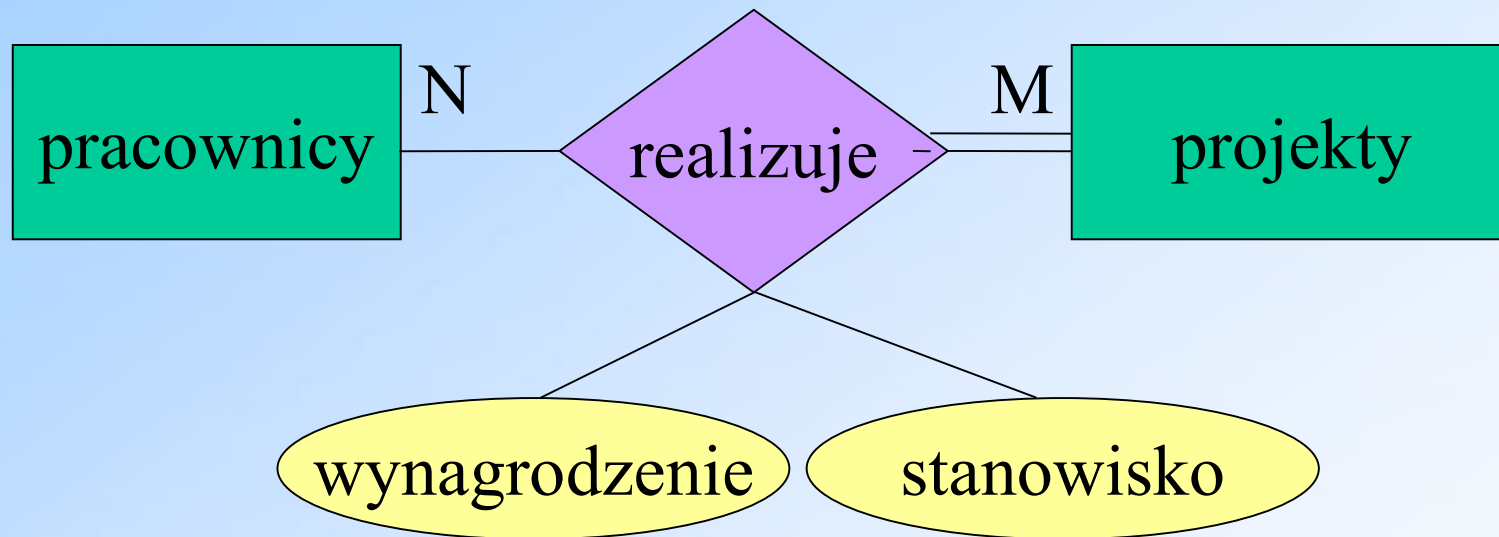
Przykłady – cechy związku:

- 0..1 – udział w związku opcjonalny, maksymalnie do jednego egzemplarza wiązanej encji
- 0..* - udział w związku opcjonalny, maksymalnie do wielu egzemplarzy wiązanej encji
- 1..1 – udział w związku obowiązkowy, z dokładnie jednym egzemplarzem wiązanej encji
- 1..* - udział w związku obowiązkowy, maksymalnie do wielu egzemplarzy wiązanej encji

Atrybuty związków

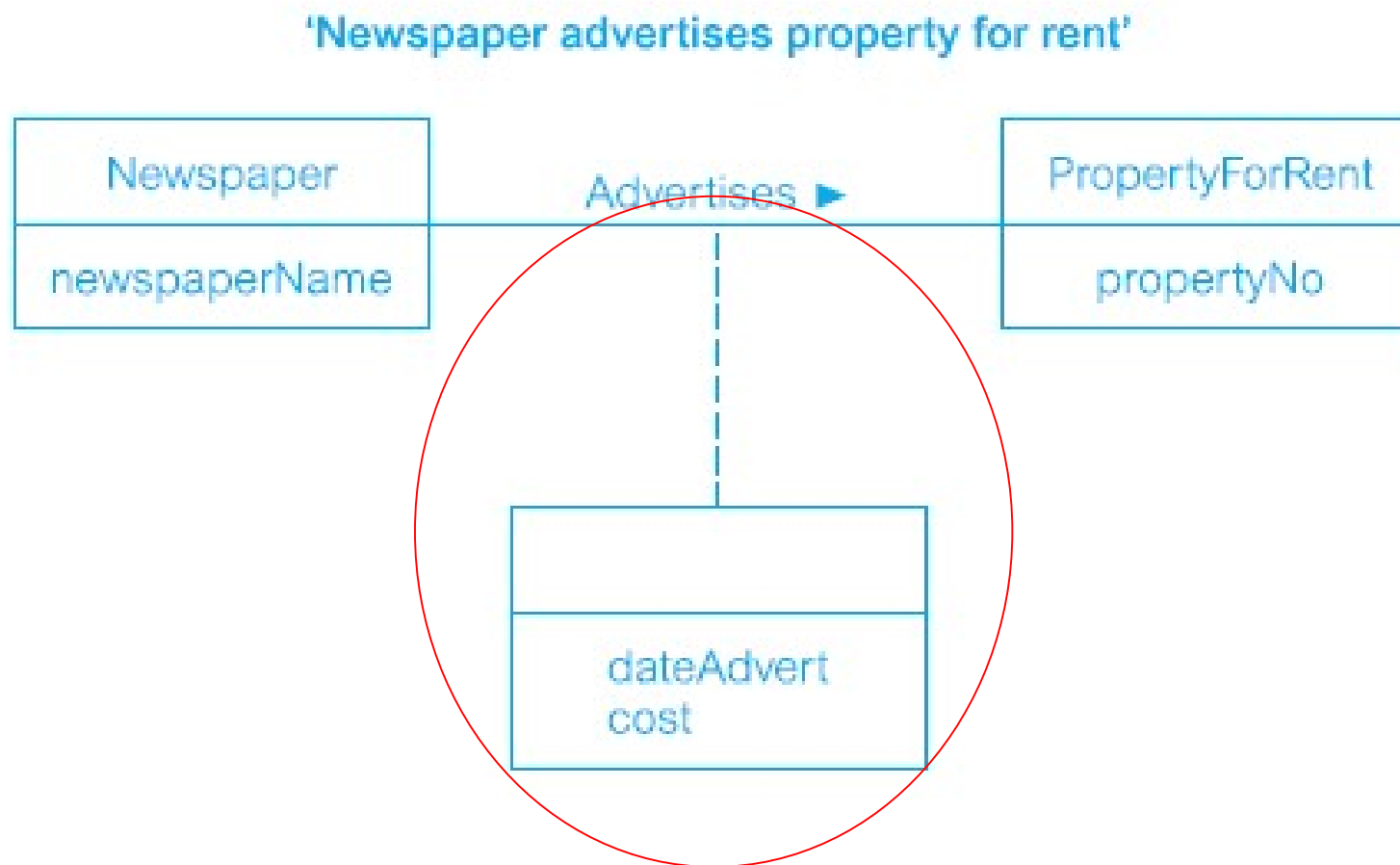
- Czasami wygodniejsze jest łączenie atrybutów ze związkiem pomiędzy zbiorami encji.

Atrybuty związków - Przykład



Dla pracowników biorących udział w projekcie należy zapamiętać stanowisko i wynagrodzenie

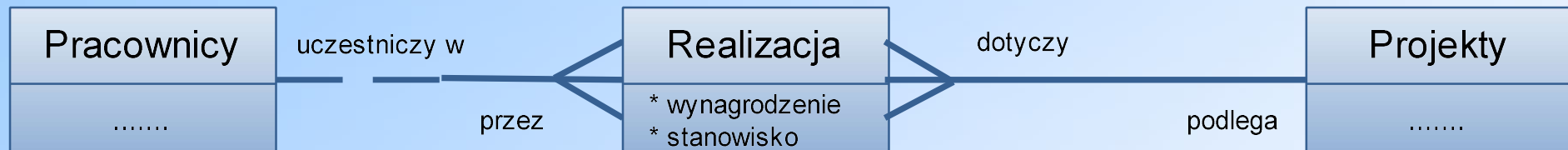
Atrybuty związków – Przykład - UML



Równoważne diagramy bez atrybutów związków

- Należy utworzyć zbiór encji reprezentujący dane atrybuty.
- Następnie należy tak utworzony zbiór encji włączyć do związku.
- Notacja Barkera nie dopuszcza wiązania atrybutów ze związkami

Atrybuty związków – notacja Barkera



- Jeżeli istnieje wystąpienie encji realizacja to musi ono dotyczyć jakiegoś projektu i pracownika
- nie może istnieć realizacja bez projektu i pracownika

Słabe zbiory encji

- ❓ Czasami encje z jednego zbioru encji wymagają innych zbiorów encji w celu umożliwienia ich jednoznacznej identyfikacji.
- ❓ Zbiór encji E jest *słabym* jeśli do unikalnej identyfikacji jego encji należy prześledzić jeden lub kilka związków jeden-do-wielu i dołączyć klucz z innego, powiązanego zbioru encji.

Przykład

- **Imię** jest „prawie” kluczem dla zbioru encji piłkarze, ale może być dwóch piłkarzy o tym samym imieniu.
- **numer** nie jest kluczem, ponieważ w dwóch drużynach piłkarskich numery mogą się powtarzać.
- **Numer** razem z **nazwą** drużyny związaną z zawodnikiem związkiem **gra-w** jest kluczem.

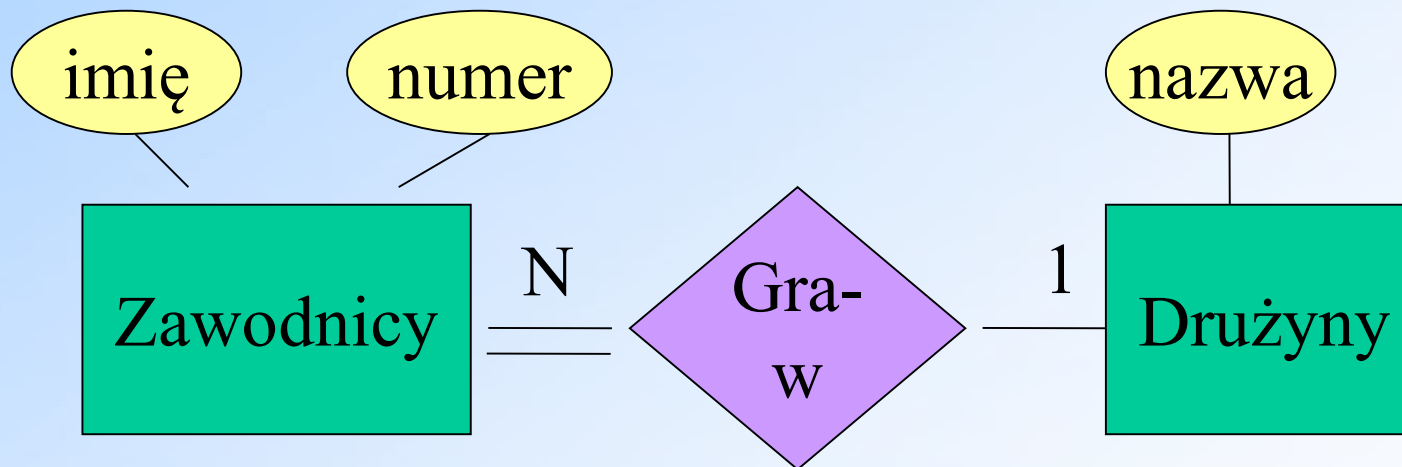
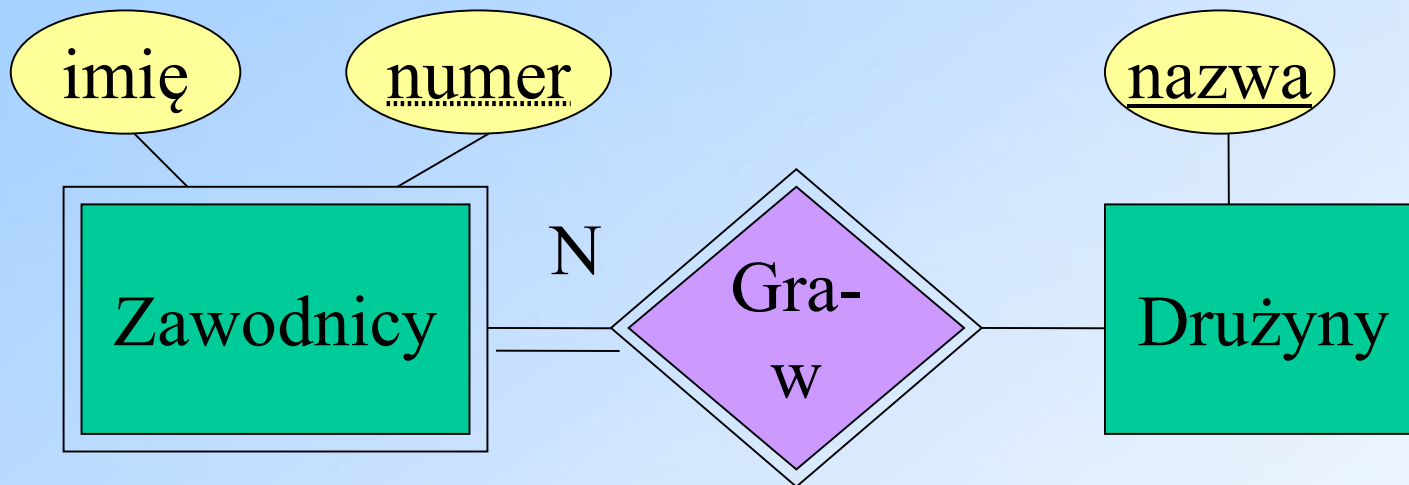
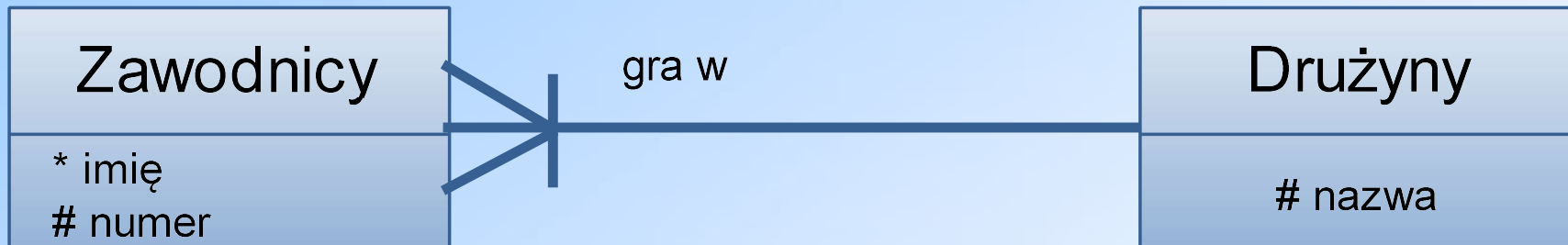


Diagram E/R



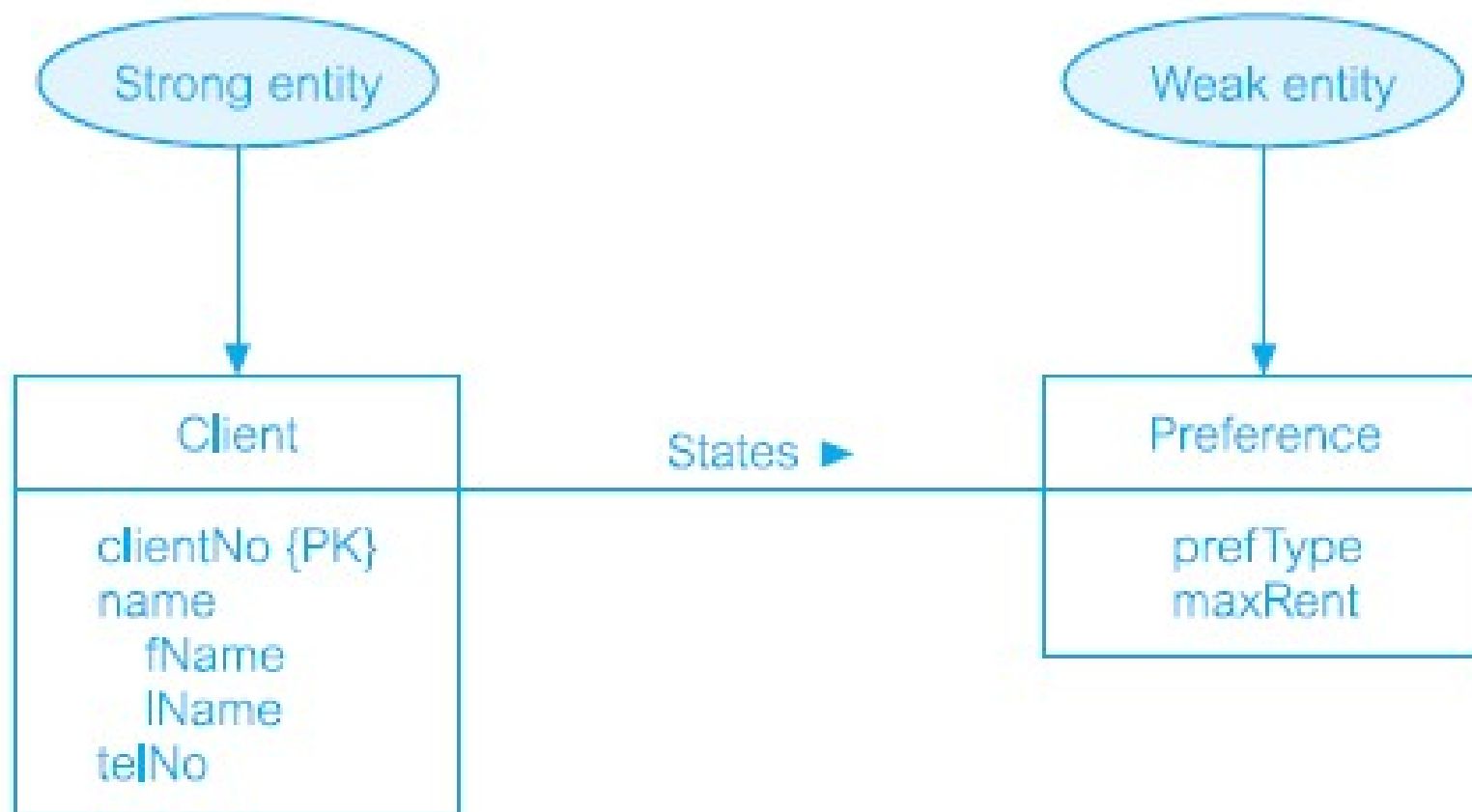
- Związek *wspierający* oznaczany jest podwójnym rombem.
- Słaby zbiór encji oznaczany jest podwójnym prostokątem.

Słabe zbiory encji – notacja Barkera



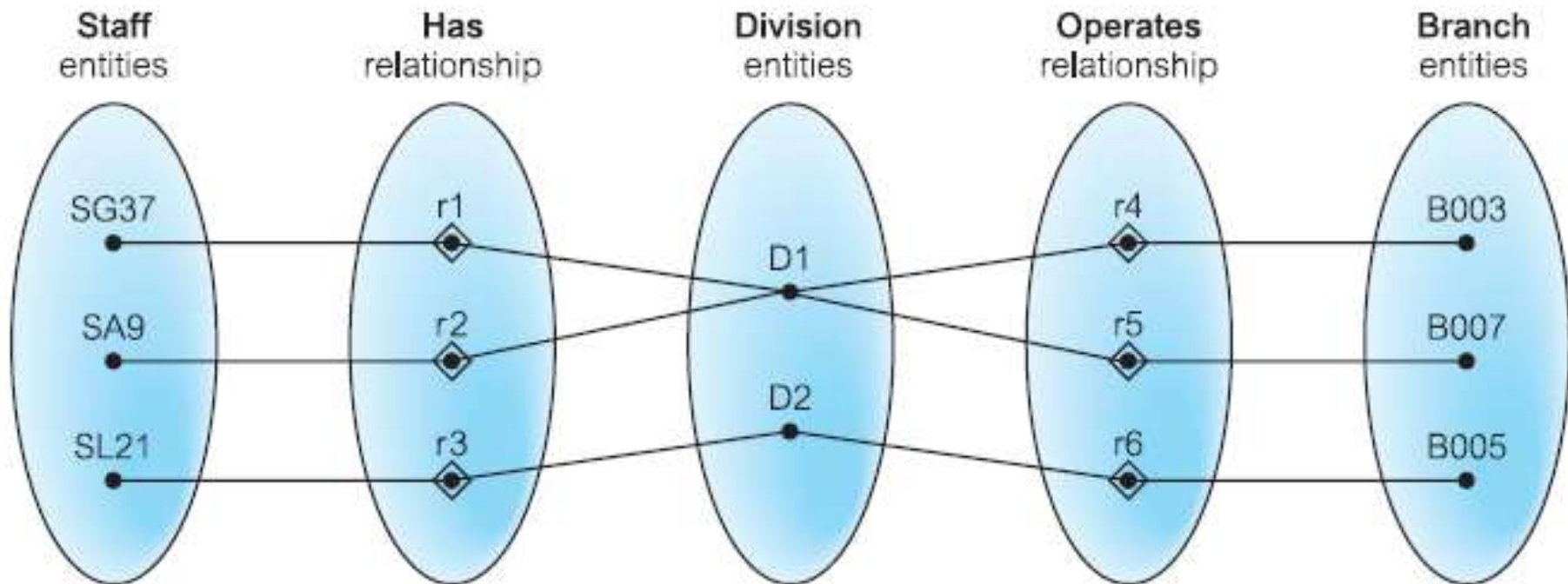
- Związek wchodzący w skład identyfikatora encji oznaczany jest jako krótka prostopadła linia.

Słabe zbiory encji – notacja UML



Potencjalne problemy w modelowaniu ER

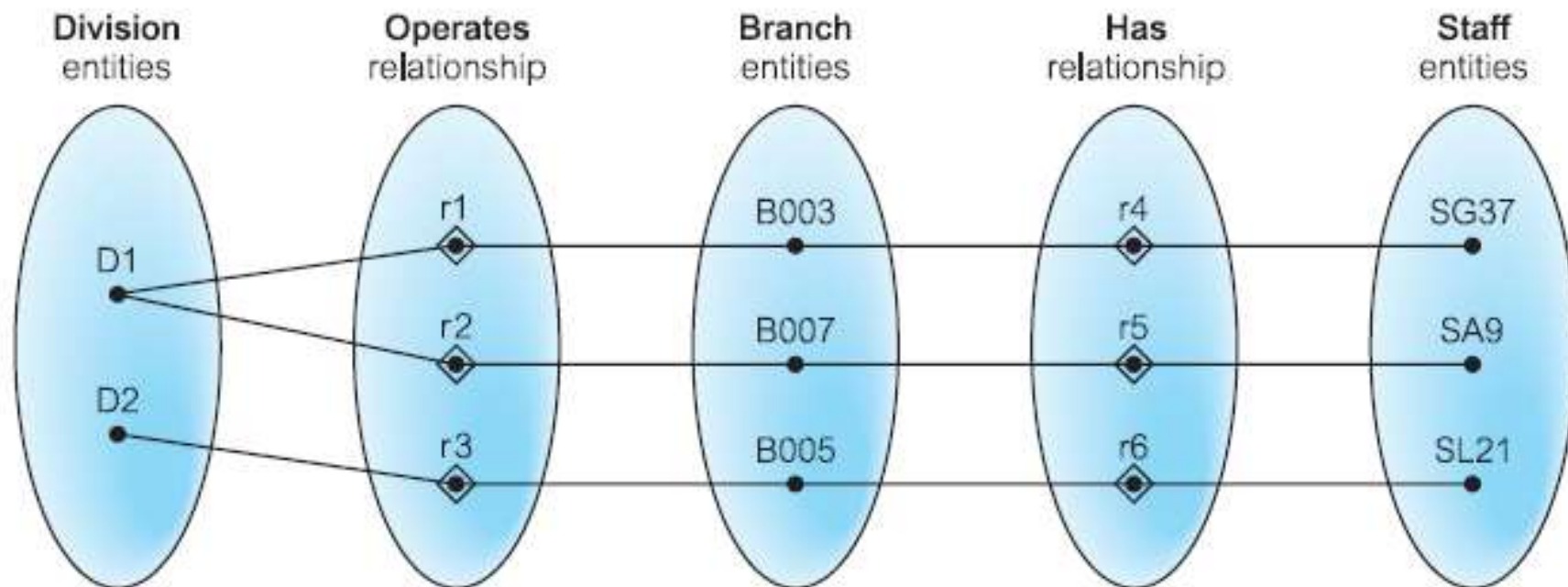
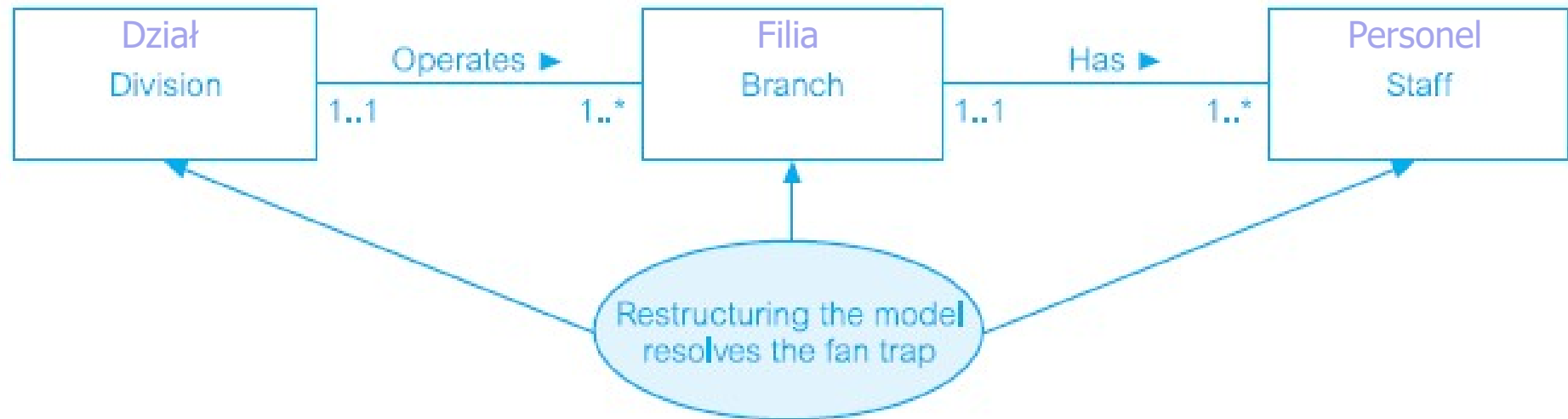
- Wiatrak (Fan trap)



W której filii pracuje SG37 ?

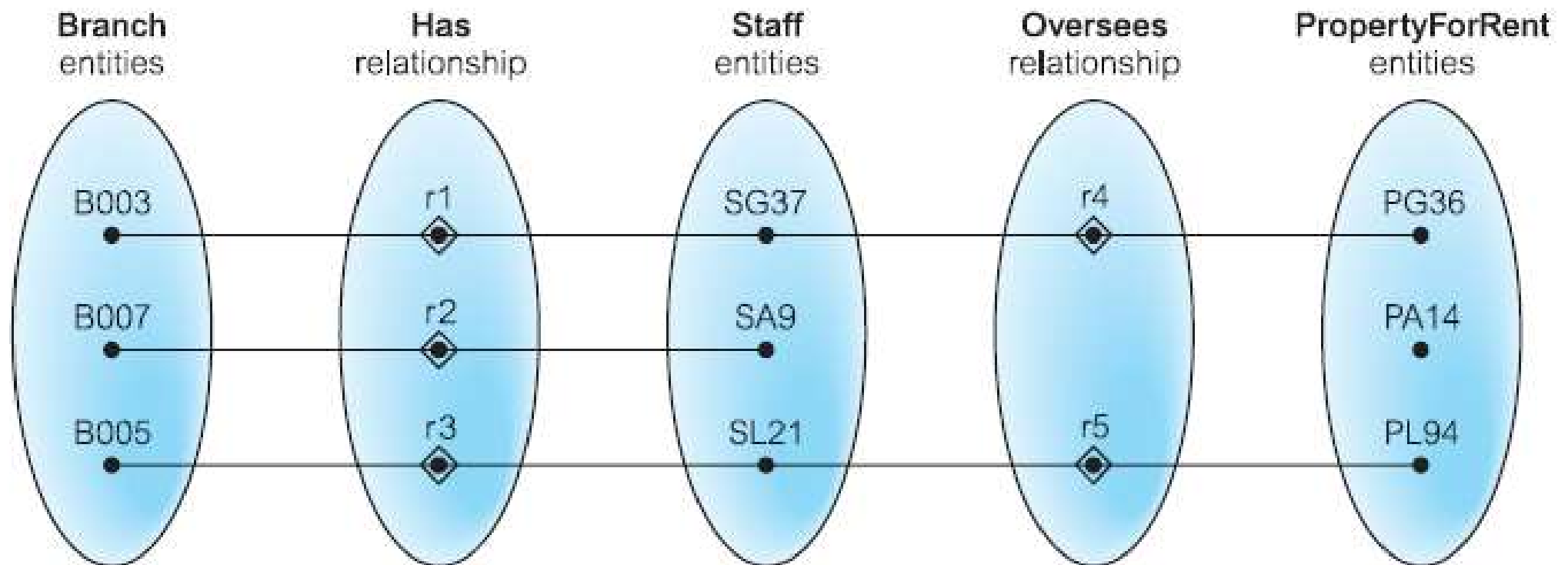
Potencjalne problemy w modelowaniu ER

- Wiatrak (Fan trap) - rozwiązanie



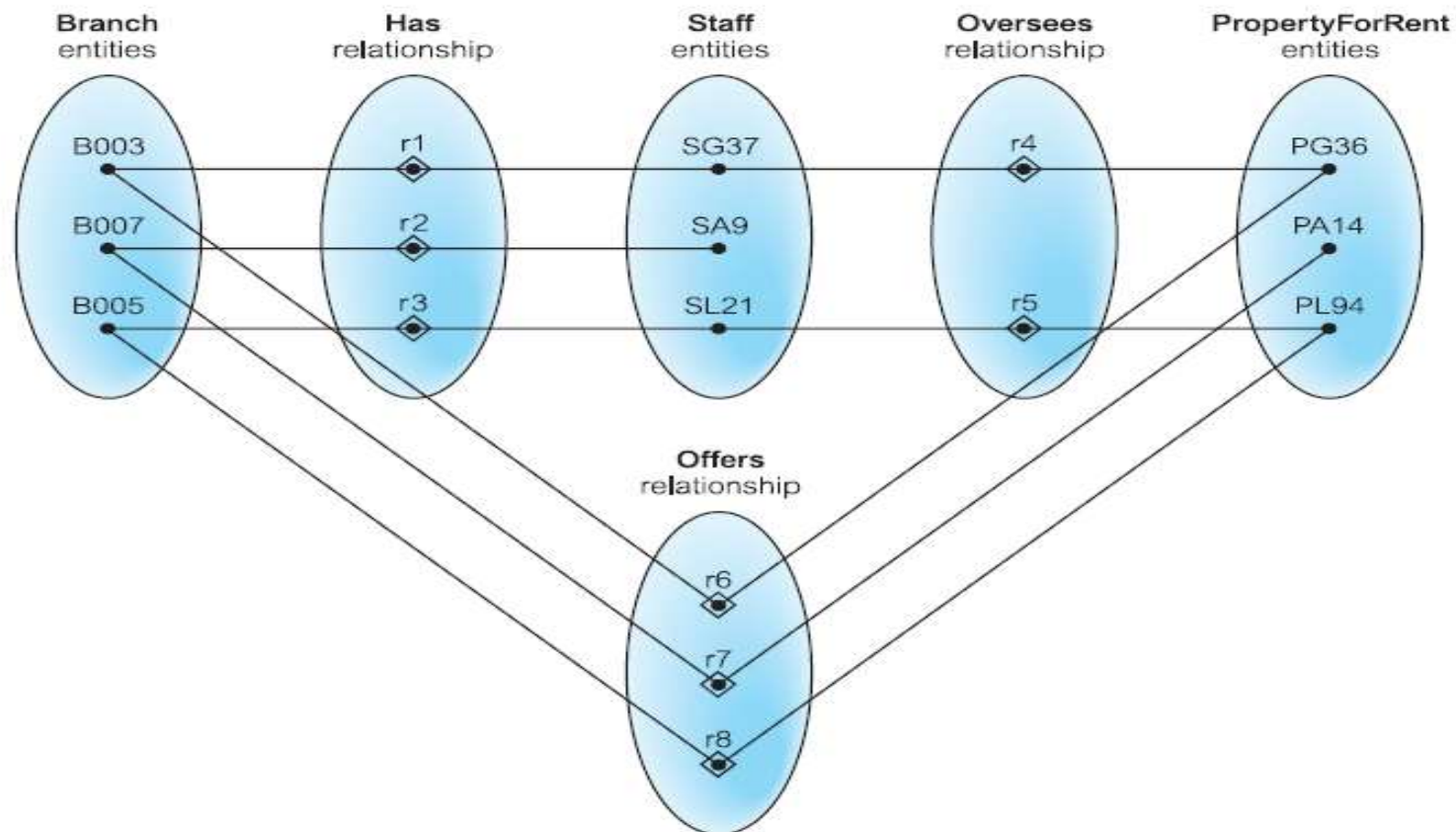
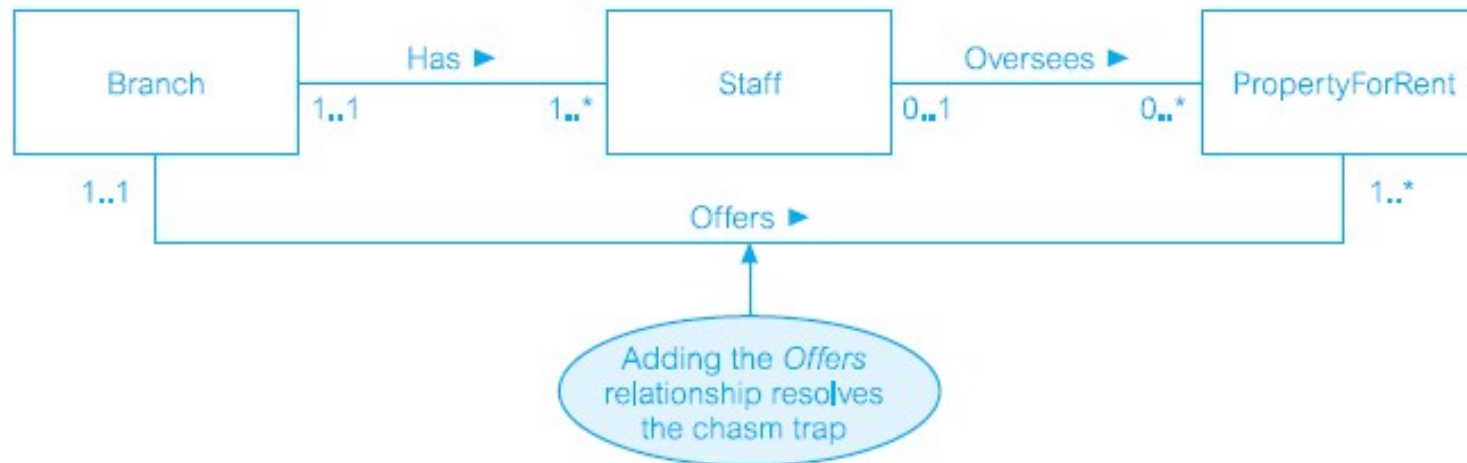
Potencjalne problemy w modelowaniu ER

- Przepaść (chasm trap)



Potencjalne problemy w modelowaniu ER

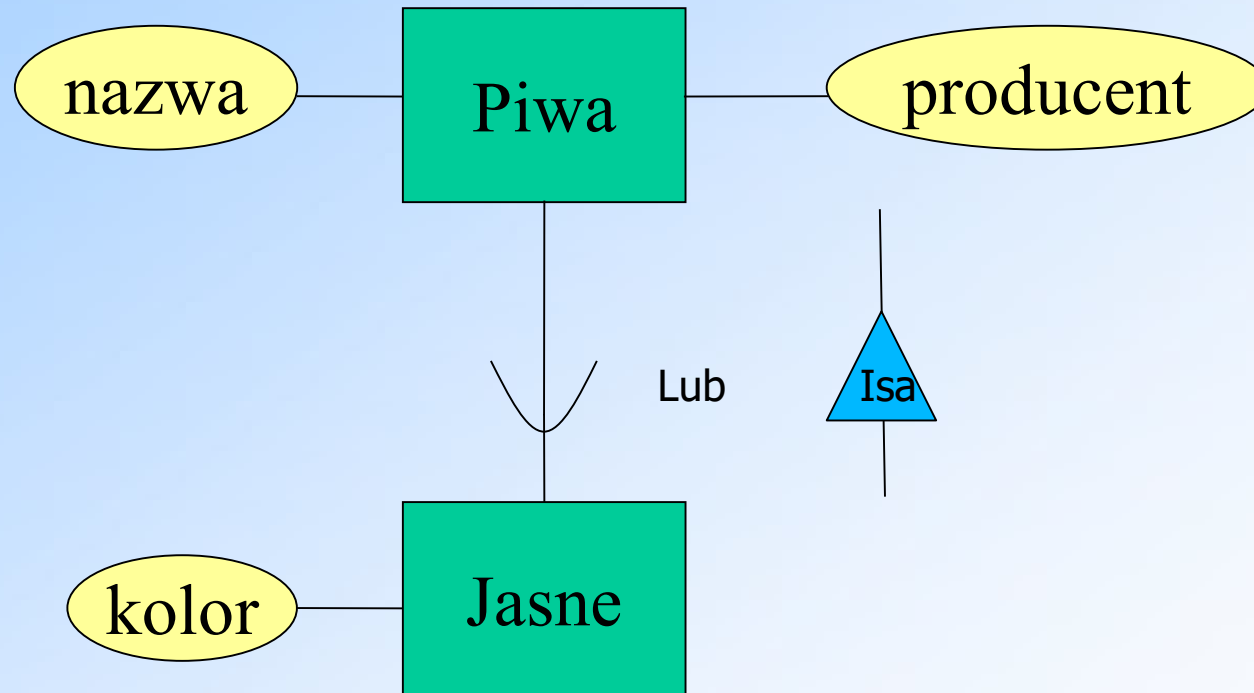
- Przepaść (chasm trap) - rozwiązanie



Podklasy w modelu E/R

- *Podklasa* = wyjątkowy zbiór encji posiadający dodatkowe, specjalizowane atrybuty.
- Np: Piwa jasne są pewną grupą piw.
 - Dla tej grupy piw, oprócz standardowych dla piw atrybutów, można dodać właściwość **kolor**.

Przykład



Specjalizacja i generalizacja

□ *Specjalizacja*

- definiowanie zbioru encji podklasy na podstawie encji nadklasy

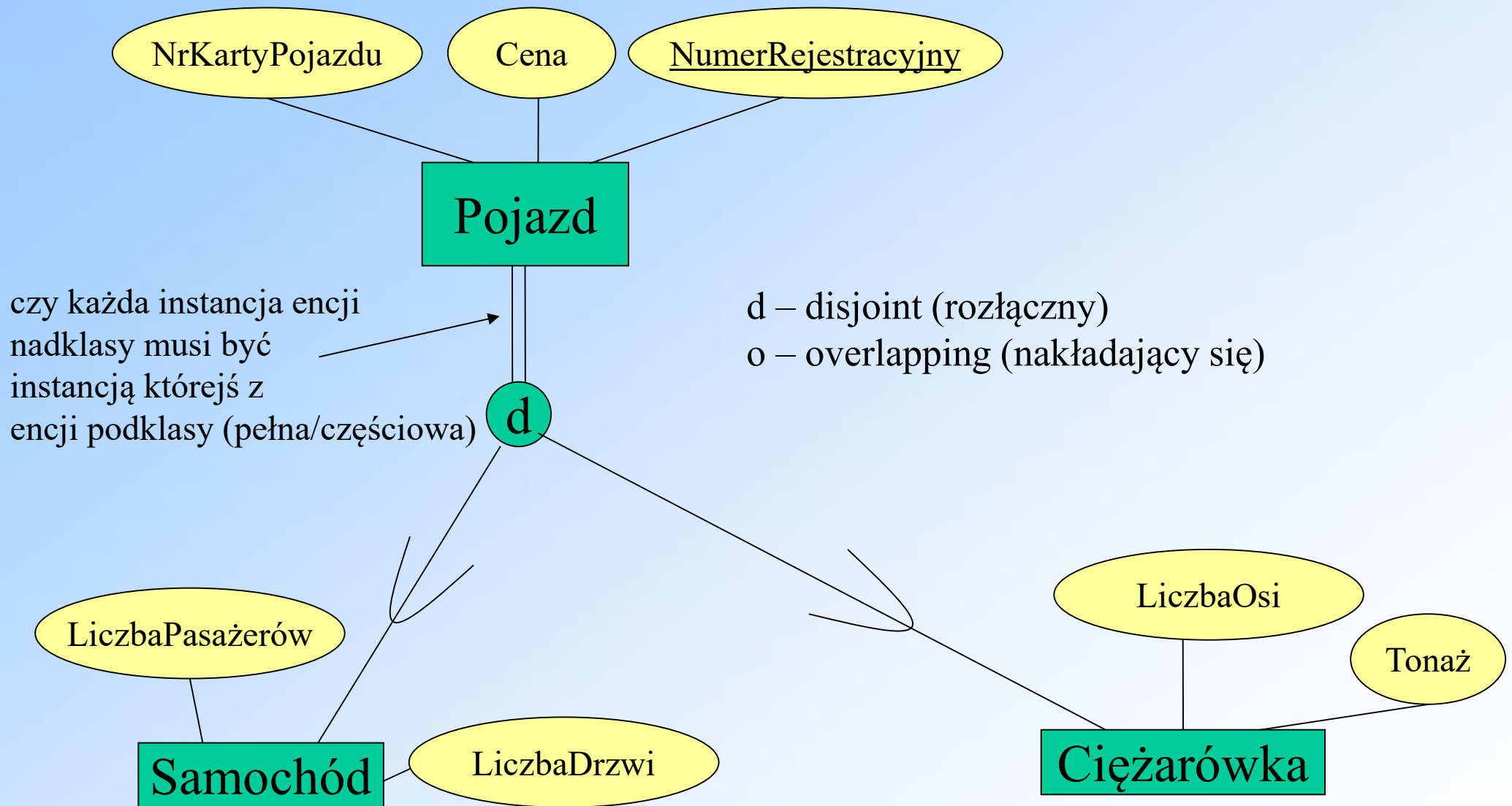
□ *Generalizacja*

- definiowanie ogólnej encji nadklasy na podstawie istniejącego zbioru encji, które stają się encjami podklas

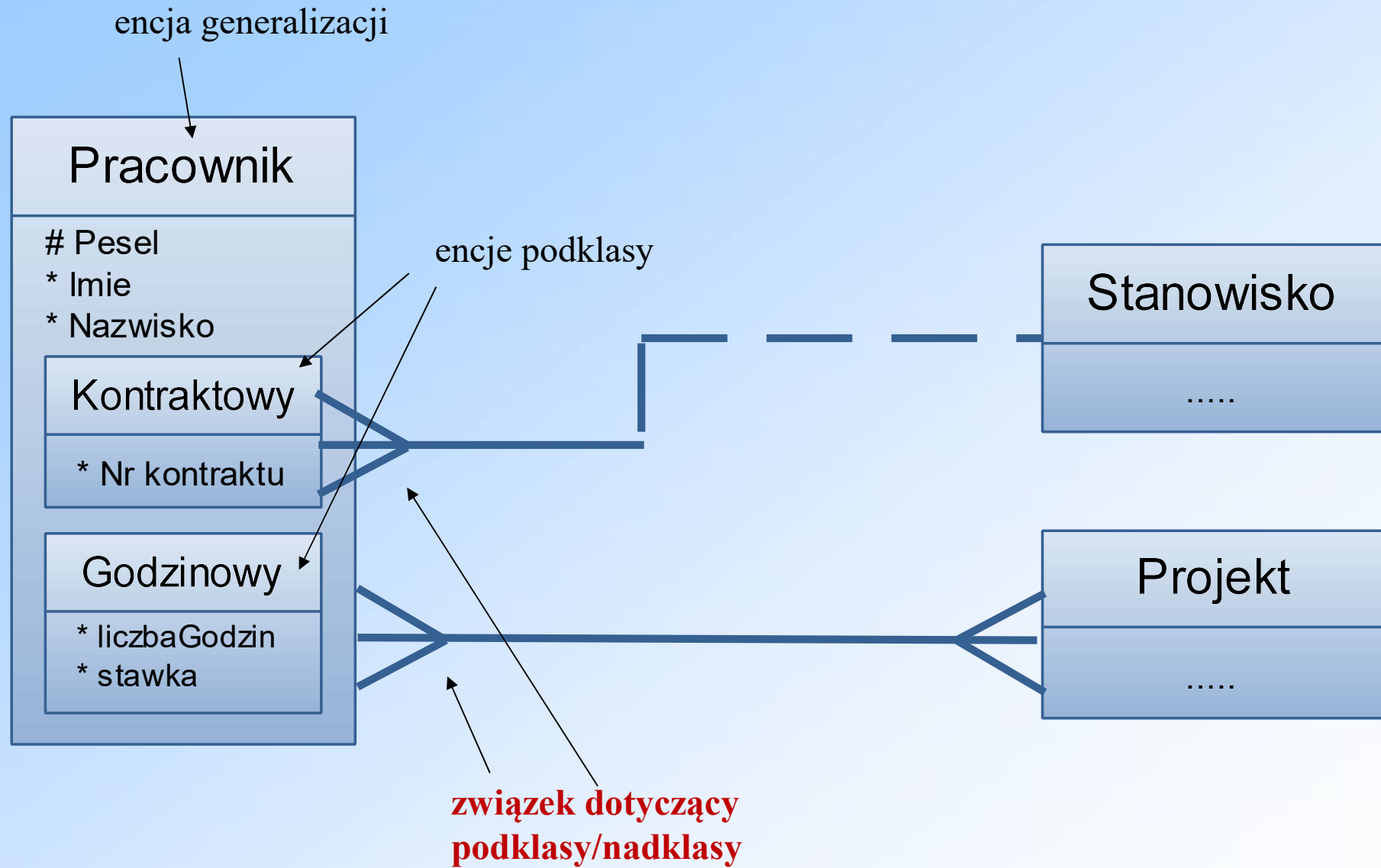
□ *Cechy specjalizacji / generalizacji*

- specjalizacja może być rozłączna lub nakładająca się
- specjalizacja może być pełna lub częściowa

Hierarchia encji

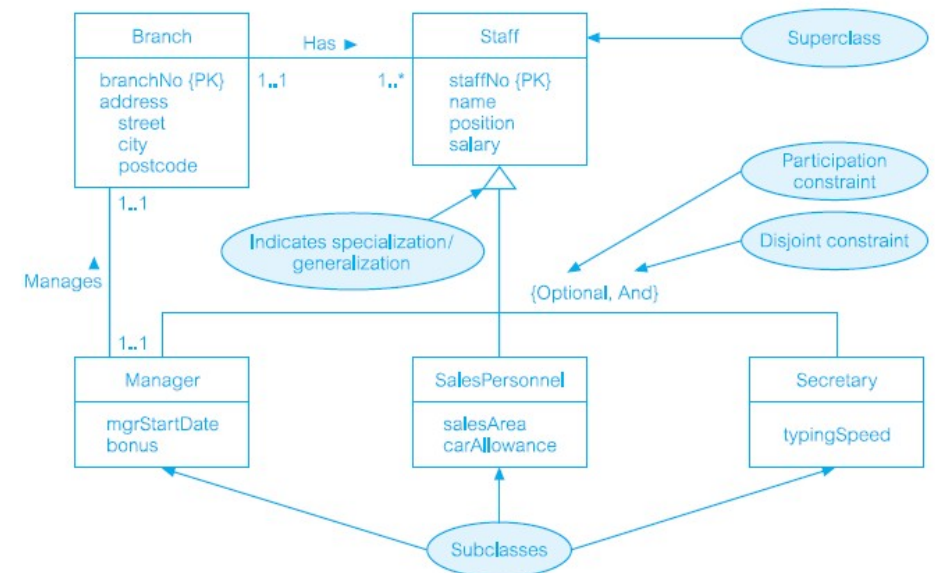


Hierarchia encji



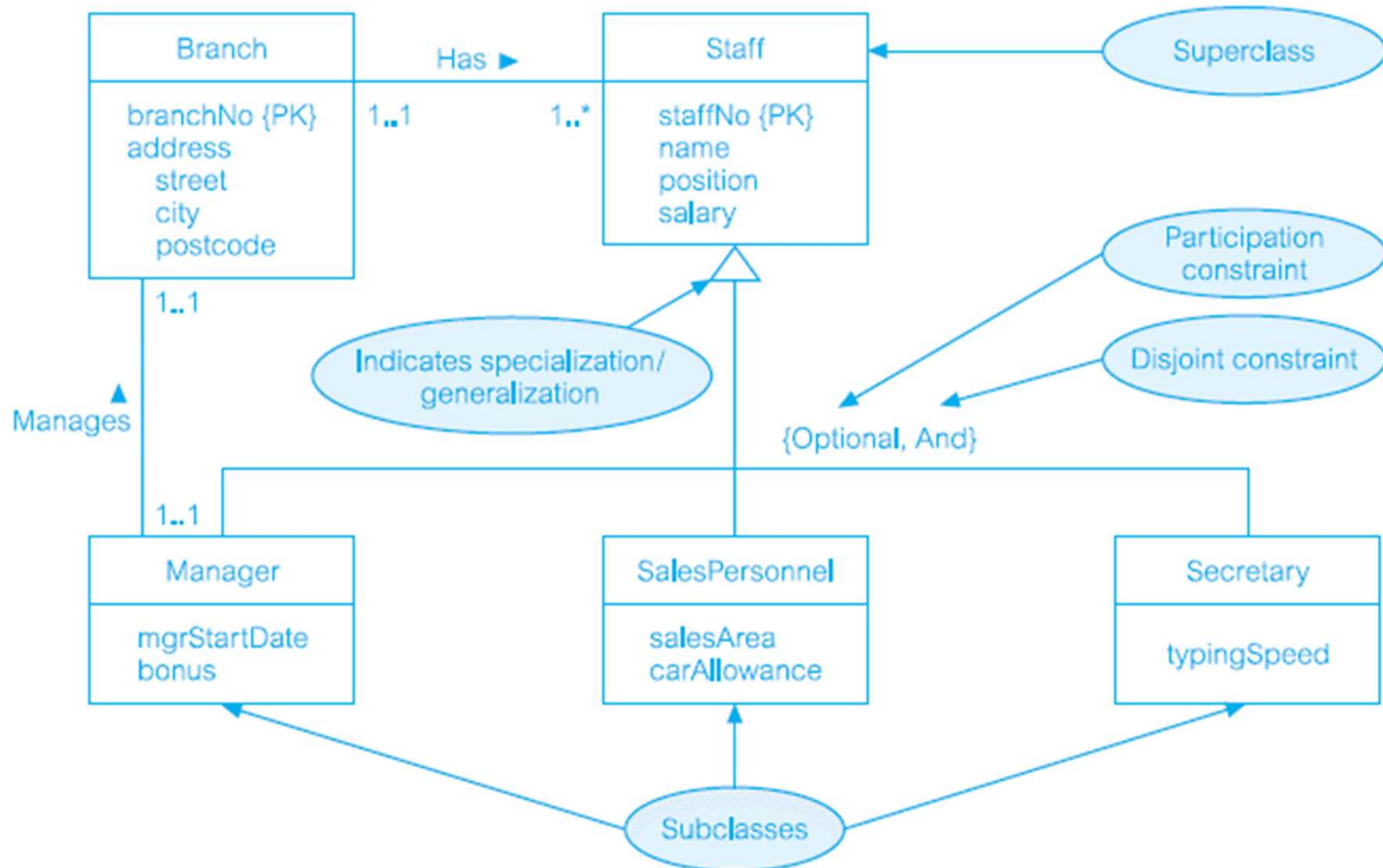
Hierarchia encji – nadklasy/podklasy - UML

Attributes appropriate for all staff				Attributes appropriate for branch Managers		Attributes appropriate for Sales Personnel		Attribute appropriate for Secretarial staff
staffNo	name	position	salary	mgrStartDate	bonus	sales Area	car Allowance	typing Speed
SL21	John White	Manager	30000	01/02/95	2000	SA1A	5000	100
SG37	Ann Beech	Assistant	12000					
SG66	Mary Martinez	Sales Manager	27000					
SA9	Mary Howe	Assistant	9000					
SL89	Stuart Stern	Secretary	8500	01/06/91	2350	SA2B	3700	100
SL31	Robert Chin	Snr Sales Asst	17000					
SG5	Susan Brand	Manager	24000					

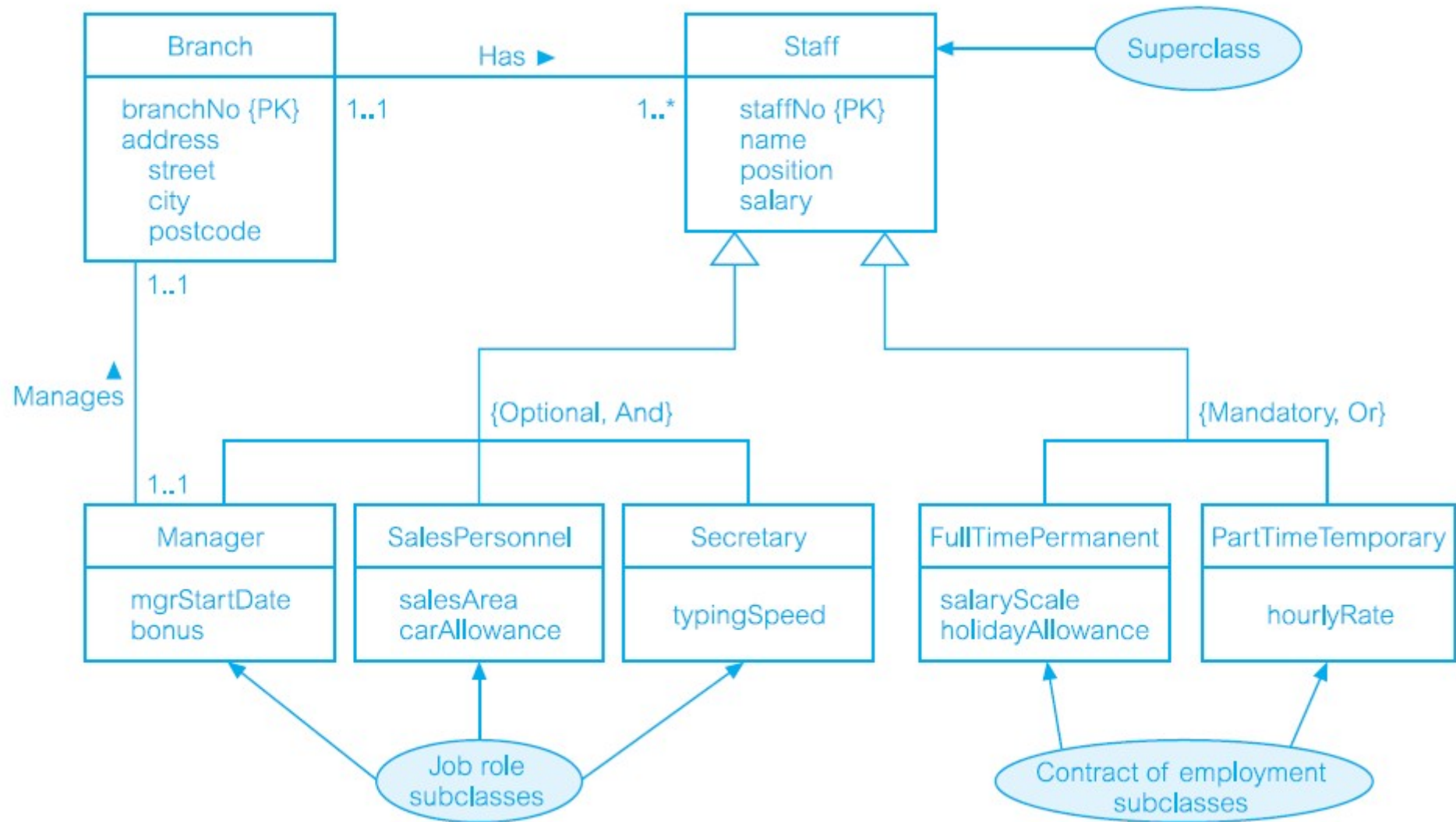


Hierarchia encji – nadklasy/podklasy - UML

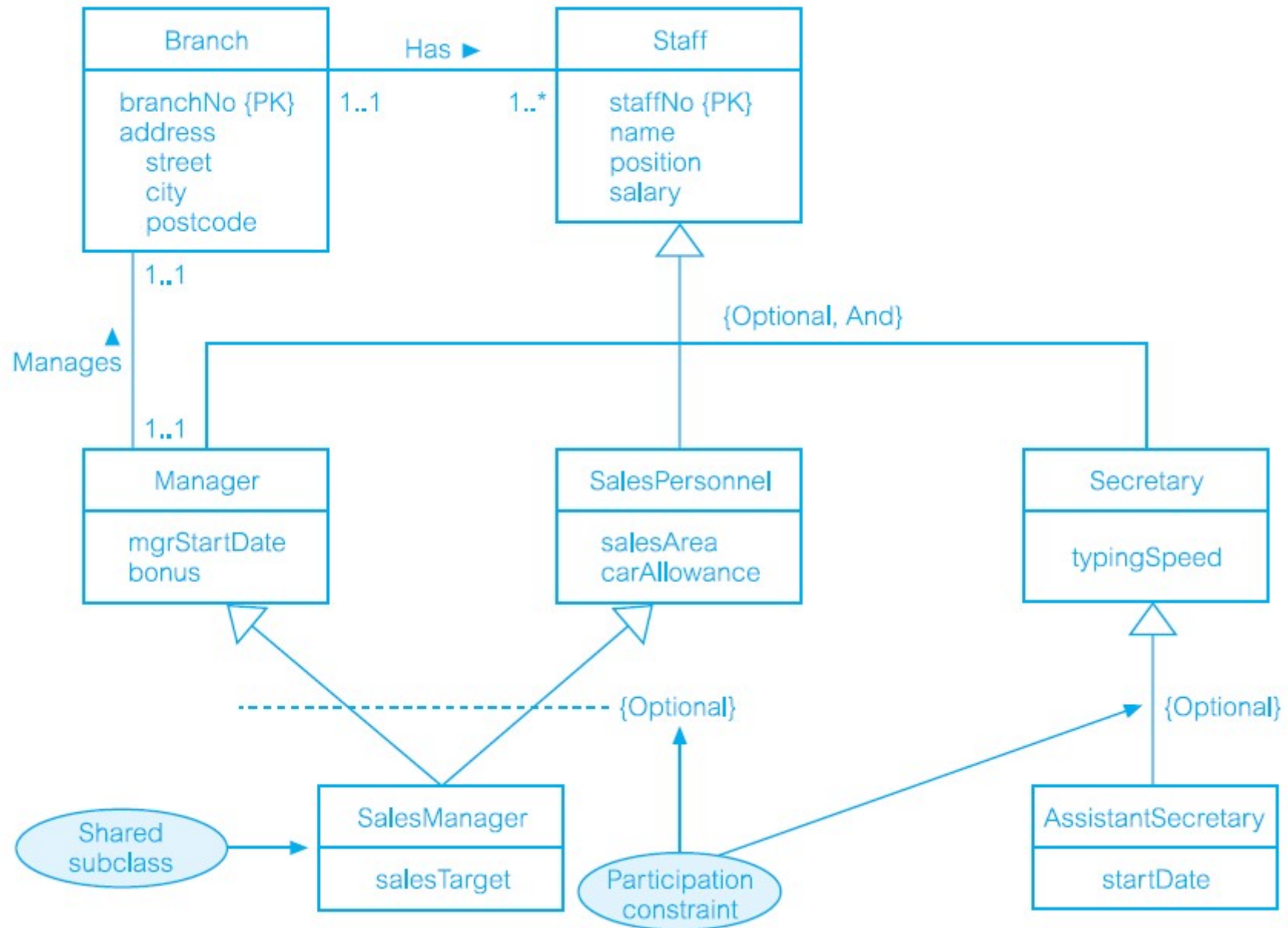
Attributes appropriate for all staff				Attributes appropriate for branch Managers		Attributes appropriate for Sales Personnel		Attribute appropriate for Secretarial staff
staffNo	name	position	salary	mgrStartDate	bonus	sales Area	car Allowance	typing Speed
SL21	John White	Manager	30000	01/02/95	2000	SA1A	5000	100
SG37	Ann Beech	Assistant	12000					
SG66	Mary Martinez	Sales Manager	27000					
SA9	Mary Howe	Assistant	9000					
SL89	Stuart Stern	Secretary	8500	01/06/91	2350	SA2B	3700	100
SL31	Robert Chin	Snr Sales Asst	17000					
SG5	Susan Brand	Manager	24000					



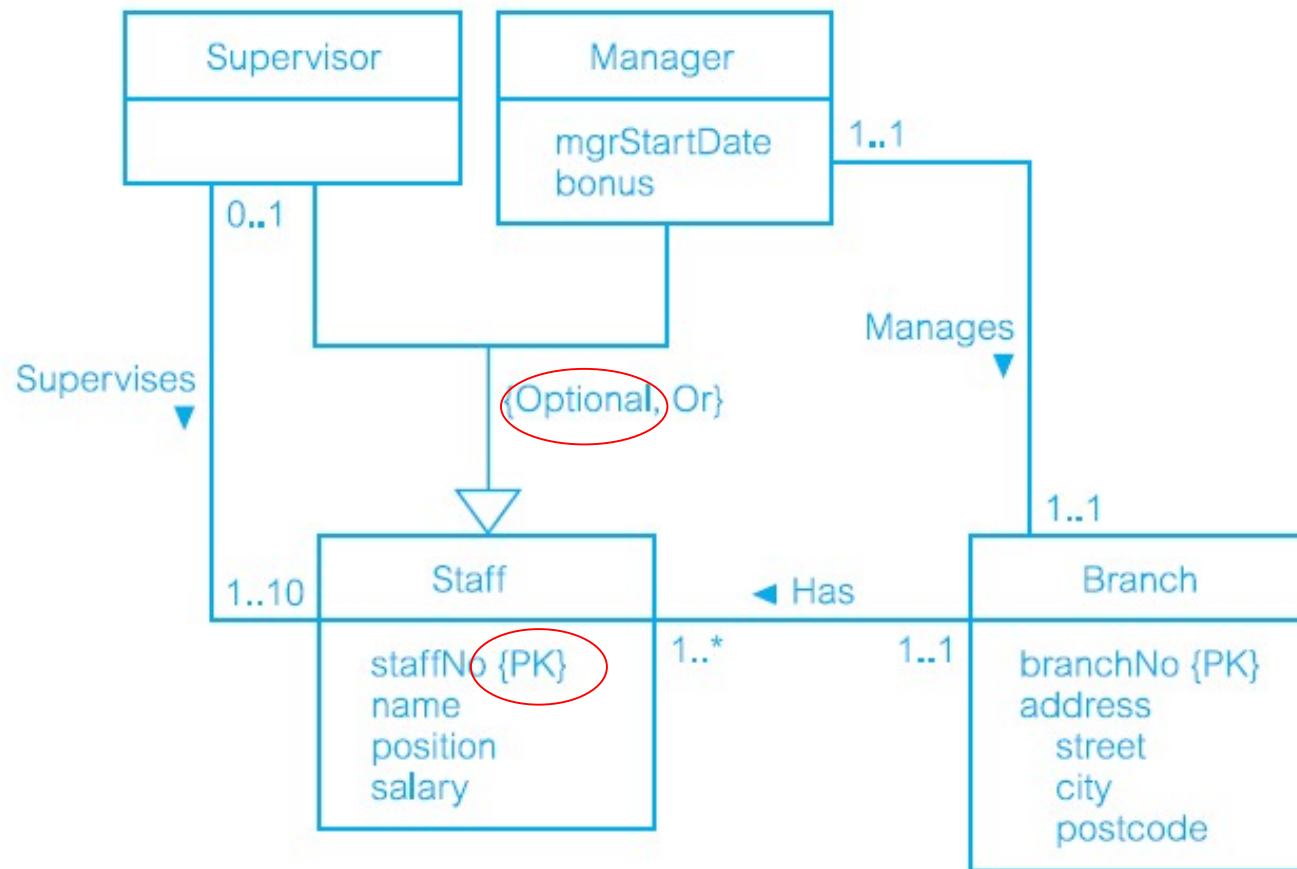
Hierarchia encji – nadklasy/podklasy – UML (przykł. 2)



Hierarchia encji – nadklasy/podklasy – UML (przykł. 3)



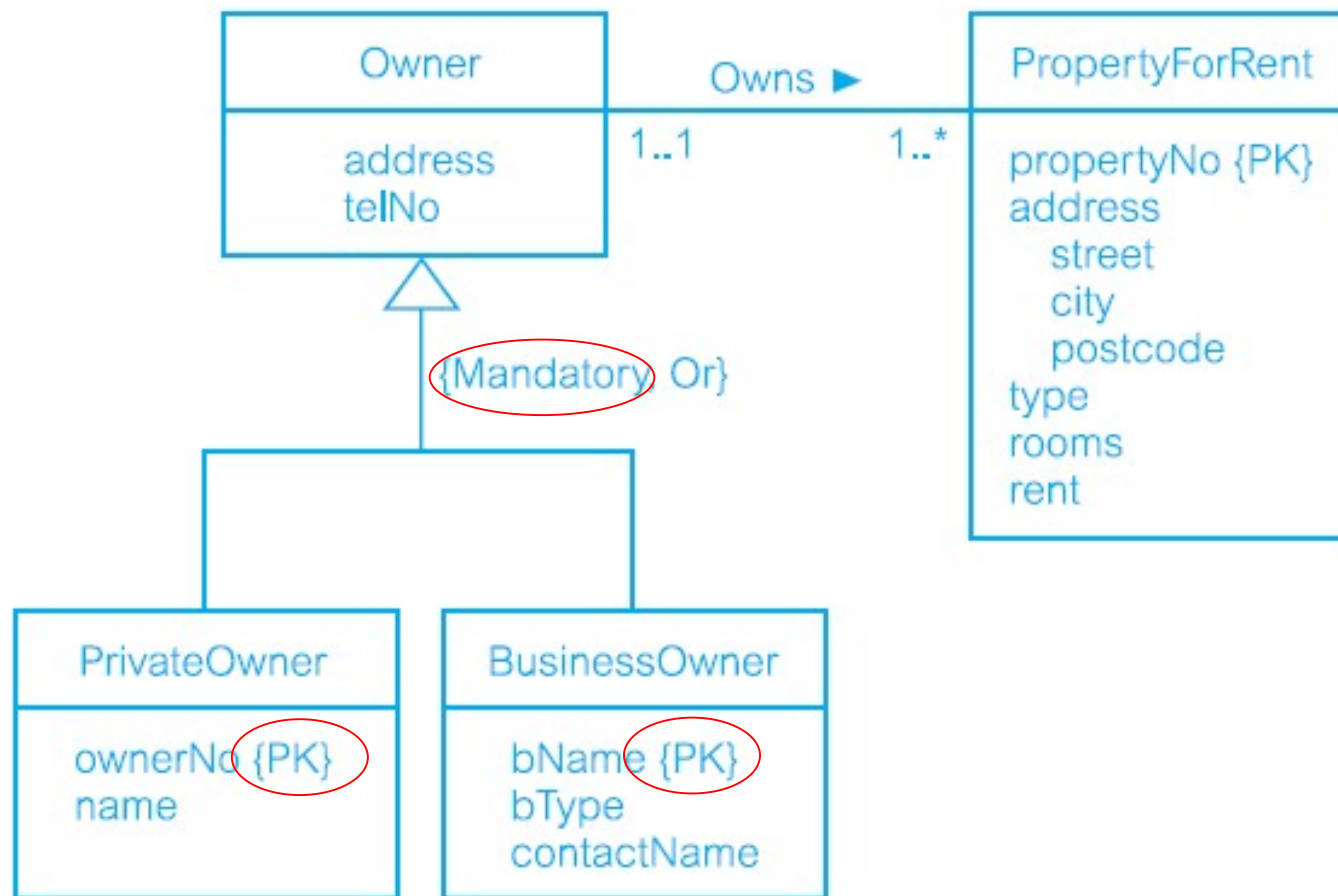
Hierarchia encji – specjalizacja/generalizacja – UML (p. 4)



3 sposoby reprezentacji kluczowych stanowisk:

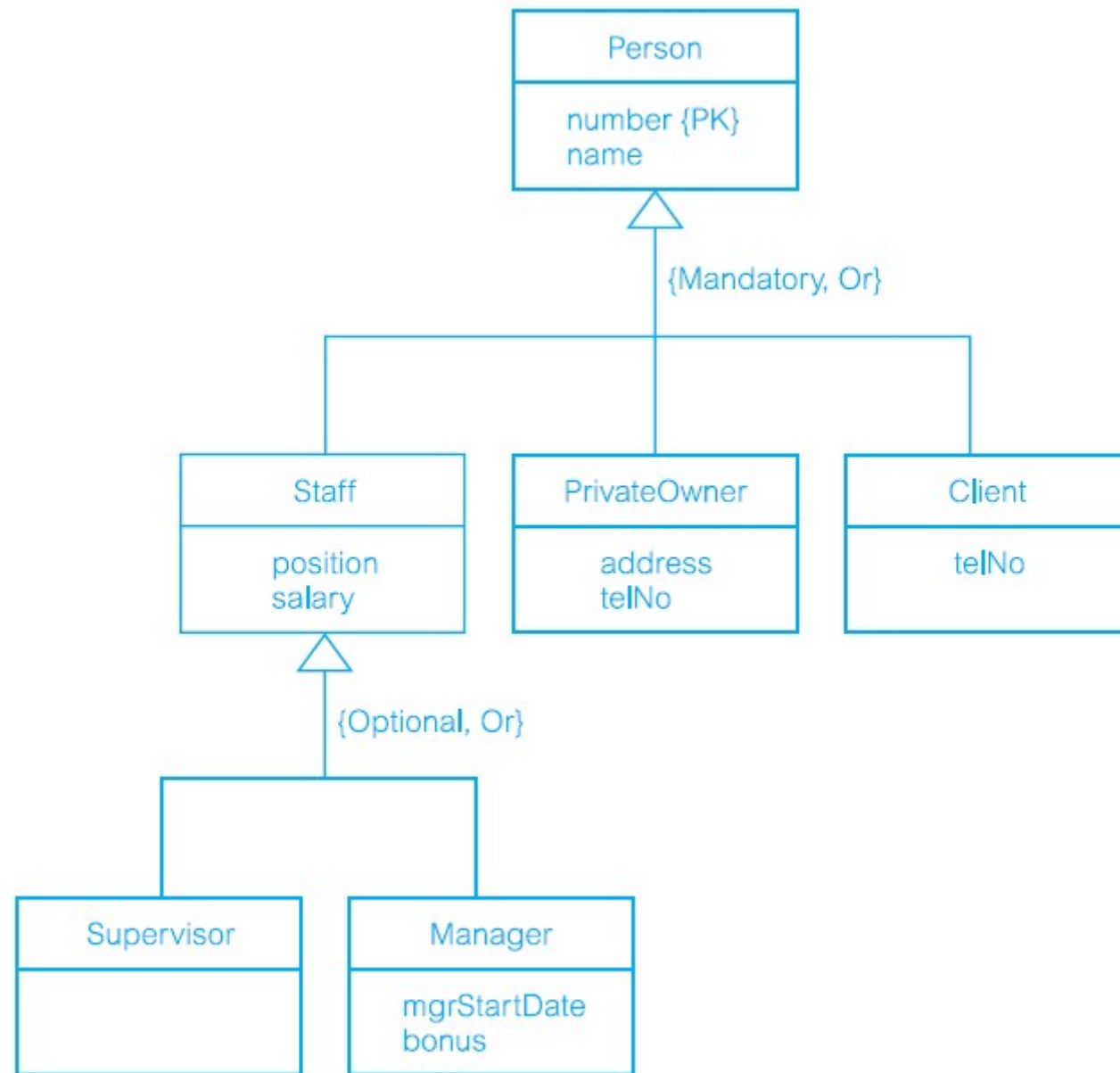
- Wszyscy pracownicy w jednej, ogólnej encji
- 3 osobne encje dla pracowników, menedżerów, nadzorców
- **Menedżerowie i nadzorcy jako podklasy encji pracownicy**

Hierarchia encji – specjalizacja/generalizacja – UML (p. 5)



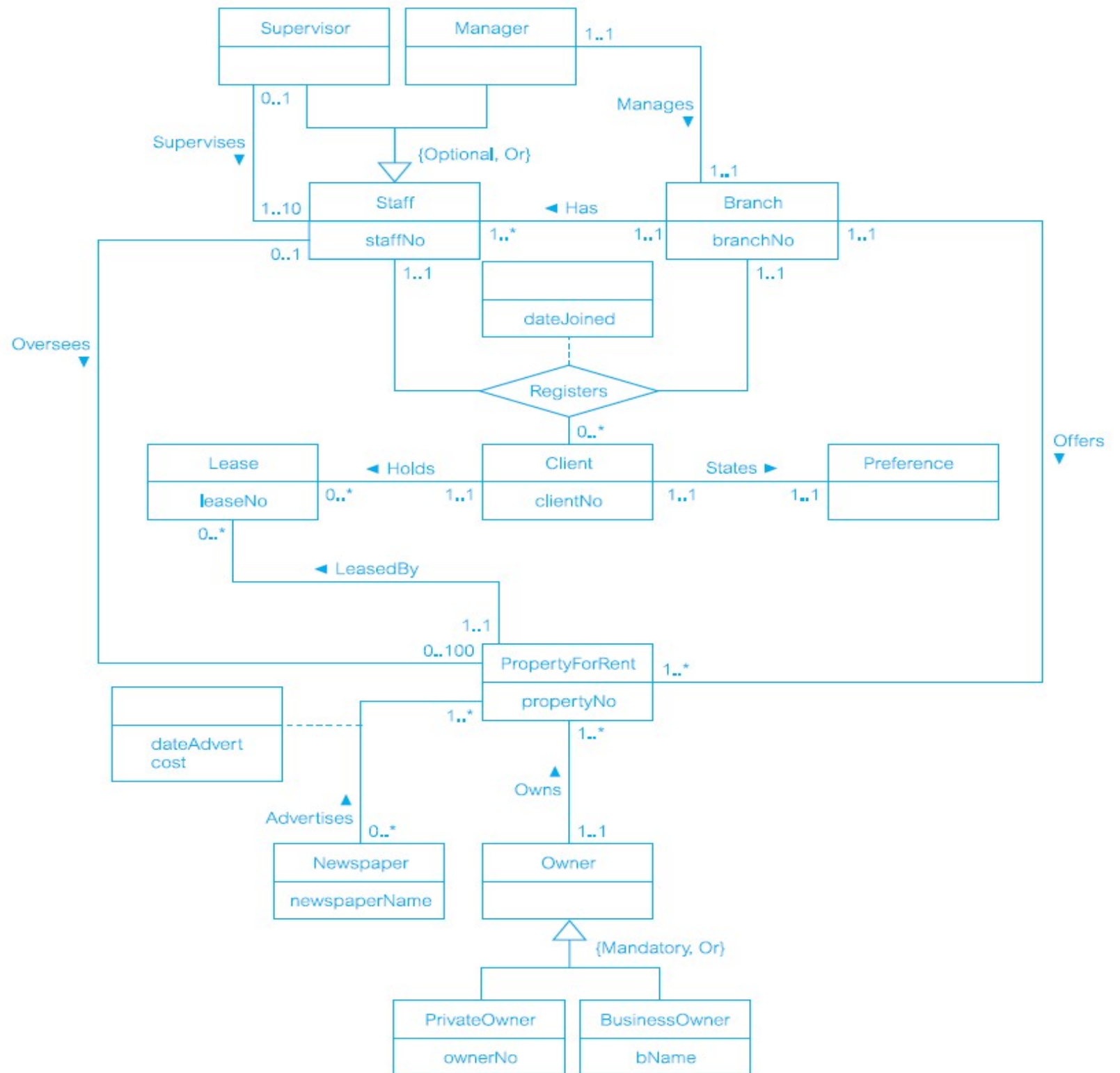
Podobnie jak poprzednio 3 sposoby reprezentacji

Hierarchia encji – specjalizacja/generalizacja – UML (p. 6)



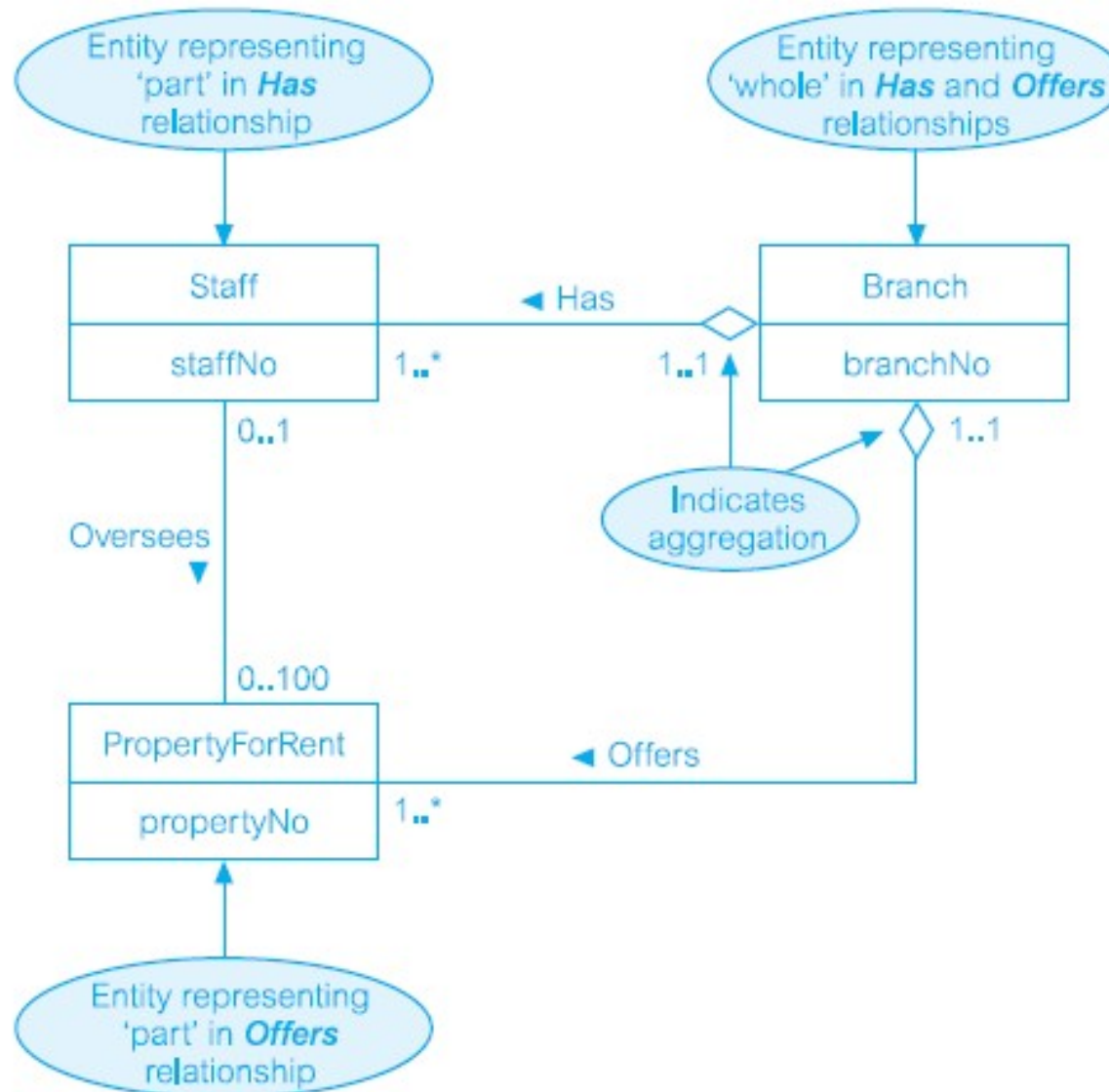
Kolejne poziomy dziedziczenia

Do analyzy ...



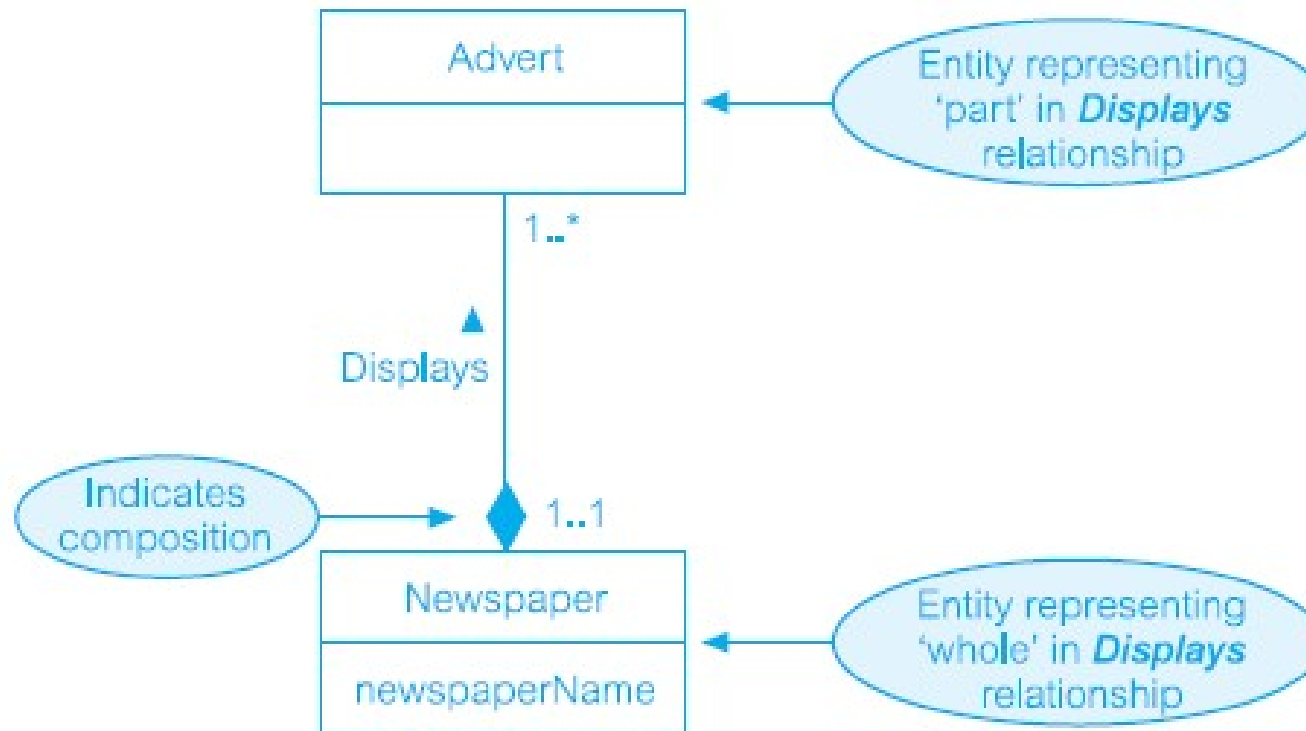
Hierarchia encji – **agregacja** -UML

Reprezentuje związek „**posiada**” lub „**jest częścią**” pomiędzy „**dużą**” encją w skład której wchodzi „**mniejsze**” encje - **elementy**



Hierarchia encji – **kompozycja** -UML

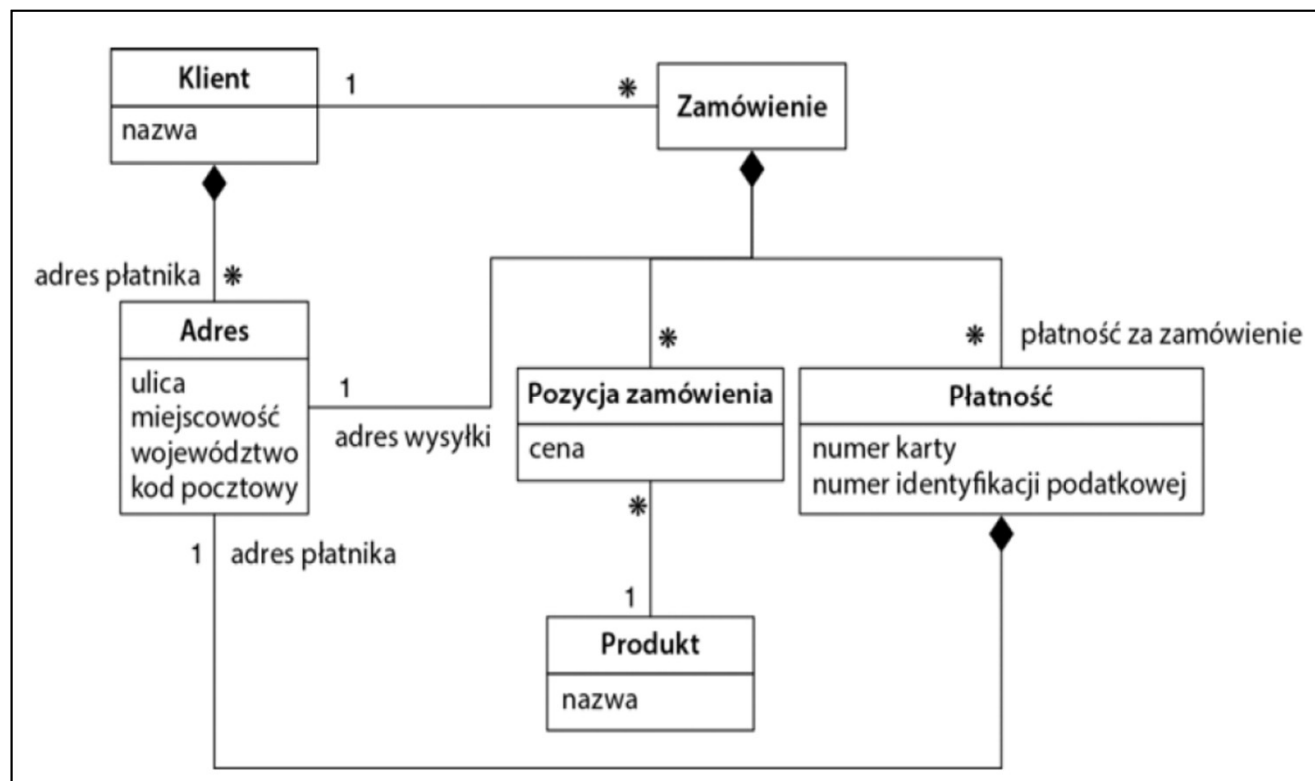
Specjalny przypadek agregacji, w którym składowe posiadane przez „całość” nie mogą bez niej istnieć.



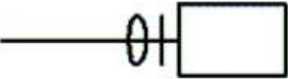
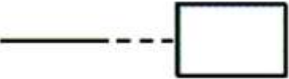
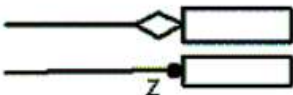
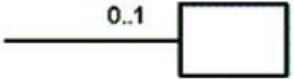
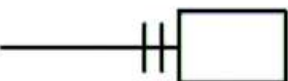
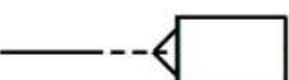
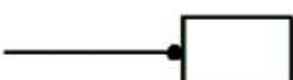
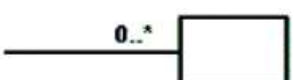
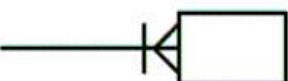
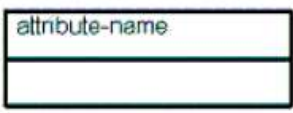
```
// w klientach
```

```
{  
  "klient": {  
    "id": 1,  
    "nazwa": "Marcin",  
    "adresPlatnika": [{"miasto": "Warszawa"}],  
    "zamowienia": [  
      {  
        "id": 99,  
        "klientId": 1,  
        "pozycjeZamowienia": [  
          {  
            "produktId": 27,  
            "cena": 32.45,  
            "produktNazwa": "NoSQL. Kompedium"  
          }  
        ],  
        "adresWyslki": [{"miasto": "Warszawa"}],  
        "zamowieniePlatnosc": [  
          {  
            "numerKarty": "1000-1000-1000-1000",  
            "NIP": "abelif879rft",  
            "adresPlatnika": {"city": "Warszawa"}  
          }  
        ],  
      }  
    ],  
  },  
}
```

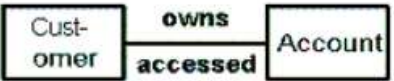
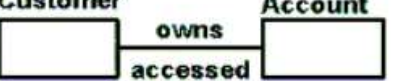
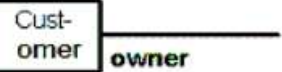
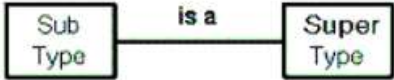
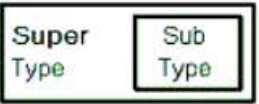
Hierarchia encji - przykład zastosowania – kompozycja –UML(uproszczony)



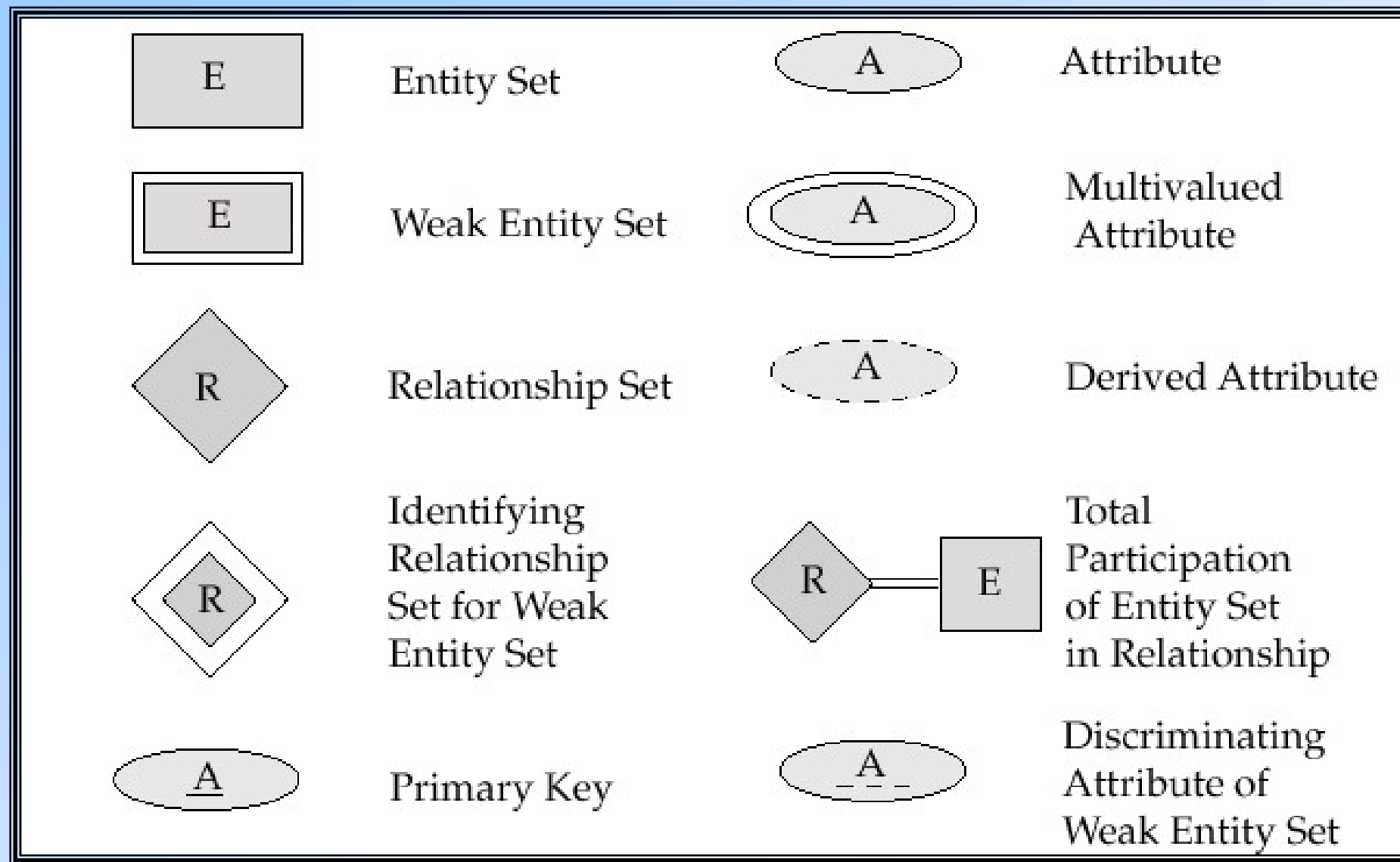
Porównanie notacji - podsumowanie

Notation	Information Engineering	Barker Notation	IDEF1X	UML
<u>Multiplicities:</u>				
- Zero or one				
- One only				
- Zero or more				
- One or more				
- Specific range	N/A	N/A	N/A	
<u>Attributes:</u>				
Names	N/A	Attribute Name: Type	attribute-name: Type	attributeName: Type
Primary key/unique identifier	N/A	# Attribute Name		attributeName <<PK>> {order=#}

Porównanie notacji – podsumowanie (2)

Notation	Information Engineering	Barker Notation	IDEF1X	UML
Associations:				
Labels				
Entity roles	N/A	N/A	N/A	
Subtyping				

Notacja Chen-a



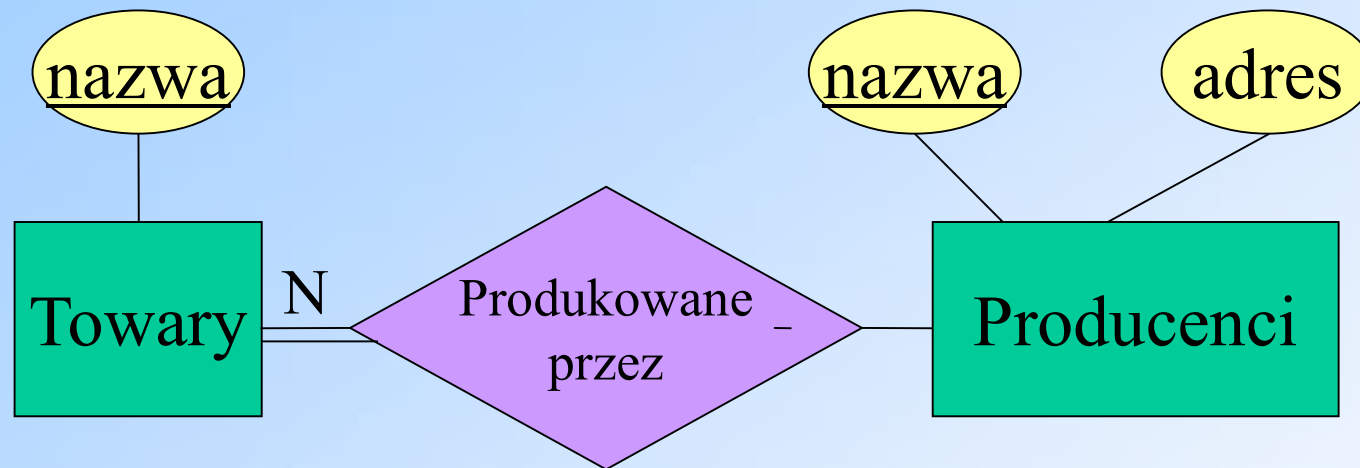
Technika projektowania

1. Unikanie nadmiarowości.
2. Unikanie używania słabych zbiorów encji.
3. Unikanie niepotrzebnego tworzenia zbiorów encji zamiast dodawania atrybutów.

Unikanie nadmiarowości

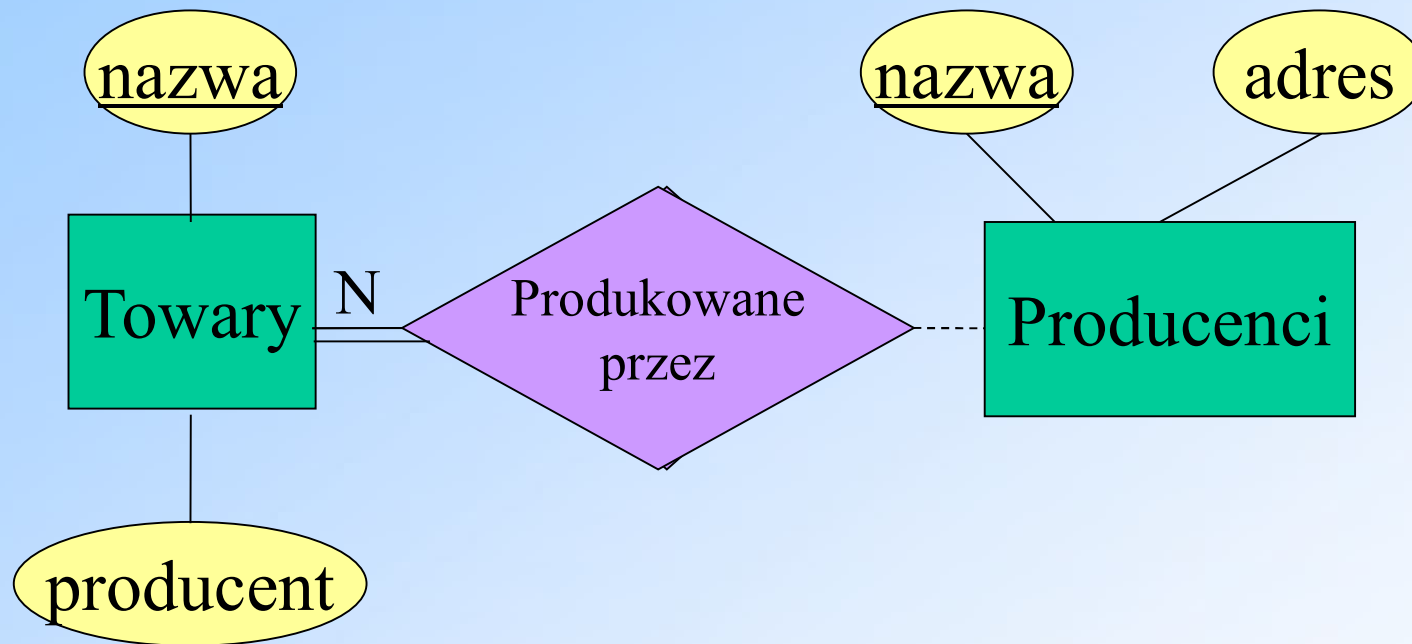
- *Nadmiarowość* występuje wtedy, gdy ta sama rzecz opisywana jest na różne sposoby.
- Redundancja powoduje marnowanie przestrzeni, a co gorsza grozi niespójnością bazy danych.
 - Dwa wystąpienia tego samego faktu mogą zostać różnie zaktualizowane, co doprowadzi do niespójności bazy danych .

Przykład: dobrze



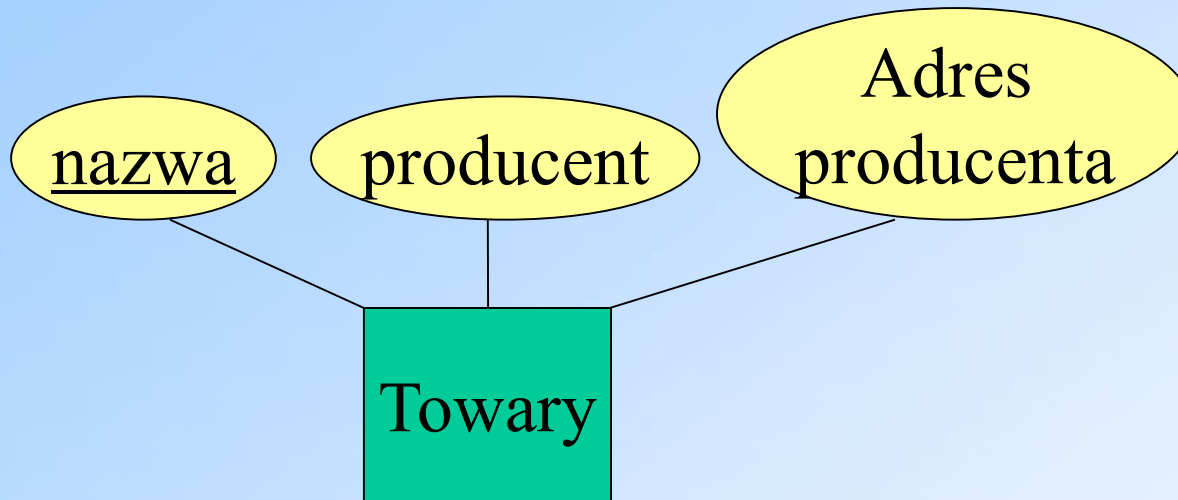
W tym projekcie adres producenta zapamiętywany jest dokładnie raz.

Przykład: źle



W tym projekcie producent towaru zapamiętywany jest 2 razy

Przykład: źle



W tym projekcie adres producenta jest powtarzany dla każdego z towarów, i całkowicie tracony, jeśli chwilowo nie ma żadnego towaru danego producenta.

Zbiory encji a atrybuty

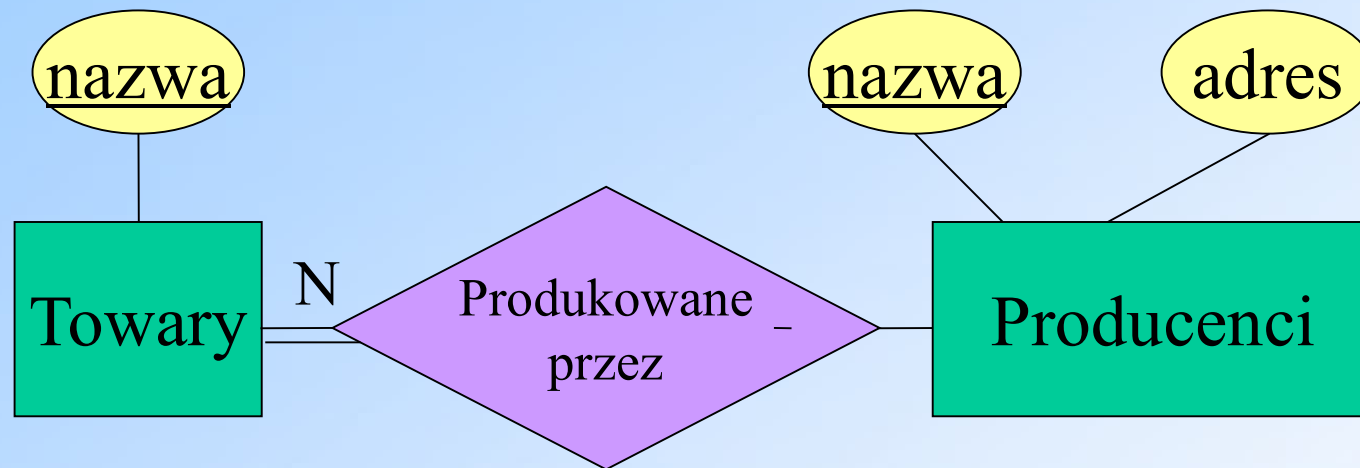
- Zbiór encji powinien spełniać przynajmniej jeden z poniższych warunków:

- Powinien zawierać co najmniej jeden atrybut nie należący do klucza.

lub

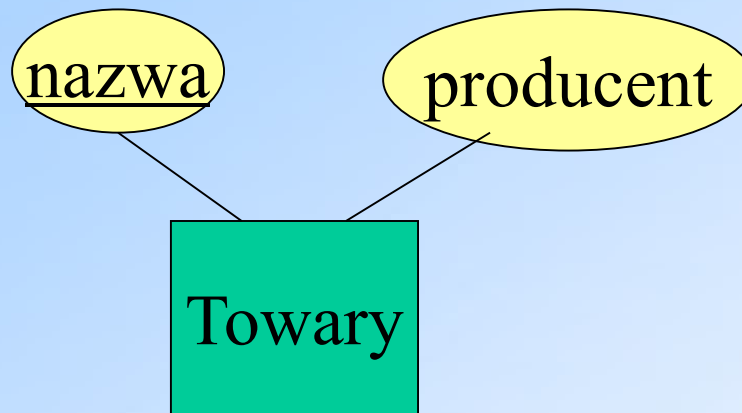
- Jest stroną „wiele” w związku wiele-do-wielu lub jeden-do-wielu.

Przykład: dobrze



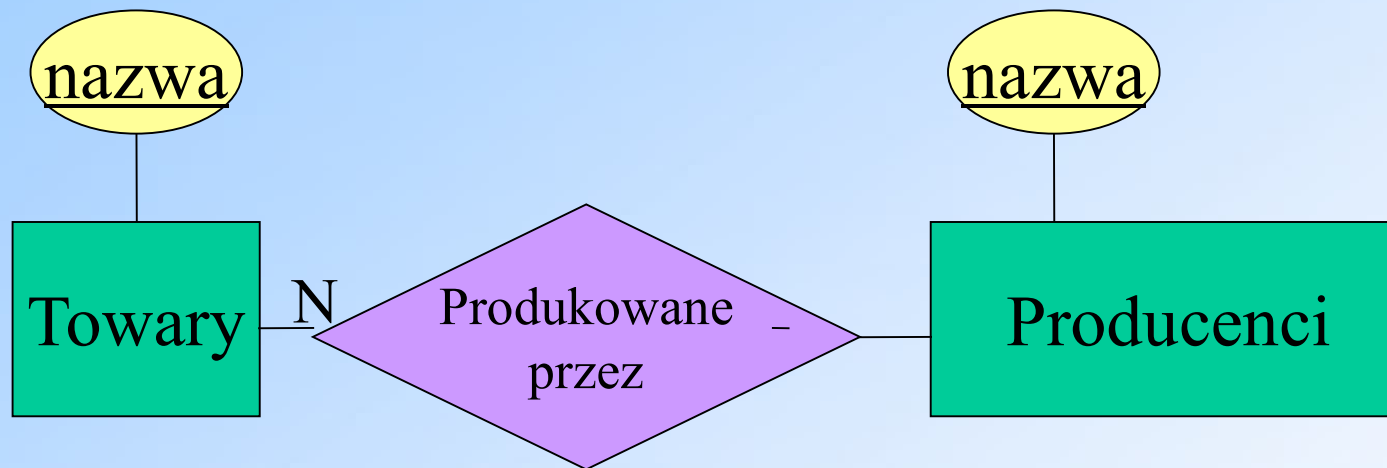
- **Producenci** są prawidłowo zbiorem encji, ponieważ posiadają atrybut niekluczowy **adres**.
- **Towary** są zbiorem encji, ponieważ są stroną „wiele” w związku **produkowane_przez**.

Przykład: dobrze



Nie ma potrzeby aby producent stał się zbiorem encji, ponieważ nie jest zapamiętywane nic oprócz samej nazwy producenta.

Przykład: źle



W zbiorze encji producenci zawarty jest tylko atrybut kluczowy, nie jest on stroną wiele w związku, więc nie ma potrzeby aby był on zbiorem encji.

Narzędzia CASE

- Narzędzia CASE pozwalają na tworzenie diagramów związków encji, które są następnie automatycznie przekształcane w model relacyjny:
- Umożliwiają synchronizację bazy danych z modelem E-R
- Przykłady:
 - Oracle Designer,
 - DBDesigner 4, MySQL Workbench
 - TOAD Data Modeler
 - Visual Paradigm

Notacja Martina



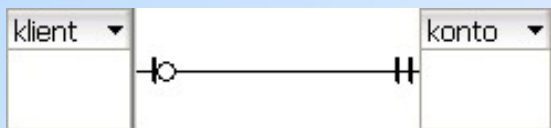
Klient musi posiadać jedno lub wiele kont
Konto musi należeć do jednego i tylko jednego klienta



Klient może posiadać jedno lub wiele kont
Konto musi należeć do jednego i tylko jednego klienta



Klient może posiadać jedno lub wiele kont
Konto może należeć do jednego klienta



Klient musi posiadać jedno konto
Konto może należeć do jednego klienta

Diagram uzyskany narzędziem DBDesigner 4

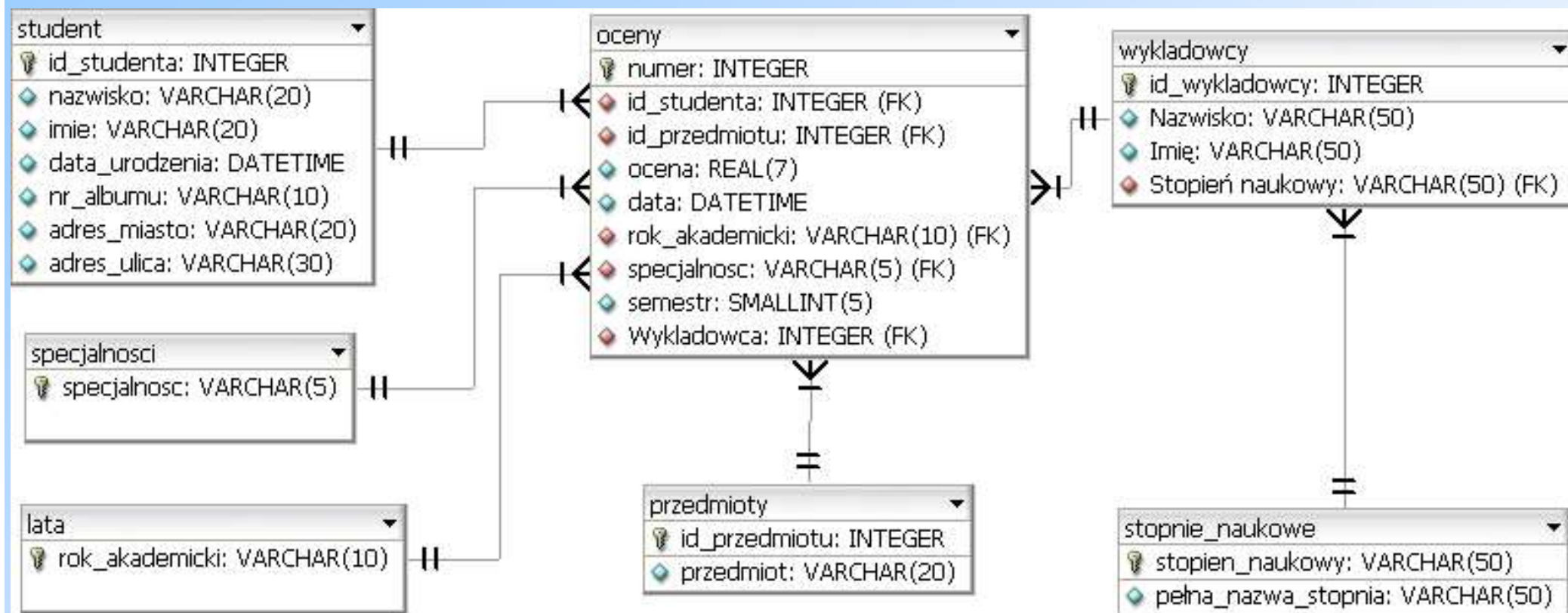
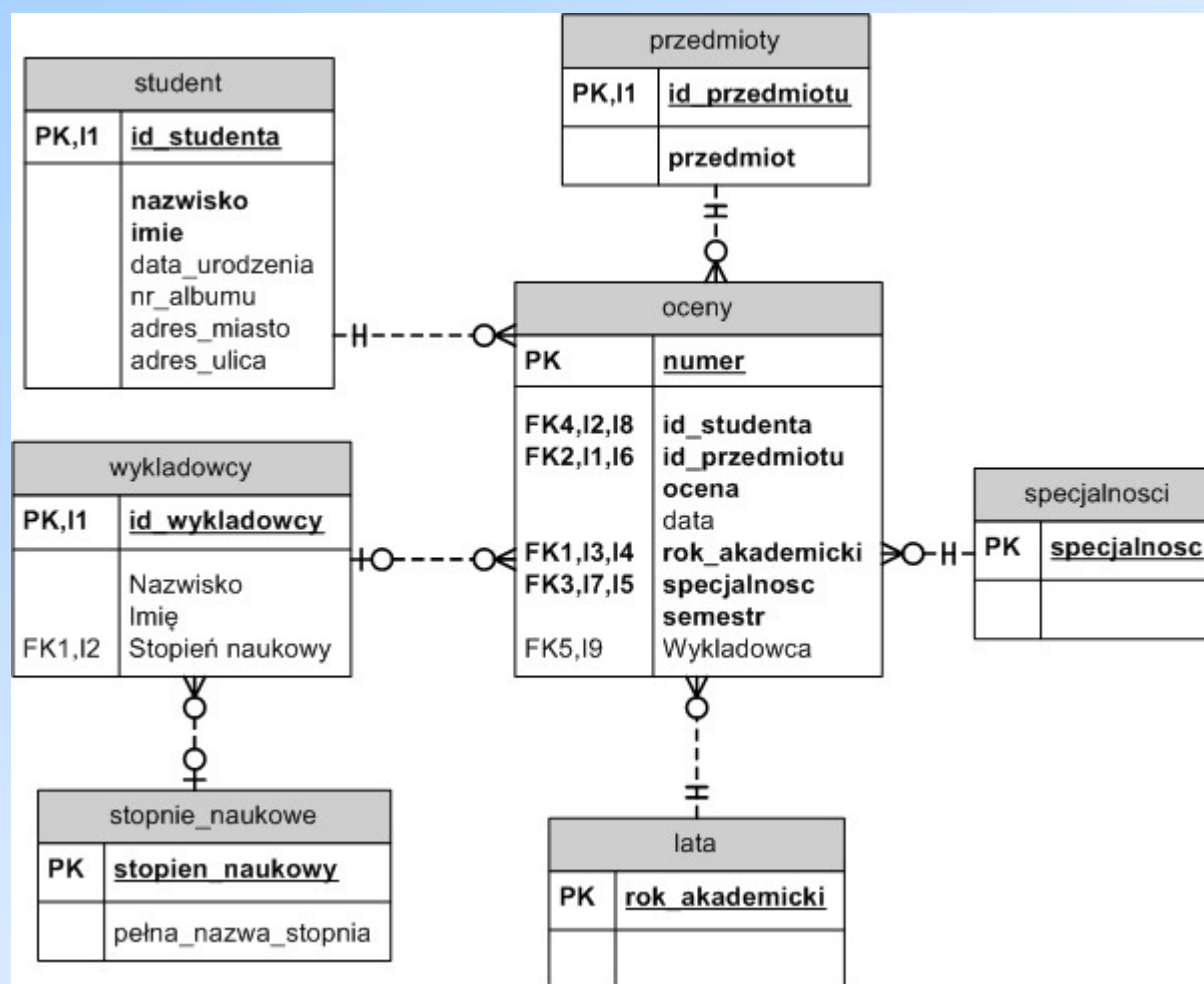
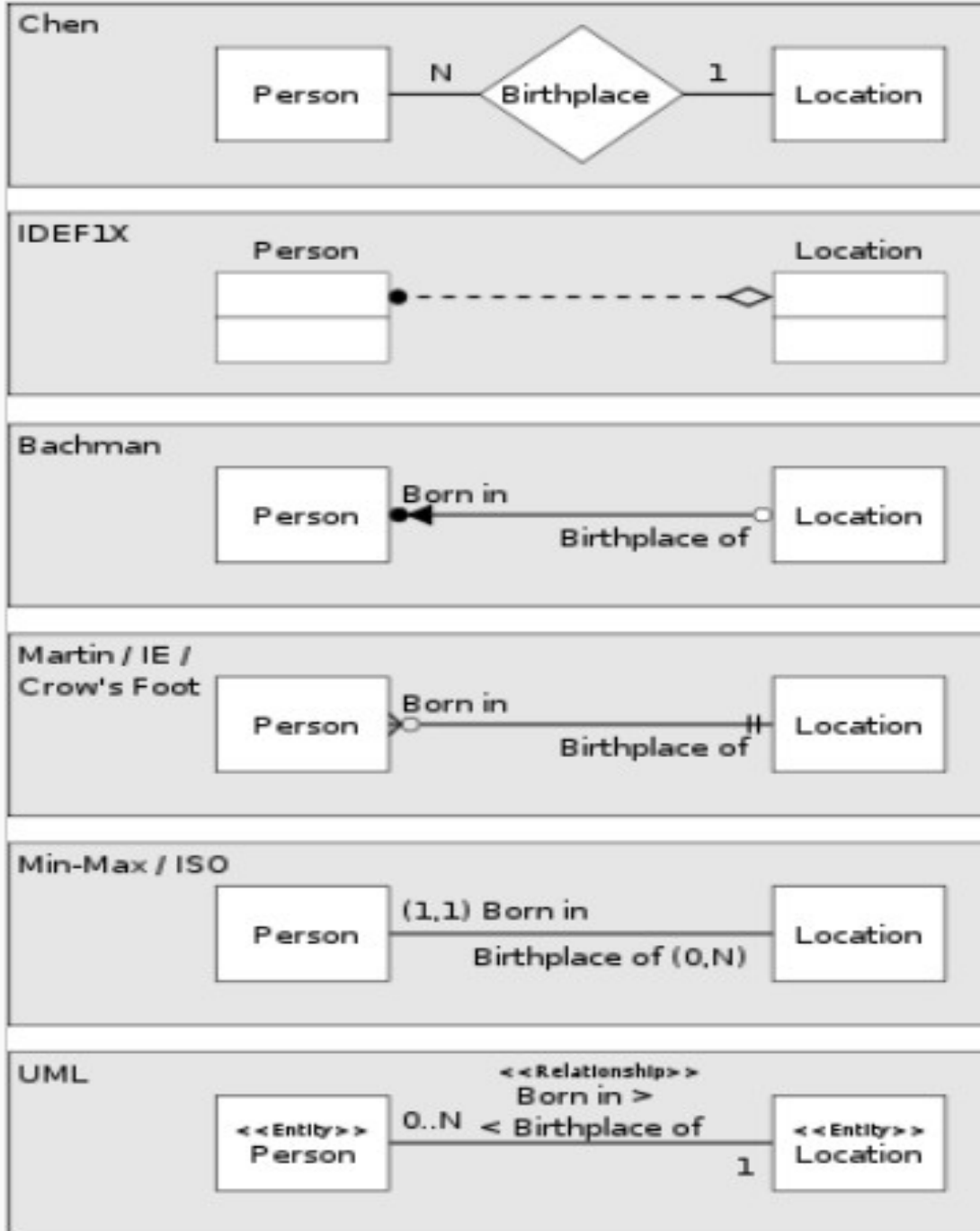


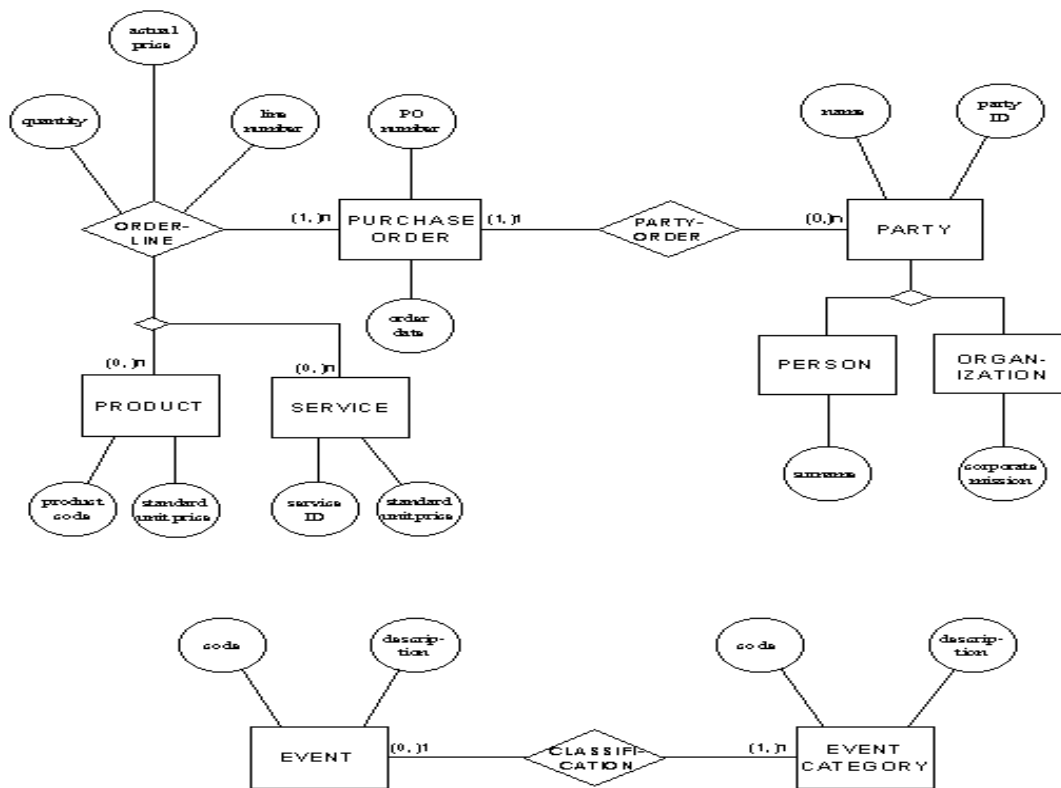
Diagram uzyskany programem MS VISIO



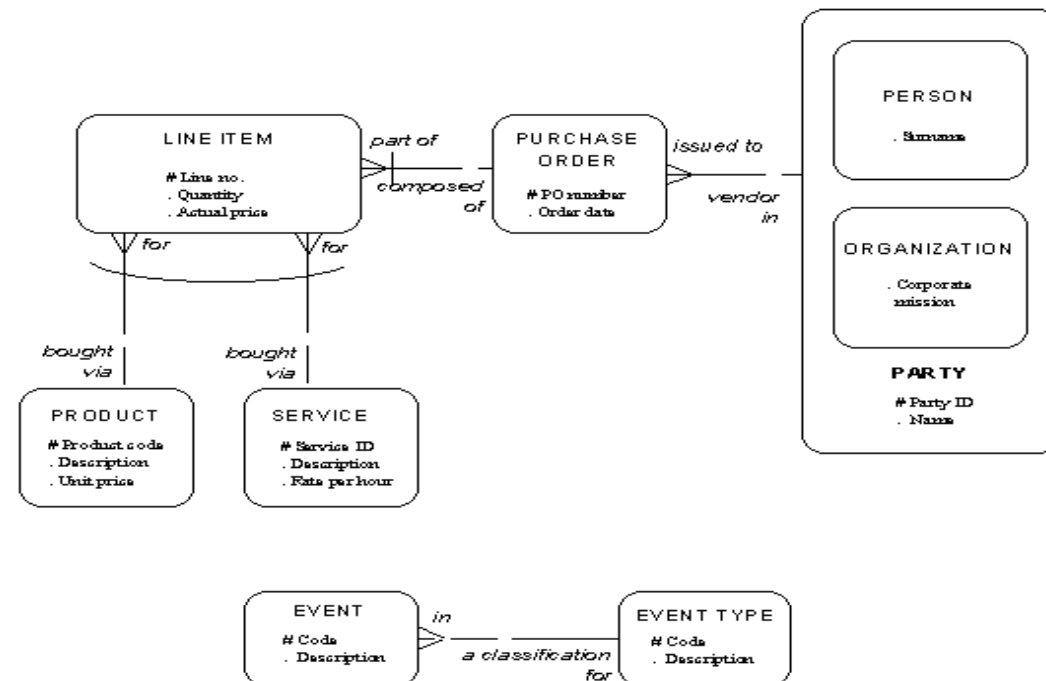
Porównanie notacji



Notacja Chena

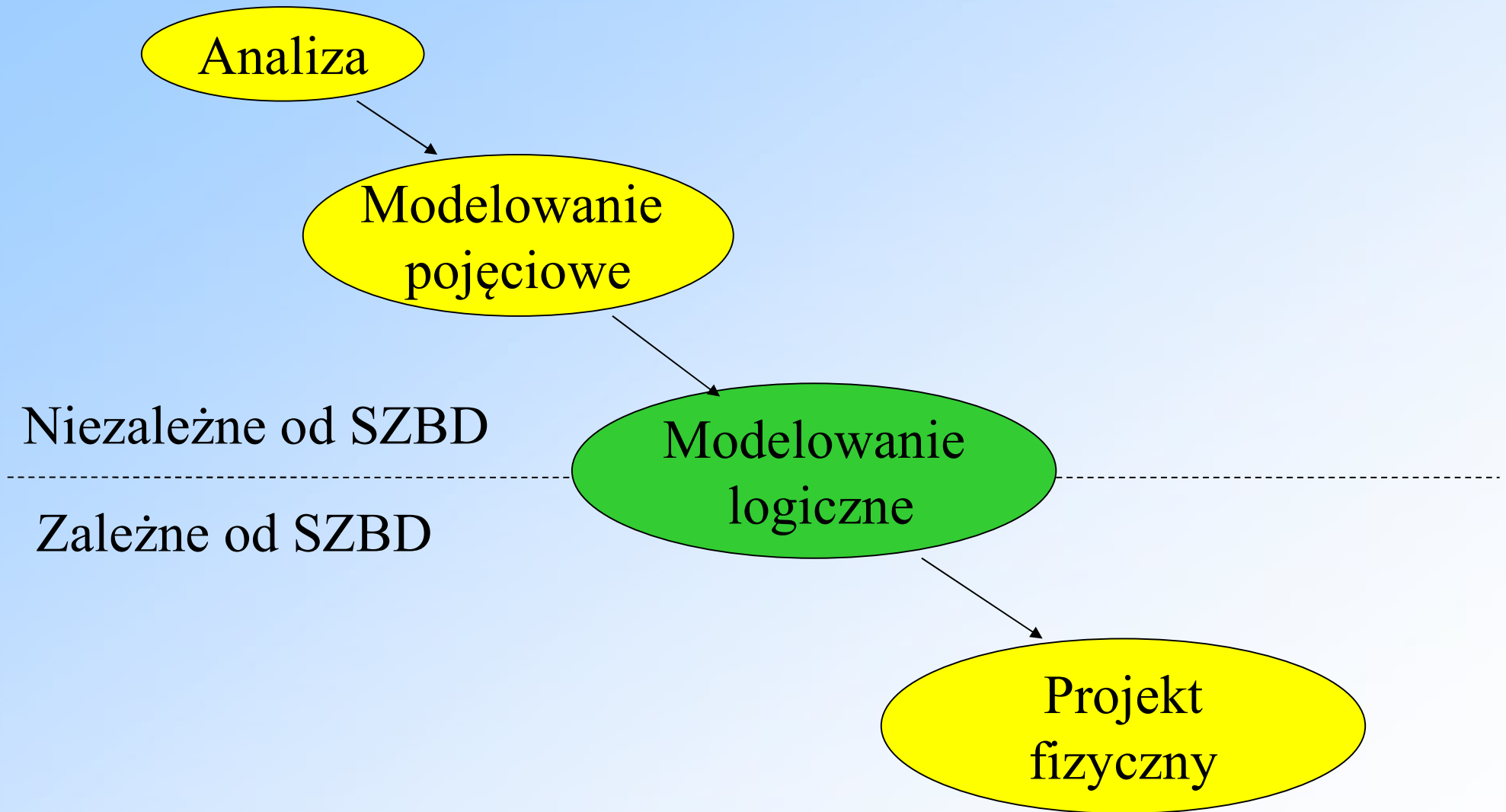


Notacja Barkera



Relacyjny model danych

Etapy projektowania bazodanowego systemu informatycznego



Model danych

Definiuje 3 podstawowe elementy:

- **Struktury danych** – w jaki sposób obiekty świata rzeczywistego są reprezentowane w bazie danych
- **Operacje** (operatory modelu danych)
- **Ograniczenie integralnościowe**

Zalety modelu relacyjnego

- Prostota.
- Systemy zarządzania bazami danych najczęściej używają tego modelu do reprezentacji projektu bazodanowego.
- Model ten jest odpowiedni do współdziałania z językiem zapytań SQL (Structured Query Language).

Relacyjny model danych

□ *Model danych definiuje*

- struktury danych.
- operacje
- ograniczenia integralnościowe

□ *Model relacyjny*

- relacje
- selekcja, projekcja, połączenie, operacje na zbiorach
- klucz podstawowy, klucz obcy, zawężenie dziedziny, unikalność, wartość pusta/niepusta

Struktury danych

- Baza danych jest **zbiorem relacji**
- Schemat relacji R , oznaczony przez $R(A_1, A_2, \dots, A_n)$ składa się z **nazwy relacji** R oraz **listy atrybutów** $A_1, A_2, \dots, A_n$
- **Stopień relacji** – liczba atrybutów składających się na schemat relacji R
- Każdy atrybut schematu relacji R posiada domenę oznaczoną jako **$\text{dom}(A_i)$**
- Domena definiuje zbiór wartości atrybutu relacji poprzez podanie typu danych

Struktury danych (2)

- **Relacją r** o schemacie **$R(A_1, A_2, \dots, A_n)$** oznaczoną **$r(R)$** nazywamy **zbiór krotek postaci** $r = \{t_1, t_2, \dots, t_n\}$
- Pojedyncza krotka t jest uporządkowaną listą **n** wartości **$t = \langle v_1, v_2, \dots, v_n \rangle$** gdzie v_i , $1 \leq i \leq n$, jest elementem **$\text{dom}(A_i)$** lub wartością pustą (NULL)

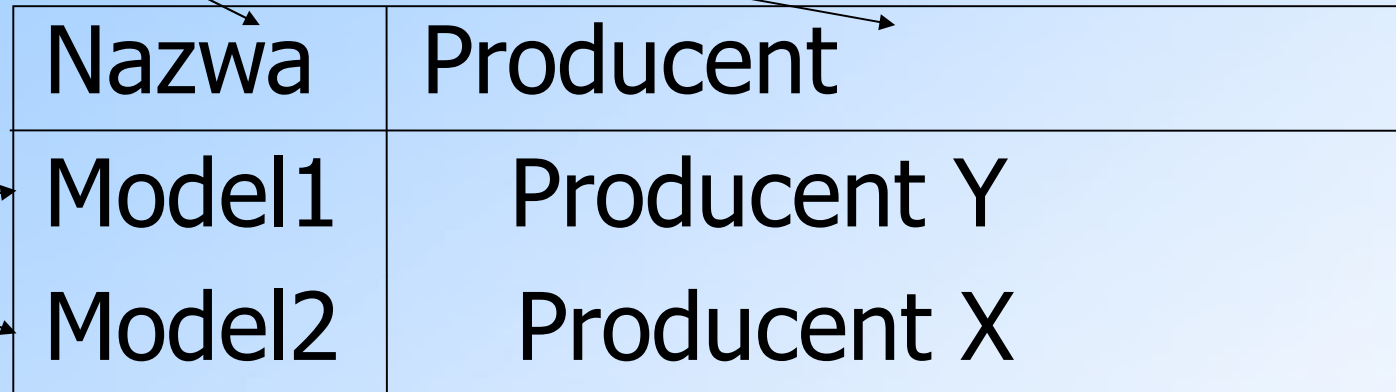
Właściwości relacji

- nie istnieją powtarzające się krotki
- krotki nie są uporządkowane (kolejność nie ma znaczenia)
- porządek atrybutów w relacji nie jest istotny
- wszystkie wartości atrybutów są atomowe
- w ramach bazy danych dziedziny mają jednoznaczne nazwy
- w ramach bazy danych relacje mają jednoznaczne nazwy
- w ramach relacji atrybuty posiadają jednoznaczne nazwy

Relacja

Atrybuty
(nagłówki
kolumn)

Krotki
(wiersze)



Nazwa	Producent
Model1	Producent Y
Model2	Producent X

Samochody

- Liczebność relacji – liczba krotek w tabeli
- Stopień relacji – liczba atrybutów

Schematy

- ❑ *Schemat relacji* = nazwa relacji z listą atrybutów.
 - ❑ Opcjonalnie: typy atrybutów.
 - ❑ Przykład: Samochody(*nazwa*, *producent*) lub Samochody(*nazwa*: string, *producent*: string)
- ❑ *Baza danych* = zbiór relacji.
- ❑ *Schemat bazy danych* = zbiór wszystkich schematów relacji w danym projekcie.

Unikalność krotek relacji - klucze

Ograniczenie na unikalność krotek relacji

- ❑ każdy podzbiór S atrybutów relacji R , taki że dla każdych dwóch krotek ze zbioru $r(R)$ zachodzi $t1[S] \neq t2[S]$ jest **superkluczem** relacji R
- ❑ cały schemat relacji jest superkluczem
- ❑ superklucz może posiadać nadmiarowe atrybuty
- ❑ **kluczem K** schematu relacji R nazywamy taki superklucz, dla którego usunięcie dowolnego atrybutu A z K powoduje, że K' nie jest już superkluczem
- ❑ schemat relacji może posiadać więcej niż jeden klucz

Ograniczenia integralnościowe

Reguły gwarantujące spełnianie przez dane zawarte w relacji nałożonych na nie warunki

- definiowane są na poziomie
 - atrybutu
 - całej relacji
- rodzaje
 - klucz podstawowy (primary key)
 - klucz obcy (foreign key)
 - unikalność (unique)
 - zawężenie domeny (check)
 - dopuszczenie/ niedopuszczenie wartości pustej (null)

Klucz podstawowy

Klucz podstawowy relacji (primary key)

- atrybut, lub zbiór atrybutów , których wartość jednoznacznie identyfikuje krotkę
- wartość ta jest unikalna w obrębie całej relacji i jest niepusta
- wyróżniony klucz – klucz podstawowy
- pozostałe klucze – klucze kandydujące lub wtórne
- Przykłady: pesel, VIN , ...

Klucz obcy

Klucz obcy relacji (foreign key)

- atrybut, lub zbiór atrybutów, który wskazuje na klucz podstawowy najczęściej w innej relacji
- służy do reprezentowania powiązań pomiędzy danymi (łączenia relacji)



Zawężenie dziedziny

Zawężenie dziedziny (ograniczenie domeny) atrybutu

- ograniczenie dozwolonych wartości do pewnego podzbioru przez wyrażenie logiczne określające przedział lub listę wartości
- Przykłady:
 - oceny: 1..6
 - pora_roku: wiosna, lato, jesień, zima

Ograniczenia integralnościowe - przykład

Etat : { 'analitik', 'dyrektor', 'referent', 'kierownik', 'sekretarka' }

Płaca : $\text{placa} > 500$

IdPrac – klucz podstawowy

IdPrac	Nazwisko	Etat	Placa	Szef	IdZesp
120	Kowalski	Analitik	850	100	10
100	Tarzan	Dyrektor	1700		10
130	Nowak	Referent	600	100	10
110	Józek	Kierownik	1200	100	20
140	Nowacki	Analitik	800	110	20
150	Bunio	Sekretarka	700	100	10

insert

{200, 'Szop', 'Księgowy', 900, 10}
{130, 'Borsuk', 'Kierownik', 1000, 20}
{210, 'Rosomak', 'Kierownik', 400, 20}

Operacja selekcji

Wyodrębnia podzbiór krotek relacji spełniających warunek selekcji

- Notacja: $\sigma_{\langle \text{warunek selekcji} \rangle} (\langle \text{Nazwa relacji} \rangle)$

warunek selekcji – zbiór predykatów w postaci:

$\langle \text{atrybut} \rangle \langle \text{operator relacyjny} \rangle \langle \text{litera} \rangle$

$\langle \text{atrybut} \rangle \langle \text{operator relacyjny} \rangle \langle \text{atrybut} \rangle$

predykaty są łączone operatorami logicznymi : AND, OR

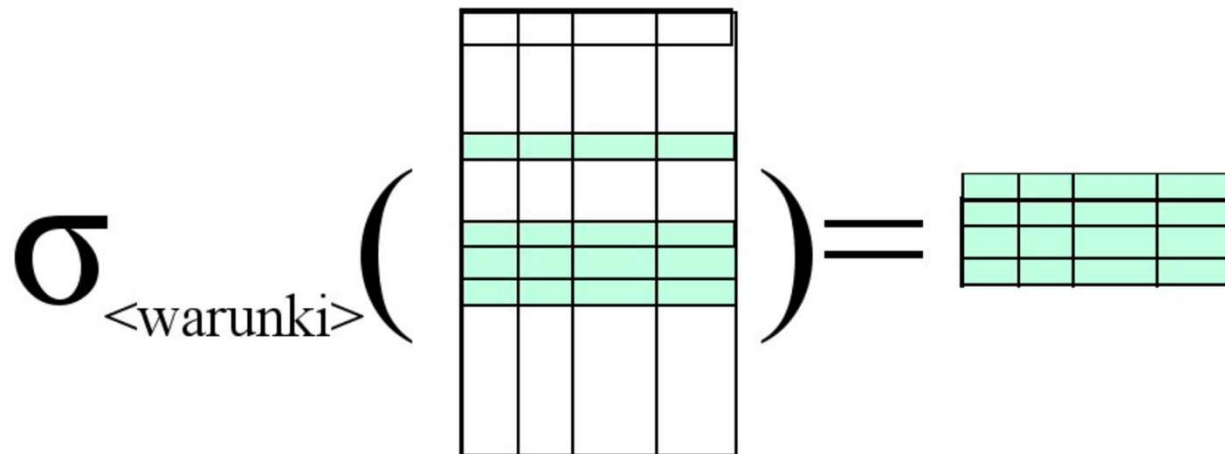
- Właściwości: operacja selekcji jest komutatywna:

$$\sigma_{\langle \text{war1} \rangle} (\sigma_{\langle \text{war2} \rangle} (\mathbf{R})) = \sigma_{\langle \text{war2} \rangle} (\sigma_{\langle \text{war1} \rangle} (\mathbf{R}))$$

Operacja selekcji (2)

Przykłady:

- $\sigma_{\langle \text{etat} = \text{'dyrektor'} \rangle}(\text{pracownicy})$
- $\sigma_{\langle \text{idZesp} = 10 \text{ AND } \text{placa} > 2000 \rangle}(\text{pracownicy})$



Operacja selekcji -przykład

ID_PRAC	NAZWISKO	ETAT	ID_SZEFA	ZATRUDNIONY	PLACA_POD	PLACA_DOD	NAZWA
100	WEGLARZ	DYREKTOR		68/01/01	1730.00	420.50	ADMINISTRACJA
110	BLAZEWICZ	PROFESOR	100	73/05/01	1350.00	210.00	ALGORYTMY
120	SLOWINSKI	PROFESOR	100	77/09/01	1070.00		SYSTEMY EKSPERCKIE
130	BRZEZINSKI	PROFESOR	100	68/07/01	960.00		SYSTEMY ROZPROSZONE
140	MORZY	PROFESOR	130	75/09/15	830.00	105.00	SYSTEMY ROZPROSZONE
150	KROLIKOWSKI	ADIUNKT	130	77/09/01	645.50		SYSTEMY ROZPROSZONE
160	KOSZLAJDA	ADIUNKT	130	85/03/01	590.00		SYSTEMY ROZPROSZONE
170	JEZIERSKI	ASYSTENT	130	92/10/01	439.70	80.50	SYSTEMY ROZPROSZONE
190	MATYSIAK	ASYSTENT	140	93/09/01	371.00		SYSTEMY ROZPROSZONE
180	MAREK	SEKRETARKA	100	85/02/20	410.20		ADMINISTRACJA
200	ZAKRZEWICZ	STAZYSTA	140	94/07/15	208.00		SYSTEMY EKSPERCKIE
210	BIALY	STAZYSTA	130	93/10/15	250.00	170.60	SYSTEMY EKSPERCKIE
220	KONOPKA	ASYSTENT	110	93/10/01	480.00		SYSTEMY ROZPROSZONE
230	HAPKE	ASYSTENT	120	92/09/01	480.00	90.00	SYSTEMY EKSPERCKIE

$\sigma_{\text{ETAT}='ASYSTENT'}(\text{PRACOWNICY})$

ID_PRAC	NAZWISKO	ETAT	ID_SZEFA	ZATRUDNIONY	PLACA_POD	PLACA_DOD	NAZWA
170	JEZIERSKI	ASYSTENT	130	92/10/01	439.70	80.50	SYSTEMY ROZPROSZONE
190	MATYSIAK	ASYSTENT	140	93/09/01	371.00		SYSTEMY ROZPROSZONE
220	KONOPKA	ASYSTENT	110	93/10/01	480.00		SYSTEMY ROZPROSZONE
230	HAPKE	ASYSTENT	120	92/09/01	480.00	90.00	SYSTEMY EKSPERCKIE

Operacja projekcji

Wyodrębnia wybrane atrybuty relacji

- Notacja: $\pi_{\text{<atrybuty>}} (\text{<Nazwa relacji>})$

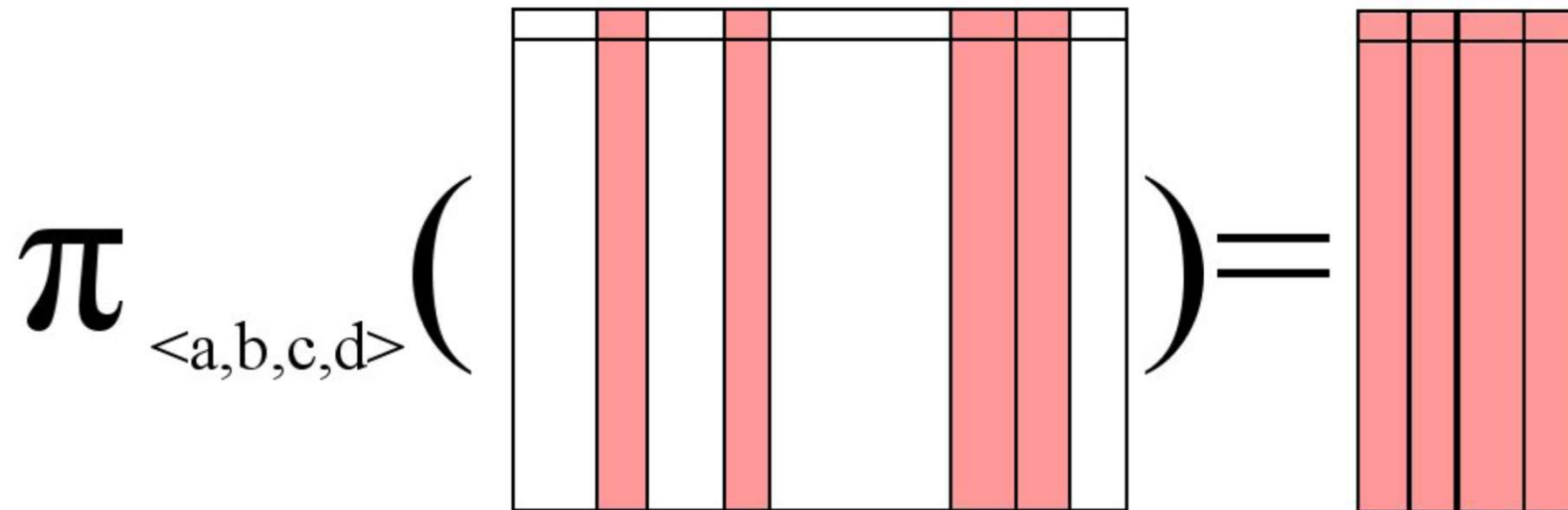
atrybuty – podzbiór atrybutów ze schematu relacji

- Właściwości: operacja projekcji nie jest komutatywna

Operacja projekcji (2)

Przykłady:

- $\pi_{\langle \text{nazwisko}, \text{etat}, \text{placa} \rangle}(\text{pracownicy})$



Operacja projekcji - przykład

ID_PRAC	NAZWISKO	ETAT	ID_SZEFA	ZATRUDNIONY	PLACA_POD	PLACA_DOD	NAZWA
100	WEGLARZ	DYREKTOR		68/01/01	1730.00	420.50	ADMINISTRACJA
110	BLAZEWICZ	PROFESOR	100	73/05/01	1350.00	210.00	ALGORYTMY
120	SLOWINSKI	PROFESOR	100	77/09/01	1070.00		SYSTEMY EKSPERCKIE
130	BRZEZINSKI	PROFESOR	100	68/07/01	960.00		SYSTEMY ROZPROSZONE
140	MORZY	PROFESOR	130	75/09/15	830.00	105.00	SYSTEMY ROZPROSZONE
150	KROLIKOWSKI	ADIUNKT	130	77/09/01	645.50		SYSTEMY ROZPROSZONE
160	KOSZLAJDA	ADIUNKT	130	85/03/01	590.00		SYSTEMY ROZPROSZONE
170	JEZIERSKI	ASYSTENT	130	92/10/01	439.70	80.50	SYSTEMY ROZPROSZONE
190	MATYSIAK	ASYSTENT	140	93/09/01	371.00		SYSTEMY ROZPROSZONE
180	MAREK	SEKRETARKA	100	85/02/20	410.20		ADMINISTRACJA
200	ZAKRZEWICZ	STAZYSTA	140	94/07/15	208.00		SYSTEMY EKSPERCKIE
210	BIALY	STAZYSTA	130	93/10/15	250.00	170.60	SYSTEMY EKSPERCKIE
220	KONOPKA	ASYSTENT	110	93/10/01	480.00		SYSTEMY ROZPROSZONE
230	HAPKE	ASYSTENT	120	92/09/01	480.00	90.00	SYSTEMY EKSPERCKIE

$\pi_{\text{NAZWISKO, ETAT, ZATRUDNIONY}}(\text{PRACOWNICY})$

NAZWISKO	ETAT	ZATRUDNIONY
WEGLARZ	DYREKTOR	68/01/01
BLAZEWICZ	PROFESOR	73/05/01
SLOWINSKI	PROFESOR	77/09/01
BRZEZINSKI	PROFESOR	68/07/01
MORZY	PROFESOR	75/09/15
KROLIKOWSKI	ADIUNKT	77/09/01
KOSZLAJDA	ADIUNKT	85/03/01
JEZIERSKI	ASYSTENT	92/10/01
MATYSIAK	ASYSTENT	93/09/01
MAREK	SEKRETARKA	85/02/20
ZAKRZEWICZ	STAZYSTA	94/07/15
BIALY	STAZYSTA	93/10/15
KONOPKA	ASYSTENT	93/10/01
HAPKE	ASYSTENT	92/09/01

Składanie operacji

Wynik jednej operacji może być zbiorem wejściowym dla drugiej operacji

- $\sigma_{\text{idZesp}=10}(\text{pracownicy}) \Rightarrow \text{pracZesp10}$

$$\pi_{\text{idPrac}, \text{Nazwisko}}(\text{pracZesp10}) \Rightarrow \text{pracZesp10Wynik}$$

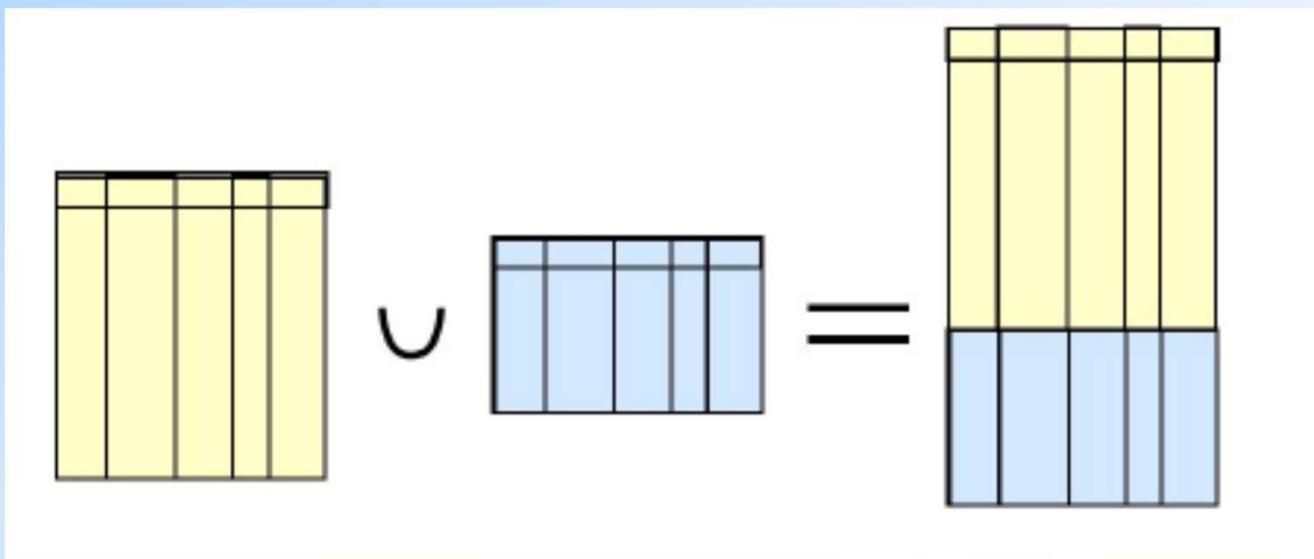
- $\text{pracZesp10Wynik} = \pi_{\text{idPrac}, \text{Nazwisko}}(\sigma_{\text{idZesp}=10}(\text{pracownicy}))$

Operacje na zbiorach

- Operacje sumy, iloczynu, różnicy są operacjami na zbiorach, które są wykonalne dla dwóch relacji jeśli występuje pomiędzy nimi **kompatybilność relacji**
- Dwie relacje $R(A_1, \dots, A_n)$ i $S(B_1, \dots, B_n)$ są **kompatybilne**, jeżeli mają ten sam stopień i $\text{dom}(A_i) = \text{dom}(B_i)$, dla $1 \leq i \leq n$

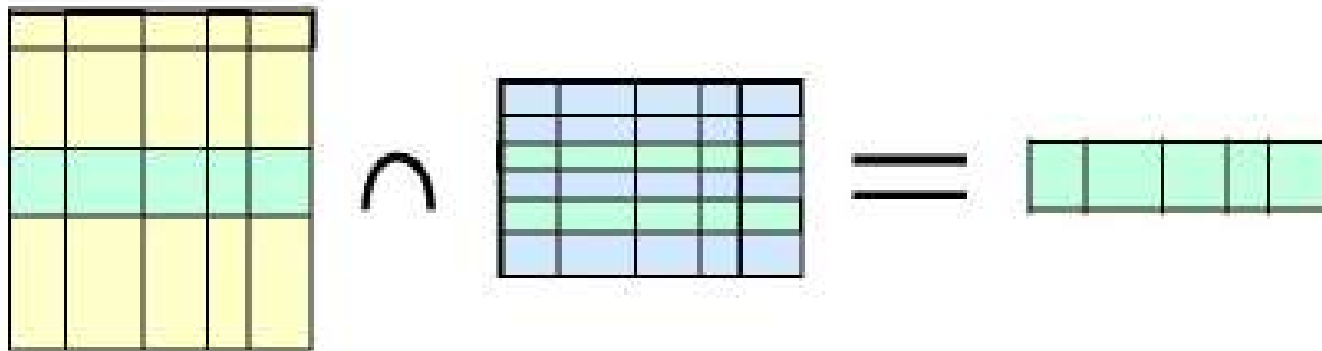
Suma

- Wynikiem tej operacji, oznaczanej przez $R \cup S$ jest relacja zawierająca wszystkie krotki, które występują w R i wszystkie krotki S, z wyłączeniem duplikatów krotek
- Operacja sumy jest operacją komutatywną $R \cup S = S \cup R$



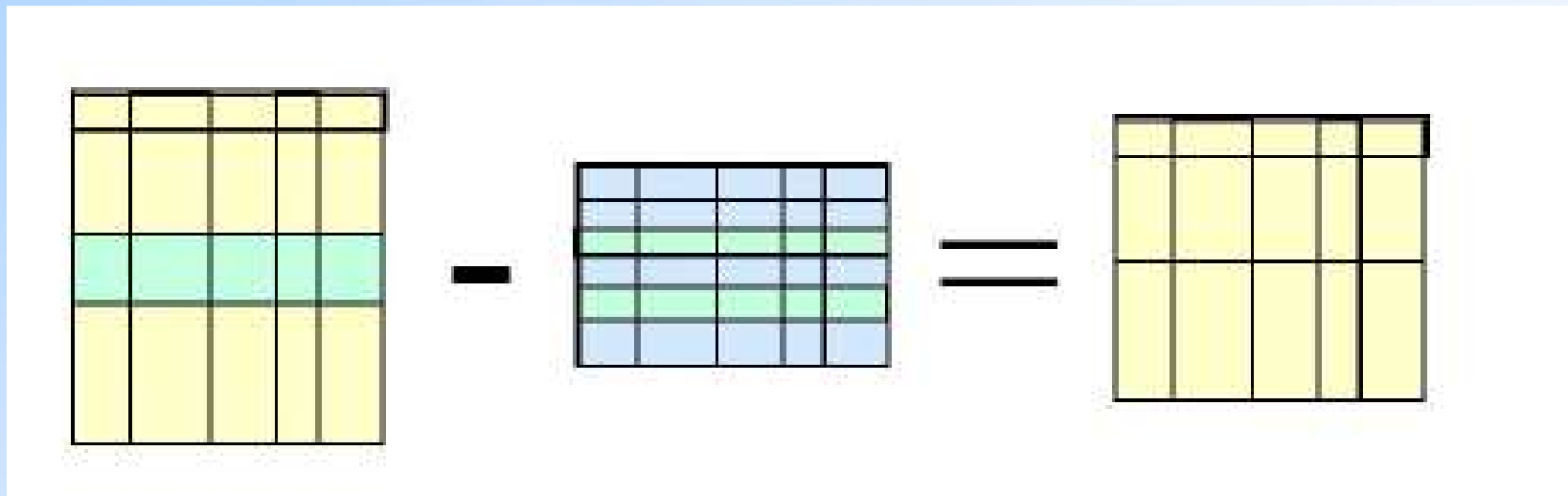
Iloczyn

- Wynikiem tej operacji, oznaczanej przez $R \cap S$ jest relacja zawierająca krotki, które występują zarówno w R jak i S
- Operacja iloczynu jest operacją komutatywną $R \cap S = S \cap R$

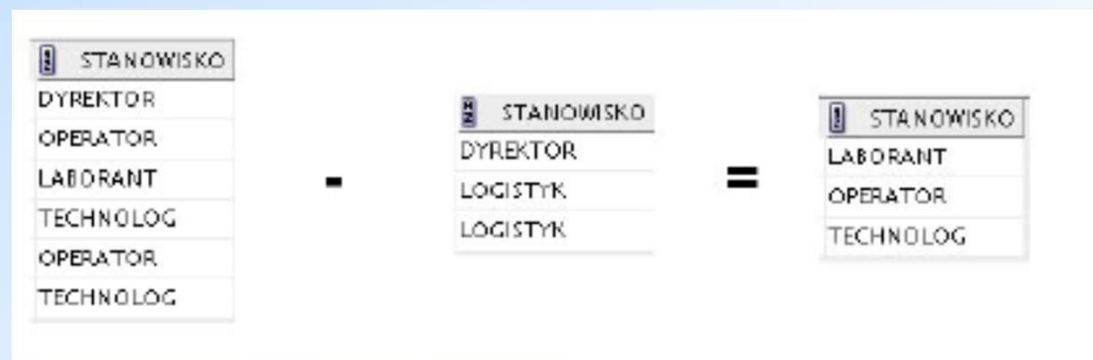


Różnica

- Wynikiem tej operacji, oznaczanej przez $R-S$ jest relacja zawierająca wszystkie krotki, które występują w R i nie występują w S
- Operacja różnicy nie jest operacją komutatywną $R - S \neq S - R$



Przykłady

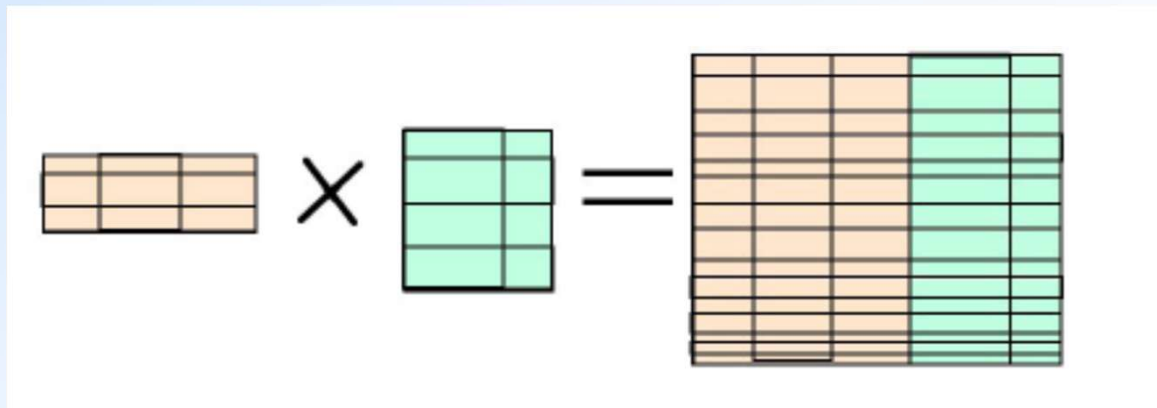


Iloczyn kartezjański

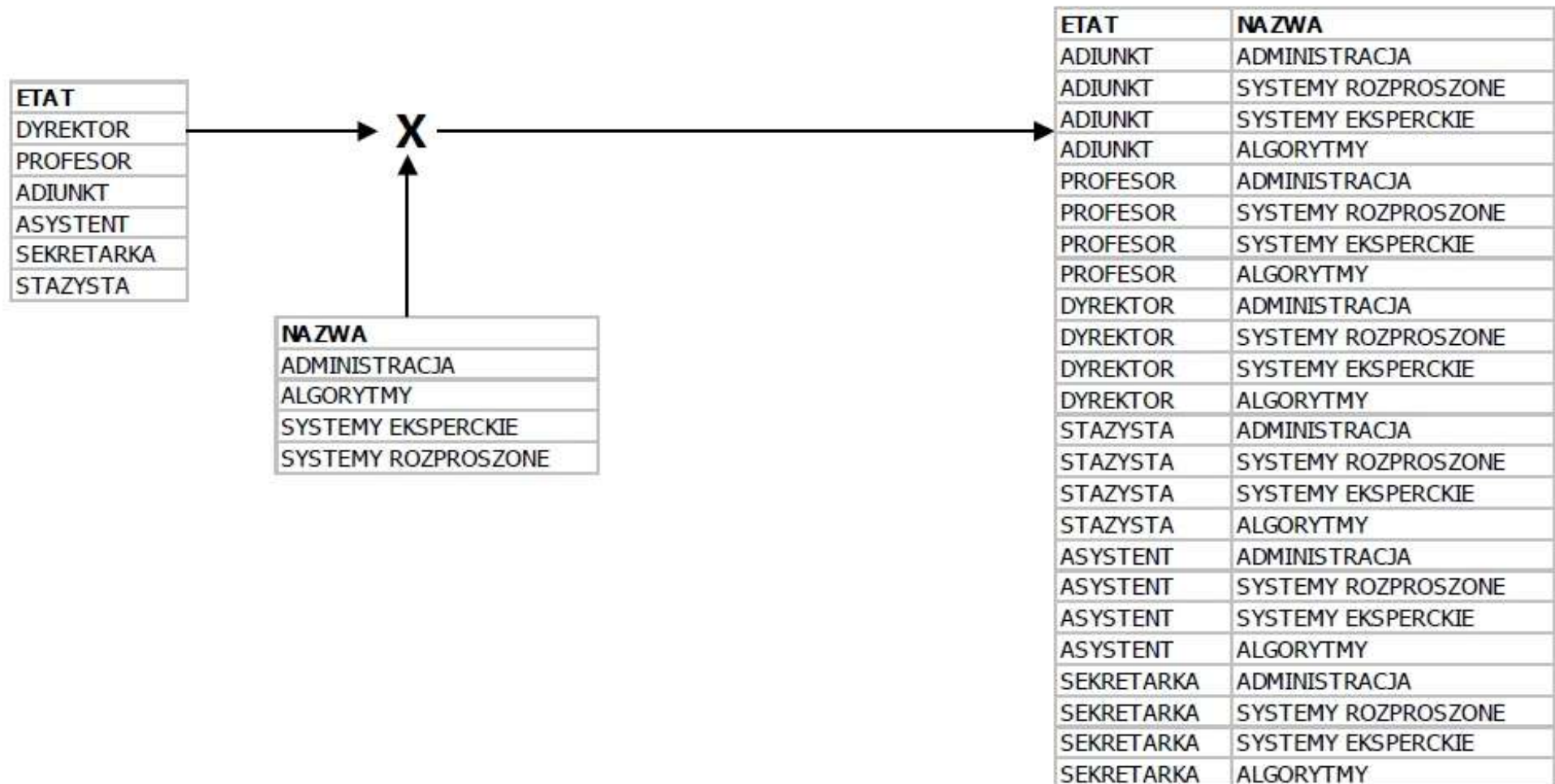
- Dane są dwie relacje $R(A_1, \dots, A_n)$ i $S(B_1, \dots, B_m)$

Wynikiem iloczynu kartezjańskiego relacji R i S , oznaczonego $R \times S$ jest relacja Q stopnia $n+m$ i schemacie $Q(A_1, \dots, A_n, B_1, \dots, B_m)$

- Krotkom w relacji Q odpowiadają **wszystkie kombinacje** krotek z relacji R i S
- Jeżeli relacja R ma N krotek, a relacja S ma M krotek to relacja Q będzie miała $N \cdot M$ krotek
- notacja: $R \bowtie S$



Iloczyn kartezjański - przykład



Operacja połączenia

Służy do łączenia na podstawie warunku połączeniowego wybranych krotek z dwóch relacji w pojedynczą krotkę

- Notacja: $R \bowtie_{\langle \text{warunek połączeniowy} \rangle} S$

warunek połączeniowy – zbiór predykatów w postaci:

$$A_i \theta B_j$$

predykaty są łączone operatorami logicznymi : AND

- A_i – atrybut R , B_j – atrybut S ,
- $\text{dom}(A_i) = \text{dom}(B_j)$

$\forall \theta$ - operator relacyjny ze zbioru $\{ =, <, >, <=, >, >= \}$

Operacja połączenia - przykład

ID_PRAC	NAZWISKO	ETAT	ID_SZEFA	ZATRUDNI	PLACA_POD	PLACA_DOI	PID_ZESP
100	WEGLARZ	DYREKTOR		68/01/01	1730.00	420.50	10
110	BLAZEWICZ	PROFESOR	100	73/05/01	1350.00	210.00	40
120	SLOWINSKI	PROFESOR	100	77/09/01	1070.00		30
130	BRZEZINSKI	PROFESOR	100	68/07/01	960.00		20
140	MORZY	PROFESOR	130	75/09/15	830.00	105.00	20
150	KROLIKOWSKI	ADIUNKT	130	77/09/01	645.50		20
160	KOSZLAJDA	ADIUNKT	130	85/03/01	590.00		20
170	JEZERSKI	ASYSTENT	130	92/10/01	439.70	80.50	20
190	MATYSIAK	ASYSTENT	140	93/09/01	371.00		20
180	MAREK	SEKRETARZ	100	85/02/20	410.20		10
200	ZAKRZEWICZ	STAZYSTA	140	94/07/15	208.00		30
210	BIALY	STAZYSTA	130	93/10/15	250.00	170.60	30
220	KONOPKA	ASYSTENT	110	93/10/01	480.00		20
230	HAPKE	ASYSTENT	120	92/09/01	480.00	90.00	30

klucz
obcy

ZID_ZESP	NAZWA	ADRES
10	ADMINISTRACJA	PIOTROWO 3A
20	SYSTEMY ROZPROSZONE	PIOTROWO 3A
30	SYSTEMY EKSPERCKIE	STRZELECKA 14
40	ALGORYTMY	WLODKOWICA 16
50	BADANIA OPERACYJNE	MIELZYNSKIEGO 30

$R \leftarrow \text{PRACOWNICY} \times \text{ZESPOLY}$

$S \leftarrow \sigma_{\text{PID_ZESP}=\text{ZID_ZESP}}(R)$

$T \leftarrow \pi_{\text{ID_PRAC}, \text{NAZWISKO}, \text{PID_ZESP}, \text{NAZWA}}(S)$

$S \leftarrow \text{PRACOWNICY} \bowtie_{\text{PID_ZESP}=\text{ZID_ZESP}} \text{ZESPOLY}$

ID_PRAC	NAZWISKO	ID_ZESP	NAZWA
100	WEGLARZ	10	ADMINISTRACJA
180	MAREK	10	ADMINISTRACJA
160	KOSZLAJDA	20	SYSTEMY ROZPROSZONE
220	KONOPKA	20	SYSTEMY ROZPROSZONE
190	MATYSIAK	20	SYSTEMY ROZPROSZONE
170	JEZERSKI	20	SYSTEMY ROZPROSZONE
150	KROLIKOWSKI	20	SYSTEMY ROZPROSZONE
130	BRZEZINSKI	20	SYSTEMY ROZPROSZONE
140	MORZY	20	SYSTEMY ROZPROSZONE
230	HAPKE	30	SYSTEMY EKSPERCKIE
210	BIALY	30	SYSTEMY EKSPERCKIE
200	ZAKRZEWICZ	30	SYSTEMY EKSPERCKIE
120	SLOWINSKI	30	SYSTEMY EKSPERCKIE
110	BLAZEWICZ	40	ALGORYTMY

Połączenie zewnętrzne

Wynikiem tradycyjnego połączenia jest zbiór krotek z relacji R i S spełniających kryteria połączenia

Połączenie zewnętrzne umożliwia pozostawienie w wyniku także tych krotek, dla których nie znaleziono odpowiednika w łączonej tabeli

Typy połączeń zewnętrznych:

- lewostronne (left outer join)
- prawostronne (right outer join)
- obustronne (full outer join)

Połączenie zewnętrzne (2)

-połączenie lewostronne $R \bowtie S$:

zawiera wszystkie krotki z relacji R połączone z odpowiadającymi im krotkami w relacji S (lub połączone z pustą krotką, jeśli nie znaleziono odpowiednika w S)

-połączenie prawostronne $R \ltimes S$:

zawiera wszystkie krotki z relacji S połączone z odpowiadającymi im krotkami w relacji R (lub połączone z pustą krotką, jeśli nie znaleziono odpowiednika w R)

-połączenie obustronne $R \ltimes\bowtie S$:

zawiera wszystkie krotki z obu łączonych relacji R i S wraz z odpowiadającymi im krotkami z drugiej relacji (lub połączone z pustą krotką)

Połączenie zewnętrzne - przykład

ID_PRAC	NAZWISKO	ETAT	ID_SZEFA	ZATRUDNI	PLACA_POD	PLACA_DOD	PID_ZESP
100	WEGLARZ	DYREKTOR		68/01/01	1730.00	420.50	10
110	BLAZEWICZ	PROFESOR	100	73/05/01	1350.00	210.00	40
120	SLOWINSKI	PROFESOR	100	77/09/01	1070.00		30
130	BRZEZINSKI	PROFESOR	100	68/07/01	960.00		20
140	MORZY	PROFESOR	130	75/09/15	830.00	105.00	20
150	KROLIKOWSKI	ADIUNKT	130	77/09/01	645.50		20
160	KOSZLAJDA	ADIUNKT	130	85/03/01	590.00		20
170	JEZIERSKI	ASYSTENT	130	92/10/01	439.70	80.50	20
190	MATYSIAK	ASYSTENT	140	93/09/01	371.00		20
180	MAREK	SEKRETARKA	100	85/02/20	410.20		10
200	ZAKRZEWICZ	STAZYSTA	140	94/07/15	208.00		30
210	BIALY	STAZYSTA	130	93/10/15	250.00	170.60	30
220	KONOPKA	ASYSTENT	110	93/10/01	480.00		20
230	HAPKE	ASYSTENT	120	92/09/01	480.00	90.00	30

ZID_ZESP	NAZWA	ADRES
10	ADMINISTRACJA	PIOTROWO 3A
20	SYSTEMY ROZPROSZONE	PIOTROWO 3A
30	SYSTEMY EKSPERCKIE	STRZELECKA 14
40	ALGORYTMY	WLODKOWICA 16
50	BADANIA OPERACYJNE	MIELZYNSKIEGO 30

???

ID_PRAC	NAZWISKO	ID_ZESP	NAZWA
100	WEGLARZ	10	ADMINISTRACJA
180	MAREK	10	ADMINISTRACJA
160	KOSZLAJDA	20	SYSTEMY ROZPROSZONE
220	KONOPKA	20	SYSTEMY ROZPROSZONE
...	...	...	...
<i>null</i>	<i>null</i>	50	BADANIA OPERACYJNE

S ← PRACOWNICY  **PID_ZESP=ZID_ZESP ZESPOLY**

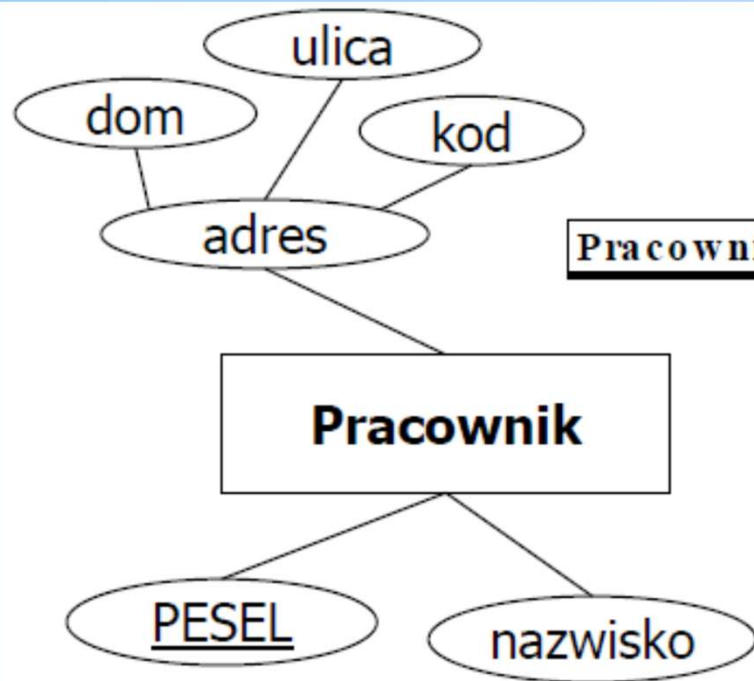
Transformacja modelu pojęciowego do logicznego

- Model ER -> schemat relacyjny
- Transformacja:
 - encji z atrybutami
 - związków
 - hierarchii encji

Transformacja encji

- Encja -> relacja
- Atrybut encji -> atrybut relacji
- Typ danych atrybutu encji -> typ danych atrybutu relacji
- Identyfikator encji -> klucz podstawowy relacji
- Obowiązkowość atrybutów encji -> ograniczenie NOT NULL
- Ograniczenia integralnościowe atrybutów encji -> ograniczenia integralnościowe atrybutów relacji

Przykład

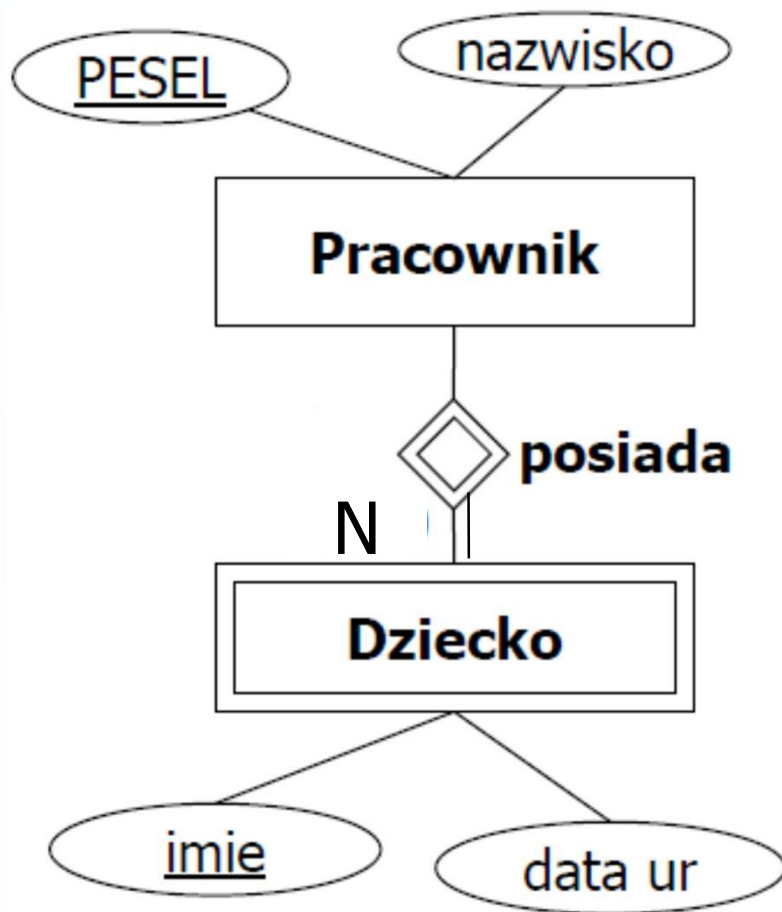


Pracownicy	<u>PESEL</u>	nazwisko	ulica	dom	kod
	1111111111	Kowalski	Koperkowa	13/2	60-234
	12345678901	Nowak	Selerowa	2a	52-892

Pracownicy (
PESEL PRIMARY KEY,
nazwisko NOT NULL,
dom NOT NULL,
ulica NOT NULL,
kod NOT NULL)

Transformacja encji słabych

- Dla każdej encji słabej **S** związanej z encją nadrzędną **N** (reprezentowaną przez relację **R_N**) stwórz relację **R_S** zawierającą wszystkie proste atrybuty **S**.
- Dodaj atrybuty klucza podstawowego relacji **R_N**, jako klucz obcy w **R_S** wskazujący na **R_N**.
- Kluczem podstawowym relacji **R_S** są atrybuty identyfikatora encji **S** i relacji **R_N**.



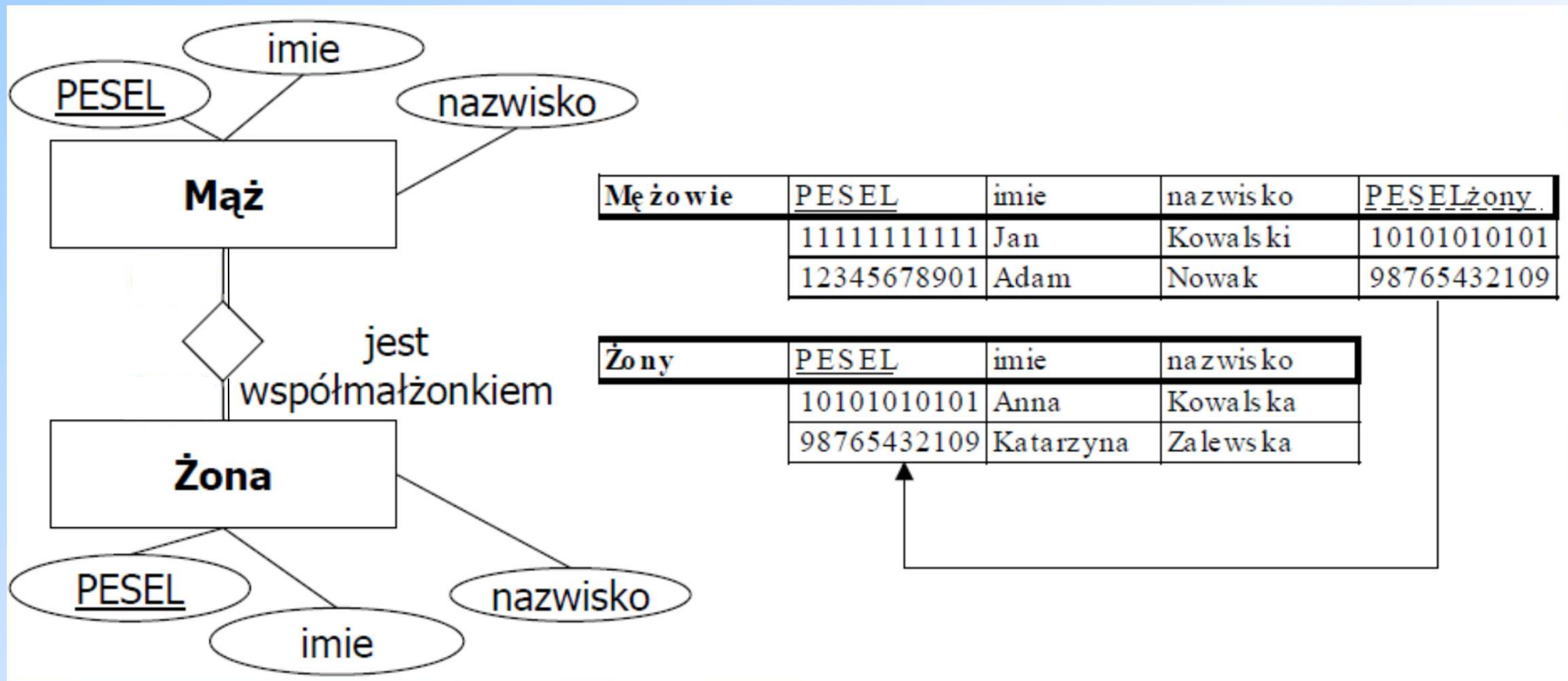
Pracownicy	<u>PESEL</u>	nazwisko
	1111111111	Kowalski
	12345678901	Nowak

Dzieci	<u>imie</u>	data ur	<u>PESEL</u>
	Anna	01-01-2003	1111111111
	Jan	05-03-2005	1111111111
	Anna	08-12-1998	12345678901

An arrow points from the PESEL attribute of the **Dzieci** table to the **Pracownicy** table, indicating a foreign key relationship.

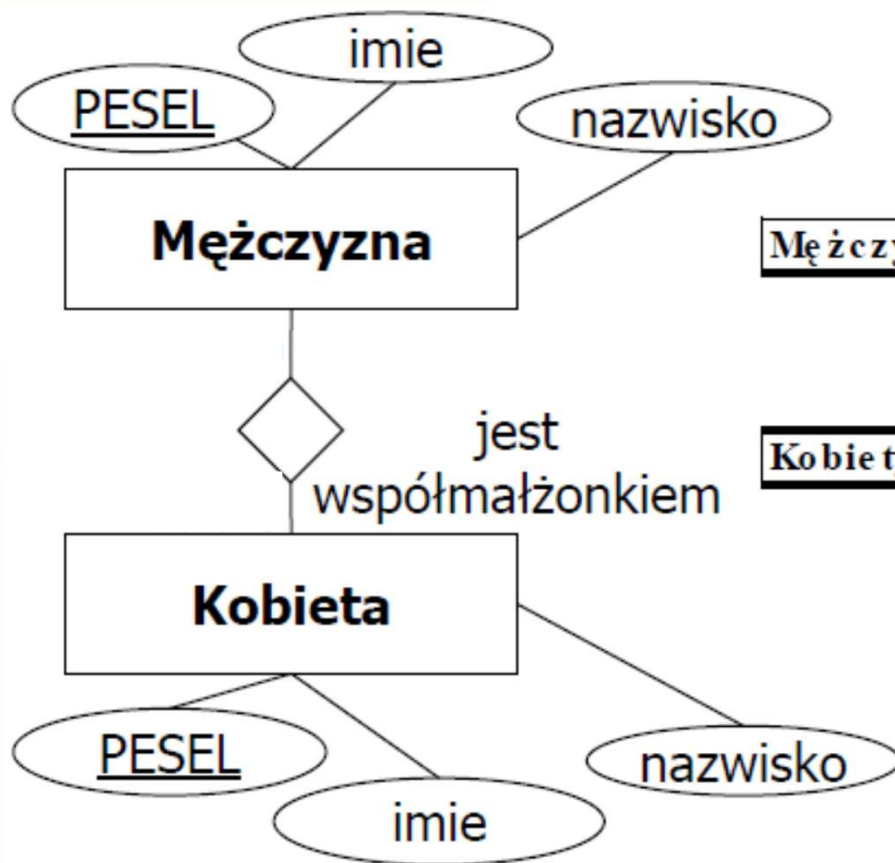
Transformacja związków binarnych 1:1, związek obustronnie wymagany

Dla każdego związku binarnego typu 1:1 obustronnie wymaganego pomiędzy encjami E i F, utwórz odpowiadające łączonym encjom relacje i połącz je za pomocą klucza obcego.



Transformacja związków binarnych 1:1, związek obustronnie opcjonalny

Dla każdego związku binarnego typu 1:1 obustronnie opcjonalnego pomiędzy encjami E i F, utwórz odpowiadające łączonym encjom relacje i połącz je za pomocą klucza obcego. Umieść klucz obcy w relacji o przewidywanej mniejszej ilości krotek.



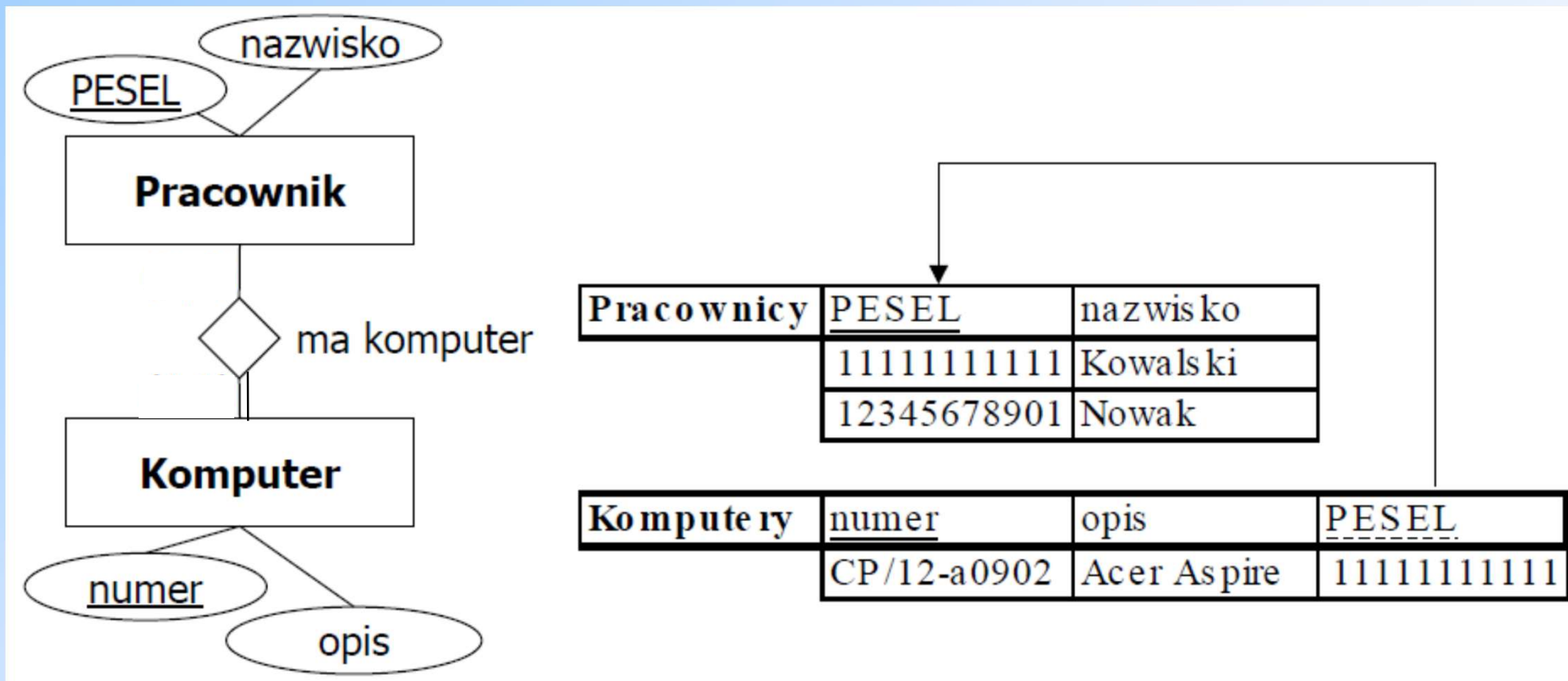
Mężczyźni	<u>PESEL</u>	imie	nazwisko	<u>PESEL</u> żony..
	1111111111	Jan	Kowalski	10101010101
	12345678901	Adam	Nowak	

Kobiety	<u>PESEL</u>	imie	nazwisko
	10101010101	Anna	Kowalska
	98765432109	Katarzyna	Zalewska



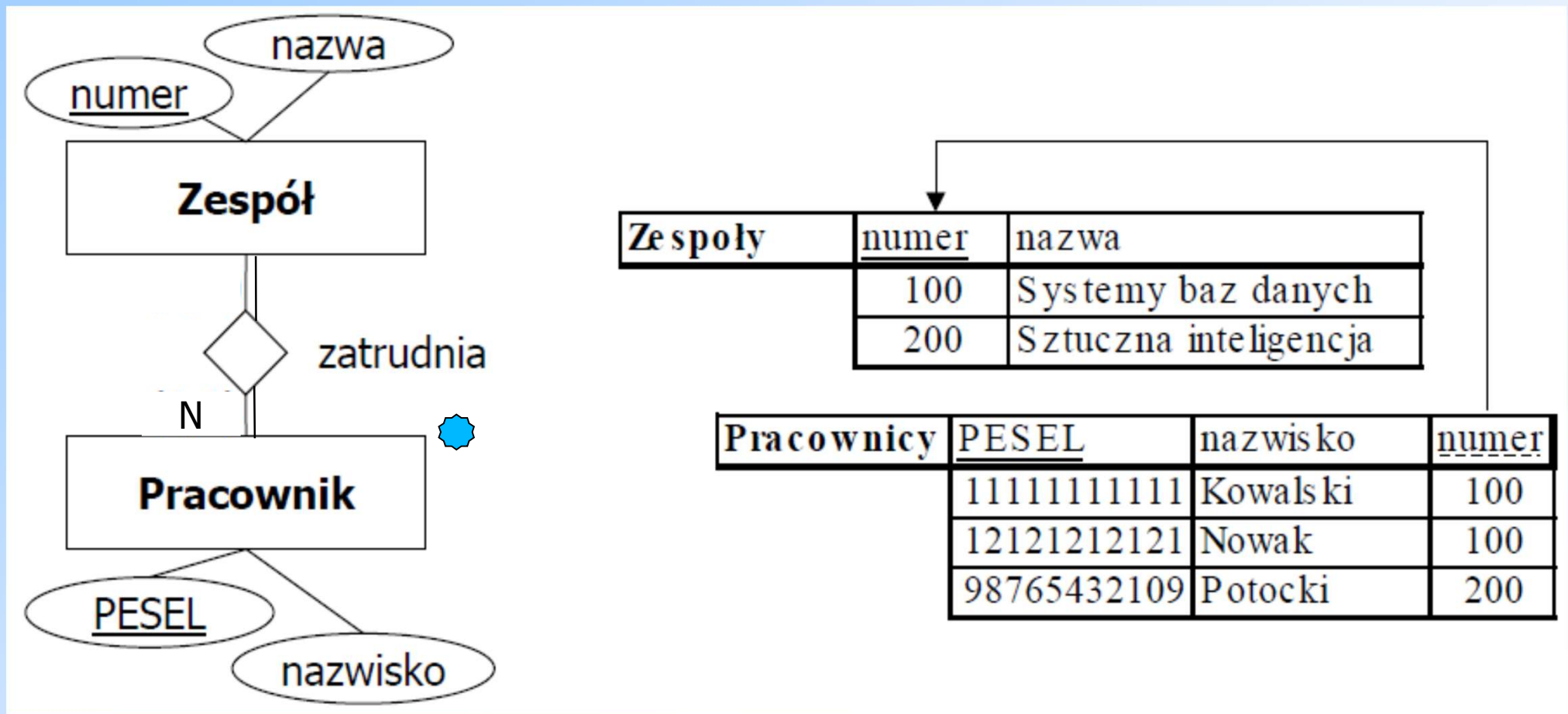
Transformacja związków binarnych 1:1, związek jednostronnie wymagany

Dla każdego związku binarnego typu 1:1 jednostronnie wymaganego pomiędzy encjami E i F, utwórz odpowiadające łączonym encjom relacje i połącz je za pomocą klucza obcego. Umieść klucz obcy w relacji reprezentującej encję, której uczestnictwo w związku jest obowiązkowe (totalne).



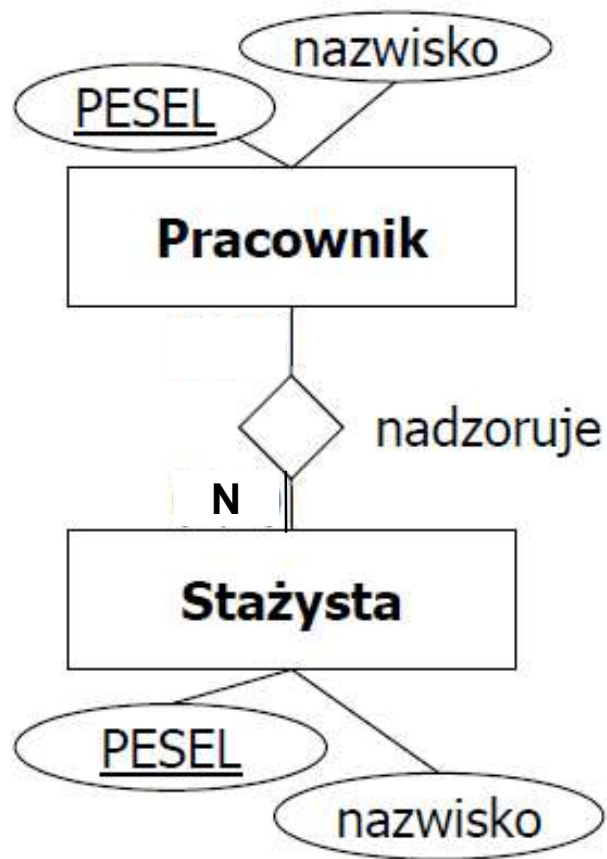
Transformacja związków binarnych 1:N, związek obustronnie wymagany

Dla każdego związku binarnego typu 1:N obustronnie wymaganego pomiędzy encjami E i F, utwórz odpowiadające łączonym encjom relacje R_E i R_F . Umieść w relacji R_F klucz obcy wskazujący na relację R_E . Atrybuty związku umieść jako proste atrybuty relacji R_F .



Transformacja związków binarnych 1:N, związek jednostronnie wymagany

Dla każdego związku binarnego typu 1:N jednostronnie wymaganego (po stronie N) pomiędzy encjami E i F , utwórz odpowiadające łączonym encjom relacje R_E i R_F . Umieść w relacji R_F **klucz obcy wymagany** wskazujący na relację R_E . Atrybuty związku umieść jako proste atrybuty relacji R_F .



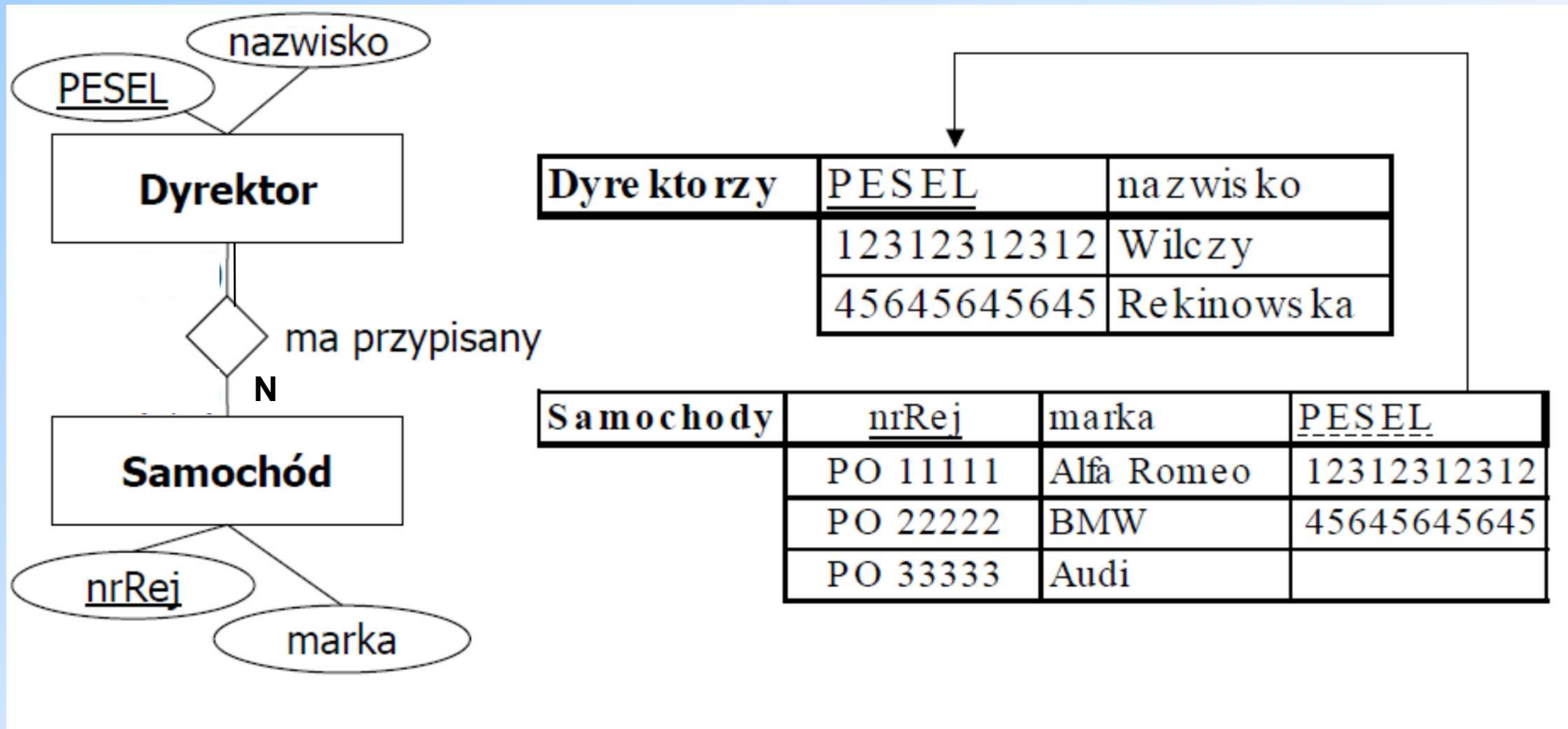
Pracownicy	<u>PESEL</u>	nazwisko
	1111111111	Kowalski
	1212121212	Nowak

Stażyci	<u>PESEL</u>	nazwisko	<u>PESELszeфа</u>
	1000000000	Śruba	1111111111
	2000000000	Nakrętka	1111111111

An arrow points from the PESELszeфа attribute in the second table to the PESEL attribute in the first table, indicating a foreign key relationship.

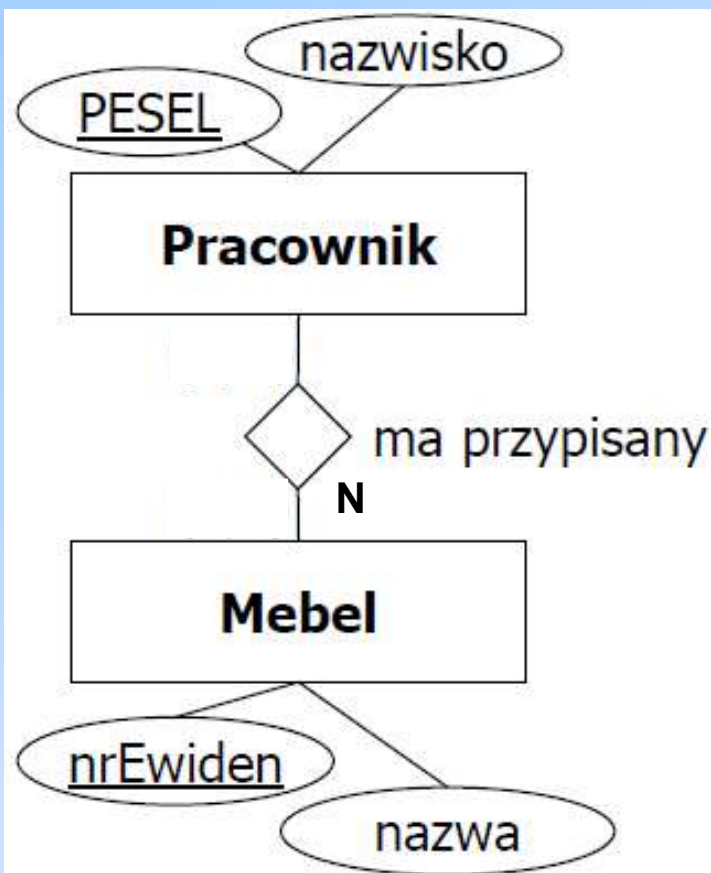
Transformacja związków binarnych 1:N, związek jednostronnie wymagany

Dla każdego związku binarnego typu 1:N jednostronnie wymaganego (po stronie 1) pomiędzy encjami E i F, utwórz odpowiadające łączonym encjom relacje R_E i R_F . Umieść w relacji R_F **klucz obcy opcjonalny** wskazujący na relację R_E . Atrybuty związku umieść jako proste atrybuty relacji R_F .



Transformacja związków binarnych 1:N, związek obustronnie opcjonalny

Dla każdego związku binarnego typu 1:N obustronnie opcjonalnego pomiędzy encjami E i F, utwórz odpowiadające łączonym encjom relacje R_E i R_F . Umieść w relacji R_F klucz obcy opcjonalny wskazujący na relację R_E . Atrybuty związku umieść jako proste atrybuty relacji R_F .



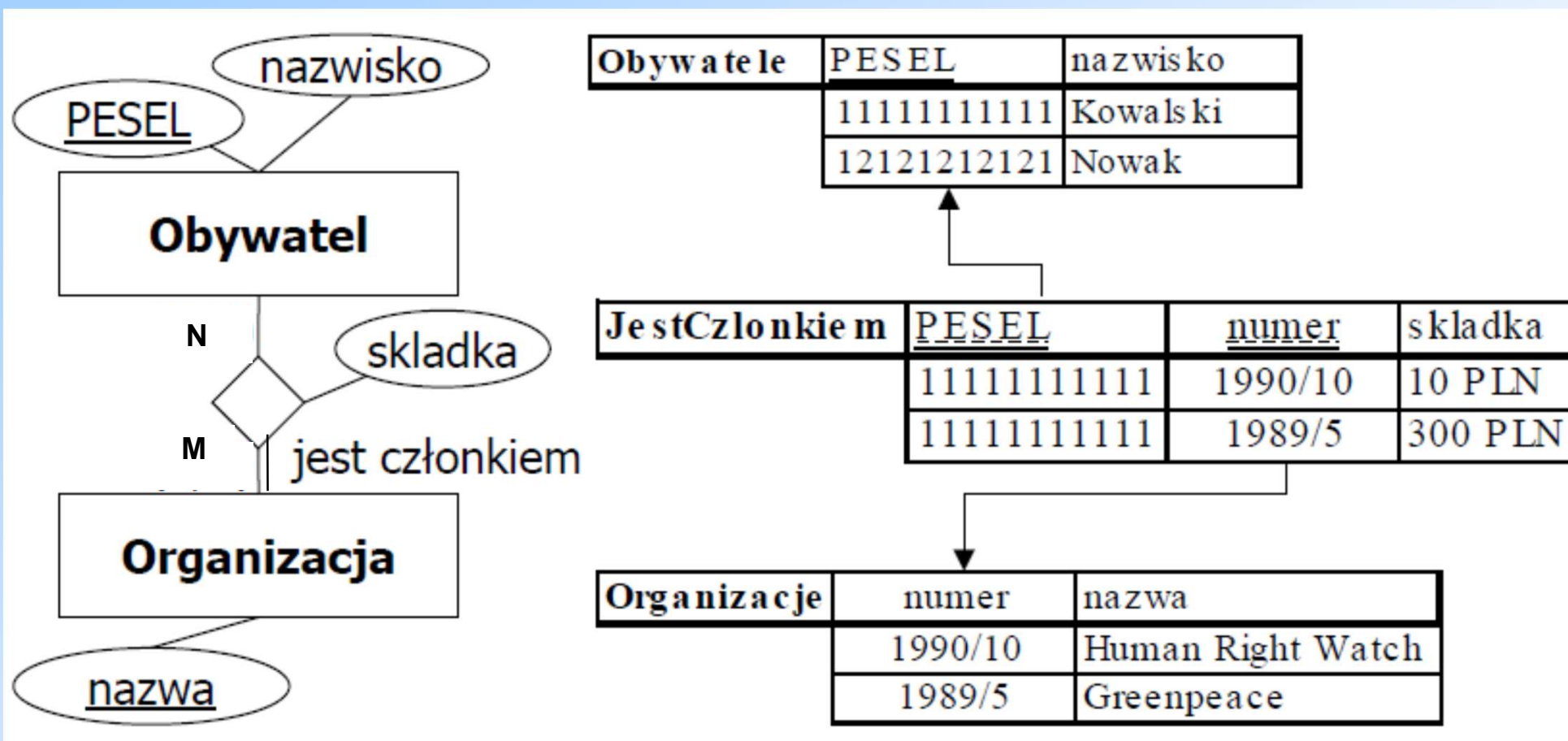
Pracownicy	<u>PESEL</u>	nazwisko
	1111111111	Kowalski
	1212121212	Nowak

Meble	<u>nrEwiden</u>	nazwa	PESEL
	90/123 DK	krzesło	1111111111
	88/001 xR	biurko	1111111111
	91/190 DK	sofa	

An arrow points from the PESEL attribute in the **Meble** table to the PESEL attribute in the **Pracownicy** table, indicating a foreign key relationship.

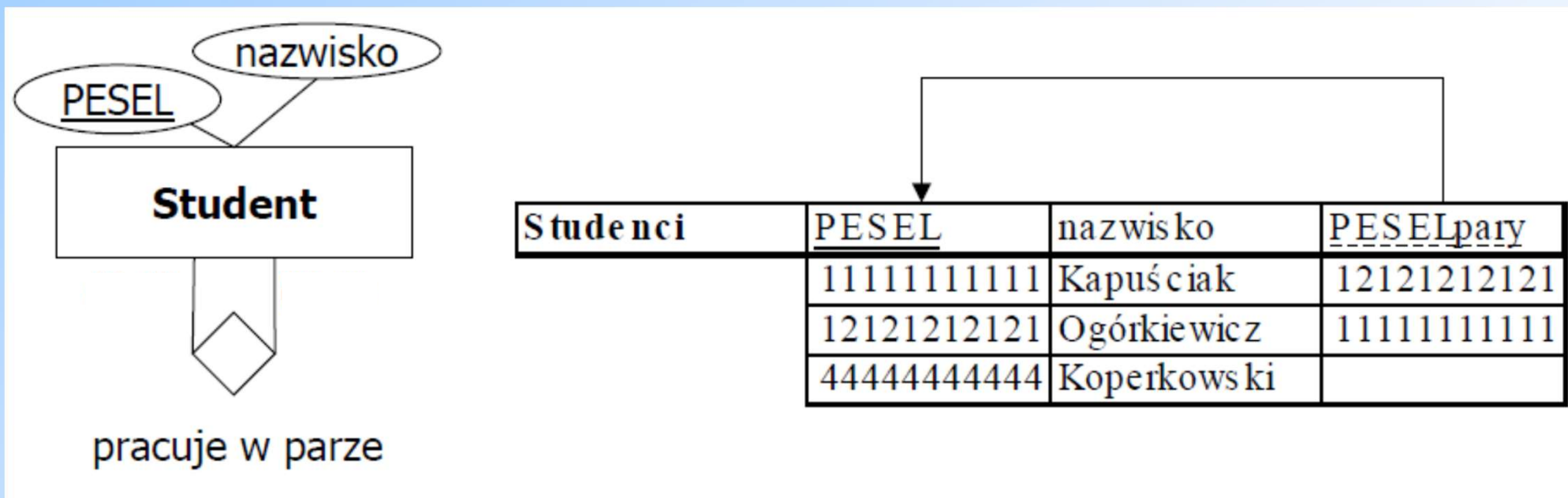
Transformacja związków binarnych N:M

Dla każdego związku binarnego typu N:M pomiędzy encjami E i F, utwórz odpowiadające łączonym encjom relacje R_E i R_F . Utwórz nową relację Z reprezentującą związek. Dodaj do Z dwa klucze obce wskazujące na relacje R_E i R_F . Kluczem podstawowym relacji Z jest kombinacja kluczy obcych. Dodaj do Z atrybuty związku.



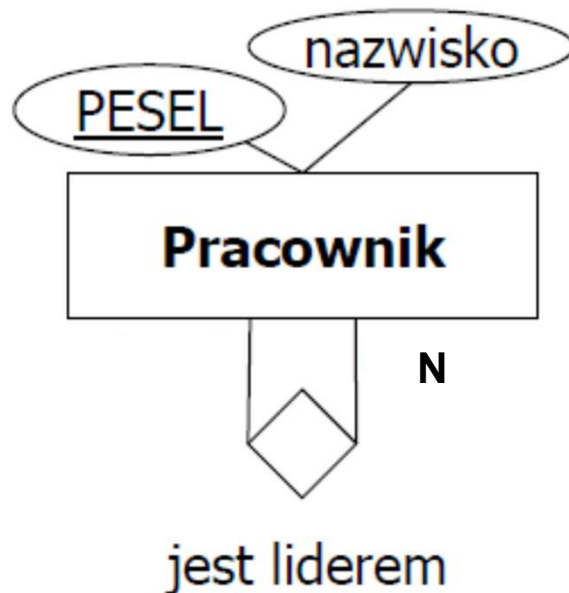
Transformacja związków unarnych 1:1

Dla każdego związku unarnego typu 1:1 w ramach encji **E** dodaj do relacji R_E reprezentującej encję **E** nowy klucz obcy wskazujący na klucz podstawowy relacji R_E . Klucz obcy jest opcjonalny jeśli cały związek jest opcjonalny. Klucz jest obowiązkowy jeśli związek jest obowiązkowy.



Transformacja związków unarnych 1:N

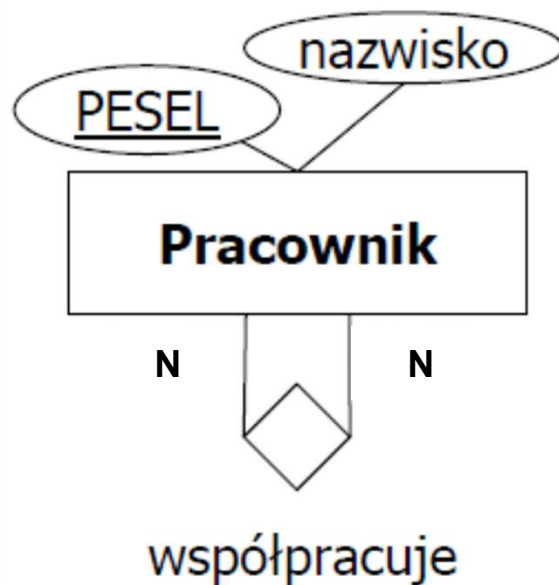
Dla każdego związku unarnego typu 1:N w ramach encji **E** dodaj do relacji **R_E** reprezentującej encję **E** nowy klucz obcy wskazujący na klucz podstawowy relacji **R_E**. Klucz obcy jest opcjonalny, ponieważ związek unarny typu 1:N musi być opcjonalny.



Pracownicy	<u>PESEL</u>	nazwisko	<u>PESEL</u> lidera
	1111111111	Kowalski	12312312312
	1212121212	Nowak	12312312312
	12312312312	Dziubutkiewicz	

Transformacja związków unarnych N:M

Dla każdego związku unarnego typu N:M w ramach encji **E** utwórz relację **R_E** reprezentującą encję **E**. Utwórz nową relację **Z** i dodaj do niej dwukrotnie wymagany klucz obcy wskazujący na relację **R_E**. Kluczem podstawowym relacji **Z** jest kombinacja obu kluczy obcych wskazujących na relację **R_E**.



Pracownicy	<u>PESEL</u>	nazwisko
	1111111111	Kowalski
	1212121212	Nowak
	12312312312	Dziubutkiewicz

Współpracuje	<u>PESEL1</u>	<u>PESEL2</u>
	1111111111	1212121212
	1111111111	12312312312
	1212121212	12312312312

Arrows indicate foreign key relationships: from the PESEL attribute of the 'Pracownicy' table to the PESEL1 attribute of the 'Współpracuje' table, and from the PESEL attribute of the 'Pracownicy' table to the PESEL2 attribute of the 'Współpracuje' table.

Transformacja atrybutów wielowartościowych

Dla każdego atrybutu wielowartościowego w encji **E** utwórz relację R_E reprezentującą encję **E**. Następnie stwórz nową relację **W** i dodaj do niej, jako wymagany, klucz obcy wskazujący na relację R_E . Dodaj do **W** wszystkie proste składowe atrybutu wielowartościowego. Kluczem podstawowym **W** jest kombinacja wartości atrybutu i klucza obcego.

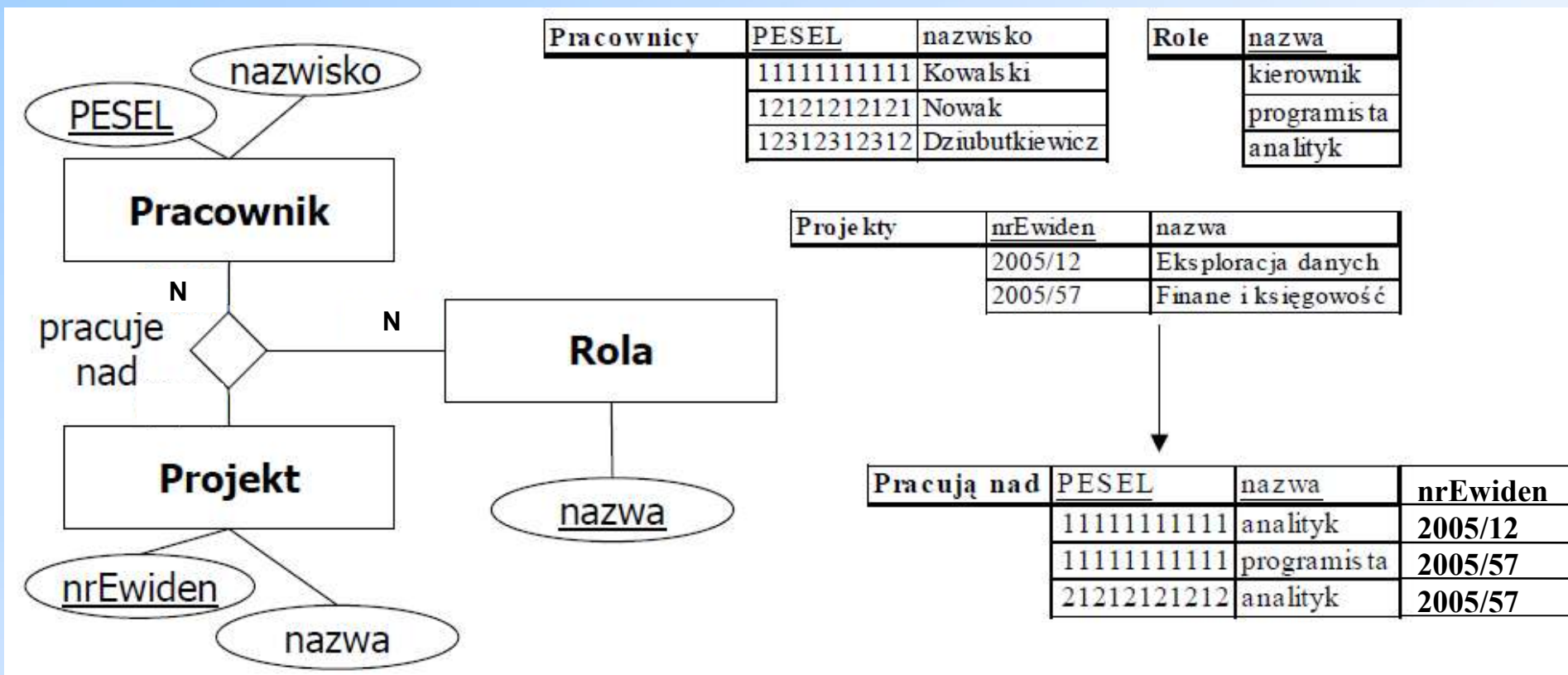


Pracownicy	<u>PESEL</u>	nazwisko
	1111111111	Kowalski
	1212121212	Nowak
	12312312312	Dziubutkiewicz

JęzykiObce	<u>nazwa</u>	<u>PESEL</u>
	angielski	1111111111
	francuski	12312312312
	angielski	12312312312
	niemiecki	12312312312

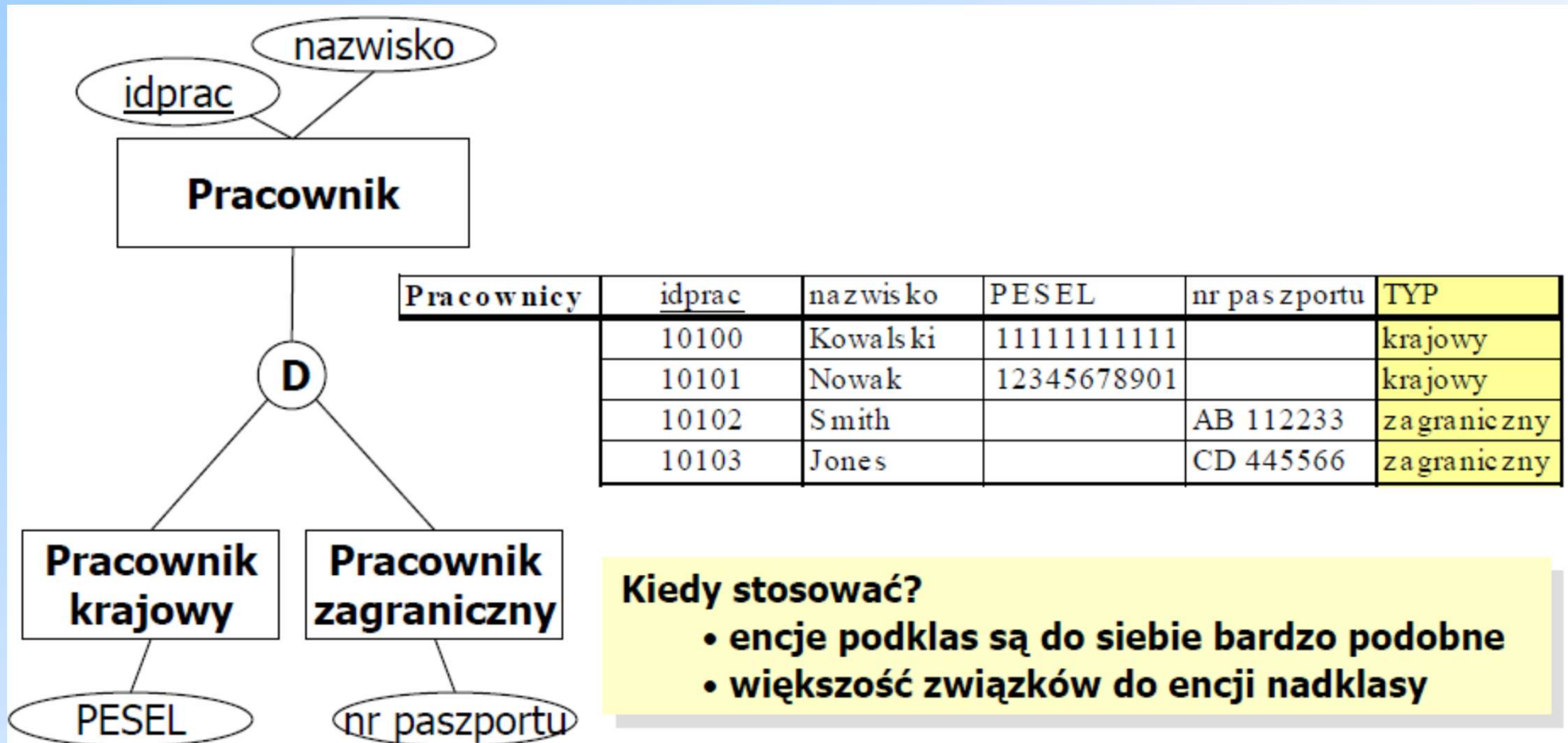
Transformacja związków ternarnych

Dla każdego związku o stopniu >2 utwórz nową relację **Z** i umieść w niej klucze obce wskazujące na klucze podstawowe relacji R, S, T reprezentujące encje wchodzące w skład związku. Kluczem podstawowym Z staje się kombinacja wszystkich kluczy obcych o stopniu asocjacji „wiele” wchodzących w skład Z.



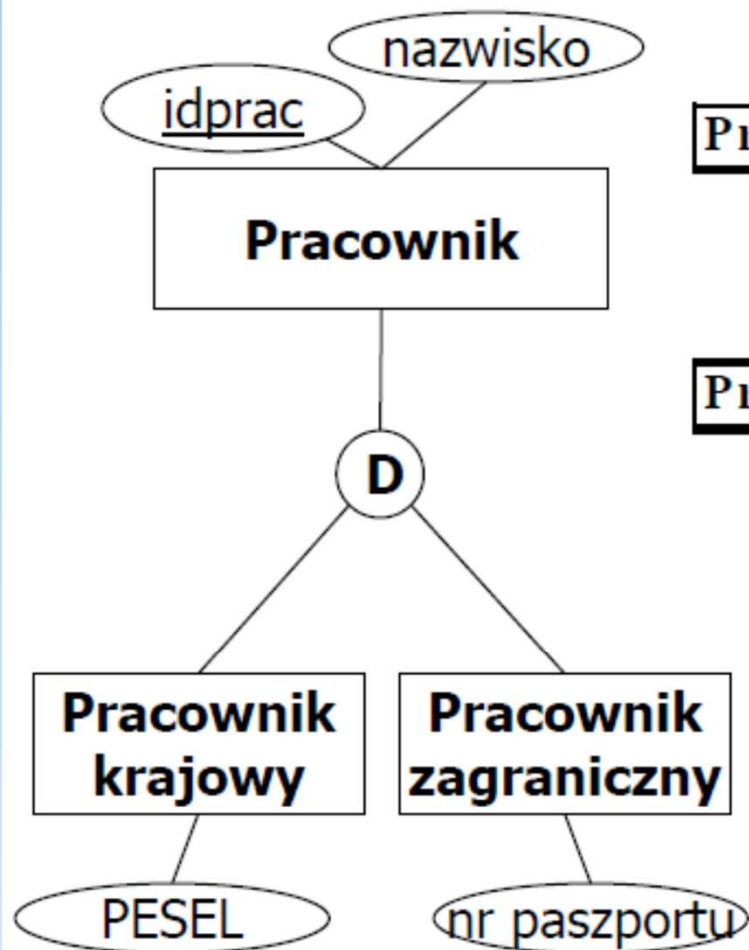
Transformacja hierarchii encji schemat 1

Dla encji nadklasy **E** i wszystkich encji podklas **E'**, **E''**, ..., utwórz nową relację **R_E** i umieść w niej wszystkie atrybuty występujące w encjach. Wszystkie związki dotyczące encji nadklasy lub podklas zaimplementuj w relacji **R_E**. Dodaj do relacji **R_E** dodatkowy atrybut **TYP**.



Transformacja hierarchii encji schemat 2

Dla encji nadklasy E i wszystkich encji podklas $E', E'', \dots$, utwórz nową relację R_E i $R_{E'}$, umieść w relacjach atrybuty właściwej encji podklasy, atrybuty encji nadklasy zduplikuj w obu relacjach (wraz ze związkami do nadklasy).



Prac Krajowi	<u>idprac</u>	nazwisko	PESEL
	10100	Kowalski	1111111111
	10101	Nowak	12345678901

Prac Zagraniczni	<u>idprac</u>	nazwisko	nr paszportu
	10102	Smith	AB 112233
	10103	Jones	CD 445566

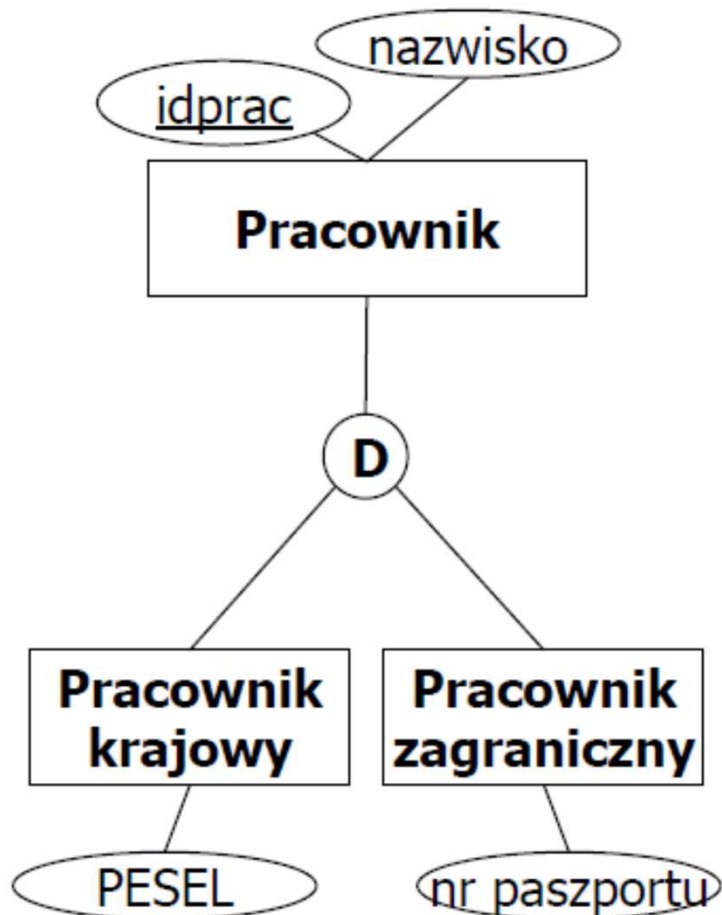
Kiedy stosować?

- encje podklas są bardzo różne
- większość związków do encji podklas

Transformacja hierarchii encji

schemat 3

Dla encji nadklasy E i wszystkich encji podklas $E', E'', \dots$, utwórz odpowiadające im relacje $R_E, R_{E'},$ i $R_{E''}$, umieść każdej z encji w odpowiadającej jej relacji, połącz relacje $R_{E'}$ i $R_{E''}$ z relacją R_E za pomocą klucza obcego. Związki zaimplementuj we właściwej relacji.



Pracownicy	<u>idprac</u>	nazwisko
	10100	Kowalski
	10101	Nowak
	10102	Smith
	10103	Jones

Krajo wi	<u>idprac</u>	PESEL
	10100	11111111111
	10101	12345678901

Za graniczni	<u>idprac</u>	nr paszportu
	10102	AB 112233
	10103	CD 445566

Kiedy stosować?

- encje podklas są dość zróżnicowane
- część związków do encji nadklasy, część do encji podklas

Wprowadzenie do SQL

Interfejs do systemów DBMS

Konieczna jest warstwa pośrednia umożliwiająca korzystanie z bazy danych programom klienckim

Język definiowania danych – DDL (Data Definition Language):

- tworzenie, modyfikacja, usuwanie struktur danych

Język operowanie danymi – DML (Data Manipulation Language):

- służy do tworzenia, modyfikacji, czytania i usuwania danych znajdujących się w strukturach bazy danych

Język kontroli danych – DCL (Data Control Language):

- definiowanie użytkowników, przysługujących im uprawnień, tworzenie grup użytkowników, wykorzystywany przez administratora b.d

Język integralności danych – DIL (Data Integrity Language):

- definiowanie więzów integralności bazy danych

Język SQL

SQL – Structured Query Language

- Pierwsze standaryzacje SQL pochodzą z lat 80-tych, w kolejnych latach rozwijane (np. SQL2 – SQL92)
- W DBMS pełni rolę interfejsu, zapewnia wszystkie 4 funkcjonalności tj. DDL, DML, DCL, DIL
- Przyjął się jako standard, dlatego też często DBMS nazywa się **serwerami SQL**

Dlaczego SQL?

- SQL jest językiem wysokiego poziomu.
 - Za jego pomocą określa się „co zrobić” a nie „jak to zrobić”
 - Pozwala na uniknięcie detali implementacyjnych zawartych w językach proceduralnych np: C++ lub Java.
- Zadaniem systemu zarządzania bazą danych (DBMS) jest ustalenie najbardziej optymalnego sposobu uzyskania odpowiedzi na zadane zapytanie

Wyrażenia Select-From-Where

SELECT wybrane atrybuty

FROM jednej lub kilku tabel (relacji)

WHERE warunek dotyczący krotek w tabelach

Przykłady

- Przykłady zapytań SQL będą oparte na poniższym schemacie bazy danych.

Towary(nazwa, producent)

Sklepy(nazwa, adres, licencja)

Klienci(nazwisko, adres, telefon)

Lubi(nazwisko, towar)

Sprzedaje(sklep, towar, cena)

Odwiedza(nazwisko, sklep)

Przykład

- Używając relacji **Towary(nazwa, producent)**, jakie towary są produkowane przez producenta 'ABC'?

```
SELECT nazwa  
FROM Towary  
WHERE producent = 'ABC';
```

W SQL stałe napisowe umieszcza się w pojedynczym apostrofie.
Poza stałymi napisowymi SQL nie rozróżnia małych i dużych liter



Wynik zapytania

nazwa
rogale
bułki
...

Odpowiedzią jest relacja z pojedynczym atrybutem, nazwą i krotkami z nazwami piw produkowanymi przez wybrany browar.

Zapytanie dotyczące pojedynczej relacji

- Wybierz relację wymienioną w klauzuli FROM.
- Zastosuj operację selekcji wyrażoną klauzulą WHERE.
- Zastosuj operację projekcji wyrażoną klauzulą SELECT.

* w klauzuli SELECT

- Jeżeli w klauzuli FROM jest jedna relacja, * w klauzuli SELECT oznacza wszystkie atrybuty tej relacji
- Przykład **Towary(nazwa, producent):**

```
SELECT *
```

```
FROM Towary
```

```
WHERE producent = 'ABC';
```

Wynik zapytania:

nazwa	producent
rogale	ABC
bułki	ABC
chleb	ABC
. . .	. . .

Zmiana nazw atrybutów

- Zmiany nazwy atrybutu w relacji wynikowej dokonują się stosując składnię "AS <nowa nazwa>".

- Przykład Towary(nazwa, producent):

```
SELECT nazwa AS pieczywo, producent  
FROM Towary  
WHERE producent = 'CDE'
```



Wynik zapytania:

pieczywo	producent
bulki	CDE
rogale	CDE
kajzerki	CDE
. . .	. . .

Wyrażenia w klauzuli SELECT

- W klauzuli SELECT może pojawić się dowolne wyrażenie.
- Przykład: **Sprzedaje(sklep, towar, cena):**

```
SELECT sklep, towar,  
       cena / 4.7 AS cena_w_EUR  
FROM Sprzedaje;
```



Przykład: Wyrażenia stałe

□ Relacja **Lubi(nazwisko, towar)** :

```
SELECT nazwisko,  
       'lubi chleb' AS ktoLubiChleb  
FROM Lubi  
WHERE towar = 'chleb';
```

Wynik zapytania

nazwisko	ktoLubiChleb
Xiński	lubi chleb
Abacki	lubi chleb
...	...

Definiowanie schematu bazy danych

- Schematem bazy danych są deklaracje relacji (tabel) wchodzących w skład bazy danych.
- Elementami schematu bazy danych są również widoki, indeksy, procedury składowane, wyzwalane itd...

Tworzenie (deklarowanie) relacji

- Najprostsza forma ma postać:

```
CREATE TABLE <nazwa> (  
  <lista elementów> );
```

Gdzie pojedynczy element to para nazwa atrybutu i jego typ

- Usuwanie relacji:

```
DROP TABLE <nazwa>;
```

Przykład: tworzenie relacji

```
CREATE TABLE Sprzedaje (  
    sklep          CHAR(20),  
    towar          VARCHAR(20),  
    cena          FLOAT  
);
```


Deklaracja kluczy

- Atrybut lub lista atrybutów może mieć przypisaną właściwość PRIMARY KEY lub UNIQUE.
- Obydwie właściwości oznaczają klucze pierwotne relacji.

Klucze składające się z pojedynczych atrybutów

- Dla takich kluczy właściwość PRIMARY KEY lub UNIQUE umieszcza się po deklaracji typu atrybutu.
- Przykład:

```
CREATE TABLE Towary (  
    nazwa CHAR(20) UNIQUE,  
    producent CHAR(20)  
);
```

Klucze wieloatrybutowe

- Deklaracja klucza może być również osobnym elementem w wyrażeniu `CREATE TABLE`.
- Takiej składni używa się w przypadkach, kiedy na klucz składa się więcej niż jeden atrybut.

Przykład: klucz wieloatrybutowy

- Sklep i towar są razem kluczem pierwotnym dla relacji Sprzedaje:

```
CREATE TABLE Sprzedaje (  
    sklep      CHAR(20),  
    towar      VARCHAR(20),  
    cena       FLOAT,  
    PRIMARY KEY (sklep, towar)  
);
```

Różnice pomiędzy właściwościami PRIMARY KEY i UNIQUE

- Standard SQL pozwala implementatorom DBMS na wprowadzanie własnych różnic pomiędzy PRIMARY KEY i UNIQUE.
- Przykładowo niektóre DBMS mogą automatycznie tworzyć indeks dla atrybutów z właściwością PRIMARY KEY, ale nie z UNIQUE.

Różnice wymagane przez standard SQL

1. W definicji relacji może istnieć tylko jeden PRIMARY KEY, natomiast właściwość UNIQUE może mieć kilka atrybutów.
2. Żaden z atrybutów z właściwością PRIMARY KEY nie może przyjmować wartości NULL. Dla właściwości UNIQUE jest to dopuszczalne.

Pozostałe właściwości atrybutów

1. Właściwość NOT NULL oznacza, że dany atrybut nie może przyjąć tej wartości.
2. Właściwość DEFAULT <wartość> powoduje nadawanie domyślnej wartości danemu atrybutowi.

Przykład: wartości domyślne

```
CREATE TABLE Klienci (  
    imie CHAR(30) PRIMARY KEY,  
    adres CHAR(50)  
        DEFAULT 'nieznany',  
    telefon CHAR(16)  
);
```



Efekty stosowania wartości domyślnych --- (1)

- Przykładowo do bazy należy dodać klienta Franek ,ale nie znany jest jego adres ani telefon.
- Dzięki zastosowaniu składni INSERT z listą atrybutów uzyskujemy:

```
INSERT INTO Klienci(imie)  
VALUES ( ' Franek' ) ;
```



Efekty stosowania wartości domyślnych --- (2)

- Jaka krotka zostanie dopisana do relacji Klienci?

imie	adres	telefon
Franek	nieznany	NULL

- Jeżeli atrybut telefon miałby właściwość NOT NULL operacja dodawania zostałaby odrzucona.

Dodawanie atrybutów

- Dodawanie nowych atrybutów(kolumn) do schematu relacji możliwe jest poprzez zastosowanie składni:

```
ALTER TABLE <nazwa> ADD  
<deklaracja atrybutu>;
```

Przykład:

```
ALTER TABLE Sklepy ADD  
telefon CHAR(16) DEFAULT 'nieznany';
```

Usuwanie atrybutów

- Wymaga zastosowania składni:

```
ALTER TABLE <nazwa>
```

```
DROP <atrybut>;
```

- Przykład: usunięcie atrybutu licencja:

```
ALTER TABLE Sklepy DROP  
parking;
```

SQL – typy danych - 1

Nazwa	Rozmiar	Zakres	Uwagi
CHAR(n) NCHAR(n)	n znaków	1 .. 32768 bajtów, 1 znak 1 .. 3 bajty	Stała długość pola, 'ATR'
VARCHAR(n) NVARCHAR(n)	N znaków	tjw	Zmienna długość pola
INTEGER	32 bity	-2147483648 .. 2147483647	212
SMALLINT	16 bitów	-32768 .. 32767	Tjw
FLOAT	32 bity	$1.175 \cdot 10^{-38} \dots 3.402 \cdot 10^{38}$ Uwaga na precyzję!	3.1415
DOUBLE PRECISION	64 bity	$2.225 \cdot 10^{-308} \dots 1.797 \cdot 10^{308}$	tjw

SQL – typy danych - 2

Nazwa	Rozmiar	Zakres	Uwagi
DECIMAL (precyzja,skala)	16,32,64,128 ... bity - zmienny	precyzja = 0..255 – <u>minimalna</u> liczba przechowywanych cyfr skala = 0..31 – liczba cyfr po przecinku (<= precyzji)	2.877 DECIMAL(10,3) – pppppppp.sss
NUMERIC (precyzja,skala)	Tjw	Precyzja tjw, określa <u>dokładną</u> liczbę przechowywanych cyfr	tjw
VARBINARY (BLOB)	zmienny		

SQL – typy danych -3

Nazwa	Rozmiar	Zakres	Uwagi
DATE	3 bajty	01.01.0001 .. 31.12.9999 (1 dzień)	'mm-dd-yy' – np '09-23-05',
TIME	5 bajtów	0:00 .. 23:59.99999999 (100ns)	'hh:mm:ss' – np '22:53:02'
DATETIME	8 bajtów	Tjw	Czas z datą

Konwersje typów danych ukryte i jawne

From:	binary	varbinary	char	varchar	nchar	nvarchar	datetime	smalldatetime	date	time	datetimeoffset	datetime2	decimal	numeric	float	real	bigint	int(INT4)	smallint(INT2)	tinyint(INT1)	money	smallmoney	bit	timestamp	uniqueidentifier	image	ntext	text	sql_variant	xml	CLR UDT	hierarchyid
binary																																
varbinary																																
char																																
varchar																																
nchar																																
nvarchar																																
datetime																																
smalldatetime																																
date																																
time																																
datetimeoffset																																
datetime2																																
decimal																																
numeric																																
float																																
real																																
bigint																																
int(INT4)																																
smallint(INT2)																																
tinyint(INT1)																																
money																																
smallmoney																																
bit																																
timestamp																																
uniqueidentifier																																
image																																
ntext																																
text																																
sql_variant																																
xml																																
CLR UDT																																
hierarchyid																																

- Explicit conversion
- Implicit conversion
- Conversion not allowed
- * Requires explicit CAST to prevent the loss of precision or scale that might occur in an implicit conversion.
- Implicit conversions between xml data types are supported only if the source or target is untyped xml. Otherwise, the conversion must be explicit.

SQL – konwersje typów danych

Konwersji dokonuje się operatorem CAST (kompatybilne ze standardami ANSI)

CAST(source-value AS destination-type)

CAST(count AS float)

lub CONVERT (MS SQL)

CONVERT (data_type [(length)], expression [,style])

CONVERT(nvarchar(10), OrderDate, 101)

SQL - operatory

- Arytmetyczne

+ , - , * , / , (,)

- Znakowe

|| - łączenie ciągów znaków, np. 'miasto:' || MIASTA

- Logiczne

Operator	Opis	Przykład
NOT	Zaprzeczenie wyrażenia logicznego	... WHERE NOT (zawod IS NULL)
AND	Logiczne „i”	... WHERE A=1 AND B=1
OR	Logiczne „lub”	... WHERE A=1 OR B=1

Operator IN

- Warunek logiczny **<krotka> IN <relacja>** ma wartość PRAWDA tylko jeśli krotka należy do relacji.
 - Istnieje również operator **<krotka> NOT IN <relacja>**.
- Operator IN stosuje się w klauzuli WHERE.
- <relacja> jest często podzapytaniem.

Przykład

- Znajdź nazwę i producenta wszystkich towarów, które lubi Kowalski, relacje **Towary(nazwa, producent)** i **Lubi(nazwisko, towar)**.

```
SELECT *
```

```
FROM Towary
```

```
WHERE nazwa IN (
```

```
SELECT towar  
FROM Lubi  
WHERE nazwisko = 'Kowalski');
```

Zbiór towarów,
które lubi
Kowalski



Operator EXISTS

- **EXISTS(<relacja>)** zwraca PRAWDĘ jeśli <relacja> nie jest pusta.
- Przykład: Znajdź te towary z relacji **Towary(nazwa, producent)** , które są jedynymi towarami produkowanymi przez swojego producenta.

Przykładowe zapytanie z EXISTS

```
SELECT nazwa  
FROM Towary t1  
WHERE NOT EXISTS(  

```

Zbiór towarów
takiego
samego
producenta
jak p1

```
SELECT *  
FROM Towary  
WHERE producent = t1.producent  
AND  
nazwa <> t1.nazwa);
```



Operator ANY

- $x = \text{ANY}(\langle \text{relacja} \rangle)$ zwraca PRAWDĘ jeśli x jest równe przynajmniej jednej krotce z relacji.
- Operator $=$ może być zastąpiony przez dowolny operator porównania.
- Przykład: $x \geq \text{ANY}(\langle \text{relacja} \rangle)$ oznacza że x nie jest najmniejszą krotką w relacji.
 - Krotki muszą mieć tylko jeden atrybut.

Operator ALL

- Podobnie, $x \neq \text{ALL}(\langle \text{relacja} \rangle)$ jest PRAWDĄ jeśli dla każdej krotki k z relacji, x jest różne od k .
 x nie należy do relacji.
- Operator $\neq$ można zastąpić dowolnym operatorem porównania.
- Przykład: $x \geq \text{ALL}(\langle \text{relacja} \rangle)$ oznacza że w relacji nie ma krotki większej niż x .

Przykład

- W relacji **Sprzedaje**(sklep, towar, cena),
znajdź towar (towary) sprzedawane po
najwyższej cenie.

```
SELECT towar  
FROM Sprzedaje  
WHERE cena >= ALL(  
  SELECT cena  
  FROM Sprzedaje);
```

Cena z zewnętrznego
Sprzedaje musi być
nie mniejsza niż
dowolna cena.



SQL – operatory porównania

Operator	Opis	Przykład
=	Dwa wyrażenia są równe	... WHERE placa=1000
!=, <>, ^=	Dwa wyrażenia są różne	... WHERE placa <> 1000
>, <, >=, <=	Dwa wyrażenia większe , mniejsze	... WHERE placa > 1000
[NOT] BETWEEN x AND y	Wyrażenie [nie] większe lub równe od x i mniejsze lub równe y	... WHERE placa BETWEEN 1000 AND 2000
IS [NOT] NULL	Wyrażenie [nie] jest równe NULL	... WHERE zawod IS NULL

Wyrażenia złożone w klauzuli WHERE

- Relacja **Sprzedaje**(sklep, towar, cena),
znajdź ile kosztuje chleb w „Sklepie X”:

```
SELECT cena
FROM Sprzedaje
WHERE sklep = 'Sklep X' AND
      towar = 'chleb';
```

Szablony

- W klauzuli WHERE można stosować warunki, w których wartość atrybutu tekstowego porównywana jest z szablonem.
 - <Atrybut> LIKE <szablon>
lub
 - <Atrybut> NOT LIKE <szablon>
- Szablon jest stałą tekstową ze znakami specjalnymi
 - % = "dowolny napis";
 - _ = "dowolna litera"

Przykład

- Relacja **Klienci(nazwisko, adres, telefon)**
Znaleźć klientów z numerem kierunkowym 52:

```
SELECT nazwisko  
FROM Klienci  
WHERE telefon LIKE '%52- _ _ _ _ _ _ _ _ _ _';
```



Wartości NULL

- Logika w wyrażeniach SQL jest logiką 3-wartościową: TRUE, FALSE, UNKNOWN.
- Porównanie jakiegokolwiek wartości z NULL, daje w wyniku wartość UNKNOWN.
- Zapytania zwracają krotkę tylko, jeżeli w wyniku klauzuli WHERE otrzymuje się TRUE (a nie UNKNOWN)

Logika 3-wartościowa

- Dla uproszczenia można przyjąć, że:
 $\text{TRUE} = 1, \text{FALSE} = 0, \text{UNKNOWN} = \frac{1}{2}.$
- $\text{AND} = \text{MIN}; \text{OR} = \text{MAX}, \text{NOT}(x) = 1-x.$
- Przykład:
$$\begin{aligned} &\text{TRUE AND (FALSE OR NOT(UNKNOWN))} \\ &= \text{MIN}(1, \text{MAX}(0, (1 - \frac{1}{2}))) = \\ &\text{MIN}(1, \text{MAX}(0, \frac{1}{2})) = \text{MIN}(1, \frac{1}{2}) = \frac{1}{2}. \end{aligned}$$

Przykład

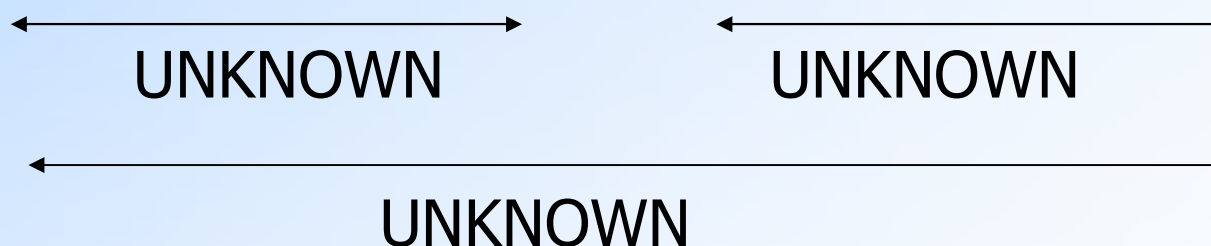
□ Relacja Sprzedaje:

sklep	towar	cena
Sklep A	chleb	NULL

SELECT towar

FROM Sprzedaje

WHERE cena < 2.00 OR cena >= 2.00;



Zapytania z wielu relacji

- Wybranie kilku relacji w jednym zapytaniu następuje poprzez ich wymienienie w klauzuli FROM.
- Atrybuty o takich samych nazwach rozróżnia się poprzez "`<relacja>.<atrybut>`"

Przykład

- Używając relacji **Lubi(nazwisko, towar)** i **Odwiedza(nazwisko, sklep)**, znajdź towary, które są lubiane przez co najmniej jedną osobę przychodzącą do sklepu 'Sklep A'.

```
SELECT towar
FROM Lubi, Odwiedza
WHERE sklep = 'Sklep A' AND
Odwiedza.nazwisko = Lubi.nazwisko;
```



Alias nazw relacji

- Czasami w zapytaniu konieczne jest zastosowanie dwóch kopii tej samej relacji.
- Każdej z relacji w zapytaniu można nadać nową nazwę (alias).

Przykład

- Relacja **Towary(nazwa, producent)**, znajdź wszystkie pary nazw towarów wybranego producenta.
 - Nie twórz par typu (chleb, chleb).
 - Pary muszą być uporządkowane alfabetycznie, np. (bułka, chleb), a nie (chleb, bułka).

```
SELECT t1.nazwa, t2.nazwa
FROM Towary t1, Towary t2
WHERE t1.producent = t2.producent AND
t1.nazwa < t2.nazwa;
```



Podzapytania

- Wyrażenie SELECT-FROM-WHERE (*podzapytanie*) może być używane jako wartość w wielu miejscach zapytania głównego, w szczególności w klauzulach FROM i WHERE.
- Np. w miejsce relacji w klauzuli FROM można umieścić inne zapytanie (podzapytanie).
 - Przydatna jest w takich przypadkach technika stosowania aliasów.

Podzapytania zwracające jedną krotkę

- Jeśli jest zagwarantowane, że podzapytanie zwróci jedną krotkę, to takie podzapytanie może być używane jak **wartość**.
- W takim przypadku krotka ma **jeden atrybut**.
- Jeżeli brak krotek, lub jest ich więcej niż jedna następuje **błąd wykonania zapytania**.

Przykład

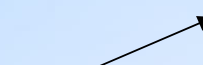
- Z relacji **Sprzedaje**(sklep, towar, cena),
znajdź sklepy sprzedające **kajzerki** w
takiej samej cenie jak sklep '**Sklep A**'
sprzedaje **bułki**.
- Wymaga to zastosowania dwóch
zapytań:
 1. Znajdź cenę kajzererek w sklepie 'Sklep A'.
 2. Znajdź sklepy sprzedające bułki w tej cenie.

Rozwiązanie z podzapytaniem

```
SELECT sklep  
FROM Sprzedaje  
WHERE towar = 'bułka' AND
```

```
cena = (SELECT cena  
        FROM Sprzedaje  
        WHERE sklep = 'Sklep A'  
        AND towar = 'kajzerka');
```

Cena w jakiej
'Sklep A' sprzedaje
kajzerki



Eliminacja duplikatów

- Duplikaty eliminuje się poprzez zastosowanie składni

SELECT DISTINCT . . .

- W relacji **Sprzedaje(sklep, towar, cena)** znajdź wszystkie różne ceny:

```
SELECT DISTINCT cena  
FROM Sprzedaje;
```

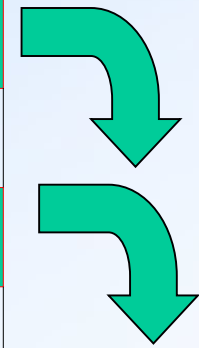
- Bez klauzuli DISTINCT, każda cena pojawiłaby się tyle razy, ile było par bar/piwo o danej wartości.



Operacje relacyjne - selekcja

Inaczej - podzbiór poziomy

PESEL	IMIE	NAZWISKO	WYKSZT
72091811928	Wojciech	Kowalski	WY
55013098922	Adam	Nowak	SR
12324232324	Hieronim	Rycki	PO
89898772772	August	Abacki	SR
73963876399	Włodzimierz	Cabacki	WY



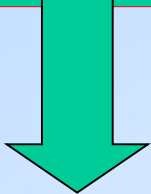
```
SELECT * FROM pracownicy WHERE WYKSZT='SR'
```



Operacje relacyjne - projekcja

Inaczej - podzbiór pionowy

PESEL	IMIE	NAZWISKO	WYKSZT
72091811928	Wojciech	Kowalski	WY
55013098922	Adam	Nowak	SR
12324232324	Hieronim	Rycki	PO
89898772772	August	Abacki	SR
73963876399	Włodzimierz	Cabacki	WY


`SELECT PESEL, WYKSZT FROM pracownicy`

Operacje relacyjne – suma (unia)

Pozwala na zsumowanie zbioru krotek 2 relacji (tabel)

pracownicy

PESEL	IMIE	NAZWISKO	WYKSZT
72091811928	Wojciech	Kowalski	WY
55013098922	Adam	Nowak	SR



zarząd

PESEL	IMIE	NAZWISKO	WYKSZT
89898772772	August	Abacki	SR
73963876399	Włodzimierz	Cabacki	WY

```
SELECT * FROM pracownicy  
UNION  
SELECT * FROM zarząd
```

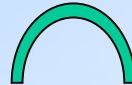


Operacje relacyjne - przekrój

Pozwala na znalezienie części wspólnej - iloczynu zbioru krotek 2 relacji (tabel)

PESEL	IMIE	NAZWISKO	WYKSZT
72091811928	Wojciech	Kowalski	WY
55013098922	Adam	Nowak	SR

pracownicy



PESEL	IMIE	NAZWISKO	WYKSZT
89898772772	August	Abacki	SR
73963876399	Włodzimierz	Kowalski	WY

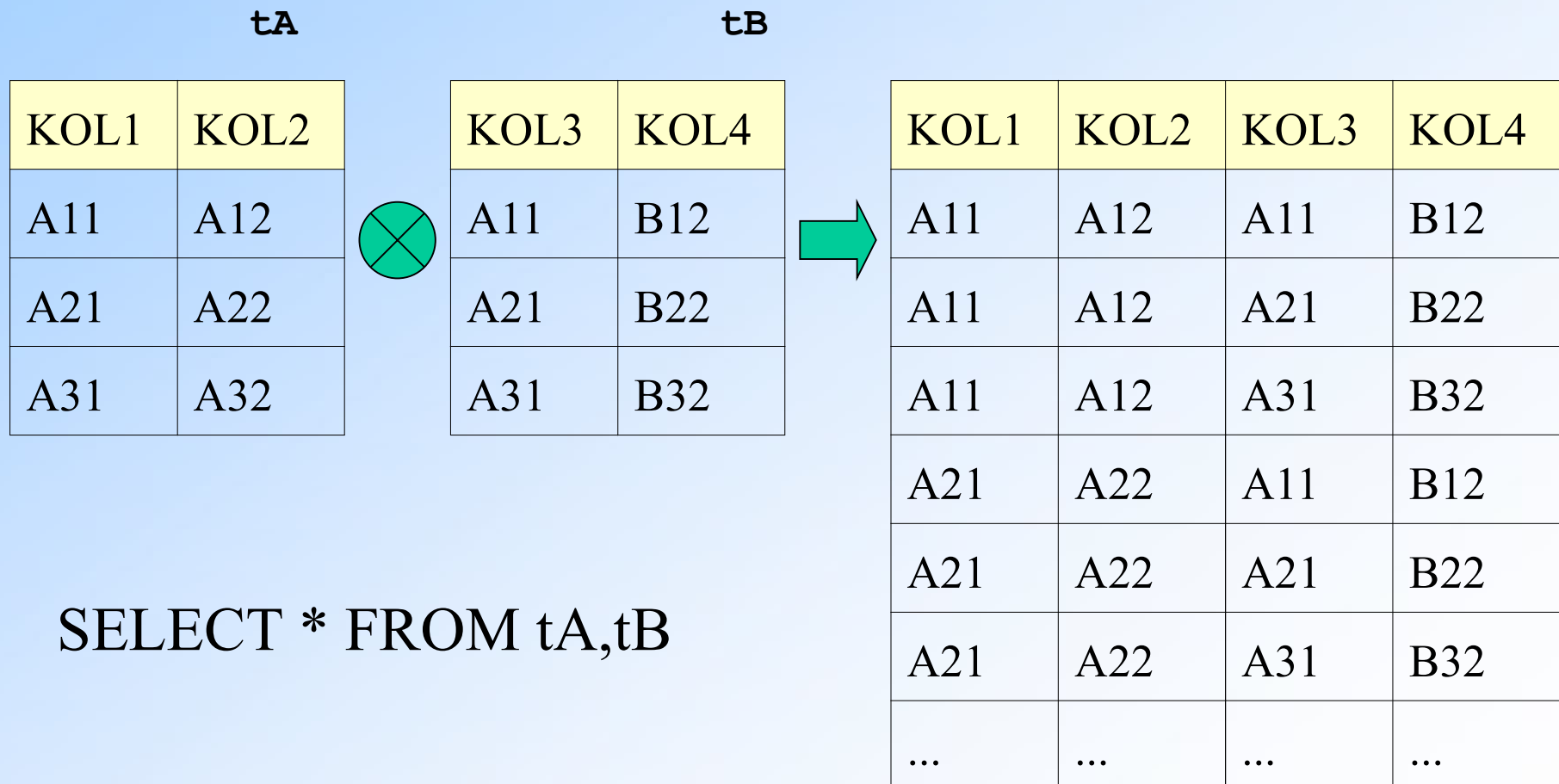
zarząd

```
SELECT Nazwisko FROM pracownicy  
INTERSECT  
SELECT Nazwisko FROM zarząd
```

Operacje relacyjne - złączenia

Pozwalają na łączenie krotek 2 relacji (tabel)

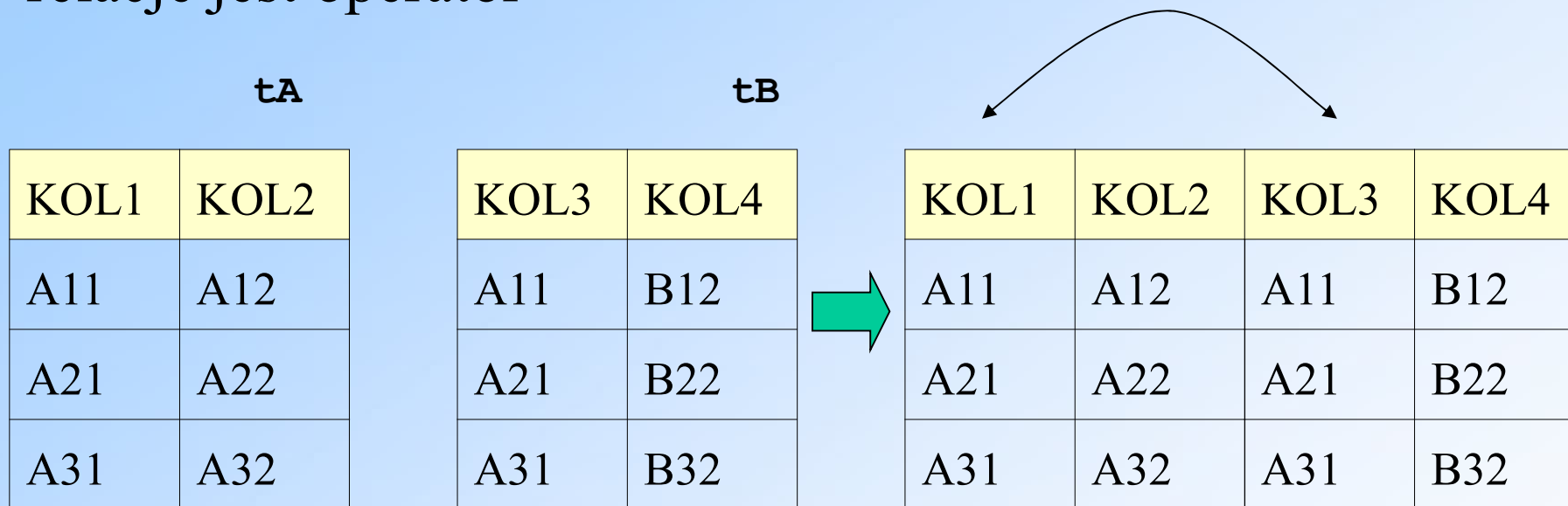
Iloczyn kartezjański – każda krotka z pierwszej relacji łączona jest z każdą krotką z drugiej relacji



SELECT * FROM tA,tB

Operacje relacyjne – równozłączenia

Równozłączenie (equi –join) – warunkiem łączącym 2 relacje jest operator =



SELECT * FROM tA,tB where tA.KOL1= tB.KOL3

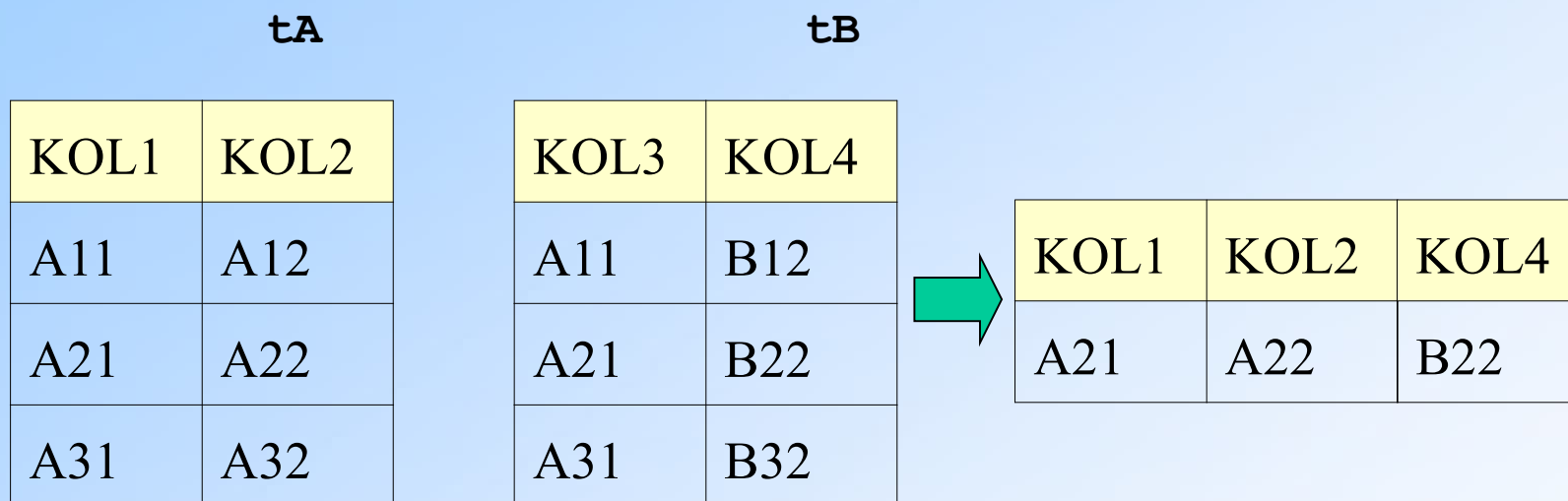
równoważne

SELECT * FROM tA JOIN tB ON tA.KOL1= tB.KOL3



Operacje relacyjne – równozłączenia ₂

Zawężanie wyników złączeń – zastosowanie projekcji i selekcji



**SELECT KOL1,KOL2,KOL4 FROM tA,tB WHERE
tA.KOL1= tB.KOL3 AND KOL1='A21'**

**SELECT KOL1,KOL2,KOL4 FROM tA JOIN tB ON
tA.KOL1= tB.KOL3 WHERE KOL1='A21'**

Operacje relacyjne – złączenie wewnętrzne

Domyślnym rodzajem złączeń są złączenia wewnętrzne

tA		tB				
KOL1	KOL2	KOL3	KOL4			
A11	A12	A11	B12			
A21	A22	A21	B22			
A31	A32	A43	B32			
		A21	B11			



KOL1	KOL2	KOL4
A11	A12	B12
A21	A22	B22
A21	A22	B11

**SELECT KOL1,KOL2,KOL4 FROM tA JOIN tB ON
tA.KOL1= tB.KOL3**

JOIN = INNER JOIN

Operacje relacyjne – złączenie zewnętrzne

tA		tB					
KOL1	KOL2	KOL3	KOL4		KOL1	KOL2	KOL4
A11	A12	A11	B12		A11	A12	B12
A21	A22	A21	B22		A21	A22	B22
A31	A32	A43	B32		A31	A32	NULL
		A21	B11		A21	A22	B11

**SELECT KOL1,KOL2,KOL4 FROM tA
LEFT OUTER JOIN tB ON tA.KOL1= tB.KOL3**

Lewe złączenie zewnętrzne (LEFT OUTER JOIN) zwraca wszystkie wiersze relacji znajdującej się po lewej stronie wyrażenia JOIN, zaś prawe (RIGHT OUTER JOIN)- wszystkie wiersze relacji po prawej stronie wyrażenia JOIN

Operacje relacyjne – pełne złączenie zewnętrzne

tA		tB					
KOL1	KOL2	KOL3	KOL4				
A11	A12	A11	B12				
A21	A22	A21	B22				
A31	A32	A43	B32				
		A21	B11				

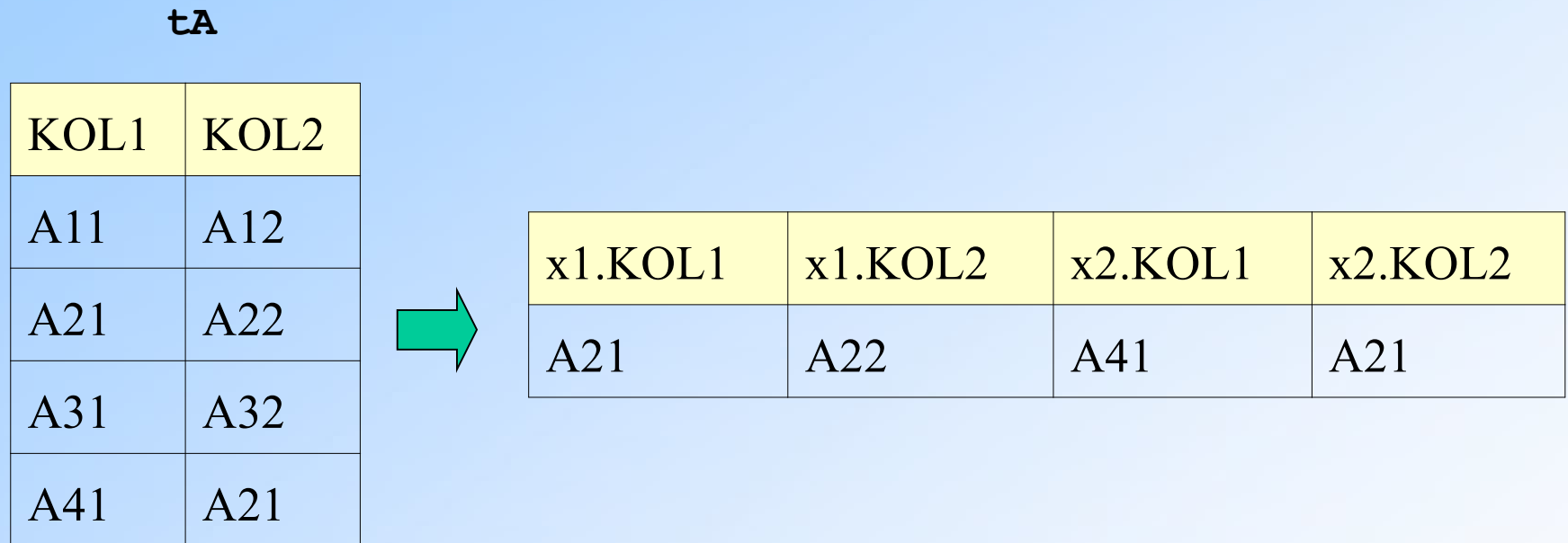


KOL1	KOL2	KOL3	KOL4
A11	A12	A11	B12
A21	A22	A21	B22
A21	A22	A21	B11
A31	A32	NULL	NULL
NULL	NULL	A43	B32

**SELECT * FROM tA
FULL OUTER JOIN tB ON tA.KOL1= tB.KOL3**

Pełne złączenie zewnętrzne (FULL OUTER JOIN) zwraca wszystkie wiersze relacji znajdujących się po obu stronach wyrażenia JOIN

Operacje relacyjne – samozłączenia



SELECT * **tA x1** JOIN **tA x2** ON x1.KOL1= x2.KOL2

Operatory agregowania

- Operatory SUM, AVG, COUNT, MIN, i MAX stosuje się w klauzuli SELECT aby uzyskać wartości zagregowane wybranej kolumny.
- Operator COUNT można zastosować również w postaci COUNT(*) , która służy do zliczenia liczby krotek w relacji.

Przykład: operatory Agregowania

- W relacji **Sprzedaje**(sklep, towar, cena),
znajdź średnią cenę chleba:

```
SELECT AVG(cena)
FROM Sprzedaje
WHERE towar = 'chleb';
```



Eliminacja duplikatów w operatorach agregujących

- Duplikaty eliminuje się operatorem DISTINCT wewnątrz funkcji agregującej.
- Przykład: Znajdź ile różnych cen ma piwo Żywiec:

```
SELECT COUNT(DISTINCT cena)
FROM Sprzedaje
WHERE towar = 'chleb';
```



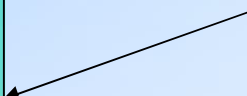
NULL w operatorach agregujących

- NULL nie jest zaliczany do średniej lub sumy, nie może być również minimum lub maksimum danej kolumny.
- Jeżeli nie ma wartości różnych od NULL w kolumnie, to wynikiem funkcji agregującej jest NULL.

Przykład: NULL

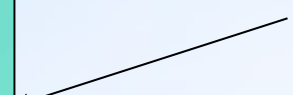
```
SELECT count(*)  
FROM Sprzedaje  
WHERE towar = 'chleb';
```

Liczba sklepów
sprzedających chleb.



```
SELECT count(cena)  
FROM Sprzedaje  
WHERE towar = 'chleb';
```

Liczba sklepów
sprzedających chleb
i mających na niego
ustaloną cenę



Grupowanie

- Na końcu wyrażenia SELECT-FROM-WHERE można dodać klauzulę GROUP BY z listą atrybutów.
- Wynikowa relacja z klauzuli SELECT-FROM-WHERE jest grupowana według wartości wyspecyfikowanych atrybutów, operacje agregujące wykonywane są wewnątrz tych grup.

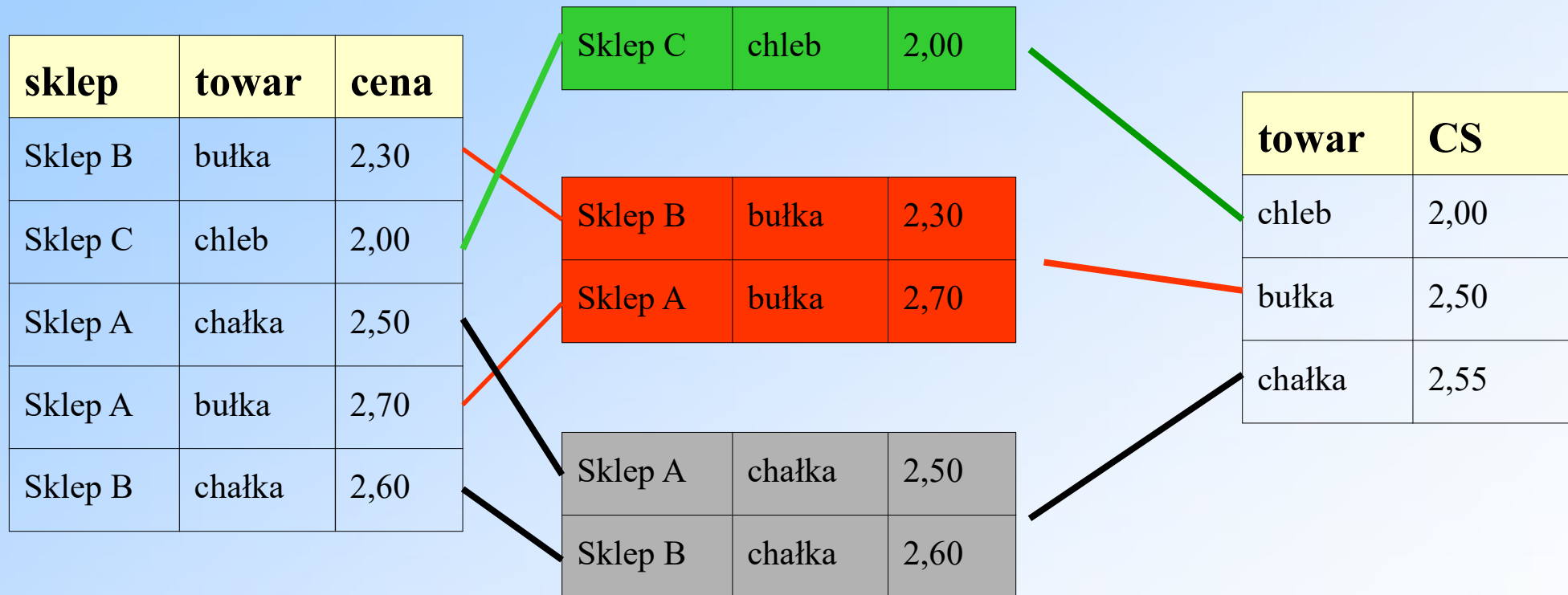
Przykład: Grupowanie

- W relacji **Sprzedaje**(sklep, towar, cena),
znajdź średnią cenę każdego z towarów:

```
SELECT towar, AVG(cena) as CS  
FROM Sprzedaje  
GROUP BY towar;
```



Przykład: Grupowanie



Przykład: Grupowanie

- Z relacji **Sprzedaje**(sklep, towar, cena) i **Odwiedza**(nazwisko, bar), znajdź dla każdego klienta średnią cenę chleba w sklepach które on odwiedza:

```
SELECT nazwisko, AVG(cena)
FROM Odwiedza, Sprzedaje
WHERE towar = 'chleb' AND
      Odwiedza.sklep = Sprzedaje.sklep
GROUP BY nazwisko;
```



Ograniczenia klauzuli SELECT w operacjach grupowania

- Jeżeli w klauzuli SELECT używa się grupowania to wynikowe kolumny muszą być:
 1. Wynikiem funkcji agregującej, lub
 2. Atrybutem wymienionym w liście GROUP BY.

Błędne zapytanie

- Znajdź sklep, który sprzedaje chleb najtaniej :

```
SELECT sklep, MIN(cena)  
FROM Sprzedaje  
WHERE towar = 'chleb';
```

Klauzule HAVING

- Klauzulę **HAVING** <warunek> umieszcza się po klauzuli GROUP BY.
- Grupy nie spełniające podanego warunku są eliminowane z relacji wynikowej.

Przykład: HAVING

- Z relacji **Sprzedaje(sklep, towar, cena)** i **Towary(nazwa, producent)** wyszukaj średnią cenę tych towarów, które są sprzedawane przynajmniej w 2 sklepach, lub są produkowane w 'Piekarnia B'.

Rozwiązanie

```
SELECT towar, AVG(cena)  
FROM Sprzedaje  
GROUP BY towar
```

Grupy z przynajmniej 2
sklepami lub produkowane w
Piekarni B.

```
HAVING COUNT(sklep) >= 2 OR  
    towar IN (SELECT nazwa  
              FROM Towary  
              WHERE producent = 'Piekarnia B');
```



Wymagania obowiązujące warunki HAVING

- Warunki mogą dotyczyć dowolnej relacji z klauzuli FROM.
- Mogą dotyczyć atrybutów tych relacji pod warunkiem że są to:
 1. Atrybuty po których następuje grupowanie, lub
 2. Atrybuty zagregowane.

SQL – składnia

Stosowana notacja:

[el1] – stosowanie elementu el1 jest opcjonalne

element1 | element2 – zastosować należy element 1 albo element2

<pojęcie ogólne> - „**pojęcie ogólne**” rozwinięte jest poniżej

nazwa – dowolna nazwa wprowadzona przez użytkownika

... – powtarzanie pewnego składnika

SQL – SELECT

```
SELECT [DISTINCT | ALL] { * | <val> [, <val> ... ] } FROM  
<tableref> [, <tableref> ... ]  
[WHERE <srchcond>]  
[GROUP BY col [, col ... ]]  
[HAVING <srchcond>]  
[UNION [ALL] <sel_expr>]  
[ORDER BY <order_list>]
```

<val> = nazwa_kol | stala | wyraz

<tableref> = <joined_table> | tabela [alias]

<joined_table> = <tableref> <join_type> <tableref> ON
<srchcond>

<join_type> = [INNER] JOIN | { {LEFT|RIGHT|FULL} [OUTER] }
JOIN

<srchcond> = wyr. zwracające FALSE lub TRUE



SQL – SELECT ₂

```
<order_list> = kolumna [COLLATE stała_sort] [ASC[ENDING]  
| DESC[ENDING]] [, <order_list> ...]
```

Przykład:

```
SELECT DISTINCT nazwisko FROM uczniowie  
WHERE wiek BETWEEN 10 AND 15  
ORDER BY nazwisko DESC
```

```
SELECT nazwisko, COUNT(*) AS c FROM uczniowie  
GROUP BY nazwisko  
HAVING c > 10  
ORDER BY nazwisko ASC
```

Operacje modyfikacji danych

- Komendy modyfikujące dane nie zwracają żadnych wyników (w przeciwieństwie do zapytań).
- Istnieją trzy rodzaje modyfikacji:
 1. *Insert* - dodawanie krotek.
 2. *Delete* - usuwanie krotek.
 3. *Update* - modyfikowanie wartości istniejących krotek.

Dodawanie krotek

- W celu dodania pojedynczej krotki stosuje się składnię:

```
INSERT INTO <relacja>  
VALUES ( <lista wartości> );
```

- Przykład:

Należy dodać do relacji **Lubi(nazwisko, towar)** fakt, że Abacki lubi chleb.

```
INSERT INTO Lubi  
VALUES ( 'Abacki' , 'chleb' );
```


Specyfikowanie atrybutów w operacji INSERT

- Oprócz nazwy relacji można wyspecyfikować listę atrybutów.
- Powody:
 1. Uniknięcie konieczności pamiętania standartowej kolejności atrybutów.
 2. Nie przypisujemy wartości wszystkim atrybutom, pozwalamy systemowi na wypełnienie pozostałych atrybutów wartościami domyślnymi.

Przykład: Specyfikowanie atrybutów

- Inny sposób dodania do bazy faktu, że Franek lubi Tatra

```
INSERT INTO Lubi (towar, nazwisko)  
VALUES ('chleb', 'Abacki');
```

Dodawanie wielu krotek

- Istnieje możliwość dodania do relacji krotek będących wynikiem zapytania. Służy do tego składnia:
INSERT INTO <relacja>
(<podzapytanie>);

Przykład: wstawianie wyniku podzapytania

- Za pomocą relacji **Odwiedza(nazwisko, sklep)**, dodaj do nowej relacji **PotencjalniZanjomi(nazwisko)** wszystkich „potencjalnych znajomych” Abackiego t.j., tych klientów, którzy odwiedzili przynajmniej jeden ze sklepów odwiedzanych przez Abackiego.

Rozwiązanie

INSERT INTO PotencjalniZanjomi

(SELECT d2.nazwisko

FROM Odwiedza d1, Odwiedza d2
WHERE d1.nazwisko = 'Abacki' AND
d2.nazwisko <> 'Abacki' AND
d1.sklep = d2.sklep

);



Usuwanie

- Usuwania krotek spełniających pewien warunek umożliwia zastosowanie składni:

```
DELETE FROM <relacja>  
WHERE <warunek>;
```

Przykład: usuwanie

- Usuń z relacji **Lubi(nazwisko, towar)** informację o tym, że Dabacki lubi bułki:

```
DELETE FROM Lubi  
WHERE nazwisko = 'Dabacki' AND  
towar = 'bułka';
```

Przykład: usunięcie wszystkich krotek

- Relacja Lubi będzie pusta:

```
DELETE FROM Lubi;
```

- W tym wypadku nie jest konieczna klauzula WHERE

Przykład: usuwanie wielu krotek

- Usuń z relacji Towary(nazwa,producent) towary w ten sposób ,aby dla każdego producenta pozostało co najwyżej jeden towar.

```
DELETE FROM Towary t
WHERE EXISTS (
    SELECT nazwa FROM Towary
    WHERE producent = p.producent AND
    nazwa <> p.nazwa);
```

Semantyka usuwania --- (1)

- Załóżmy że 'Piekarnia A' produkuje tylko bułki i chleb.
- Załóżmy, że pierwszą analizowaną krotką k jest 'bułka'.
- Podzapytanie nie jest puste, ponieważ istnieje krotka 'chleb', zatem krotka 'bułka' zostanie usunięta.
- W kolejnym kroku krotką bieżącą k stanie się 'chleb' czy zostanie ona usunięta?

Semantyka usuwania --- (2)

- Odpowiedź: TAK.
- Powodem tego jest przeprowadzanie operacji kasowania w 2 etapach:
 1. Zaznacz wszystkie krotki spełniające warunek po WHERE.
 2. Usuń zaznaczone krotki.

Aktualizacje

- Do zmiany wartości określonych atrybutów w określonej krotce relacji stosuje się składnię:

UPDATE <relacja>

SET <lista przyporządkowań atrybutów>

WHERE <warunek>;

Przykład: Update

- Zmień numer telefonu Bolka na 55-12-12:

```
UPDATE Klienci  
SET nrTel = '55-12-12'  
WHERE imie = 'Bolek';
```



Przykład: aktualizacja kilku krotek

- Ustaw maksymalną cenę towarów w bazie na 4 zł:

```
UPDATE Sprzedaje  
SET cena = 4.00  
WHERE cena > 4.00;
```



Widoki

- Widok jest „wirtualną tabelą” – relacją definiowaną zawartością innych tabel lub widoków.

Stosuje się je do:

- Ograniczania dostępu do tabel w bazie danych
- Uproszczenia zapytań kierowanych do systemu
- Zapewnienia niezależności danych wewnątrz bazy danych

Widoki przechowywane są w bazie danych

- Deklarowane są składnią:

```
CREATE VIEW <nazwa> AS <zapytanie>;
```

Przykład: Definicja widoku

- **OdwiedzaneSklepy(nazwisko, towar)** jest widokiem zawierającym pary klient-towar na podstawie informacji o sklepach uczęszczanych przez danego klienta:

```
CREATE VIEW OdwiedzaneSklepy AS  
SELECT nazwisko, towar  
FROM Odwiedza, Sprzedaje  
WHERE Odwiedza.sklep = Sprzedaje.sklep;
```



Przykład: używanie widoków

- Zapytania z użyciem widoków wykonuje się tak samo jak z tabelami.

```
SELECT towar FROM OdwiedzaneSklepy  
WHERE nazwisko = 'Abacki';
```



SQL – CREATE TABLE

CREATE TABLE tabela (<def_kol>[,<def_kol> ...])

<def_kol> = nazwa {<typ_danych>|

COMPUTED [BY] (<wyr>) }

[DEFAULT {stała | NULL}]

[NOT NULL]

<typ_danych> = dowolny z typów danych

<wyr> = dowolne, prawidłowe wyrażenie SQL zwracające pojedynczą wartość

Przykład:

```
CREATE TABLE uczen ( imie CHAR(20),  
                      nazwisko VARCHAR(40) NOT NULL  
                      wiek SMALLINT DEFAULT 2 )
```

SQL – INSERT

```
INSERT INTO tabela (kol1 [, kol2 ...])  
{ VALUES (<war> [, <war> ...]) | <wyr_select> }
```

<war>= {stała | <wyrażenie> | **NULL**}

<wyrażenie> = poprawne wyrażenie SQL zwracające pojedynczą wartość

Przykład:

```
INSERT INTO uczen ( imie, nazwisko) VALUES ('Wojciech','Kowalski')
```

SQL – COUNT

COUNT (* | [ALL] kol | DISTINCT kol)

kol – nazwa kolumny lub wyrażenie

* – zlicza liczbę wierszy razem z wartościami NULL

ALL – zlicza ilość wartości różnych od NULL w wybranej kolumnie

DISTINCT – zlicza w wybranej ilość wartości niepowtarzalnych i różnych od NULL

Przykład:

```
SELECT COUNT(DISTINCT miasto) FROM dane_osobowe;
```

SQL – MAX, MIN, AVG

MAX (kol)

MIN (kol)

AVG ([ALL] kol | DISTINCT kol)

kol – nazwa kolumny lub wyrażenie

MAX, MIN – jeśli nie ma kwalifikujących się do zliczania wierszy zwraca NULL

ALL – zlicza średnią wszystkich wartości

DISTINCT – przed wyznaczeniem średniej eliminuje duplikaty

Generatory

- Generatory umożliwiają nadawanie unikalnych wartości polom kluczowym – szczególnie ważne w środowisku wielodostępowym
- Tworzone są instrukcją SQL **CRATE SEQUENCE** lub **GENERATOR**
- Pobieranie kolejnych, unikalnych wartości umożliwia funkcja **GEN_ID** lub **NEXT VALUE FOR**
- Możliwe jest też ustawienie generatora na żadaną wartość – **SET GENERATOR**
- **-ALTER SEQUENCE nazwa RESTART WITH wartosc;**

```
INSERT INTO pracownicy(id_prac,nazwisko) VALUES  
(GEN_ID(gen_idprac,1),'Kowalski')
```

SQL – Generator

CREATE GENERATOR nazwa

W Firebird generatory generują 64-bitowe liczby całkowite, dlatego też pole kluczowe musi być typu DECIMAL(18,0)

Przykład:

CREATE GENERATOR gen_IDPrac;

GEN_ID (nazwa, krok)

SET GENERATOR nazwa **TO** wartosc



Indeksy

Pole1	Pole2
	1
	6
	3
	8
	4

SELECT * FROM test WHERE pole2 BETWEEN 3 AND 6

- Indeksy są mechanizmem przyspieszającym wyszukiwanie danych przy pewnych kryteriach zapytania
- Służą także do wymuszania unikalności wartości pól dodawanych rekordów

Indeksy (2)

Kiedy należy tworzyć indeksy

- Jeśli dana kolumna często uczestniczy jako warunek wyszukiwania
- Jeśli kolumna używana jest w złączeniach tabel (JOIN)
- Jeśli kolumna używana jest jako pole sortowane (ORDER BY)

Kiedy nie jest konieczne tworzenie indeksów

- Kolumna jest często aktualizowana
- Zbiór możliwych wartości kolumny jest mały
- Kolumny są rzadko używane jako warunek wyszukiwania

SQL – CREATE INDEX

```
CREATE [UNIQUE] [ASC[ENDING] | DESC[ENDING]] INDEX nazwa  
ON nazwa_tabeli (kol [, kol...])
```

Przykład:

```
CREATE ASC INDEX ind_nazwiska ON pracownicy (nazwisko);
```

Integralność danych

Integralność encji

- Każda tabela musi posiadać klucz główny
- Wartości klucza głównego danej tabeli muszą być unikalne i różne od NULL
- Wartość każdego z pól tabeli musi należeć do zbioru wartości dopuszczalnych dla danego pola np.: ocena 2..6

Integralność referencyjna

- Każda wartość klucza obcego musi być równa jednej z wartości klucza głównego lub może być równa NULL

Integralność danych – mechanizmy DBMS₍₁₎

Mechanizmy zapewniające integralność encji

- Indeksowanie kluczy głównych
- Definiowalne restrykcje (*constraints*) definiujące zakres dopuszczalnych wartości pól

Integralność danych – mechanizmy DBMS₍₂₎

Usuwanie lub edycja klucza głównego

Nowoczesne DBMS posiadają wbudowane mechanizmy zapewniające zachowanie w takich przypadkach integralności referencyjnej.

- Aktualizacja i usuwanie kaskadowe
- Usunięcie i edycja klucza głównego jest zabroniona dopóki istnieją powiązane z nim klucze obce
- Unieważnione wartości kluczy obcych zastępowane są wartością NULL lub wartością domyślną

SQL – ALTER TABLE

ALTER TABLE tabela <operacja>[,<operacja> ...]

<operacja> = { **ADD** <def_kol>

| **ADD** <restrykcja>

| **DROP** kolumna

| **DROP CONSTRAINT** restrykcja }

<def_kol> = nazwa_kol {<typ_danych> | ... tj. w CREATE TABLE }

<restrykcja> = [**CONSTRAINT** nazwa_r]

{{ **PRIMARY KEY** | **UNIQUE** } (nazwa_kol[,nazwa_kol ...])

| **FOREIGN KEY** (nazwa_kol[,nazwa_kol ...]) **REFERENCES**

nazwa_tabeli

[**ON DELETE** {NO ACTION|CASCADE|SET DEFAULT|SET NULL}]

[**ON UPDATE** {NO ACTION|CASCADE|SET DEFAULT|SET NULL}]

| **CHECK** (<warunek>)

SQL – ALTER TABLE₍₂₎

NO ACTION – zmiana wartości klucza pierwotnego nie powoduje żadnych operacji na kluczu obcym, jednak wykonywane jest sprawdzanie integralności referencyjnej, w przypadku jej niedotrzymania operacja jest anulowana

CASCADE – zmiana wartości klucza pierwotnego powoduje zmianę wartości klucza obcego, usunięcie klucza pierwotnego powoduje usunięcie klucza obcego

SET NULL – zmiana wartości lub usunięcie klucza pierwotnego powoduje zmianę wartości klucza obcego na NULL

SET DEFAULT – tjw. z tym, że nową wartością klucza obcego będzie wartość domyślna

Kolumna, która ma być kluczem pierwotnym musi posiadać restrykcję NOT NULL

Dla danej tabeli może istnieć tylko jedna definicja PRIMARY KEY i dowolna ilość definicji UNIQUE

SQL – ALTER TABLE₍₃₎

Przykład:

```
ALTER TABLE PANSTWA  
    ADD STOLICA VARCHAR(30) NOT NULL  
DROP WALUTA;
```

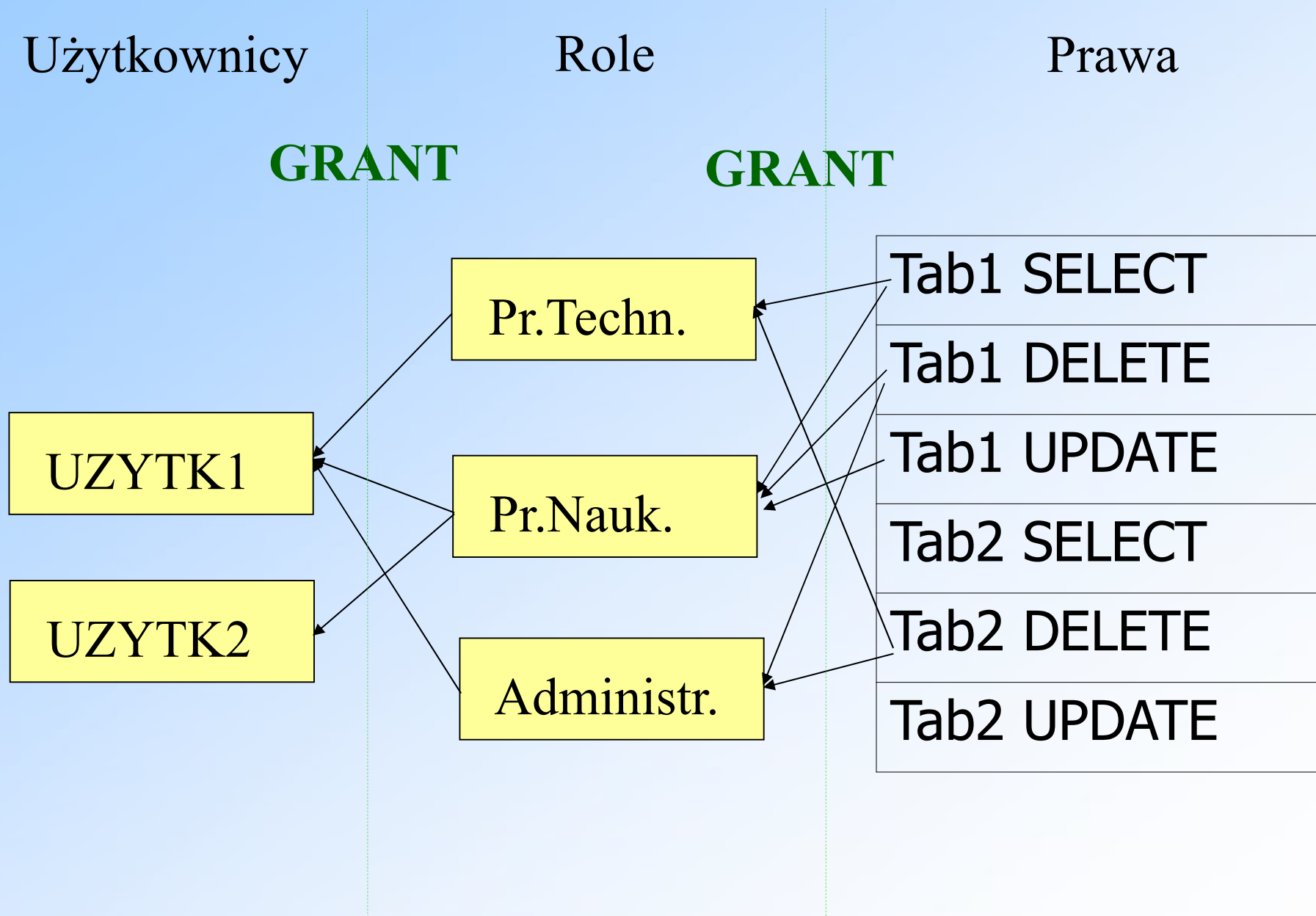
```
ALTER TABLE DZIENNIK  
ADD OCENA NUMERIC(2,1)  
DEFAULT 2  
NOT NULL  
CHECK (OCENA BETWEEN 1 AND 6);
```

```
ALTER TABLE KSIAZKI  
ADD CONSTRAINT K1 PRIMARY KEY (ID);
```

```
ALTER TABLE WYPOZYCZONE  
ADD CONSTRAINT K2 FOREIGN KEY (ID) REFERENCES  KSIAZKI ON  
DELETE CASCADE ON UPDATE CASCADE;
```



Kontrolowanie dostępu do danych



SQL – CREATE ROLE

CREATE ROLE nazwa;

DROP ROLE nazwa;

Przykład:

CREATE ROLE pr_techniczni ;

SQL – GRANT

```
GRANT { <uprawnienia> ON [TABLE] {nazwa_tabeli} TO
<lista_rol> }
| {<rola> TO { PUBLIC | <lista_uzytk> } }
| EXECUTE ON PROCEDURE nazwa TO <lista_rol>

<uprawnienia>= ALL | SELECT | DELETE | INSERT |
UPDATE [(kol[,kol ...])]

<lista_rol>= nazwa_rol [, nazwa_rol ...] [WITH GRANT
OPTION]

<lista_uzytk>= nazwa_uzytk [, nazwa_uzytk ...] [WITH
ADMIN OPTION]
```

Przykład:

```
GRANT ALL ON TABLE ksiazki TO obsluga_tech WITH GRANT OPTION ;
GRANT obsluga_tech TO Wlodek, Waldek;
```

Transakcje

Transakcja – sekwencja logicznie powiązanych operacji na bazie danych przeprowadzających bazę danych z jednego do drugiego stanu spójnego

Np: Przelew bankowy

Transakcja

- 1) Zmniejszenie stanu konta 1
- 2) Zwiększenie stanu konta 2



AWARIA /
WSPÓLBIEŻNOŚĆ

Transakcje₍₂₎

Przelew kwoty z konta A na konto B

```
SET TRANSACTION;
```

```
UPDATE konta SET stan = stan - N
```

```
WHERE id_konta=A;
```

```
UPDATE konta SET stan = stan + N
```

```
WHERE id_konta=B;
```

```
COMMIT;
```

Transakcje -ACID

Każda transakcja posiada cechy:

Niepodzielności (*Atom ic*) - wykonane są wszystkie elementy składowe transakcji albo żaden – zbiór operacji wchodzących w skład transakcji jest niepodzielny.

Spójności (*Consistent*) – transakcja przeprowadza bazę z jednego stanu spójnego do innego stanu spójnego, transakcja nie może naruszać ograniczeń integralnościowych.

Transakcje -ACID

Kolejne cechy:

Izolacji (*Isolation*) – transakcje są od siebie logicznie odseparowane, oddziałują na siebie poprzez dane. Mimo współbieżnego wykonywania, transakcje widzą stan bazy danych tak, jak gdyby były wykonywane w sposób sekwencyjny.

Trwałości (*Durability*) – wyniki zatwierdzonych transakcji nie mogą zostać utracone w wyniku wystąpienia awarii systemu. Zatwierdzone dane w bazie danych, w przypadku awarii, muszą być odtwarzalne.

Transakcje logiczne i fizyczne

```
SET TRANSACTION;
```

```
UPDATE towary SET cena =  
cena*0.9 WHERE  
typ_tow='buty';
```

```
COMMIT;
```

transakcja logiczna

```
begin_transaction;
```

```
Read(X);
```

```
.....
```

```
Write(Y);
```

```
Read(Z);
```

```
commit;
```

transakcja fizyczna

Współbieżność - problemy

- Problem utraconej modyfikacji (*dirty read*)

Czas (T)	Transakcja A	Transakcja B	Rekord 21 Mjsc
0	Odczyt rekordu 21 (Mjsc=10)		Mjsc=10
1	Mjsc=Mjsc+1	Odczyt rekordu 21 (Mjsc=10)	Mjsc=10
2	Zapis rekordu 21 (Mjsc)	Mjsc=Mjsc+5	Mjsc=11
3	Zatwierdź	Zapis rekordu 21 (Mjsc)	Mjsc=15
4		Zatwierdź	Mjsc=15

Zapobieganie – zablokowanie innym transakcjom dostępu do rekordu 21 aż do zatwierdzenia transakcji A

Współbieżność - problemy

- Problem niezatwierdzonej zależności

Czas (T)	Transakcja A	Transakcja B	Rekord 21 Mjsc
0	Odczyt rekordu 21 (Mjsc=10)		Mjsc=10
1	Mjsc=Mjsc+1		Mjsc=10
2	Zapis rekordu 21 (Mjsc)		Mjsc=11
3		Odczyt rekordu 21 (Mjsc=11)	Mjsc=11
4	Wycofaj	Mjsc=Mjsc+5	Mjsc=10
5		Zapis rekordu 21 (Mjsc)	Mjsc=16
6		Zatwierdź	Mjsc=16

Zapobieganie – ?

Współbieżność - problemy

- Problem niespójności analizy – błędna suma

Czas (T)	Transakcja A	Transakcja B	Suma	Rekord 20 Mjsc	Rekord 21 Mjsc
0	Odczyt rekordu 20 (Mjsc=10)	Suma=0	0	Mjsc=10	Mjsc=15
1	Mjsc=Mjsc+1	Odczyt rekordu 20 (Mjsc=10)	0	Mjsc=10	Mjsc=15
2	Zapis rekordu 20 (Mjsc)	Suma = suma + 10	10	Mjsc=11	Mjsc=15
3	Zatwierdź	Odczyt rekordu 21 (Mjsc=15)	10	Mjsc=11	Mjsc=15
4		Suma=suma + 15	25	Mjsc=10	Mjsc=15

- Problem niepowtarzalnego odczytu (*unrepeateble read*)

Mechanizmy odtwarzania - awarie

Możliwe awarie można podzielić na:

- transakcyjne
- systemowe
- nośników

Najczęstsze przyczyny awarii

-awaria komputera (błąd sprzętowy, sieciowy bądź programowy w czasie wykonywania transakcji), awarie sprzętowe to najczęściej awarie nośników (dyski, pamięć główna)

-błędy transakcyjne – dzielenie przez 0, błędne parametry transakcji, błędy logiczne, przerwanie transakcji przez użytkownika

Mechanizmy odtwarzania - awarie

Najczęstsze przyczyny awarii cd.

-wymuszenie sterowania współbieżnego – zastosowana metoda sterowania współbieżnego przerywa transakcję

-problemy fizyczne – brak zasilania, awarie chłodzenia, pożar, włamanie ..

Dziennik transakcji

Menadżer transakcji zapisuje historię transakcji wykonywanych na bazie danych – **dziennik transakcji**, w którym przechowywane są wpisy:

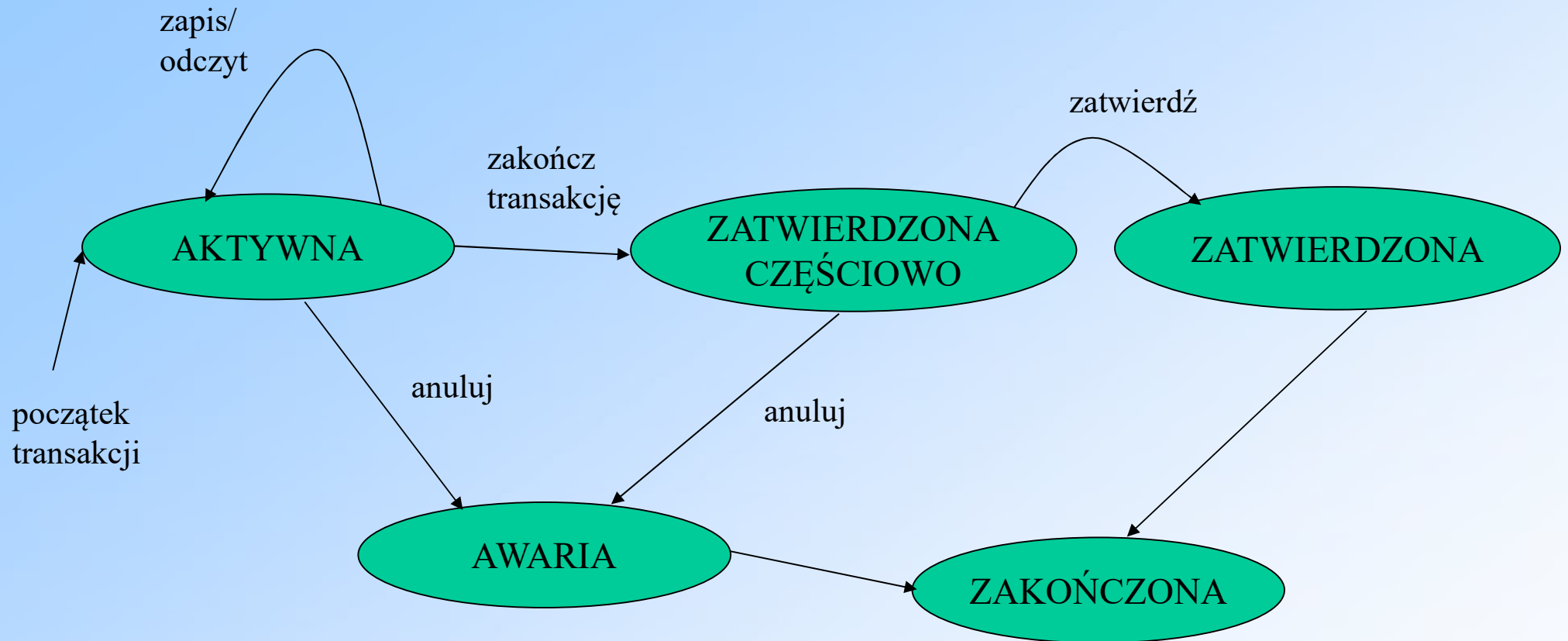
- *początek transakcji T*
- *zapis elementu, T , x , stara wartość, nowa wartość*
- *odczyt elementu, T , x*
- *zatwierdź T*
- *anuluj T*

Dziennik transakcji

Dzięki *dziennikowi transakcji* po awarii systemu możliwe jest odtworzenie prawidłowego stanu bazy danych poprzez:

- cofnięcie efektów operacji w transakcjach nie zatwierdzonych (brak wpisu *zatwierdź* lub *anuluj*)
- ponowne wykonanie transakcji zatwierdzonych na podstawie wpisów z dziennika

Diagram stanów transakcji



Harmonogramy transakcji

Harmonogram (*schedule*) S zbioru n transakcji $T_1, T_2, \dots, T_n$ jest uporządkowaniem operacji transakcji podlegającym ograniczeniu określającym, że dla każdej transakcji T_i w S kolejność jej operacji jest **taka sama** jak w T_i .

Skrócona notacja opisu harmonogramów:

litery ***r, w, c, a*** dla operacji **odczyt, zapis, zatwierdź, anuluj**

S_1 :

$r_A(21,10); w_A(21,11);$

$r_B(21,11); a_A;$

$w_B(21,16); c_B$

Czas (T)	Transakcja A	Transakcja B	Rekord 21 Mjsc
0	Odczyt rekordu 21 (Mjsc=10)		Mjsc=10
1	Mjsc=Mjsc+1		Mjsc=10
2	Zapis rekordu 21 (Mjsc)		Mjsc=11
3		Odczyt rekordu 21 (Mjsc=11)	Mjsc=11
4	Wycofaj	Mjsc=Mjsc+5	Mjsc=10
5		Zapis rekordu 21 (Mjsc)	Mjsc=16
6		Zatwierdź	Mjsc=16

Harmonogramy transakcji - operacje konfliktowe

Dwie operacje $o_i(x)$ i $o_j(y)$ w harmonogramie są **konfliktowe** wtedy i tylko wtedy, gdy są spełnione następujące trzy warunki:

- 1) $x = y$ – operacje dotyczą tej samej danej
- 2) $i \neq j$ – operacje konfliktowe muszą należeć do różnych transakcji
- 3) jedna z dwóch operacji musi być operacją zapisu

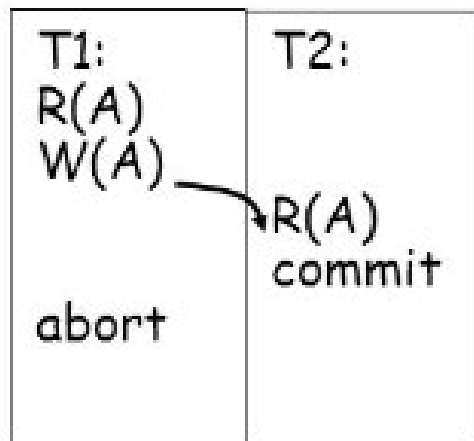
Przykład: Z harmonogramu S_1 operacje $r_A(21,10)$; $w_B(21,16)$ są konfliktowe, a $w_A(21,11)$; $r_A(21,11)$ nie są

Harmonogramy transakcji – harmonogramy pełne

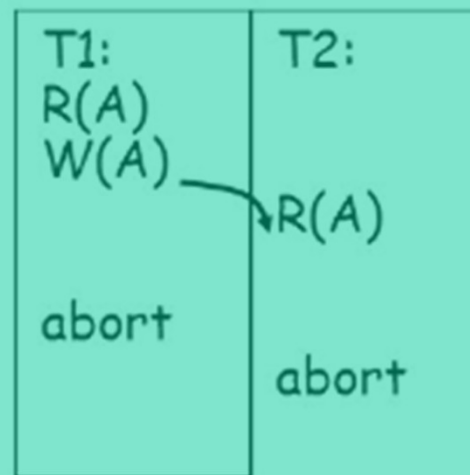
Harmonogram S składający się z n transakcji $T_1, T_2, \dots, T_n$ jest **harmonogramem pełnym** (*total shedule*) jeśli są spełnione warunki:

- 1) Operacje harmonogramu S są dokładnie takimi samymi operacjami jak w transakcjach $T_1, T_2, \dots, T_n$ włączając w to operacje anulowania bądź zatwierdzenia transakcji
- 2) Dla dowolnej pary operacji w transakcji T_i kolejność ich występowania jest taka sama jak w T_i
- 3) Dla dowolnych dwóch operacji konfliktowych musi być określona w harmonogramie kolejność ich występowania

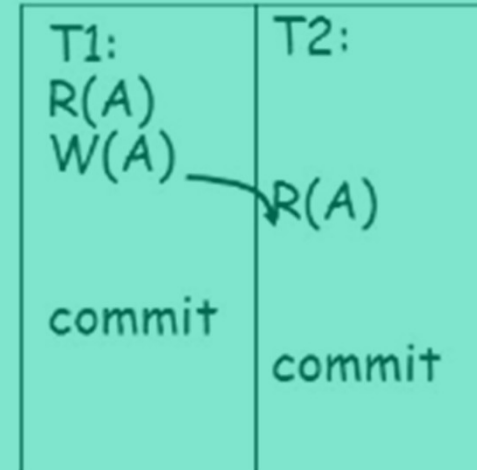
Harmonogramy transakcji



Non-recoverable
schedule



Recoverable schedule
with cascading abort



Recoverable schedule
with commit

Harmonogramy odtwarzalne

Harmonogram S jest **odtworzalny** (*recoverable schedule*), jeśli żadna transakcja T w harmonogramie S nie jest zatwierdzana do momentu, aż wszystkie transakcje T' , które zapisały element x **odczytywany** przez transakcję T , zostaną zatwierdzone.

Transakcja T **czyta** z transakcji T' , jeśli pewien element x jest najpierw zapisywany przez T' a potem odczytywany przez T .

Spełnienie tego warunku powoduje, że po zakończeniu transakcji T już nigdy nie będzie konieczne

Harmonogram S_a jest odtwarzalny mimo, że występuje w nim problem utraty aktualizacji

Przykład:

S_a : $r_1(x)$; $r_2(x)$; $w_1(x)$; $r_1(y)$; $w_2(x)$; c_2 ; $w_1(y)$; c_1 ;

Harmonogramy odtwarzalne

Harmonogram nie odtwarzalny:

$S_c: r_1(x); w_1(x); r_2(x); r_1(y); w_2(x); c_2; a_1;$

Anulowanie transakcji T_1 po operacji c_2 , pociąga za sobą konieczność anulowania już zatwierdzonej transakcji T_2

Poprawka harmonogramu:

$S_d: r_1(x); w_1(x); r_2(x); r_1(y); w_2(x); w_1(y); c_1; c_2;$

W przypadku anulowania T_1 :

$S_d: r_1(x); w_1(x); r_2(x); r_1(y); w_2(x); w_1(y); a_1; a_2;$

Harmonogramy bezkaskadowe

Wycofanie kaskadowe (*cascading rollback*) zachodzi w momencie konieczności wycofania niezatwierdzonej transakcji z powodu niepowodzenia innej transakcji z której dokonano odczytu elementu.

W **harmonogramach bezkaskadowych** zachodzi zjawisko wycofywania kaskadowego, w którym wycofujemy nie tylko elementów zapisanych przez zatwierdzoną transakcję, ale także operacje, które zostały wykonane przez transakcję, która została wycofana.

Operacja $r_2(x)$ musi zostać wstrzymana aż do momentu zatwierdzenia, bądź anulowania transakcji T_1 .

Harmonogram bezkaskadowy (na przykładzie S_d):

S_d : $r_1(x)$; $w_1(x)$; $r_1(y)$; $w_1(y)$; c_1 ; $r_2(x)$; $w_2(x)$; c_2 ;

Harmonogramy ścisłe

W harmonogramach **ścisłych** (*strict schedule*) transakcje nie mogą ani odczytywać, ani zapisywać elementu x do momentu, aż zostanie zatwierdzona (lub anulowana) ostatnia transakcja, która go zapisywała

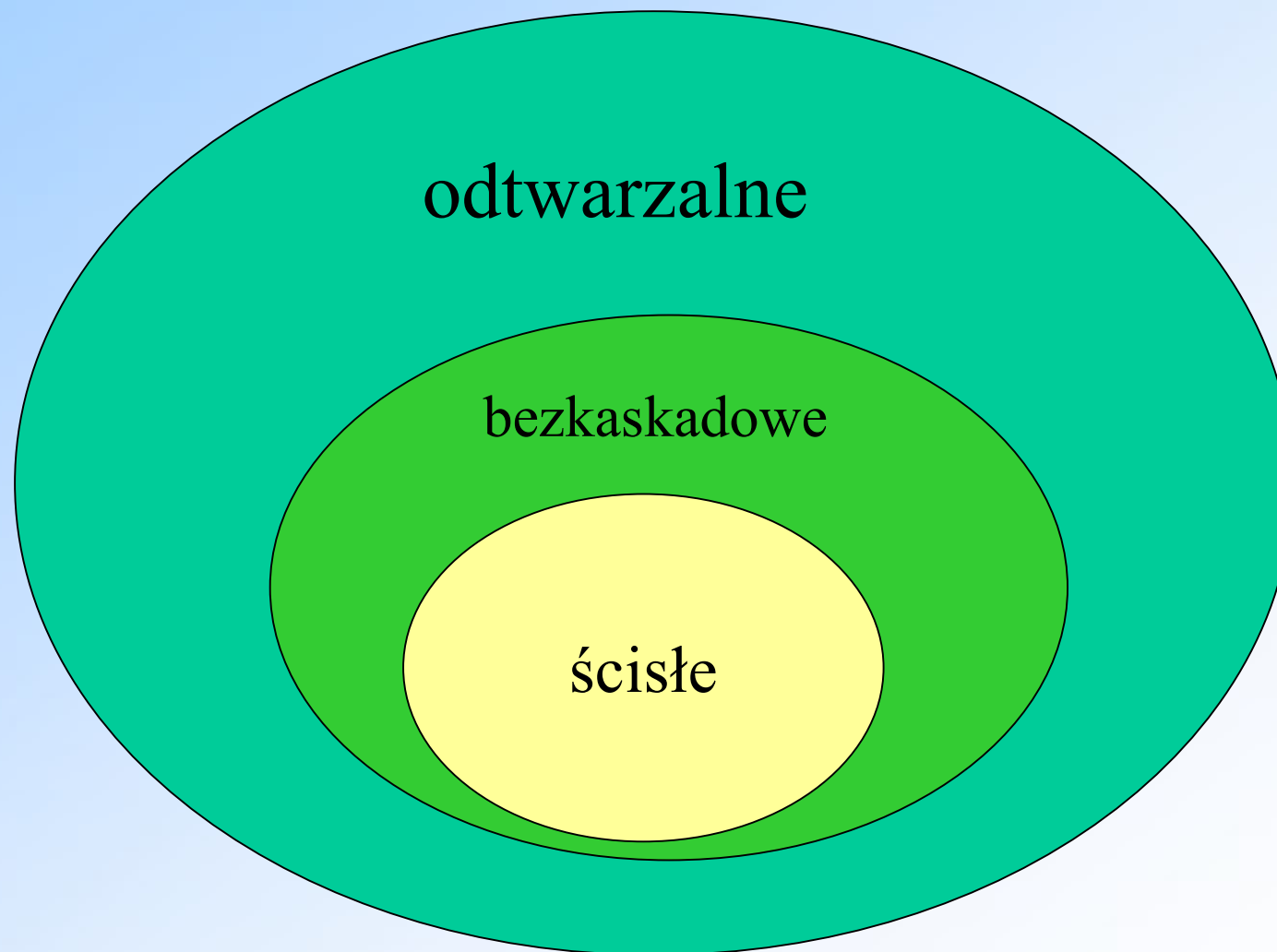
Zaletą harmonogramów ścisłych jest prosta procedura odtwarzania w przypadku cofnięcia operacji polegająca na odtworzeniu starej wartości elementu x

Przykład:

$S_f: w_1(x,5); a_1; r_2(x,8);$

W harmonogramach bezkaskadowych lub odtwarzalnych przywrócenie pierwotnej wartości x z powodu anulowania T_1 prowadzi do błędnych wyników ponieważ wykonana została już operacja w_2 (harmonogram bezkaskadowy, ale nie ścisły)

Klasyfikacja harmonogramów według możliwości odtwarzania



Harmonogramy szeregowe

Harmonogram jest **szeregowym**, jeśli operacje każdej z transakcji należących do harmonogramu są wykonywane po kolei – bez przeplotu z operacjami należącymi do innych transakcji

$x=5$

T_1	T_2
$r(x)$	
$x:=x-2$	
$w(x)$	
$r(y)$	
$y:=y+3$	
$w(y)$	
	$r(x)$
	$x:=x+3$
	$w(x)$

$x=6$

T_1	T_2
	$r(x)$
	$x:=x+3$
	$w(x)$
$r(x)$	
$x:=x-2$	
$w(x)$	
$r(y)$	
$y:=y+3$	
$w(y)$	

-ograniczają współbieżność i przeplot wykonywanych operacji

Harmonogramy szeregowalne

Harmonogram S składający się z n transakcji jest **szeregowalny** (*serializable*), jeśli jest równoważny pewnemu harmonogramowi szeregowemu tych samych n transakcji

T_1	T_2	x
$r(x)$		5
$x := x - 2$		3
	$r(x)$	5
	$x := x + 3$	8
$w(x)$		3
$r(y)$		
	$w(x)$	8
$y := y + n$		
$w(y)$		

T_1	T_2	x
$r(x)$		5
$x := x - 2$		3
$w(x)$		3
	$r(x)$	3
	$x := x + 3$	6
	$w(x)$	6
$r(y)$		
$y := y + n$		
$w(y)$		

Harmonogramy równoważne konfliktowo

Dwa harmonogramy są **równoważne konfliktowo** (*conflict equivalent*) jeśli kolejność dowolnych dwóch operacji konfliktowych jest taka sama w obu harmonogramach

$S_1: \mathbf{r_1(x)}; w_2(x)$

$S_2: w_2(x); \mathbf{r_1(x)}$

wartość x odczytana w operacji $r_1(x)$
może być różna w obu harmonogramach

Harmonogram S jest **szeregowalny konfliktowo** jeśli jest on konfliktowo równoważny pewnemu szeregowemu harmonogramowi S''

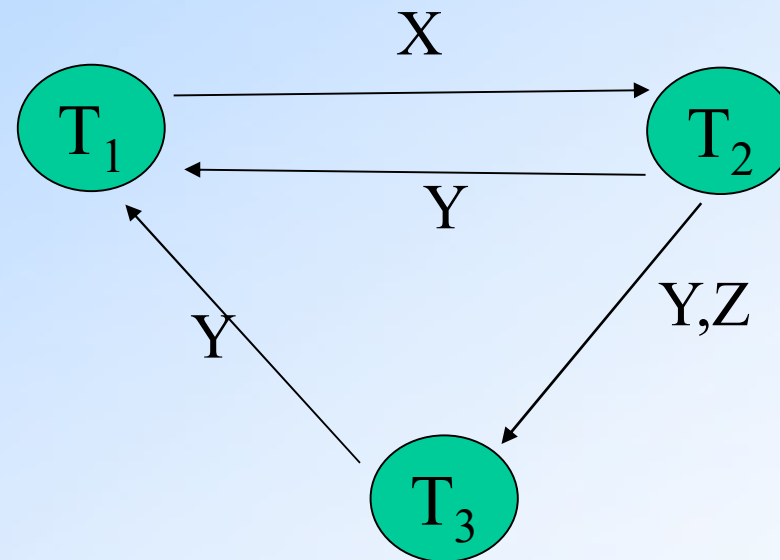
Testowanie szeregowalności konfliktowej harmonogramu

Algorytm testowania:

1. Dla każdej transakcji T_i należącej do harmonogramu S utwórz w grafie pierwszeństwa wierzchołek opatrzony etykietą T_i
2. Dla każdego przypadku gdy transakcja T_j wykonuje operację $r_j(x)$, $w_j(x)$, $w_j(x)$ po wykonaniu przez transakcję T_i operacji odpowiednio $w_i(x)$, $r_i(x)$, $w_i(x)$ utwórz w grafie pierwszeństwa krawędź $(T_i \rightarrow T_j)$
3. Harmonogram **jest konfliktowo szeregowalny**, jeśli graf pierwszeństwa **nie zawiera cykli**

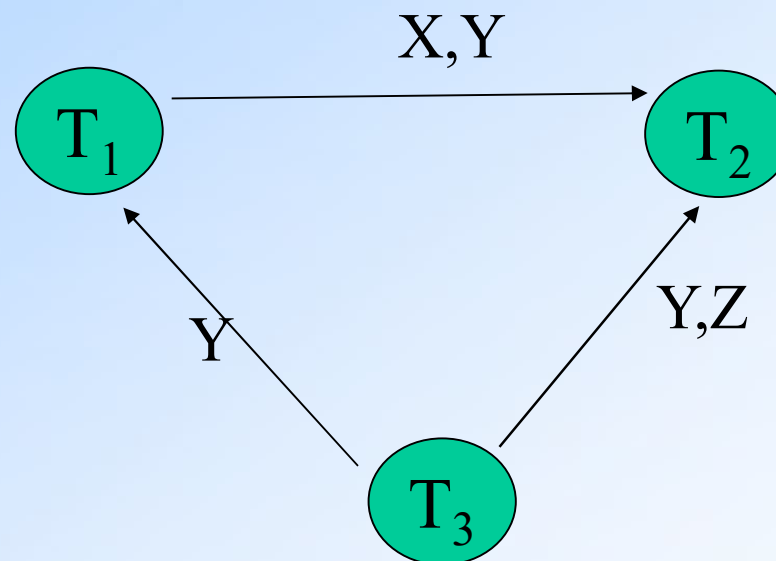
Testowanie szeregowalności konfliktowej harmonogramu

T_1	T_2	T_3
	$r(z)$	
	$r(y)$	
	$w(y)$	
		$r(y)$
		$r(z)$
$r(x)$		
$w(x)$		
		$w(y)$
		$w(z)$
	$r(x)$	
$r(y)$		
$w(y)$		
	$w(x)$	



Testowanie szeregowalności konfliktowej harmonogramu

T_1	T_2	T_3
		$r(y)$
		$r(z)$
$r(x)$		
$w(x)$		
		$w(y)$
		$w(z)$
	$r(z)$	
$r(y)$		
$w(y)$		
	$r(y)$	
	$w(y)$	
	$r(x)$	
	$w(x)$	



Równoważne harmonogramy szeregowe:

$$T_3 \rightarrow T_1 \rightarrow T_2$$

Transakcje w SQL

- Rozpoczynana niejawnie w momencie wystąpienia określonych instrukcji SQL
- Rozpoczynana jest przez **BEGIN**, **START TRANSACTION**, parametry transakcji ustawiane przez **SET TRANSACTION**
- Kończona przez **COMMIT** – zatwierdzenie wszystkich zmian wnoszonych przez transakcję lub **ROLLBACK** – odtworzenie stanu bazy danych sprzed transakcji
- W wielu DBMS dostępny jest tryb automatycznych transakcji – zatwierdzanie po każdej instrukcji SQL

SQL – SET TRANSACTION

SET TRANSACTION [NAME nazwa] [READ WRITE | READ ONLY]
[WAIT | NO_WAIT] [[ISOLATION LEVEL] { SNAPSHOT [TABLE
STABILITY] | READ COMMITTED [[NO] RECORD VERSION]

SNAPSHOT – zmiany dokonane przez konkurujące transakcje nie są widoczne (powtarzalny odczyt), nie można dokonywać zapisu

SNAPSHOT TABLE STABILITY – powtarzalny odczyt jest zapewniany poprzez zabronienie konkurującym transakcjom operacji zapisu do danej tabeli, odczyt jest możliwy

SQL – SET TRANSACTION₍₂₎

READ COMMITTED – transakcja „widzi” ostatnio zatwierdzone zmiany przez transakcje konkurujące, sama może również dokonywać zmian

R.C. NO RECORD VERSION – transakcja czyta ostatnią wersję krotki, jeśli nie jest jeszcze ona zatwierdzona, czeka na zatwierdzenie przez transakcje konkurujące (z opcją WAIT) – wersja domyślna

R.C. RECORD VERSION – transakcja czyta ostatnią zatwierdzoną wersję krotki, nawet jeśli kolejne jej wersje oczekują na zatwierdzenie

Transakcje w SQL

Parametry transakcji:

- Tryb dostępu (*access level*): READ ONLY, READ WRITE
- Poziom izolacji (*isolation level*) :

Poziom izolacji	Rodzaj naruszenia		
	O.zmodyfikowany	O.niepowtarzalny	Fantom
READ UNCOMMITTED	tak	tak	tak
READ COMMITED	nie	tak	tak
REPEATABLE READ	nie	nie	tak
SERIALIZABLE	nie	nie	nie

SQL – ROLLBACK

ROLLBACK [TRANSACTION nazwa]

Przykład:

ROLLBACK TRANSACTION t1

COMMIT [TRANSACTION nazwa]

Przykład:

COMMIT TRANSACTION t1

Zalecane jest zamykanie transakcji dokonujących tylko odczytu komendą COMMIT – lepsza wydajność

Algorytmy zarządzania współbieżnym wykonywaniem transakcji

algorytmy blokowania - uszeregowanie transakcji wynika z kolejności uzyskiwanych blokad (algorytm blokowania dwufazowego – 2PL);

algorytmy znaczników czasowych - uszeregowanie transakcji wynika z wartości znaczników czasowych związanych z transakcjami;

Algorytmy blokowania

Blokada jest zmienną skojarzoną z każdą daną w bazie danych, określającą dostępność danej ze względu na możliwość wykonania na niej określonych operacji. Ogólnie, z każdą daną mamy skojarzoną jedną blokadę.

Ze względu na proces blokowania, dane w bazie danych mogą występować w jednym z trzech stanów:

- **dana nie zablokowana (0),**
- **dana zablokowana dla odczytu R (współdzielona S),**
- **dana zablokowana dla zapisu W (wyłączna X).**

Algorytmy blokowania

Kompatybilność blokad:

Dwie blokady są **kompatybilne** jeśli mogą być założone na tej samej danej przez różne transakcje

Kompatybilne są blokady do odczytu

Zakładanie blokady do odczytu

```
B: if BLOKADA(x) = 'ODBLOKOWANY' then
  begin
    BLOKADA(x) = 'ZABLOKOWANY R' ;
    liczbaOdczytów(x)=1;
  end else
    if BLOKADA(x) = 'ZABLOKOWANY R'
    then liczbaOdczytów(x) = liczbaOdczytów(x) +1;
    else begin
      czekaj (dopóki BLOKADA(x)='ODBLOKOWANY' i
        menedżer blokad uruchomi transakcję);
      goto B;
    end
```


Zakładanie blokady do zapisu

```
B: if BLOKADA(x) = 'ODBLOKOWANY' then
BLOKADA(x) = 'ZABLOKOWANY W' ;
  else begin
    czekaj (dopóki BLOKADA(x)='ODBLOKOWANY' i
    menedżer blokad uruchomi transakcję);
    goto B;
  end
```

Zdejmowanie blokady

```
if BLOKADA(x) = 'ZABLOKOWANY W' then
begin
    BLOKADA(x) = 'ODBLOKOWANY';
    uruchom którąś z oczekujących transakcji;
end else
if BLOKADA(x) = 'ZABLOKOWANY R'
then begin
    liczbaOdczytów(x) = liczbaOdczytów(x) - 1;
    if liczba_odczytów(x)=0 then
    begin
        BLOKADA(x) = 'ODBLOKOWANY';
        uruchom którąś z oczekujących transakcji;
    end;
end;
end;
```

Blokowanie dwufazowe

T_1	T_2
Lr(y)	
r(y)	
U(y)	
	Lr(x)
	r(x)
	U(x)
	Lw(y)
	r(y)
	y:=x+y
	w(y)
	U(y)
Lw(x)	
r(x)	
x:=x+y	
w(x)	
U(x)	

Samo stosowanie blokad odczytu-zapisu nie gwarantuje szeregowalności harmonogramu

Przykład:

wartości początkowe $x=20$, $y=30$,
harmonogram szeregowy T_1, T_2 daje
 $x=50$, $y=80$

Blokowanie dwufazowe

W algorytmie **blokowania dwufazowego** wszystkie operacje blokowania (odczyt, zapis) poprzedzają pierwszą operację odblokowania

Transakcje mają

- **fazę rozszerzania** – sukcesywne pobieranie blokad i
- **fazę kurczenia** – tylko zdejmowanie blokad

Blokowanie dwufazowe

W algorytmie **blokowania dwufazowego** wszystkie operacje blokowania (odczyt, zapis) poprzedzają pierwszą operację odblokowania

Transakcje mają

- **fazę rozszerzania** – sukcesywne pobieranie blokad i
- **fazę kurczenia** – tylko zdejmowanie blokad

Warianty blokowania dwufazowego (*Two-Phase Locking - 2PL*)

Konserwatywne blokowanie dwufazowe (*conservative, static 2PL*)

transakcja blokuje wszystkie potrzebne jej elementy przed rozpoczęciem swojego wykonywania

Ścisłe blokowanie dwufazowe (*strict 2PL*)

transakcja nie zwalnia żadnej ze swoich blokad do zapisu aż do momentu jej zatwierdzenia bądź anulowania – gwarantuje harmonogramy ścisłe

Rygorystyczne blokowanie dwufazowe (*rigorous 2PL*) –

transakcja nie zwalnia żadnej ze swoich blokad aż do momentu jej zatwierdzenia bądź anulowania – gwarantuje ścisłość harmonogramu

Kontrola współbieżności – przykład

- Problem utraconej modyfikacji

Czas (T)	Transakcja A	Transakcja B	Rekord 21 Mjsc
0	Blokada do zapisu (rek. 21)		Mjsc=10
1	Odczyt rekordu 21 (Mjsc=10)	Blokada do zapisu (rek. 21)	Mjsc=10
2	Mjsc=Mjsc+1	Czekaj	Mjsc=11
3	Zapis rekordu 21 (Mjsc)	Czekaj	Mjsc=11
4	Odblokowanie (rek 21)	Czekaj	Mjsc=11
5	Zatwierdź	Odczyt rekordu 21 (Mjsc=11)	Mjsc=11
6		Mjsc=Mjsc+5	Mjsc=11
7		Zapis rekordu 21 (Mjsc)	Mjsc=16
		Odblokowanie (rek 21)	Mjsc=16
		Zatwierdź	Mjsc=16

Kontrola współbieżności - zakleszczenia

- Mimo stosowania blokowania dwufazowego możliwe jest powstanie zakleszczenia pomiędzy dwiema konkurującymi o zasoby transakcjami

Czas (T)	Transakcja A	Transakcja B
0	Blokada do zapisu (rek 1)	
1	Odczyt rek1	Blokada do zapisu (rek 2)
2	Blokada do zapisu (rek 2)	Odczyt rek 2
3	Czekaj	Blokada do zapisu (rek 1)
4	Czekaj	Czekaj
5	Czekaj	Czekaj
6	Czekaj	Czekaj

Kontrola współbieżności - zakleszczenia

Strategie zapobiegania zakleszczeniom

- Transakcja zdobywa wszystkie potrzebne jej blokady przed jej rozpoczęciem – w obciążonym systemie duże transakcje będą długo czekały na rozpoczęcie – **strategia pasywna**
- **Strategia aktywna** – DBMS wykrywa zakleszczenia już po ich wystąpieniu i dopiero wtedy je likwiduje – np poprzez analizę **grafu oczekiwania**

Normalizacja

Reguły projektowania relacyjnych baz danych sformułował w 1970 E.F.Codd – są to tzw. trzy postacie normalne

Przedmiot	Nr Prac	Nazwisko Prac	NrStud enta	Student	Ocena	Typ Oceny
Sys.relac. b.d.	234	Kowalski	37798	IksińskiG.	3	Lab1
Proj.rel.baz dan	234	Kowalski	34698	Abacki J.	5	Spr1
Proj.rel.baz dan	234	Kowalski	34698	Abacki J.	4	Spr2
B.d. w przemyśle	428	Xiński				

1) Usunięcie studenta 34668 = utrata informacji o przedmiocie i powiązonym z nim wykładowcą – tzw. uboczny efekt usunięcia

3) Dodanie nowego studenta jako uczęszczającego na wybrany przedmiot wiąże się z tym, że musi już on mieć przynajmniej jedną ocenę

Pierwsza postać normalna

Relacja jest w pierwszej postaci normalnej (1NF) jeśli wartości atrybutów tej relacji są atomowe (niepodzielne)

NazwaWydz	<u>NrWydzialu</u>	PESELDyrWydz	LokalizacjeWydz
Badawczy	5	78876265	{Olsztyn,Łódź,Bydgoszcz}
Administracja	3	98873773	{Toruń}
Dyrekcja	1	98772762	{Warszawa,Kielce}

Metody doprowadzenia do 1NF:

1. dekompozycja – usunięcie atrybutu wielowartościowego i utworzenie z niego dodatkowej relacji

NazwaWydz	<u>NrWydzialu</u>	PESELDyrWydz
-----------	-------------------	--------------

<u>NrWydzialu</u>	<u>LokalizacjaWydz</u>
-------------------	------------------------

Pierwsza postać normalna

Metody doprowadzenia do 1NF c.d.:

2. rozszerzenie klucza głównego tak, aby każdej lokalizacji odpowiadała jedna krotka z relacji WYDZIAŁY

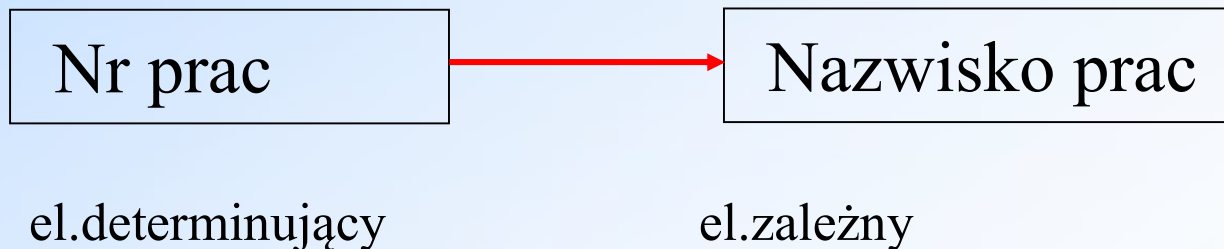
NazwaWydz	<u>NrWydzialu</u>	PESELDyrWydz	<u>LokalizacjeWydz</u>
Badawczy	5	78876265	Olsztyn
Administracja	3	98873773	Toruń
Dyrekcja	1	98772762	Warszawa
Badawczy	5	78876265	Łódź
Badawczy	5	78876265	Bydgoszcz
Dyrekcja	1	98772762	Kielce

3. przy znanej maksymalnej ilości lokalizacji stworzenie dodatkowych atrybutów (LokalizacjeWydz2, LokalizacjeWydz3 ...)

Zależności funkcyjne

- Dwa elementy danych A i B są w **związku zależności** jeśli pewne wartości elementu danych B zawsze występują z pewnymi wartościami elementu A
- **Zależność funkcyjna (jednowartościowa)** –
B jest funkcyjnie zależny od elementu A jeśli dla każdej wartości elementu A istnieje co najwyżej jedna wartość elementu B. Element A jest elementem **determinującym**, a element B jest **zależnym**

np.:



Druga postać normalna

Relacja jest w drugiej postaci normalnej (2NF) wtedy, gdy jest w 1NF i każdy atrybut niepodstawowy jest zupełnie funkcyjnie zależny od każdego klucza relacji

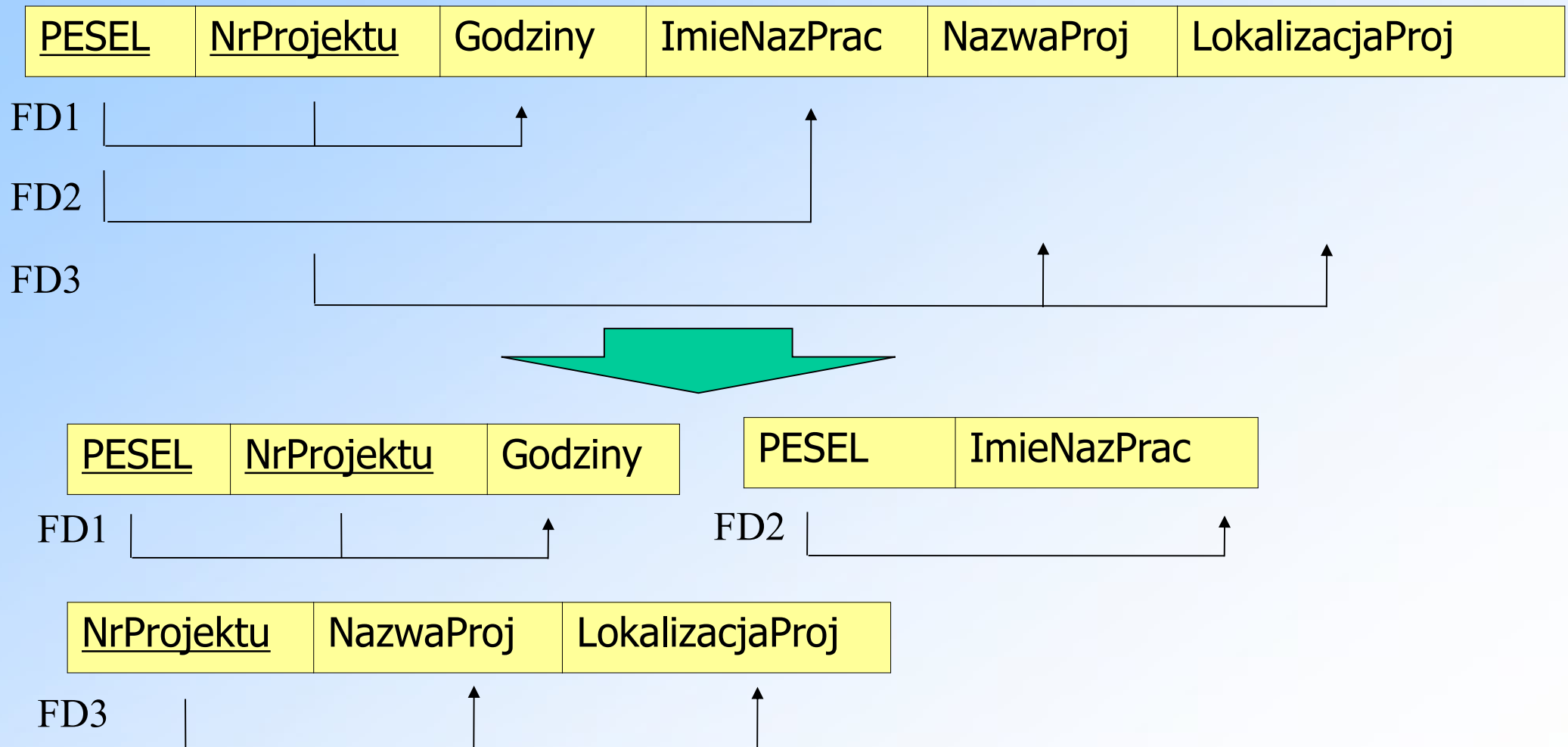
także klucze kandydujące

Atrybut podstawowy – atrybut będący składową pewnego klucza relacji

Zależność funkcyjna $X \rightarrow Y$ jest **zupełną zależnością funkcyjną** jeśli usunięcie dowolnego atrybutu ze zbioru X sprawia, że zależność przestaje obowiązywać

Druga postać normalna

Należy zbadać wszystkie niekluczowe pola w tabelach posiadających klucze złożone czy dane pole nie jest jednoznacznie identyfikowane przez część klucza złożonego

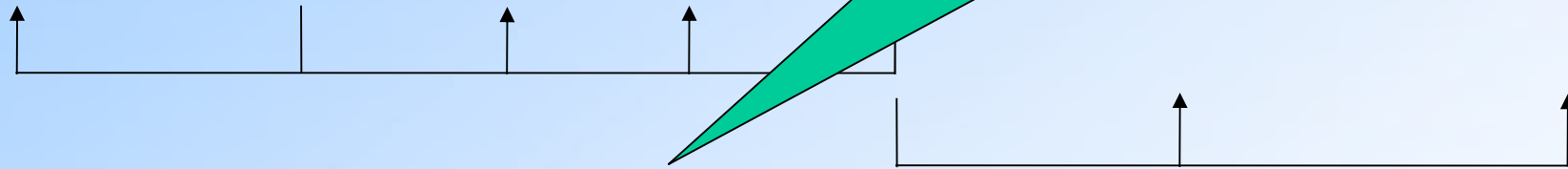


Trzecia postać normalna

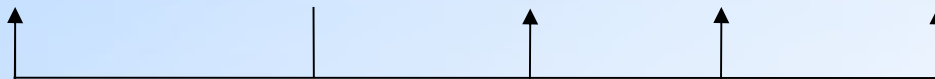
Relacja jest w trzeciej postaci normalnej, jeśli jest w 2NF oraz każdy niekluczowy nieprzechodni zależny od klucza

Test – czy nie ma atrybutów niekluczowych determinujących inne atrybuty niekluczowe

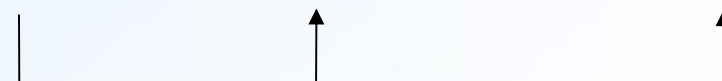
ImieNazwPrac	<u>PESEL</u>	DataUr	Adres	NrWydz	NazwaWydz	PESELDyrWydz
--------------	--------------	--------	-------	--------	-----------	--------------



ImieNazwPrac	<u>PESEL</u>	DataUr	Adres	NrWydz
--------------	--------------	--------	-------	--------

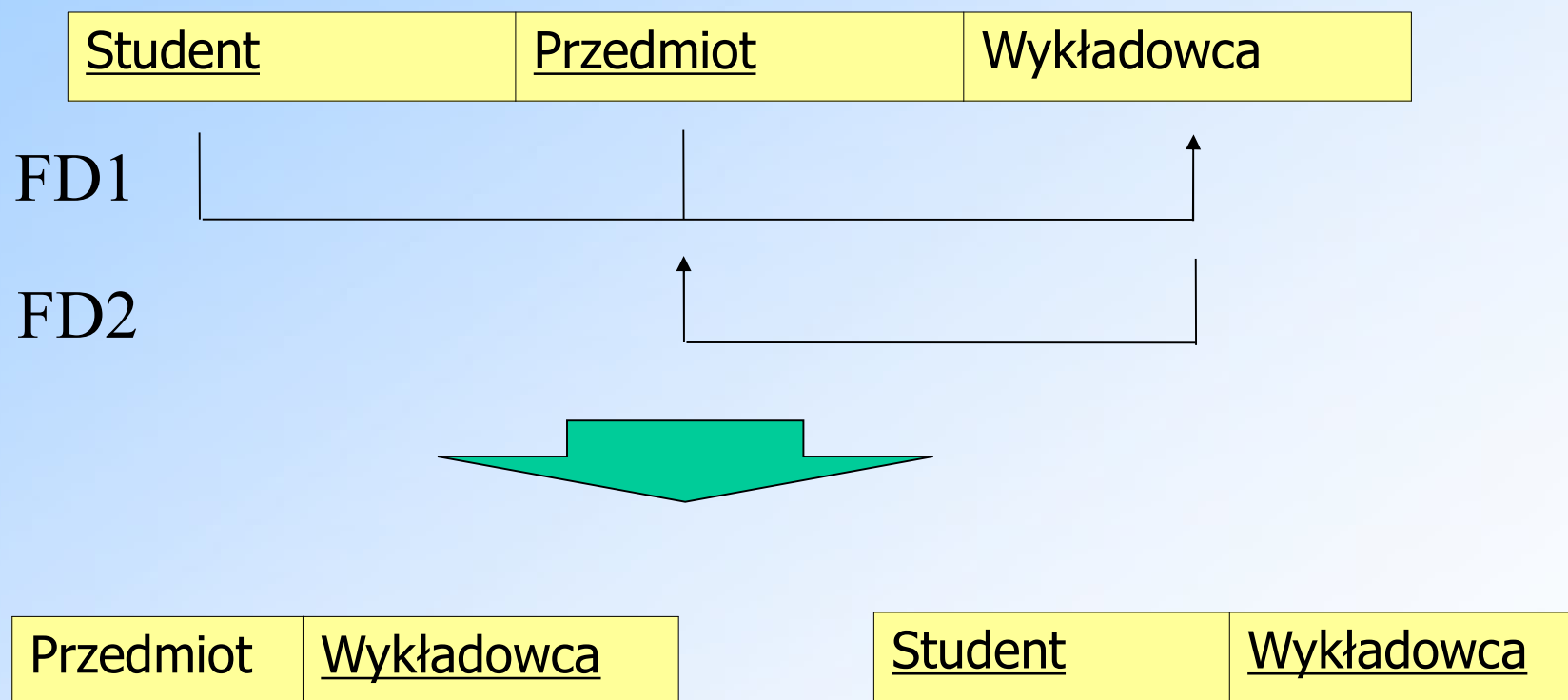


<u>NrWydz</u>	NazwaWydz	PESELDyrWydz
---------------	-----------	--------------



Postać normalna Boyce'a-Codda (BCNF)

Relacja znajduje się w postaci normalnej Boyce'a-Codda jeżeli zawsze, kiedy nietrywialna zależność funkcyjna $X \rightarrow A$ jest zachowana w R to X jest nadkluczem w R



Wyzwalacze i procedury składowane

- Są to procedury związane z tabelą , napisane w języku będącym rozszerzeniem SQL
- Przechowywane są bezpośrednio na serwerze SQL
- Wykorzystywane są w nich tzw. wyjątki, które służą do obsługi błędów pojawiających się podczas wykonywania wyzwalacza
- Wyzwalacze (triggers) wykonywane automatycznie w momencie aktualizacji, dodawania, usuwania wiersza z danej tabeli
- Wyzwalacze nigdy nie są wywoływane bezpośrednio
- Procedury składowane (stored procedures) wykonywane są na życzenie użytkownika

Wyzwalacze

Zalety stosowania wyzwalaczy

- Automatyczne stosowanie restrykcji zapewniających właściwe zakresy danych w kolumnach
- Redukcja nakładów pracy niezbędnych do konserwacji aplikacji korzystających z danej bazy danych
- Mechanizmy automatycznego logowania zmian w tabelach
- Automatyczne powiadamianie o zmianach w bazie za pomocą zdarzeń generowanych w wyzwalaczach

Procedury składowane

Zalety stosowania procedur składowanych

- Redukcja ruchu w sieci – procedury wykonywane są bezpośrednio na serwerze, do klienta odsyłane są tylko wyniki
- Zwiększenie wydajności zapytań – p.s. są już po etapie parsowania, optymalizacji i przetwarzania na język wewnętrzny serwera (execution plan)
- Umożliwiają tworzenie wielowarstwowej logiki aplikacji klienckich
- Pomagają w tworzeniu skutecznego systemu zabezpieczeń – np. zwracają tylko wyniki przetwarzania bez dostępu do danych szczegółowych

Tworzenie procedur składowanych

P.S. Można podzielić na dwa typy

- Procedury typu „SELECT” – można je używać w zapytaniach w miejsce zwykłej tabeli lub widoku. Definiowane są tak aby zwracać jedną lub więcej wartości
- Procedury wykonywalne – mogą być bezpośrednio wykonywane za pomocą wyrażenia EXECUTE PROCEDURE, nie muszą zwracać żadnych wyników
- Aby móc wykonywać P.S. Użytkownik musi mieć prawo EXECUTE

SQL – CREATE PROCEDURE (Firebird)

```
CREATE PROCEDURE nazwa
[( param typ [, param typ ...])]
[RETURNS ( param typ [, param typ ...])]
AS
<procedure_body>;

< procedure_body> = [<lista zmiennych>]
< blok>

<lista zmiennych> =
DECLARE VARIABLE var datatype;
[DECLARE VARIABLE var datatype; ...]

<blok> =
BEGIN
< wyrażenie>
[< wyrażenie> ...]
END
```

Tworzenie procedur składowanych

- Każde wyrażenie oprócz BEGIN i END musi być zakończone średnikiem, dlatego trzeba ustalić za pomocą SET TERM (lub DELIMITER) osobny znak kończący całą P.S.
- Dostępne są wszystkie konstrukcje SQL oprócz części DDL (definiowanie danych, praw ...)

```
SET TERM #;  
CREATE PROCEDURE SP_SUM_UP (  
    VAL_1    INTEGER,  
    VAL_2    INTEGER  
)  
RETURNS ( NEW_SUM INTEGER )  
AS BEGIN  
    NEW_SUM = VAL_1 + VAL_2;  
END #  
SET TERM ;#
```

Tworzenie procedur składowanych - Podstawowe konstrukcje 1

- **BEGIN END** – definiuje blok wyrażeń wykonywanych jako jedno wyrażenie
- Podstawianie wartości – operator =
- **/* */** - komentarz
- Zmienne wewnątrz zapytań SQL – poprzedzane dwukropkiem :

/* przykład */

IF (:zm_id_klienta IS NULL) THEN

BEGIN

klient = 'nieznany'

END

Tworzenie procedur składowanych - Podstawowe konstrukcje 2

- **IF** (warunek) **THEN** *poj_wyraż* [**ELSE** *poj_wyraż*]
- **FOR** *wyr_select* **DO** *pojedyncze_wyrażenie*
wykonuje *pojedyncze_wyrażenie* dla każdego wiersza zwracanego przez *wyr_select*, *wyr_select* - wyrażenie SELECT ... zakończone klauzulą INTO
- **DECLARE VARIABLE** *nazwa typ* [= *wartPocz*]
po słówku AS, przed pierwszym BEGIN
- **SUSPEND** – wstrzymuje wykonanie procedury aż aplikacja kliencka dokona odczytu danych (fetch), zwraca wartości procedury, zapobiega zakończeniu procedury przed odczytaniem danych przez klienta

Tworzenie procedur składowanych – parametry, zwracanie wartości

```
SET TERM #;  
CREATE PROCEDURE SP_GET_COUNTRY_DATA (ID    INTEGER )  
RETURNS (   
    ISO                CHAR(2) ,  
    CNAME              VARCHAR(40) ,  
    CARPLATE           VARCHAR(3)  
)  
AS BEGIN  
    SELECT  CT_ISO, CT_NAME, CT_CAR  
    FROM    COUNTRY  
    WHERE   CT_ID = :ID  
    INTO    :ISO, :CNAME, :CARPLATE;  
END #  
SET TERM ;#
```

Tworzenie procedur składowanych – Procedury typu „executable” i „selectable”

- EXECUTE **PROCEDURE** (<INPUT_PARAMS>)

Procedura tego typu zwraca jeden zestaw zmiennych

- SELECT <FIELD_LIST> FROM **PROCEDURE**
<WHERE>
<ORDER BY>

*Procedura „selectable” zwraca dowolną ilość wierszy paramatrów
– jak tabela*

Tworzenie procedur składowanych – Procedury typu „executable” i „selectable”

„executable”

```
CREATE PROCEDURE SP_SUM_UP (  
    VAL_1    INTEGER,  
    VAL_2    INTEGER,  
)  
RETURNS ( NEW_SUM INTEGER )  
AS BEGIN  
    NEW_SUM = VAL_1 + VAL_2;  
END
```

”selectable”

```
CREATE PROCEDURE SEL_CUSTOMER_IN_CITY ( CITY_NAME  VARCHAR(40) )  
RETURNS (  
    CUST_NAME  VARCHAR(40),  
    CUST_NO    VARCHAR(10)  
)  
AS BEGIN  
    FOR SELECT          CUST_NAME, CUST_NO  
    FROM CUSTOMERS  
    WHERE               CUST_CITY = :CITY_NAME  
    INTO                :CUST_NAME, :CUST_NO  
    DO BEGIN  
        SUSPEND;  
    END  
END
```

Tworzenie procedur składowanych - **Podstawowe konstrukcje 6**



Tworzenie procedur składowanych - Przykład

P.S. Zwracająca kilka wierszy – „selectable”

```
SET TERM ^;
```

```
CREATE PROCEDURE LISTA_ZAM(ID_KLI INTEGER)
```

```
RETURNS (NUMER_POZ CHAR(8))
```

```
AS
```

```
BEGIN
```

```
FOR SELECT NUMER_POZ FROM SPRZEDAZE WHERE  
ID_KLI=:ID_KLI INTO :NUMER_POZ
```

```
DO SUSPEND;
```

```
END ^
```

```
SET TERM ; ^
```

Tworzenie procedur składowanych - Podstawowe konstrukcje 5

```
CREATE PROCEDURE sprawdz( idKli CHAR(8) ) AS  
DECLARE VARIABLE idZam CHAR(8);  
BEGIN  
    FOR SELECT idZamowienia FROM sprzedaze WHERE  
    idKlienta = :idKli INTO :idZam  
    DO BEGIN  
        UPDATE klienci SET znacznik = '*' WHERE idKlienta = :idKli;  
    END  
END
```



Tworzenie procedur składowanych - Podstawowe konstrukcje 3

- **WHILE** (warunek) **DO** poj_wyraz

WHILE (i <=5) **DO**

BEGIN

SELECT count(*) FROM oceny WHERE

ocena = :i INTO :wynik;

IF (wynik IS NULL) THEN wynik='NULL';

i=i +1;

SUSPEND;

END

Tworzenie procedur składowanych - Podstawowe konstrukcje 4

- **EXCEPTION** nazwa – wywołuje wyjątek zwracający błąd do wywołującej aplikacji

```
CREATE EXCEPTION nieznanyId 'nieznany identyf. ID_KL !'
```

```
.
```

```
.
```

```
IF (status = 'brak') THEN  
BEGIN  
EXCEPTION nieznanyId;  
END
```



Tworzenie procedur składowanych - Podstawowe konstrukcje 5

• **WHEN {EXCEPTION x | SQLCODE x | ANY } DO** poj_wyraż

– obsługa ewentualnych błędów powstałych podczas wykonania P.S.,
musi wystąpić bezpośrednio przed kończącym END procedury

```
INSERT INTO klienci (nazwisko, id_kl) VALUES (:nazw, :id);  
WHEN SQLCODE -530 DO  
BEGIN  
  EXCEPTION nieznanyId;  
END
```

kody np:

-530 – foreign key violation

-803 – primary key duplicate

Tworzenie procedur składowanych – przykład MariaDB

```
DELIMITER //
CREATE PROCEDURE PokazZarobki()
BEGIN
    DECLARE koniec INT DEFAULT FALSE;
    DECLARE v_zarobki DOUBLE;
    DECLARE kursor_zarobkow CURSOR FOR SELECT zarobki FROM kl2;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET koniec = TRUE;

    CREATE TEMPORARY TABLE IF NOT EXISTS TymczasoweZarobki (
        zarobki_po_zmianie DOUBLE
    );
    TRUNCATE TABLE TymczasoweZarobki; -- Czyszczenie tabeli przed każdym uruchomieniem
    OPEN kursor_zarobkow;
    petla_start: LOOP
        FETCH kursor_zarobkow INTO v_zarobki;
        IF koniec THEN
            LEAVE petla_start;
        END IF;
        INSERT INTO TymczasoweZarobki VALUES (v_zarobki * 1.1);
    END LOOP petla_start;
    CLOSE kursor_zarobkow;
    SELECT * FROM TymczasoweZarobki;
END //

DELIMITER ;
```

Tworzenie procedur składowanych – obsługa błędów

```
SET TERM #;  
CREATE PROCEDURE SEL_PROCEDURE  
RETURNS (  
    FIELD_1 VARCHAR(40) ,  
    ...  
    ERROR_AT INTEGER  
)  
AS BEGIN  
    ERROR_AT = 0;  
    /* Step 1 */  
    ERROR_AT = 1;  
    ... CODE ...  
  
    /* Step 2 */  
    ERROR_AT = 2;  
    ... CODE ...  
  
    /* End of Code */  
  
    WHEN ANY DO  
    BEGIN  
        /* Return Variable ERROR_AT */  
        SUSPEND;  
    END  
END #  
SET TERM ;#
```

SQL – CREATE TRIGGER

```
CREATE TRIGGER nazwa FOR {tabela,widok}  
[ACTIVE | INACTIVE]  
{BEFORE | AFTER } {DELETE | INSERT | UPDATE}  
[POSITION liczba]  
AS  
<trigger_body>;  
  
< trigger_body> = [<lista zmiennych>]  
< blok>  
  
<lista zmiennych> =  
DECLARE VARIABLE var datatype;  
[DECLARE VARIABLE var datatype; ...]  
  
<blok> =  
BEGIN  
< wyrażenie>  
[< wyrażenie> ...]  
END
```

Tworzenie wyzwalaczy

Wyzwalacze

- Nie mają parametrów wejściowych
- Nie zwracają wyników
- Wzywane są automatycznie
- Dostęp do wartości atrybutów, które mogły ulec zmianie poprzez modyfikatory **new** i **old**

	INSERT	UPDATE	DELETE
:NEW	+	+	-
:OLD	-	+	+

Tworzenie procedur składowanych - Podstawowe konstrukcje 7

NEW.nazwaKolumny – nowa wartość kolumny (wyrażenia INSERT, UPDATE) - tylko wyzwalacze

OLD.nazwaKolumny – wartość przed wykonaniem UPDATE lub DELETE

Tworzenie wyzwalaczy - Przykład

Przykład

```
CREATE TRIGGER USTAW_ID_KLIENTA FOR KLIENCI  
ACTIVE BEFORE INSERT POSITION 0  
  
AS BEGIN  
  
IF new.ID_KLIENTA IS NULL THEN new.ID_KLIENTA  
= gen_id(GEN_ID_KLIENTA, 1);  
  
END ^
```



Tworzenie wyzwalaczy – Przykład (MariaDB)

Przykład

```
DELIMITER //
```

```
CREATE TRIGGER t_zarobki BEFORE UPDATE ON k12
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    IF NEW.zarobki < 3000 THEN
```

```
        SET NEW.zarobki = 3000;
```

```
    ELSEIF NEW.zarobki > 30000 THEN
```

```
        SET NEW.zarobki = 30000;
```

```
    END IF;
```

```
END //
```

```
delimiter ;
```



Procedury składowane i wyzwalacze - zdarzenia

Mechanizm zdarzeń (events) umożliwia powiadamianie aplikacji klienckich o zaistnieniu pewnych warunków – np dodanie nowego wiersza do tabeli przez innego klienta.

- Aplikacja kliencka deklaruje zdarzenie, którym jest zainteresowana:

```
EVENT INIT ev_dod_kli ( 'wstawiono_klienta' )
```

- Następnie w odpowiednim momencie oczekuje na zajście pożądanego zdarzenia

```
EVENT WAIT ev_dod_kli
```

Procedury składowane i wyzwalacze - zdarzenia

Przykład wyzwalacza generującego zdarzenie:

```
SET TERM !! ;  
  
CREATE TRIGGER POST_N_KLI FOR KLIENCI  
AFTER INSERT AS  
  
BEGIN  
  
    POST_EVENT 'wstawiono_klienta';  
  
END !!  
  
SET TERM ; !!
```